



AiNEKKO.

Erbium Minion Isotope TRM

Document Status: Release Candidate

Created By

Vijayvithal

✉ jvs@nekkko.ai

[AiFoundry](#)

Ref: 246c4e0+post1

June 23, 2026

Contents

Overview	8
Features	9
Signals	11
Test Signals	11
xSPI Target Interface	11
UART Target Interface	11
I2C Controller Interface	11
QSPI Controller Interface	12
GPIO Interface.	12
JTAG Signals	12
Pin Muxing	12
Bring up sequence.	14
First boot on the tester.	14
Second Boot.	15
Program Mode	15
Application Code	15
Power Reset & Clock Management.	16
Clock and reset signals.	16
System Clocks	16
OSC Clock	17
xSPI Clock	18
CPU Subsystem External Resets	18
Reset Sequencing	19
Valid Power State	20
Power State Controller FSM.	21
Interrupts	24
CPU Handling of platform interrupts	24
Interrupt handling process.	25



Memory Map	26
Interconnect: Transaction Initiators and Targets	26
CPU and UART Memory Map.	27
xSPI Memory Map.	27
XSPI Datasheet	29
Commands	29
Default Mode.	31
Not supported in Phase 1	32
Verification Requirement.	32
Bootting.	32
Error handling.	33
Errata:	33
RWDS Behavior	34
SFDP_Reg address map	34
SCCR_Reg address map	82
Erbium CPU Subsystem: High Level Architecture and Programming	
Model	87
Table of Contents	87
1. CPU Subsystem hierarchy	88
2. Programming Model	92
3 ISA changes (Erbium)	100
4. Erbium Memory	100
Minion	105
ISA changes	105
Errata fixes	105
CSR Changes	106
GPIO Datasheet	108
GPIO Registers	108
Register Descriptions	108
Pin Multiplexer	109
Program Flow	110
I2C Controller Datasheet	111
Memory Map	111
Registers	111
Enabling the I2C Peripheral	114
Program Flow	114
Error Handling	115
MRAM	116
Address Space	116
Top-Level Data Path	116



CONTENTS

AXI Front End	117
Read Flow	118
Write Flow	118
Exclusive Accesses	119
Bank Translator	119
ET Controller Path	120
ECC	120
Address Translation And Layout	121
Functional Guarantees	121
BIST And Maintenance Traffic	122
Clocking, Reset, And Power	122
Register Visibility	123
QSPI Datasheet	124
Features	124
Ports	124
QSPI registers	125
Register Descriptions	125
Features Tested	132
Program Flow	132
UART Datasheet	135
UART Registers	135
Register Descriptions	136
Program Flow	139
Steps for running verification	141
Simulation Environment	143
DV Master Plan	144
Architecture Freeze - DV Plan	144
Mission	144
Verification Strategy	144
DV Testbenches	144
DV Testbenches - EVL	145
DV Testbenches - ETL2AXI	145
DV Testbenches - CocoTB	146
DV Testbenches - MRAM	146
DV Testbenches - xSPI	147
DV Testbenches - I2C, QSPI, UART	148
DV Testbenches - SOC	148
DV Testbenches - Validation	149
DV Co-Simulation	149
Constrained Randoms	150



Verification Flows	150
DV tools	150
Verification Goals	151
DV Master Plan	151
Architecture Freeze - DV Plan	151
ErbiumxSPI_MemoryMap address map	152
mram memory	152
system_registers address map	152
mram_registers address map	168
bridge_regs register file	168
bank0_tregs register file	172
bank1_tregs register file	180
bank2_tregs register file	188
bank3_tregs register file	196
i2c_registers address map	204
qspi_registers address map	208
uart_registers address map	216
ROMRAM address map	223
ROM memory	223
SRAM memory	223
cpu_registers address map	223
User_neigh address map	224
User_cpu address map	224
Supervisor_cpu address map	239
D_hart_esr address map	241
dbg_hart register file	241
dbg_hart register file	245
dbg_hart register file	249
dbg_hart register file	253
dbg_hart register file	257
dbg_hart register file	261
dbg_hart register file	265
dbg_hart register file	269
dbg_hart register file	273
dbg_hart register file	277
dbg_hart register file	281
dbg_hart register file	285
dbg_hart register file	289
dbg_hart register file	293
dbg_hart register file	297



CONTENTS

dbg_hart register file	301
D_neigh address map	305
D_cpu address map	310
Machine_neigh address map	315
Machine_cpu address map	320
plic address map	327
nic_config address map	373
ni700_registers memory	373
ErbiumCPU_MemoryMap address map	374
system_registers address map	374
mram_registers address map	389
bridge_regs register file	390
bank0_tregs register file	394
bank1_tregs register file	402
bank2_tregs register file	410
bank3_tregs register file	418
i2c_registers address map	426
qspi_registers address map	429
uart_registers address map	437
ROMRAM address map	444
ROM memory	445
SRAM memory	445
xspi_registers address map	445
mram memory	449
cpu_registers address map	449
User_neigh address map	450
User_cpu address map	450
Supervisor_cpu address map	465
D_hart_esr address map	466
dbg_hart register file	467
dbg_hart register file	471
dbg_hart register file	475
dbg_hart register file	479
dbg_hart register file	483
dbg_hart register file	487
dbg_hart register file	491
dbg_hart register file	495
dbg_hart register file	499
dbg_hart register file	503
dbg_hart register file	507



dbg_hart register file	511
dbg_hart register file	515
dbg_hart register file	519
dbg_hart register file	523
dbg_hart register file	527
D_neigh address map	531
D_cpu address map	536
Machine_neigh address map	541
Machine_cpu address map	546
plic address map	553
nic_config address map	599
ni700_registers memory	599
Packaging	600
Changelog	605



Overview

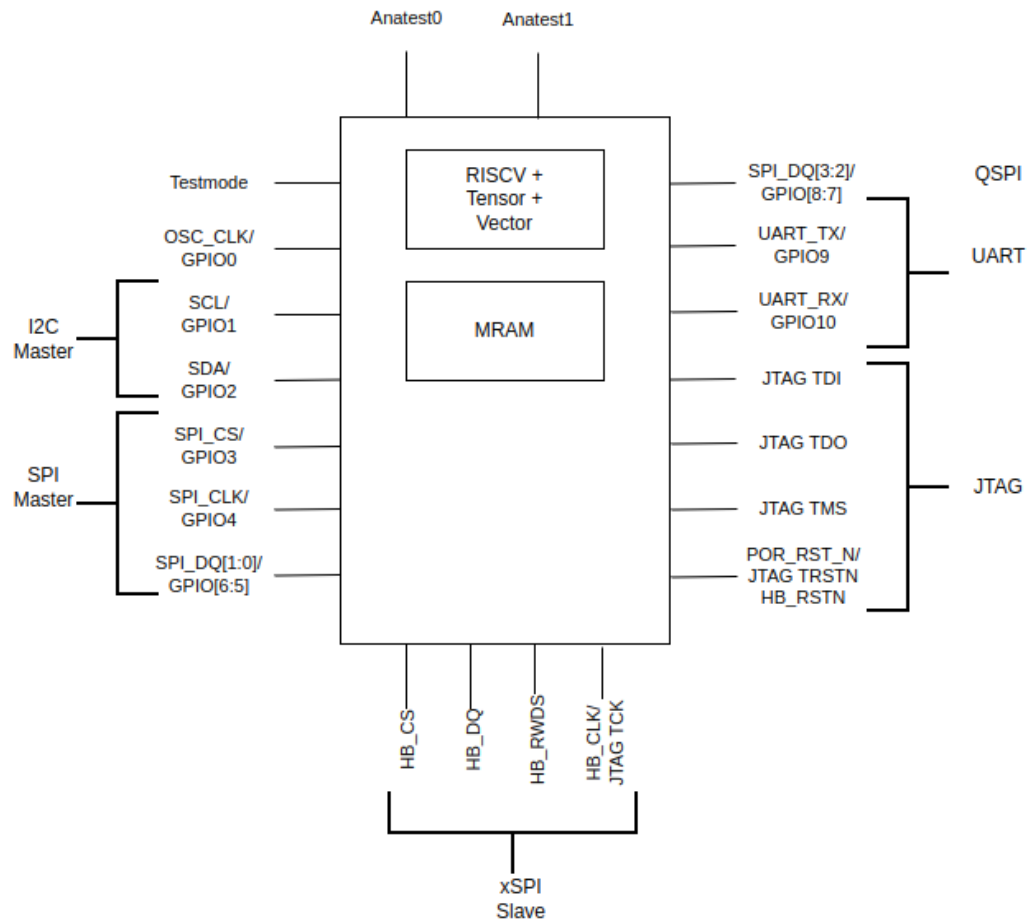


Figure 1: Erbuim RISC-V Isotope

Erbium is

- 8C,16T RISC-V Core, ISA(RV64IMFC_Zicsr_Zifencei +Tensor, +Vector +Atomic)
- I2C and QSPI Controllers
- xSPI and UART Target interfaces
- 16MB MRAM (L2) with 16KB of OTP region with 12KB of user usable OTP's
- Multiple Boot Options
 - Boot from Bootrom.



- Boot from MRAM. (via OTP setting)
- Boot from SRAM (in test mode)

Features

- External Interfaces
 - JTAG
 - UART
 - xSPI Target,
 - I2C Controller
 - (Q)SPI Controller
 - 11 GPIO
 - Chiplet Interfaces
 - ★ AHB
 - ★ AXI

xSPI Interface

- JESD251C and JESD216H compliant
- Supports
 - 1S-1S-1S
 - 4S-4D-4D
 - 4D-4D-4D
 - 8S-8S-8S
 - 8D-8D-8D (Octal)
 - 8D-8D-8D (Hyperbus)
- Operating frequency 200 Mhz
- 64 bit register and memory access
- Linear Burst capable.

Chiplet Interface

- 2 Interfaces AXI, AHB
- AXI and AHB in 32bit and 64 bit variants.
- Chiplet interface is not available in testchip tapeouts.

Note: While the chiplet interface is a part of the design it is not available (optimised away during PD) in testchips. i.e. erbium will not have the chiplet interface.

CPU Subsystem

- ET-Minion
 - RV64IMFC + Zicsr + Zifencei
 - In-Order execution with 2 Hardware Threads
- 8 ET-Minions
- L1 Instruction cache
 - Shared among the 8 ET-Minions
 - 128 sets, 4 way (set associative)
 - 64Byte cache line
- L0 Micro Instruction cache
 - Shared among 4 ET-Minions
 - 1 set, 16 ways (fully associative)
- Cooperative Tensor Load
 - Coalesces multiple Tensor Load requests to the same memory location (from different minions) into a single request to provide better performance and power.
- L1 Data Cache (**L1D\$**) - 4KB configurable



CONTENTS

- 16 sets, 4 ways, 64 Bytes cache line
- Non-coherent
- Configuration through mcache_control CSR
- Configurable **modes** :
 - ★ Shared(4KB)
 - ★ Split (2K Each)
 - ★ Scratchpad (3K Scratchpad, 512B Split)
- Vector Processing Unit (**VPU**)
 - 8 lanes, 32 bits per lane
 - 1 FMA unit, 2 IMA unit, 1 INT unit, 1 TRANS unit
 - Extends the RISC-V FP registers from 32b to 256b
- Each FP register is like a vector with **8** elements of **32b** each
- Related **custom extensions** :
 - ★ Packed Single (**PS**) –> FP32, memory and compute operations
 - ★ Packed Integer (**PI**) –> Int32/uInt32, compute operations
 - ★ Atomic (**AMO**) –> PS and PI AMO operations
 - ★ **Tensor** –> Memory, FMA32, FMA16A32, IMA8A32, QUANT, REDUCE operations



Signals

Test Signals

Signal	Direction	Note
ANATEST0	INOUT	Analog Test Pins
ANATEST1	INOUT	Analog Test Pins
TestMode	INPUT	Digital Testmode Enable. For internal use only. Tie to 0 in production.
OSC_CLK_OUT	OUTPUT	Oscillator clk div N

xSPI Target Interface

Port	Direction	Note
XS_CS	INPUT	Chip Select
XS_DQ,	INPUT	Data, Bidirectional
XS_RWDS,	INOUT	Data Strobe

UART Target Interface

Port	Direction	Note
Tx	OUTPUT	Transmit Pin
Rx	INPUT	Receive Pin

I2C Controller Interface

Port	Direction	Note
SCL	OUTPUT	Clock
SDA	INOUT	Data. Needs external pullup



QSPI Controller Interface

Port	Direction	Note
CLK	OUTPUT	Clock
DQ[3:0]	INOUT	Data
CS	OUTPUT	Chip select

GPIO Interface.

Port	Direction	Note
GPIO[10:0]	INOUT	All GPIO Pins are muxed with other IO protocols.

JTAG Signals

Port	Direction	Note
TCK,	INPUT	Clock. Shared with xSPI
TRSTn	INPUT	Reset. Active Low Shared with xSPI
JTAG_TDI	INPUT	
JTAG_TDO	OUTPUT	
JTAG_TMS	INPUT	

Pin Muxing

A Port	B Port	A_SEL
OSC_CLK_OUT	GPIO[0]	TestMode PIN
SCL	GPIO[1]	system_register.SystemConfig.i2c_enable
SDA	GPIO[2]	system_register.SystemConfig.i2c_enable
SPI_CS	GPIO[3]	system_register.SystemConfig.spi_enable
SPI_CLK	GPIO[4]	system_register.SystemConfig.spi_enable
SPI_DQ[1:0]	GPIO[6:5]	system_register.SystemConfig.spi_enable
SPI_DQ[3:2]	GPIO[8:7]	system_register.SystemConfig.qspi_enable



A Port	B Port	A_SEL
UART_TX	GPIO[9]	system_register.SystemConfig.uart_enable
UART_RX	GPIO[10]	system_register.SystemConfig.uart_enable

Note At boot SPI_enable=1 I2C_enable=0, QSPI_enable=0;



CONTENTS

Bring up sequence.

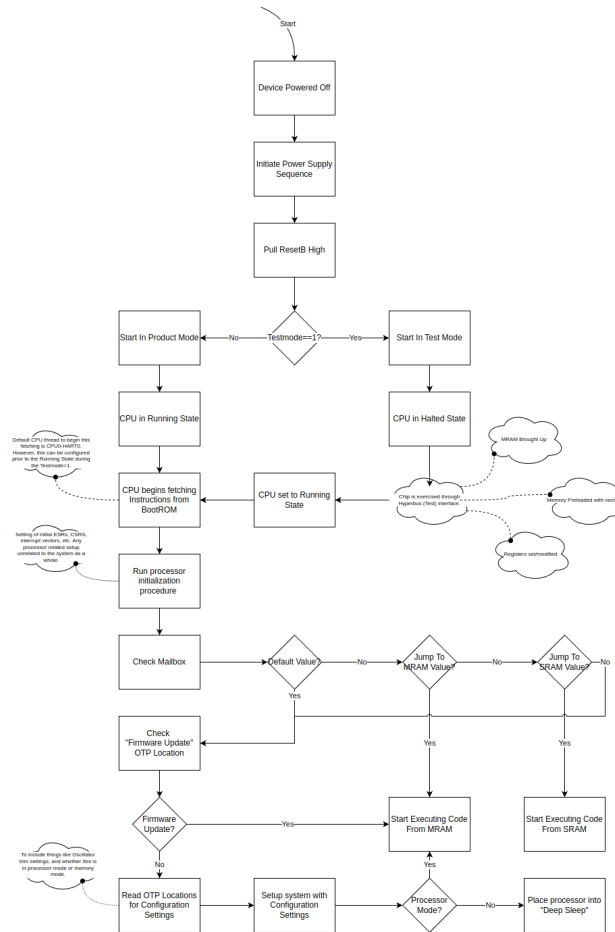


Figure 2: Boot sequence

Question: How to select single CPU for boot sequence?

Options:

- ✓ - Disabling All threads except for 1
- Powering down CPUs except for 1

How are we saving the values that say which threads are disabled for yield and which thread is being used for bootup?

- The yielding/disabled threads are chosen through an MRAM OTP value
- We'll always run CPU0-HART0 during boot up, but this can be overridden if desired during Testmode=1.

In the event of disabled cores, certain instructions that require a fully functioning Neighborhood will also be disabled.

First boot on the tester.

- TestMode pin is held high. CPU is halted (in reset mode?)
- POR is toggled.
- OSC_CLK_OUT is measured, the PPM difference from expected frequency is calculated.
- Osc TRIM Parameters are calculated. (Index in Trim table?)
- Part is speed graded.
- Osc TRIM parameters and speed grade is written to OTP



Second Boot.

- TestMode is held low.
- CPU Starts executing bootrom code.
 - OTP is read(Trim Param, Speed grade, ROM Update).
 - If ROM Update bit set, jump to updated firmware in MRAM.
 - Trim Ring Oscillator.
 - Program RO-CLK Div for highest clk freq supported by device speed grade.
 - Calculate Clk Div for Periph and MRAM.
 - Program Periph clkdiv and mram clkdiv.
 - Read CHIPMODE.bootloader register to determine location of application code (TCM,MRAM(default))
 - Jump to application code.

Program Mode

- Bring MRAM out of deep sleep.
- Load MRAM/TCM with the application code
- Set CHIPMODE.bootloader register.

Application Code

- If MRAM is in deep sleep mode it needs to be brought out of sleep.
 - Write to `system_registers.PowerDomainReq.mram_dsleep_en` to bring MRAM out of deep sleep.
- Use `system_registers.Mailbox0.mbox0` and `system_registers.Mailbox1.mbox1` for communication from CPU to host chip and vice versa.



Power Reset & Clock Management.

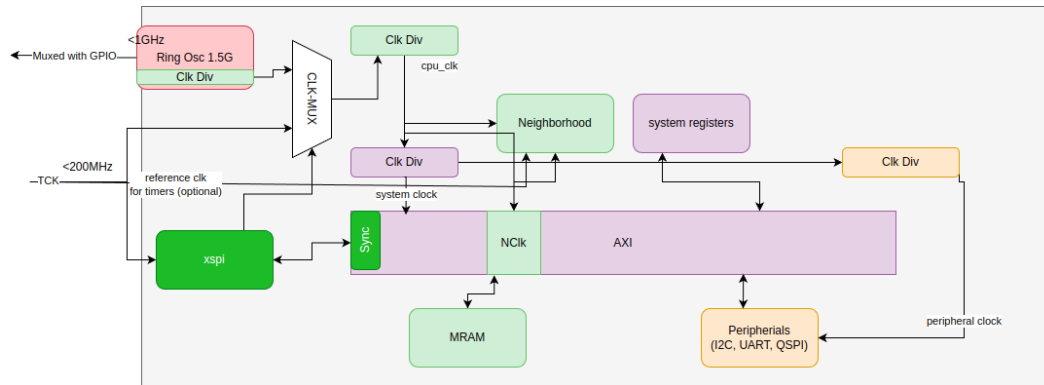


Figure 3: Clock Domains

Clock and reset signals.

System Clocks

The Device has two clock sources

Clock	Source	Notes
TCK	Pin	XSPI Clock Max 200 Mhz.
OSC_CLK	Ring Oscillator	Internal Clock Max 1GHz +- 500 MHz

The OSC_CLK goes through a series of clock Dividers to generate 3 clocks:

- CPU Clock,
- System Clock
- Peripheral Clock

Clock	Connected IP
CPU Clock	Neighbor, MRAM, MRAM Registers, CPU Registers
System Clock	aon logic, PRCM, system reg, BOOTROM, SRAM
Peripheral Clock	UART, I2C, QSPI
XSPI CLK	XSPI

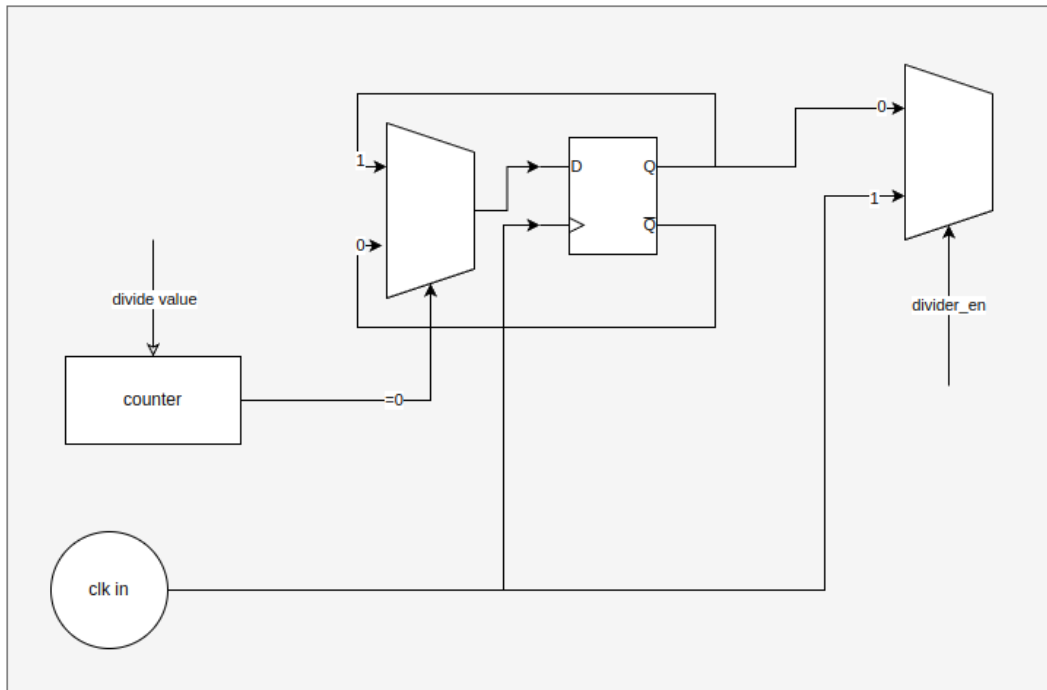


Figure 4: Clock Divider

OSC Clock

- This is a high speed clock generated internally from a ring oscillator.
- The raw ring oscillator(RO) frequency is process dependent and can vary by a few hundred MHz from the designed frequency.
- The first level clock divider is programmed to divide the output of the RO by 32 resulting in a <31 MHz boot clock.
- The tester measures the RO frequency, and process variation and writes it to the OTP. This is used to
 - Trim the RO to generate a 1GHz clock.
 - Program the clock divider to provide the required highspeed CPU Clock.
 - Clock dividers can be reprogrammed later as per the system load.
- A second clock divider divides the **high speed cpu clock** to generate the **system clock**
- Serial protocols require a clock programmed to N times the required baud rate. The Peripheral clock is used as this sampling clock.



xSPI Clock

- This is an external clock with frequency upto 200MHz.
- If required a bit in the xspi register can be written to, to make this the system clock.

CPU Subsystem External Resets

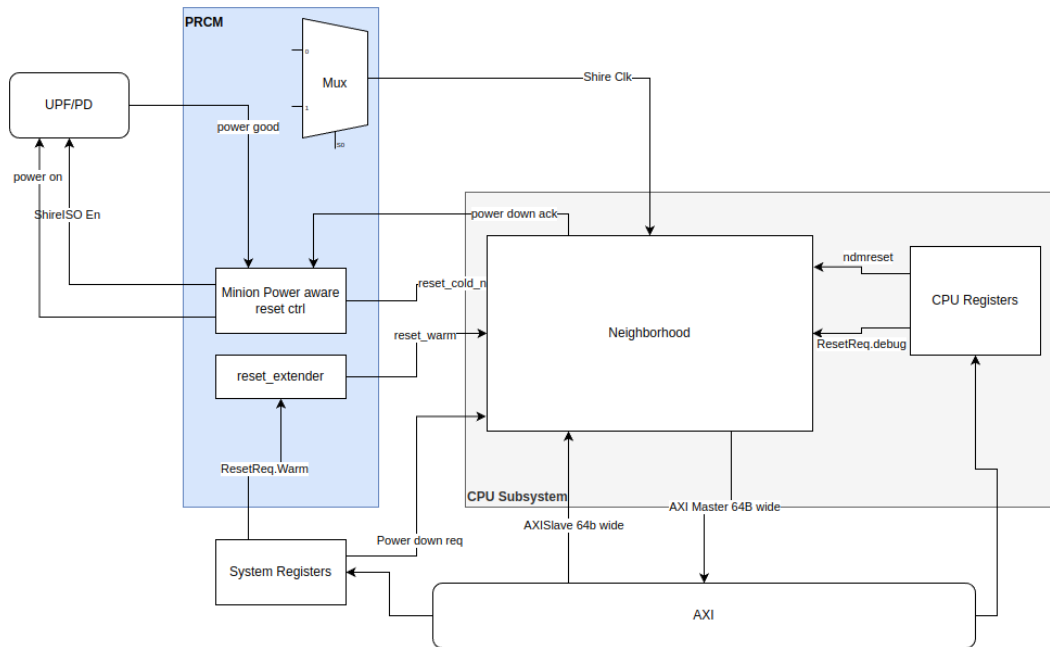


Figure 5: CPU Subsystem Integration

Reset	Description	Register Programming
reset_cold	Full Power on reset	From PRCM (POR,soft_reset, brownout)
reset_warm	Retains state: Does not reset ESR, VC FIFOs etc.	SoftReset.cpu_warm_reset = 0

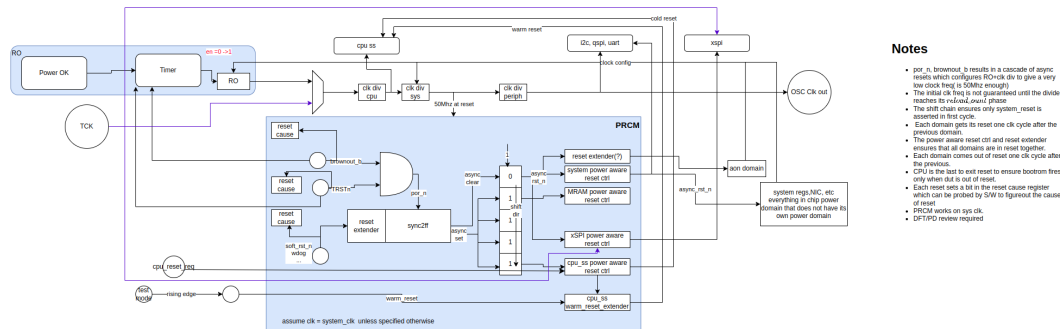


Figure 6: Reset Sequence

Reset Sequencing

At power on the Ring oscillator and clock dividers have unknown configuration. After reset is applied

- The ring oscillator immediately switched to the default configuration (clock divider enabled, slowest speed).
- Each clock divider latches its configuration(div by 32) only after its internal counter reaches the reload phase.
- The first reset extender ensures that the device remains in reset state until the system divider reloads.
- Power on reset asynchronously resets system registers and xspi ensuring that the correct configuration reaches the ring oscillator, clock dividers and tck/osc_clk mux.
- A Shift register sequences the reset to other modules one by one ensuring that all modules do not enter the reset stage at the same time resulting in a large power drop.
- The reset extender of the power aware reset module ensures that all modules remain in reset for atleast a few common cycles.
- Cpu_system is the last entity to be brought out of reset to ensure that it does not start accessing memory regions that are still in reset.
 - At this point cpu subsystem can start programming the ring oscillator and clock dividers to speed up the clock. ## Power Domains.

The device contains the following power domains.

- Always On.
- System Domain.
- CPU Subsystem Domain.



CONTENTS

- Per Minion Power Domain.
- MRAM Digital Domain
- Each Memory (Bootrom, SRAM and MRAM) has its own power domains.
- Each powerdomain is controlled via a power switch.
- At Boot every Digital Domain powerswitch is turned on.
- The boot routine will check the OTP register for domains that need to be powered down after boot and write to the corresponding system_registers.PowerDownReq bits.

Valid Power State

Simple Power States

State	Sys PS	MRAM Digital PS	MRAM PS	cpu subsys-tem PS	ROM/SRAM PS	minion PS[7:0]	Notes
All On	On	On	On	On	On	On	
System Off	Off	Off	Off	Off	Off	Off	Extreme low power. Toggle TMS to wake the device
CPU Off	On	X	X	Off	X	Off	
MRAM Off	On	Off	Off	X	X	X	
1-8 Minions Off	On	X	X	On	On	On/Off	
Controlling PowerDomainReq register	system poweroff	mram pd	mram dsleep en	cpu pd	sram pd	minion pd__[0-7]	

Note:

- X: Dont care. The domain is usually on. But can be off if a complex Power State is used.



- Other than the system off state all other power switches can be turned on by clearing the corresponding pd_req bit.
- Complex Power States are:
 - CPU OFF + MRAM OFF
 - Minion_x OFF + MRAM OFF

Power State Controller FSM.

- At boot the FSM is in normal state.
- Writing to pd_req initiates the power down sequence.
- The corresponding domain is informed about the request.
- The domain completes the current outstanding transaction and frees up the system bus and goes into a halt state.
- Domain generates the PD ack signal.
- PD ack signal triggers the Power State Controller which executes the following state changes:
 - ISO_EN State: drives the isolation enable signal to the isolation cells placed on the output wires of the power domain.
 - POWEROFF State: turns off the power switch and remains in this state until pd req goes low, This can be due to:
 - ★ Toggling of TMS signal in case of system power off
 - ★ Clearing the corresponding PD Register bit.
 - When the pd req goes low this state turns on the power switch and transitions to WTGood State.
 - WTGOOD State: It takes some time to ramp up the voltage from 0 to VDD. In this state:
 - ★ A counter is initialized with the system_registers.PowerGood.counter register value and counts down to zero.
 - ★ State changes to ISO_OFF when the counter is zero.
 - ★ soft_reset_req is asserted and stays asserted until we enter the NORMAL state.
 - ISO_OFF: the isolation enable signal to the isolation cells is de-asserted.
 - ★ Note: It is expected the reset values of the domain output signals do not initiate or accept any transaction.
 - RESET: We stay in this state for the number of cycles required to propagate the reset to all sub modules in the power domain.



CONTENTS

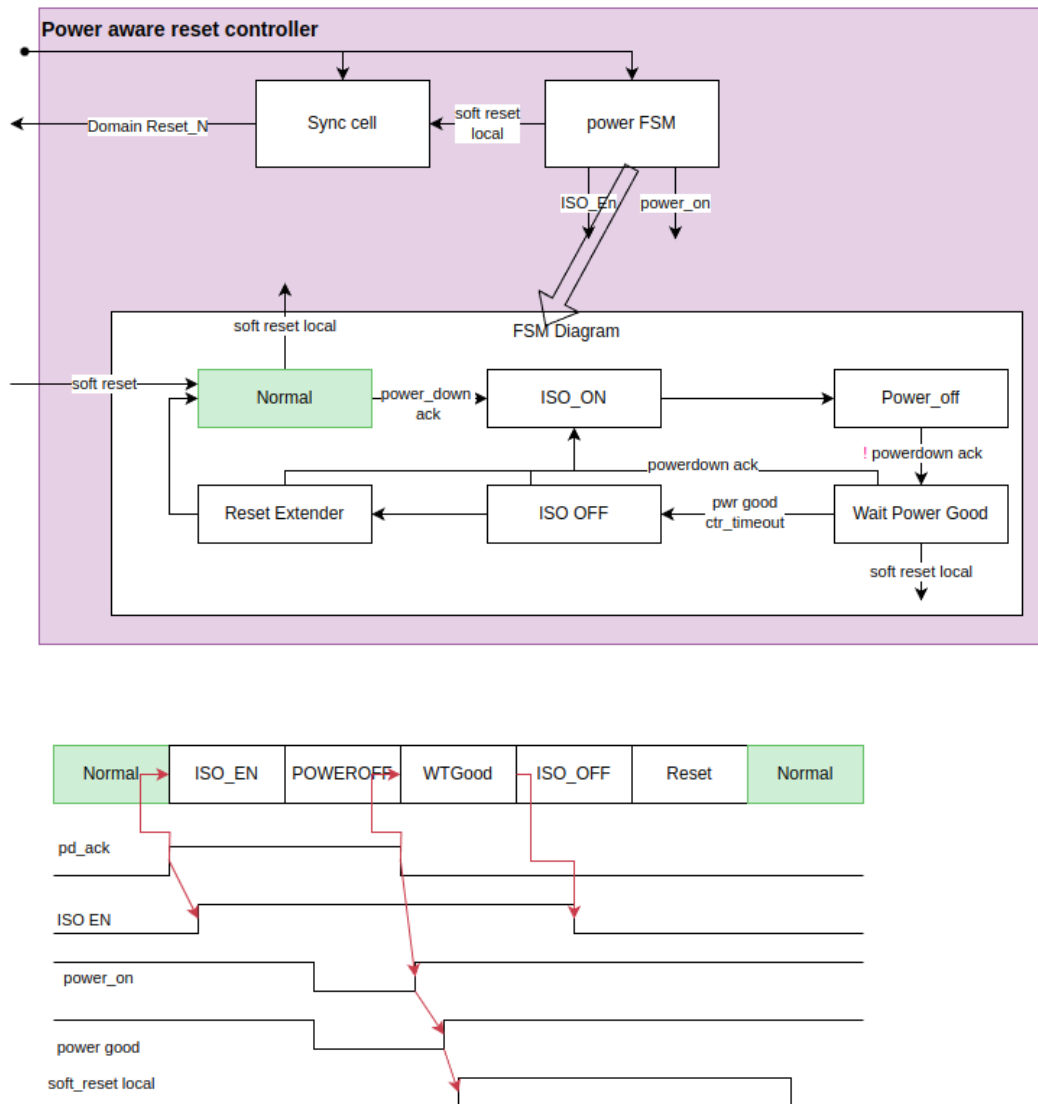


Figure 7: Power Aware reset



Erbium Minion Isotope TRM

- NORMAL: Normal operation resumes.



Interrupts

There are two sets of interrupts, CLINT (Timer and software interrupts) are described in the CLINT specification.

This chapter describes the platform interrupts.

The platform interrupts are generated from the following endpoints

- MRAM Interrupt: Raised by MRAM on ECC failure, to trigger a wellness check
- GPIO Interrupt: Raised by the GPIO module when any of the input signal changes state.
- Register Interrupt: A specific bit in the system register that can be written to to trigger an interrupt, e.g. this can be used by xSPI to interrupt the CPU and process the mailbox.
- Periph Interrupt: These interrupts are generated by QSPI, UART, I2C and xSPI Hardware to indicate state changes like TxDone, RxDone, FIFO under/over flow etc.

CPU Handling of platform interrupts

System interrupts are handled by the CPU subsystem PLIC which implements the RISC-V PLIC 1.0 specification. The implementation details are:

- The PLIC gateway assumes the connected interrupt sources are level-signals and they are synchronized to the CPU clock domain.
- Interrupts priorities are not configurable:
 - Priorities registers are hardwired to 1
 - Threadhold registers are hardwired to 0
- From the CPU point of view, PLIC base address is 0xA0000000
- Supports 32 contexts (2 contexts per hart, one M-mode and one S-mode).

The following table describes the IDs for each interrupt source.

Interrupt	ID
<i>Reserved</i>	0
MRAM interrupt	1
QSPI	2
UART	3



Interrupt	ID
System Register	4
xSPI	5
GPIO	6

Interrupt handling process.

This is a short overview. Refer to the RISC-V privileged ISA for details

- A interrupt from a peripheral sets the corresponding bit in the corresponding Interrupt Pending register
- This is ANDed with the per-context (hart+priv mode) enable bits and forwarded to the context
- If the context have external interrupts enable, it will trap to their handler and read their Claim register
- The PLIC will respond with the enabled interrupt with the lowest id that is pending and mark it as claimed.
- Multiple harts can try to claim the same interrupt. Only one will get the id as a response.
- The hart that claimed the interrupt will handle it. After this process, the level-signal from the interrupt source must go low.
- The hart will mark the interrupt as completed by writing into the Claim register. This will free up the interrupt and can be claimed again if the pending bit is set.



Memory Map

Interface	Type
CPU_SUBSYSTEM	AXI
CPU_SUBSYSTEM	APB
xSPI	AXI
xSPI	APB
I2C	AXI4Lite
QSPI	AXI4

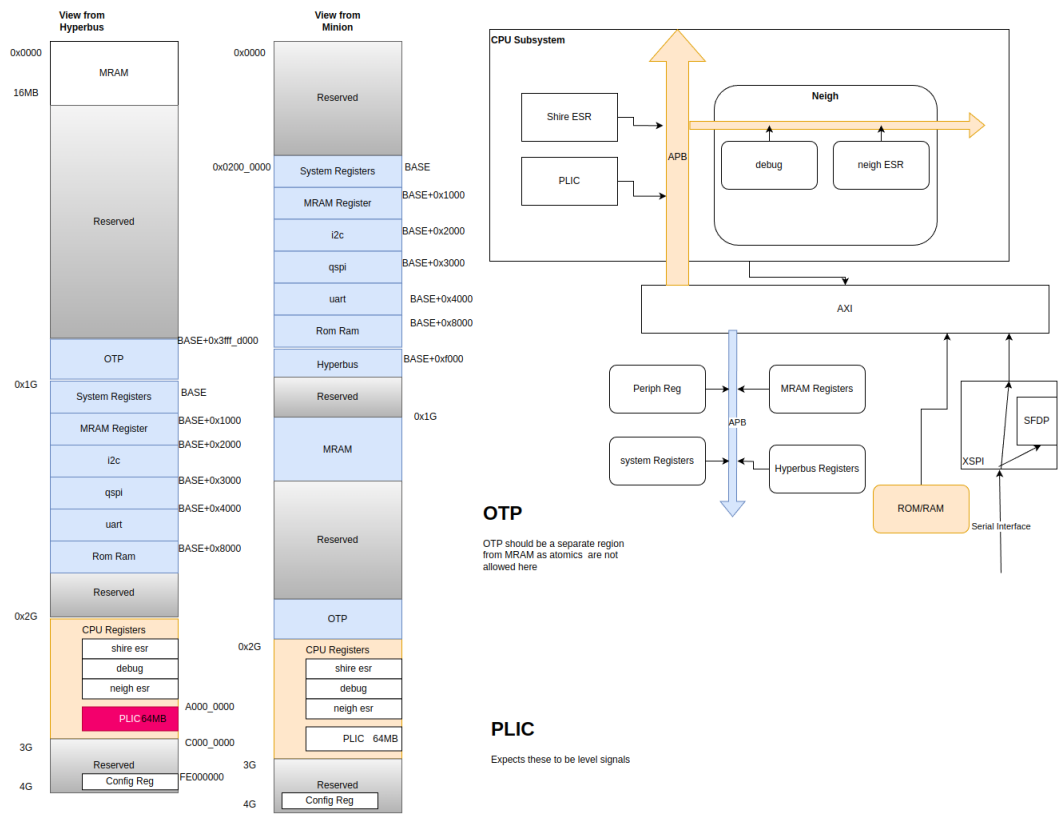


Figure 8: Busmatrix Pathways

Interconnect: Transaction Initiators and Targets

There are 3 Major Transaction Initiators in the System.



1. xSPI
2. CPU
3. UART

Initiator	Interface	Targets Accessible
xSPI	AXI4	MRAM, SRAM, System Registers, Peripheral Registers, HyperBusRegisters, CPURegisters, MRAMRegisters
CPU	AHB-Lite	MRAM, SRAM, Bootrom, System Registers, Peripheral Registers, HyperBusRegisters, CPURegisters, MRAMRegisters
UART	AXI4	MRAM, SystemRegisters

This device can be used as either an Edge AI device or as a simple Flash replacement. The memory Map seen by CPU and UART (Edge AI Mode) is different than the memory map seen by xSPI(Flash replacement device)

CPU and UART Memory Map.

Offset	Identifier	Name
0x02000000	system_registers	System Registers
0x02001000	mram_registers	Erbium MRAM Block Test Register map.
0x02002000	i2c_registers	—
0x02003000	qspi_registers	—
0x02004000	uart_registers	—
0x0200A000	SRAM	—
0x0200F000	qspi_registers	sccr
0x0E000000	nic_configuration_space	—
0x40000000	mram	MRAM
0x80000000	cpu_registers	—

xSPI Memory Map.

Customers accessing Erbium through xSPI interface are expected to use it as a MRAM Memory device. In such a case it is natural to assume the memory



CONTENTS

starts from address 0. Hence in Hyperbus case the addressmap is rearranged to facilitate this world view.

Offset	Identifier	Name
0x00000000	mram	MRAM
0x0E000000	(nic_configuration_space	—
0x40000000	system_registers	System Registers
0x40001000	mram_registers	Erbium MRAM Block Test Register map.
0x40002000	i2c_registers	—
0x40003000	qspi_registers	—
0x40004000	uart_registers	—
0x40005000	SRAM	—
0x80000000	cpu_registers	—

Note All Register address spaces are 64 bit aligned.



XSPI Datasheet

- xSPI bus interface supporting
 - 1S-1S-1S (SPI)
 - 4S-4D-4D
 - 4D-4D-4D
 - 8D-8D-8D (Octal profile 1)
 - 8D-8D-8D (Hyperbus Octal Profile 2)
- Boot configuration SPI, Quad, Octal Profile 1, Hyperbus.
- Hardware Reset.
- Deep Powerdown mode
- Based on JESD251C, JESD215C-1, JESD216H

Commands

SPI, Quad SPI, Octal SPI Mode

Commands	Code	1S-1S-1S	4S-4D-4D	8D-8D-8D(Octal)
Read SFDP	5Ah	0.E	1.B	1.B
Read (0L)	NA	NA	NA	NA
Read Register	65	0.F	1.B	1.B
Write Register	71	0.K	1.D	1.D
Reset Device	99h	0.A	1.A	1.A
and Enter default mode(1S)				
Read Memory	0Bh	0.F	1.B	1.B
Write Memory	02h	0.K	1.D	1.D
setRate	52h	0.G	1.A	1.A

SetRate command writes a tuple of 3 bytes, each byte specifies the rate for cmd, addr, data. The encoding is

- S1=0,
- D1=1,
- S2=not Implemented,
- D2 =NotImplemented,
- S4=4,
- D4=5,



CONTENTS

- S8=6,
- D8=7,
- HB=8

Format 1S-1S-1S

The following formats are applicable to 1S-1S-1S mode of operation.

Format	Ext	Address Bytes	Latency	Read Txn	Write Txn	Note
0.A	✗	✗	✗	✗	✗	CMD Only
0.E	✗	3	✓	✓	✗	Read Reg
0.F	✗	4	✓	✓	✗	Read Mem
0.G	✗	✗	✗	✗	✓	setRate
0.J	✗	3	✗	✗	✓	Write Reg
0.K	✗	4	✗	✗	✓	Write Mem

Format Other rates

- The following formats are applicable to 4S/4D/8S/8D modes of operation.
- These modes require Ext and address is always 4.

Format	Ext	Address Bytes	Latency	Read Txn	Write Txn	Note
1.A	✓	✗	✗	✗	✗	CMD Only
1.B	✓	4	✓	✓	✗	Read Reg/Mem
1.D	✓	4	✗	✗	✓	Write Reg/Mem
1.G	✓	✗	✗	✗	✓	setRate



Default Mode.

The standard allows the datarate at reset to be something other than SPI(1S-1S-1S). This can be controlled via the `default_mode` pin.

Default Mode	Default Rate
00	Hyperbus
01	Octal SPI
10	Quad SPI
11	SPI

For Hyperbus mode all 3 data bytes of setrate should be HB.

- SPI Memory access uses 4B addressing.
- SPI Register access uses 3B addressing.
- Some commands marked 1.A do not require an address field. e.g. Enter Pd, Exit PD, Reset Device.

Note

- 1.G is a non standard format for set rate
- All writes have zero latency
- All Reads have latency cycles.
- 1s can have 3 or 4 byte address, use 3 byte for xspi-register and 4 byte for memory transactions
- non 1S only support 4 byte address.
- The 1.X commands follow the pattern of CMD-Ext, 4B Address, X cycles of latency, Data
- The 0.X commands do not have Ext
- For Register Transaction the Data is 32 bit.
- For Memory Transaction Data is multiple of 64 bit.
- Behavior when data width is not multiple of 64 (for mem) or 32 for reg is not defined.



- We will not implement read zero latency as that will not work with our pipeline.
- Supported(Tested) Modes are (ref 6.10.18 of SFDP)
 - * 1S-1S-1S
 - * 8D-8D-8D
 - * 8S-8S-8S
 - * 4S-4D-4D
 - * 4D-4D-4D
 - * 4S-4S-4S

Not supported in Phase 1

Mode	Plan
Wrapped bursts	(Phase 2)
XIP	(Phase 2)
Write Enable	Not Supported
Page/Sector Erase/Program	Not Supported

Verification Requirement.

- Verify all supported mode.
- For spi, quadspi, octalspi do a cross coverage of command, rates, latency, burst length, mem/reg, read/write
- Verify SFPD.
- Verify contents and behavior of register space.
- Ensure register space is accessible from both minion and xspi.
- verify hyperbus ensure cross coverage for latency and burst
- Ensure all functionality is implemented.

Booting.

- The device requires a few cycles of xspi clock to complete the reset process.
- Whenever the XSPI IP is resetted e.g. por, soft_reset etc. Run TCK for 8 cycles.
- If the xspi controller is not capable of running TCK without a transaction, issue a transaction with cmd=0



Error handling.

Errors are generated in cases where

1. Host tries to access invalid memory space.
2. Host transfer does not complete due to network congestion.

`sccr.interrupt_status` register should be checked after each transaction for errors. The fields are as follows.

- `axi_resp`: The last known Error from the interconnect.
- `axi_read_underflow`: Set when a read transaction is issued and read data is not available before end of latency period or the next word of a burst is not available at the time when the last bit of current burst is sent on wire.
- `axi_write_overflow`: Set when the host issues a write burst transaction and the next word is presented before the current word is consumed by the interconnect.

Once set these bits remain set and can be cleared by reading the interrupt register.

If the `sccr.xspi_control.interrupt_enable` bit is set, a non zero value in the `interrupt_status` register can generate an interrupt to the minion. It is recommended to keep the interrupt disabled and let the host manage the exception. Only in case where the host cannot manage exceptions should minion step in.

Recommended host flow.

1. For write operations, Poll `sccr.xsip_status.wip` until the bit is cleared.
2. Check the value of `sccr.interrupt_status` if it is non zero initiate error recovery (e.g. retry the transaction at a lower data rate, increase latency, reduce burst size, reduce clock frequency, check the destination address etc.)
3. If it is zero, write to the mailbox to inform the minion the transaction is over and it can take control of the buffer.

Errata:

There are errors in the JEDC 251C Specification.



- The Profile2 Hyperbus bit map assigns A31:A3 to a 24 bit field instead of a 29 bit field. The upper 5 bits are marked as reserved. We follow the Cypress Hyperbus bitfield here and use the upper 5 bits to form the required 29 bit address address.
- In 251C-1 the Specification for QuadSPI defines a format for Quad SPI but does not use it. We are ignoring the format and using 1.X format for Quad mode.

RWDS Behavior

State	DQ	RWDS	RWDS_Val
Idle	In	Out	0
Cmd	In	Out	0
Ext	In	Out	0
Latency	In/Out	Out	0
Latency to Read	out	out	o
Read	out	ou	toggle
Latency to Write	In	In	0
Write	In	In	toggle
HB-CMD	In	Out	0
HB-Address	In	Out	0
HB-Latency-Write	In	In	0
HB-Latency-Read	Out	Out	0
HB-Write	In	In	Toggle
HB-Read	Out	Out	Toggle

SFDP_Reg address map

- Absolute Address: 0x0
- Base Offset: 0x0
- Size: 0xB3C

Offset	Identifier	Name
0x000	reg6_2_1	—
0x004	reg6_2_2	—



Erbium Minion Isotope TRM

Offset	Identifier	Name
0x008	reg6_4_1	—
0x00C	reg6_4_2	—
0x010	reg6_7_1	—
0x014	reg6_7_2	—
0x018	reg6_9_1	—
0x01C	reg6_9_2	—
0x020	reg6_10_1	—
0x024	reg6_10_2	—
0x028	reg6_11_1	—
0x02C	reg6_11_2	—
0x030	reg6_17_1	—
0x034	reg6_17_2	—
0x400	reg_6_4_4	—
0x404	reg_6_4_5	—
0x408	reg6_4_6	—
0x40C	reg6_4_7	—
0x410	reg6_4_8	—
0x414	reg6_4_9	—
0x418	reg6_4_10	—
0x41C	reg6_4_11	—
0x420	reg6_4_12	—
0x424	reg6_4_13	—
0x428	reg6_4_14	—
0x42C	reg6_4_15	—
0x430	reg6_4_16	—
0x434	reg6_4_17	—
0x438	reg6_4_18	—
0x43C	reg_6_4_19	—
0x440	reg_6_4_20	—
0x444	reg_6_4_21	—
0x448	reg_6_4_22	—
0x44C	reg_6_4_23	—
0x450	reg_6_4_24	—
0x454	reg_6_4_25	—
0x458	reg_6_4_26	—
0x700	reg_6_7_3	—
0x704	reg_6_7_4	—
0x900	reg_6_9_3	—



CONTENTS

Offset	Identifier	Name
0x904	reg_6_9_4	—
0x908	reg_6_9_5	—
0xA00	reg_6_10_3	—
0xA04	reg_6_10_4	—
0xA08	reg_6_10_5	—
0xA0C	reg_6_10_6	—
0xA10	reg_6_10_7	—
0xA14	reg_6_10_8	—
0xA18	reg_6_10_9	—
0xA1C	reg_6_10_10	—
0xA20	reg_6_10_11	—
0xA24	reg_6_10_12	—
0xA28	reg_6_10_13	—
0xA2C	reg_6_10_14	—
0xA30	reg_6_10_15	—
0xA34	reg_6_10_16	—
0xA38	reg_6_10_17	—
0xA3C	reg_6_10_18	—
0xA40	reg_6_10_19	—
0xA44	reg_6_10_20	—
0xA48	reg_6_10_21	—
0xA4C	reg_6_10_22	—
0xA50	reg_6_10_23	—
0xA54	reg_6_10_24	—
0xA58	reg_6_10_25	—
0xA5C	reg_6_10_26	—
0xA60	reg_6_10_27	—
0xA64	reg_6_10_28	—
0xA68	reg_6_10_29	—
0xA6C	reg_6_11_3	—
0xA70	reg_6_11_4	—
0xB00	reg_6_11_5	—
0xB04	reg_6_11_6	—
0xB08	reg_6_11_7	—
0xB0C	reg_6_11_8	—
0xB10	reg_6_11_9	—
0xB14	reg_6_11_10	—
0xB18	reg_6_11_11	—



Offset	Identifier	Name
0xB1C	reg_6_11_12	—
0xB20	reg_6_11_13	—
0xB24	reg_6_11_14	—
0xB28	reg_6_11_15	—
0xB2C	reg_6_17_3	—
0xB30	reg_6_17_4	—
0xB34	reg_6_17_5	—
0xB38	reg_6_17_6	—

reg6_2_1 register

- Absolute Address: 0x0
- Base Offset: 0x0
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	signature	r	0x50444653	—

signature field SFDP Signature**reg6_2_2 register**

- Absolute Address: 0x4
- Base Offset: 0x4
- Size: 0x4

Bits	Identifier	Access	Reset	Name
7:0	minor	r	0xC	—
15:8	major	r	0x1	—
23:16	numHdr	r	0x6	—
31:24	access_protocol	r	0xFA	—

minor field SFDP Version Minor**major field** SFDP Version Major



numHdr field Num Parameter Header TBD

access_protocol field TBD SFDP Access protocol

reg6_4__1 register

- Absolute Address: 0x8
- Base Offset: 0x8
- Size: 0x4

Bits	Identifier	Access	Reset	Name
7:0	id	r	0x0	—
15:8	minor_rev	r	0x9	—
23:16	major_rev	r	0x1	—
31:24	tbl_len	r	0x17	—

id field parameter id lsb

minor_rev field minor rev

major_rev field major_rev

tbl_len field Tbl Len

reg6_4__2 register

- Absolute Address: 0xC
- Base Offset: 0xC
- Size: 0x4

Bits	Identifier	Access	Reset	Name
23:0	param_tbl_ptr	r	0x400	—
31:24	param_id_msbr	r	0xFF	—

param_tbl_ptr field Parameter Table Pointer TBD



param_id_msb field Parameter Table ID MSB

reg6_7__1 register

- Absolute Address: 0x10
- Base Offset: 0x10
- Size: 0x4

Bits	Identifier	Access	Reset	Name
7:0	id	r	0x84	—
15:8	minor_rev	r	0x1	—
23:16	major_rev	r	0x1	—
31:24	tbl_len	r	0x2	—

id field parameter id lsb

minor_rev field minor rev

major_rev field major_rev

tbl_len field Tbl Len

reg6_7__2 register

- Absolute Address: 0x14
- Base Offset: 0x14
- Size: 0x4

Bits	Identifier	Access	Reset	Name
23:0	param_tbl_ptr	r	0x700	—
31:24	param_id_msb	r	0xFF	—

param_tbl_ptr field Parameter Table Pointer TBD

param_id_msb field Parameter Table ID MSB



CONTENTS

reg6_9__1 register

- Absolute Address: 0x18
- Base Offset: 0x18
- Size: 0x4

Bits	Identifier	Access	Reset	Name
7:0	id	r	0x6	—
15:8	minor_rev	r	0x0	—
23:16	major_rev	r	0x1	—
31:24	tbl_len	r	0x3	—

id field parameter id lsb

minor_rev field minor rev

major_rev field major_rev

tbl_len field Tbl Len

reg6_9__2 register

- Absolute Address: 0x1C
- Base Offset: 0x1C
- Size: 0x4

Bits	Identifier	Access	Reset	Name
23:0	param_tbl_ptr	r	0x900	—
31:24	param_id_msb	r	0xFF	—

param_tbl_ptr field Parameter Table Pointer TBD

param_id_msb field Parameter Table ID MSB

**reg6_10__1 register**

- Absolute Address: 0x20
- Base Offset: 0x20
- Size: 0x4

Bits	Identifier	Access	Reset	Name
7:0	id	r	0x87	—
15:8	minor_rev	r	0x1	—
23:16	major_rev	r	0x1	—
31:24	tbl_len	r	0x1C	—

id field parameter id lsb

minor_rev field minor rev

major_rev field major_rev

tbl_len field Tbl Len

reg6_10__2 register

- Absolute Address: 0x24
- Base Offset: 0x24
- Size: 0x4

Bits	Identifier	Access	Reset	Name
23:0	param_tbl_ptr	r	0xA00	—
31:24	param_id_msb	r	0xFF	—

param_tbl_ptr field Parameter Table Pointer TBD

param_id_msb field Parameter Table ID MSB

**reg6_11__1 register**

- Absolute Address: 0x28
- Base Offset: 0x28
- Size: 0x4

Bits	Identifier	Access	Reset	Name
7:0	id	r	0x9	—
15:8	minor_rev	r	0x0	—
23:16	major_rev	r	0x1	—
31:24	tbl_len	r	0xD	—

id field parameter id lsb

minor_rev field minor rev

major_rev field major_rev

tbl_len field Tbl Len

reg6_11__2 register

- Absolute Address: 0x2C
- Base Offset: 0x2C
- Size: 0x4

Bits	Identifier	Access	Reset	Name
23:0	param_tbl_ptr	r	0xB00	—
31:24	param_id_msb	r	0xFF	—

param_tbl_ptr field Parameter Table Pointer TBD

param_id_msb field Parameter Table ID MSB

**reg6_17__1 register**

- Absolute Address: 0x30
- Base Offset: 0x30
- Size: 0x4

Bits	Identifier	Access	Reset	Name
7:0	id	r	0xF	—
15:8	minor_rev	r	0x1	—
23:16	major_rev	r	0x1	—
31:24	tbl_len	r	0xA	—

id field parameter id lsb

minor_rev field minor rev

major_rev field major_rev

tbl_len field Tbl Len

reg6_17__2 register

- Absolute Address: 0x34
- Base Offset: 0x34
- Size: 0x4

Bits	Identifier	Access	Reset	Name
23:0	param_tbl_ptr	r	0x1100	—
31:24	param_id_msb	r	0xFF	—

param_tbl_ptr field Parameter Table Pointer TBD

param_id_msb field Parameter Table ID MSB

**reg_6_4_4 register**

- Absolute Address: 0x400
- Base Offset: 0x400
- Size: 0x4

Bits	Identifier	Access	Reset	Name
1:0	erase_size	r	0x3	—
2	write_granularity	r	0x0	—
3	always_volatile_csr	r	0x1	—
4	we_instruction	r	0x1	—
7:5	unused1	r	0x7	—
15:8	erase_4kb	r	0xFF	—
16	fs_1s1s2s	r	0x0	—
18:17	addrBytes	r	0x2	—
19	dtr_mode	r	0x1	—
20	fs_1s2s2s	r	0x0	—
21	fs_1s4s4s	r	0x0	—
22	fs_1s1s1s	r	0x0	—
31:23	unused0	r	0xFF	—

erase_size field Block/Sector Erase size.

write_granularity field Write Granularity (1byte).

always_volatile_csr field Volatile Status Register Block Protect.

we_instruction field Write Enable Instruction select for writing to volatile status reg.

unused1 field Unused

erase_4kb field Does not support 4KB Erase

fs_1s1s2s field Does not support 1S-1S-2S Fast Read

addrBytes field Address Bytes (4-Byte Only Addressing)



dtr_mode field Supports DTR mode

fs_1s2s2s field Does not support 1S-2S-2S Fast Read

fs_1s4s4s field Does not support 1S-4S-4S Fast Read

fs_1s1s1s field Does not support 1S-1S-4S Fast Read

unused0 field Unused

reg_6_4_5 register

- Absolute Address: 0x404
- Base Offset: 0x404
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	mem_density	r	0xFFFFFFFF	—

mem_density field memory Density(in bits)

reg6_4_6 register

- Absolute Address: 0x408
- Base Offset: 0x408
- Size: 0x4

Bits	Identifier	Access	Reset	Name
4:0	waitstate_1s4s4s	r	0x2	—
7:5	mode_1s4s4s	r	0x0	—
15:8	fr_inst_1s4s4s	r	0xAB	—
20:16	waitstate_1s1s4s	r	0x2	—
23:21	mode_1s1s4s	r	0x0	—
31:24	fr_inst_1s1s4s	r	0xAA	—

waitstate_1s4s4s field 1s-4s-4s fast read wait states



CONTENTS

mode_1s4s4s field 1s-4s-4s fast read num mode clocks

fr_inst_1s4s4s field 1s-4s-4s fast read wait states

waitstate_1s1s4s field 1s-1s-4s fast read wait states

mode_1s1s4s field 1s-1s-4s fast read num mode clocks

fr_inst_1s1s4s field 1s-1s-4s fast read instruction

reg6_4_7 register

- Absolute Address: 0x40C
- Base Offset: 0x40C
- Size: 0x4

Bits Identifier Access Reset Name			
31:0	reserved	r	0x0 —

reserved field 2S mode is not supported. Reserved

reg6_4_8 register

- Absolute Address: 0x410
- Base Offset: 0x410
- Size: 0x4

Bits Identifier Access Reset Name			
0	mode_2s	r	0x0 —
4	mode_4s	r	0x1 —

mode_2s field Supports 2S-2S-2S fast read mode(not supported)

mode_4s field Supports 4S-4S-4S fast read mode

**reg6_4_9 register**

- Absolute Address: 0x414
- Base Offset: 0x414
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	reserved	r	0x0	—

reserved field 2S mode is not supported. Reserved

reg6_4_10 register

- Absolute Address: 0x418
- Base Offset: 0x418
- Size: 0x4

Bits	Identifier	Access	Reset	Name
20:16	waitstate_4s4s4s	r	0x2	—
23:21	mode_4s4s4s	r	0x0	—
31:24	fr_inst_4s4s4s	r	0xAC	—

waitstate_4s4s4s field 4s-4s-4s fast read wait states

mode_4s4s4s field 4s-4s-4s fast read num mode clocks

fr_inst_4s4s4s field 4s-4s-4s fast read instruction

reg6_4_11 register

- Absolute Address: 0x41C
- Base Offset: 0x41C
- Size: 0x4



Bits	Identifier	Access	Reset	Name
31:0	reserved	r	0x0	—

reserved field Erase instruction 2,1 is not supported. Reserved

reg6_4_12 register

- Absolute Address: 0x420
- Base Offset: 0x420
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	reserved	r	0x0	—

reserved field Erase instruction 4,3 is not supported. Reserved

reg6_4_13 register

- Absolute Address: 0x424
- Base Offset: 0x424
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	reserved	r	0x0	—

reserved field Erase instruction is not supported. Reserved

reg6_4_14 register

- Absolute Address: 0x428
- Base Offset: 0x428
- Size: 0x4



Bits Identifier Access Reset Name		
31:0 reserved	r	0x0 —

reserved field Erase/Program instruction is not supported. Reserved

reg6_4_15 register

- Absolute Address: 0x42C
- Base Offset: 0x42C
- Size: 0x4

Bits Identifier Access Reset Name		
31:0 reserved	r	0x0 —

reserved field Suspend/Resume for Erase/Program instruction is not supported. Reserved

reg6_4_16 register

- Absolute Address: 0x430
- Base Offset: 0x430
- Size: 0x4

Bits Identifier Access Reset Name		
31:0 reserved	r	0x0 —

reserved field Suspend/Resume for Erase/Program instruction is not supported. Reserved

reg6_4_17 register

- Absolute Address: 0x434
- Base Offset: 0x434
- Size: 0x4



Bits	Identifier	Access	Reset	Name
1:0	reserved	r	0x0	—
7:2	dev_busy_poll	r	0x0	—
12:8	exit_delay_count	r	0x2	—
14:13	exit_delay_units	r	0x3	—
22:15	inst_deep_power_down_exit	r	0xAE	—
30:23	inst_deep_power_down_enter	r	0xAD	—
31	deep_power_down	r	0x1	—

reserved field Reserved

dev_busy_poll field device busy poll (not supported)

exit_delay_count field Exit Deep powerdown delay count

exit_delay_units field Exit Deep powerdown delay units 64us

inst_deep_power_down_exit field Exit Deep powerdown Instruction

inst_deep_power_down_enter field Enter Deep powerdown Instruction

deep_power_down field Deep powerdown is supported

reg6_4_18 register

- Absolute Address: 0x438
- Base Offset: 0x438
- Size: 0x4

Bits	Identifier	Access	Reset	Name
3:0	mode_disable444	r	0x0	—
8:4	mode_enable444	r	0x0	—
9	xip_supported_044	r	0x1	—
15:10	mode_exit044	r	0x0	—
19:16	mode_entry044	r	0x0	—
22:20	quad_enable	r	0x0	—



Bits	Identifier	Access	Reset	Name
23	hold_rst_support	r	0x0	—

mode_disable444 field 4-4-4 Mode disable sequence TBD

mode_enable444 field 4-4-4 Mode enable sequence TBD

xip_supported_044 field 0-4-4 Mode supported (XIP)

mode_exit044 field 0-4-4 Mode exit method TBD

mode_entry044 field 0-4-4 Mode entry method TBD

quad_enable field quad enable requirement

hold_rst_support field hold/reset disable (not supported)

reg_6_4_19 register

- Absolute Address: 0x43C
- Base Offset: 0x43C
- Size: 0x4

Bits	Identifier	Access	Reset	Name
6:0	volatile_status_reg_1	r	0x2	—
13:8	soft_reset_support	r	0x0	—
23:14	exit_4B_addressing	r	0x0	—
31:24	enter_4B_addressing	r	0x40	—

volatile_status_reg_1 field volatile register status1

soft_reset_support field Soft reset instruction support (not supported)

exit_4B_addressing field Exit 4 byte addressing TBD

enter_4B_addressing field Enter 4 byte addressing; Always in \$B mode

**reg_6_4_20 register**

- Absolute Address: 0x440
- Base Offset: 0x440
- Size: 0x4

Bits	Identifier	Access	Reset	Name
4:0	waitstate_1s8s8s	r	0x2	—
7:5	mode_1s8s8s	r	0x0	—
15:8	fr_inst_1s8s8s	r	0xAB	—
20:16	waitstate_1s1s8s	r	0x2	—
23:21	mode_1s1s8s	r	0x0	—
31:24	fr_inst_1s1s8s	r	0xAF	—

waitstate_1s8s8s field 1s-8s-8s fast read wait states

mode_1s8s8s field 1s-8s-8s fast read num mode clocks

fr_inst_1s8s8s field 1s-8s-8s fast read wait states

waitstate_1s1s8s field 1s-1s-8s fast read wait states

mode_1s1s8s field 1s-1s-8s fast read num mode clocks

fr_inst_1s1s8s field 1s-1s-8s fast read instruction

reg_6_4_21 register

- Absolute Address: 0x444
- Base Offset: 0x444
- Size: 0x4

Bits	Identifier	Access	Reset	Name
22:18	drive_strength	r	0x1	—
23	jedec_reset	r	0x0	—
25:24	data_strobe_str_waveform	r	0x0	—
26	data_strobe_support_4s4S4S	r	0x0	—



Bits	Identifier	Access	Reset	Name
27	data_strobe_support_4s4d4d	r	0x0	—
30:29	cmd_ext	r	0x0	—
31	byte_order_8D	r	0x0	—

drive_strength field Drive Strength TBD

jedec_reset field JEDEC reset(not supported)

data_strobe_str_waveform field Data strobe STR Waveform TBD

data_strobe_support_4s4S4S field Data strobe support in QPI 4S-4S-4S

data_strobe_support_4s4d4d field Data strobe support in QPI 4S-4D-4D

cmd_ext field Octal DTR Command/Extension

byte_order_8D field Byte order in 8D-8D-8D mode same as 1S-1S-1S mode

reg_6_4_22 register

- Absolute Address: 0x448
- Base Offset: 0x448
- Size: 0x4

Bits	Identifier	Access	Reset	Name
3:0	disable_seq_8s8s8s	r	0x1	—
8:4	enable_seq_8s8s8s	r	0x2	—
9	xip_supported_088	r	0x1	—
15:10	xip_exit_088	r	0x4	—
19:16	xip_entry_088	r	0x1	—
22:20	octal_enable_req	r	0x1	—



disable_seq_8s8s8s field 8s8s8s mode disable seq

enable_seq_8s8s8s field 8s8s8s mode enable seq

xip_supported_088 field 0-8-8 XIP supported

xip_exit_088 field 0-8-8 Mode Exit method TBD

xip_entry_088 field 0-8-8 Mode Entry method

octal_enable_req field octal_enable req

reg_6_4_23 register

- Absolute Address: 0x44C
- Base Offset: 0x44C
- Size: 0x4

Bits	Identifier	Access	Reset	Name
3:0	max_4S_speed_without_ds	r	0x8	—
7:4	max_4S_speed_with_ds	r	0x7	—
11:8	max_4D_speed_without_ds	r	0x8	—
15:12	max_4D_speed_with_ds	r	0x8	—
19:16	max_8S_speed_without_ds	r	0x8	—
23:20	max_8S_speed_with_ds	r	0x8	—
27:24	max_8D_speed_without_ds	r	0x8	—
31:28	max_8D_speed_with_ds	r	0x8	—

max_4S_speed_without_ds field max 4S-4S-4S speed not using DS
200 Mhz

max_4S_speed_with_ds field max 4S-4S-4S speed using DS 166Mhz

max_4D_speed_without_ds field max 4S-4D-4D speed not using DS

max_4D_speed_with_ds field max 4S-4D-4D speed using DS



max_8S_speed_without_ds field max 8S speed not using DS

max_8S_speed_with_ds field max 8S speed using DS 200Mhz

max_8D_speed_without_ds field max 8D speed not using DS

max_8D_speed_with_ds field max 8D speed using DS 200Mhz

reg_6_4_24 register

- Absolute Address: 0x450
- Base Offset: 0x450
- Size: 0x4

Bits Identifier		Access	Reset	Name
2:0	supports_4s4d4d r	0x1	—	
4:3	supports_1s4d4d r	0x1	—	
5	supports_1s2d2d r	0x0	—	
6	supports_1s1d1d r	0x0	—	

supports_4s4d4d field supports 4S-4D-4D Fast Read

supports_1s4d4d field supports 1S-4D-4D Fast Read

supports_1s2d2d field supports 1S-2D-2D Fast Read

supports_1s1d1d field supports 1S-1D-1D Fast Read

reg_6_4_25 register

- Absolute Address: 0x454
- Base Offset: 0x454
- Size: 0x4

Bits Identifier	Access	Reset	Name
31:0 reserved	r	0x0	—



reserved field 1S2D2D and 1S1D1D not supported

reg_6_4_26 register

- Absolute Address: 0x458
- Base Offset: 0x458
- Size: 0x4

Bits	Identifier	Access	Reset	Name
4:0	waitstate_1s4d4d	r	0x2	—
7:5	mode_1s4d4d	r	0x0	—
15:8	fr_inst_1s4d4d	r	0xB1	—
20:16	waitstate_4s4d4d	r	0x2	—
23:21	mode_4s4d4d	r	0x0	—
31:24	fr_inst_4s4d4d	r	0xB0	—

waitstate_1s4d4d field 1s-4d-4d fast read wait states

mode_1s4d4d field 1s-4d-4d fast read num mode clocks

fr_inst_1s4d4d field 1s-4d-4d fast read wait states

waitstate_4s4d4d field 4s-4d-4d fast read wait states

mode_4s4d4d field 4s-4d-4d fast read num mode clocks

fr_inst_4s4d4d field 4s-4d-4d fast read instruction

reg_6_7_3 register

- Absolute Address: 0x700
- Base Offset: 0x700
- Size: 0x4

Bits	Identifier	Access	Reset	Name
0	r_inst_1s1s1s	r	0x1	—



Bits	Identifier	Access	Reset	Name
1	fr_inst_1s1s1s	r	0x1	—
3:2	reserved_2s	r	0x0	—
4	fr_inst_1s1s4s	r	0x1	—
5	fr_inst_1s4s4s	r	0x1	—
12:6	erase_reserved	r	0x0	—
13	fr_inst_1s1d1d	r	0x0	—
14	fr_inst_1s2d2d	r	0x0	—
15	fr_inst_1s4d4d	r	0x0	—
19:16	sector_reserved	r	0x0	—
20	fr_inst_1s1s8s	r	0x0	—
21	fr_inst_1s8s8s	r	0x0	—
22	fr_inst_1s8d8d	r	0x0	—
23	page_program_support_1s1s8s	r	0x0	—
24	page_program_support_1s8s8s	r	0x0	—

r_inst_1s1s1s field 1s1s1s Read cmd inst 13h support(yes)

fr_inst_1s1s1s field 1s1s1s Fast Read cmd inst 0Ch support(yes)

reserved_2s field 2S not supported

fr_inst_1s1s4s field 1s1s4s fast Read cmd inst 34h support(yes)

fr_inst_1s4s4s field 1s4s4s fast Read cmd inst 3Eh support(yes)

erase_reserved field Erase/Page cmds not supported

fr_inst_1s1d1d field 1s1d1d DTR Read cmd inst 0E support(no)

fr_inst_1s2d2d field 1s2d2d DTR Read cmd inst BEh support(no)

fr_inst_1s4d4d field 1s4d4d DTR Read cmd inst EEh support(no)

sector_reserved field Sector cmds not supported

fr_inst_1s1s8s field 1s1s8s Fast Read cmd inst 7Ch support(no)



CONTENTS

fr_inst_1s8s8s field 1s8s8s Fast Read cmd inst CCh support(no)

fr_inst_1s8d8d field 1s8d8d DTR Read cmd inst FDh support(no)

page_program_support_1s1s8s field 1s1s8s page program support (no)

page_program_support_1s8s8s field 1s8s8s page program support (no)

reg_6_7_4 register

- Absolute Address: 0x704
- Base Offset: 0x704
- Size: 0x4

Bits Identifier	Access	Reset	Name
31:0 erase_reserved	r	0x0	—

erase_reserved field erase is not supported

reg_6_9_3 register

- Absolute Address: 0x900
- Base Offset: 0x900
- Size: 0x4

Bits Identifier	Access	Reset	Name
4:0 reserved	r	0x0	—
5 enter_spi_supported	r	0x1	—
14:6 pgm_reserved	r	0x0	—
15 deep_pd_support	r	0x1	—
16 cfg_reg_load_support	r	0x1	—
17 cfg_reg_read_support	r	0x1	—
18 sts_reg_clr_support	r	0x1	—
19 sts_reg_read_support	r	0x1	—
20 sren_support	r	0x0	—
21 wren2_support	r	0x0	—
22 wren1_support	r	0x0	—



Bits Identifier		Access Reset Name		
23	write_mem_linear	r	0x1	—
24	write_mem_wrapped	r	0x0	—
25	write_reg_linear	r	0x1	—
26	write_reg_wrapped	r	0x0	—
27	read_mem_linear	r	0x1	—
28	read_mem_wrapped	r	0x0	—
29	read_reg_linear	r	0x1	—
30	read_reg_wrapped	r	0x0	—
31	xspi_profile_2_support	r	0x1	—

reserved field reserved

enter_spi_supported field enter SPI supported

pgm_reserved field program/erase not supported

deep_pd_support field Deep Power down support(yes)

cfg_reg_load_support field Cfg register load support(yes)

cfg_reg_read_suport field Cfg register read support(yes)

sts_reg_clr_support field Status register clear support(yes)

sts_reg_read_support field Status register read support(yes)

sren_support field SREN support(no)

wren2_support field WREN2 support(no)

wren1_support field WREN1 support(no)

write_mem_linear field write memory linear support

write_mem_wrapped field write memory wrapped support



write_reg_linear field write register linear support

write_reg_wrapped field write register wrapped support

read_mem_linear field Read memory linear support

read_mem_wrapped field Read memory wrapped support

read_reg_linear field Read register linear support

read_reg_wrapped field Read register wrapped support

xspi_profile_2_support field xspi profile2 support

reg_6_9_4 register

- Absolute Address: 0x904
- Base Offset: 0x904
- Size: 0x4

Bits Identifier		Access Reset Name	
6:2	cfg_bit_pattern_num_dymmy_cycl_required r	0x2	—
11:7	num_dymmy_cycl_required	r	0x2 —

cfg_bit_pattern_num_dymmy_cycl_required field 200Mhz num cfg bit pattern to set num dummy cycl required TBD

num_dymmy_cycl_required field 200Mhz num dummy cycl required TBD

reg_6_9_5 register

- Absolute Address: 0x908
- Base Offset: 0x908
- Size: 0x4



Bits	Identifier	Access	Reset	Name
6:2	cfg_bit_pattern_num_dymmy_cycl_required_100	r	0x2	—
11:7	num_dymmy_cycl_required_100	r	0x2	—
16:12	cfg_bit_pattern_num_dymmy_cycl_required_133	r	0x2	—
21:17	num_dymmy_cycl_required_133	r	0x2	—
26:22	cfg_bit_pattern_num_dymmy_cycl_required_166	r	0x2	—
31:27	num_dymmy_cycl_required_166	r	0x2	—

cfg_bit_pattern_num_dymmy_cycl_required_100 field 100Mhz num
cfg bit pattern to set num dummy cycl required TBD

num_dymmy_cycl_required_100 field 100Mhz num dummy cycl re-
quired TBD

cfg_bit_pattern_num_dymmy_cycl_required_133 field 133Mhz num
cfg bit pattern to set num dummy cycl required TBD

num_dymmy_cycl_required_133 field 133Mhz num dummy cycl re-
quired TBD

cfg_bit_pattern_num_dymmy_cycl_required_166 field 166Mhz num
cfg bit pattern to set num dummy cycl required TBD

num_dymmy_cycl_required_166 field 166Mhz num dummy cycl re-
quired TBD

reg_6_10_3 register

- Absolute Address: 0xA00
- Base Offset: 0xA00
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	volatile_address	r	0x0	—

volatile_address field Address offset for volatile registers TBD

**reg_6_10_4 register**

- Absolute Address: 0xA04
- Base Offset: 0xA04
- Size: 0x4

Bits Identifier	Access	Reset	Name
31:0	nonvolatile__address	r	0x0 —

nonvolatile__address field Address offset for non-volatile registers not supported

reg_6_10_5 register

- Absolute Address: 0xA08
- Base Offset: 0xA08
- Size: 0x4

Bits	Identifier	Access	Reset	Name
9:6	num_dummy_cycles_8d8d8d	r	0x0	—
13:10	num_dummy_cycles_8s8s8s	r	0x0	—
17:14	num_dummy_cycles_4d4d4d	r	0x0	—
21:18	num_dummy_cycles_4s4s4s	r	0x0	—
25:22	num_dummy_cycles_2s2s2s	r	0xF	—
27:26	num_dummy_cycles_1s1s1s	r	0x0	—
29:28	num_addr_bytes	r	0x3	—
30	gen_reg_write_supported	r	0x1	—
31	gen_reg_read_supported	r	0x1	—

num_dummy_cycles_8d8d8d field Number of dummy cycles for read in 8d-8d-8d mode

num_dummy_cycles_8s8s8s field Number of dummy cycles for read in 8s-8s-8s mode

num_dummy_cycles_4d4d4d field Number of dummy cycles for read in 4d-4d-4d mode



num_dummy_cycles_4s4s4s field Number of dummy cycles for read in 4s-4s-4s mode

num_dummy_cycles_2s2s2s field Number of dummy cycles for read in 2s-2s-2s mode

num_dummy_cycles_1s1s1s field Number of dummy cycles for read in 1s-1s-1s mode

num_addr_bytes field Number address bytes(32bit)

gen_reg_write_supported field Generic Addressable Write Status/Control register command for volatile registers supported

gen_reg_read_supported field Generic Addressable Read Status/Control register command for volatile registers supported

reg_6_10_6 register

- Absolute Address: 0xA0C
- Base Offset: 0xA0C
- Size: 0x4

Bits Identifier Access Reset Name		
31:0 reserved	r	0x0 —

reserved field No Non volatile registers

reg_6_10_7 register

- Absolute Address: 0xA10
- Base Offset: 0xA10
- Size: 0x4

Bits Identifier Access Reset Name		
31:0 reserved	r	0x0 —



reserved field No Write in progress bit TBD

reg_6_10_8 register

- Absolute Address: 0xA14
- Base Offset: 0xA14
- Size: 0x4

Bits Identifier Access Reset Name		
31:0 reserved	r	0x0 —

reserved field No Write enable bit TBD

reg_6_10_9 register

- Absolute Address: 0xA18
- Base Offset: 0xA18
- Size: 0x4

Bits Identifier Access Reset Name		
31:0 reserved	r	0x0 —

reserved field No program Error bit

reg_6_10_10 register

- Absolute Address: 0xA1C
- Base Offset: 0xA1C
- Size: 0x4

Bits Identifier Access Reset Name		
31:0 reserved	r	0x0 —

reserved field No Erase Error bit

**reg_6_10_11 register**

- Absolute Address: 0xA20
- Base Offset: 0xA20
- Size: 0x4

Bits Identifier Access Reset Name		
31:0 reserved	r	0x0 —

reserved field Variable dummy cycles not supported

reg_6_10_12 register

- Absolute Address: 0xA24
- Base Offset: 0xA24
- Size: 0x4

Bits Identifier Access Reset Name		
31:0 reserved	r	0x0 —

reserved field Variable dummy cycles (nvreg)not supported

reg_6_10_13 register

- Absolute Address: 0xA28
- Base Offset: 0xA28
- Size: 0x4

Bits Identifier Access Reset Name		
31:0 reserved	r	0x0 —

reserved field Variable dummy cycles not supported

**reg_6_10_14 register**

- Absolute Address: 0xA2C
- Base Offset: 0xA2C
- Size: 0x4

Bits Identifier Access Reset Name		
31:0 reserved	r	0x0 —

reserved field Variable dummy cycles not supported

reg_6_10_15 register

- Absolute Address: 0xA30
- Base Offset: 0xA30
- Size: 0x4

Bits Identifier Access Reset Name		
31:0 reserved	r	0x0 —

reserved field Variable dummy cycles not supported

reg_6_10_16 register

- Absolute Address: 0xA34
- Base Offset: 0xA34
- Size: 0x4

Bits Identifier Access Reset Name		
31:0 reserved	r	0x0 —

reserved field QPI mode not supported TBD

**reg_6_10_17 register**

- Absolute Address: 0xA38
- Base Offset: 0xA38
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	reserved	r	0x0	—

reserved field QPI mode not supported TBD

reg_6_10_18 register

- Absolute Address: 0xA3C
- Base Offset: 0xA3C
- Size: 0x4

Bits	Identifier	Access	Reset	Name
23:16	reg_offset	r	0x0	—
26:24	bit_loc	r	0x0	—
27	local_addr_in_last_byte	r	0x0	—
28	addressable	r	0x1	—
30	polarity	r	0x0	—
31	octal_mode_enable	r	0x1	—

reg_offset field Reg Offset

bit_loc field Bit location in reg (TBD)

local_addr_in_last_byte field local address in last byte of address(yes)

addressable field Bit is addressable

polarity field Octal mode Enable polarity

octal_mode_enable field STR/DDR Octal mode Enable Volatile

**reg_6_10_19 register**

- Absolute Address: 0xA40
- Base Offset: 0xA40
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	reserved	r	0x0	—

reserved field Octal mode Enable non Volatile (not supported)

reg_6_10_20 register

- Absolute Address: 0xA44
- Base Offset: 0xA44
- Size: 0x4

Bits	Identifier	Access	Reset	Name
23:16	reg_offset	r	0x0	—
26:24	bit_loc	r	0x0	—
27	local_addr_in_last_byte	r	0x0	—
28	addressable	r	0x1	—
30	polarity	r	0x0	—
31	ddr_mode_select_available	r	0x1	—

reg_offset field Reg Offset

bit_loc field Bit location in reg (TBD)

local_addr_in_last_byte field local address in last byte of address(yes)

addressable field Bit is addressable

polarity field polarity

ddr_mode_select_available field SDR/DDR mode select volatile (supported)

**reg_6_10_21 register**

- Absolute Address: 0xA48
- Base Offset: 0xA48
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	reserved	r	0x0	—

reserved field SDR/DDR non Volatile (not supported)

reg_6_10_22 register

- Absolute Address: 0xA4C
- Base Offset: 0xA4C
- Size: 0x4

Bits	Identifier	Access	Reset	Name
23:16	reg_offset	r	0x0	—
26:24	bit_loc	r	0x0	—
27	local_addr_in_last_byte	r	0x0	—
28	addressable	r	0x1	—
30	polarity	r	0x0	—
31	octal_mode_enable	r	0x1	—

reg_offset field Reg Offset

bit_loc field Bit location in reg (TBD)

local_addr_in_last_byte field local address in last byte of address(yes)

addressable field Bit is addressable

polarity field Octal mode Enable polarity

octal_mode_enable field STR Octal mode Enable Volatile Dup of 6_10_18?
TBD

**reg_6_10_23 register**

- Absolute Address: 0xA50
- Base Offset: 0xA50
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	reserved	r	0x0	—

reserved field SDR/DDR non Volatile (not supported)

reg_6_10_24 register

- Absolute Address: 0xA54
- Base Offset: 0xA54
- Size: 0x4

Bits	Identifier	Access	Reset	Name
23:16	reg_offset	r	0x0	—
26:24	bit_loc	r	0x0	—
27	local_addr_in_last_byte	r	0x0	—
28	addressable	r	0x1	—
30	polarity	r	0x0	—
31	octal_mode_enable	r	0x1	—

reg_offset field Reg Offset

bit_loc field Bit location in reg (TBD)

local_addr_in_last_byte field local address in last byte of address(yes)

addressable field Bit is addressable

polarity field Octal mode Enable polarity

octal_mode_enable field DTR Octal mode Enable Volatile Dup of 6_10_18?
TBD

**reg_6_10_25 register**

- Absolute Address: 0xA58
- Base Offset: 0xA58
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	reserved	r	0x0	—

reserved field SDR/DDR non Volatile (not supported)

reg_6_10_26 register

- Absolute Address: 0xA5C
- Base Offset: 0xA5C
- Size: 0x4

Bits	Identifier	Access	Reset	Name
23:16	reg_offset	r	0x0	—
26:24	bit_loc	r	0x0	—
27	local_addr_in_last_byte	r	0x0	—
28	addressable	r	0x1	—
30	polarity	r	0x0	—
31	ddr_mode_select_available	r	0x1	—

reg_offset field Reg Offset

bit_loc field Bit location in reg (TBD)

local_addr_in_last_byte field local address in last byte of address(yes)

addressable field Bit is addressable

polarity field polarity

ddr_mode_select_available field Deep power down select volatile (supported)

**reg_6_10_27 register**

- Absolute Address: 0xA60
- Base Offset: 0xA60
- Size: 0x4

Bits	Identifier	Access	Reset	Name
23:16	reg_offset	r	0x0	—
26:24	bit_loc	r	0x0	—
27	local_addr_in_last_byte	r	0x0	—
28	addressable	r	0x1	—
30	polarity	r	0x0	—
31	ddr_mode_select_available	r	0x1	—

reg_offset field Reg Offset

bit_loc field Bit location in reg (TBD)

local_addr_in_last_byte field local address in last byte of address(yes)

addressable field Bit is addressable

polarity field polarity

ddr_mode_select_available field Ultra Deep power down select volatile (supported)

reg_6_10_28 register

- Absolute Address: 0xA64
- Base Offset: 0xA64
- Size: 0x4

Bits	Identifier	Access	Reset	Name
23:16	reg_offset	r	0x0	—
26:24	bit_loc	r	0x0	—
27	local_addr_in_last_byte	r	0x0	—



Bits	Identifier	Access	Reset	Name
28	addressable	r	0x1	—
31:30	num_bits	r	0x3	—

reg_offset field Reg Offset

bit_loc field Bit location in reg (TBD)

local_addr_in_last_byte field local address in last byte of address(yes)

addressable field Bit is addressable

num_bits field Output drive strength numbits(supported)

reg_6_10_29 register

- Absolute Address: 0xA68
- Base Offset: 0xA68
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	reserved	r	0x0	—

reserved field non Volatile (not supported)

reg_6_11_3 register

- Absolute Address: 0xA6C
- Base Offset: 0xA6C
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	offset	r	0x0	—

offset field Volatile reg offset TBD

**reg_6_11_4 register**

- Absolute Address: 0xA70
- Base Offset: 0xA70
- Size: 0x4

Bits Identifier Access Reset Name				
31:0	offset	r	0x0	—

offset field non Volatile reg offset TBD

reg_6_11_5 register

- Absolute Address: 0xB00
- Base Offset: 0xB00
- Size: 0x4

Bits	Identifier	Access Reset Name		
2:0	reserved	r	0x0	—
10:3	wip_address	r	0x0	—
14:11	wip_bit_location	r	0x0	—
15	wip_address_byte_location	r	0x0	—
16	wip_polarity	r	0x0	—
17	wip_supported	r	0x0	—
22:18	dummy_cycles_non_volative	r	0x0	—
27:23	dummy_cycles_volative	r	0x0	—
29:28	address_bytes	r	0x0	—
30	write_ctrl_sts	r	0x1	—
31	read_ctrl_sts	r	0x1	—

reserved field Reserved

wip_address field Address of WIP Bit(not supported)

wip_bit_location field Bit location of WIP Bit(not supported)



wip_address_byte_location field Local address of WIP Bit(not supported)

wip_polarity field WIP Polarity(not supported)

wip_supported field WIP (Device busy bit) (not supported)

dummy_cycles_non_volatile field Dummy cycles for non-volatile reg read TBD

dummy_cycles_volatile field Dummy cycles for volatile reg read TBD

address_bytes field Address Bytes(6 bytes)

write_ctrl_sts field Generic addressable write ctrl/sts registers(yes)

read_ctrl_sts field Generic addressable read ctrl/sts registers(yes)

reg_6_11_6 register

- Absolute Address: 0xB04
- Base Offset: 0xB04
- Size: 0x4

Bits Identifier Access Reset Name			
31:0	p_error	r	0x0 —

p_error field Program Error (Not Implemented)

reg_6_11_7 register

- Absolute Address: 0xB08
- Base Offset: 0xB08
- Size: 0x4



CONTENTS

Bits	Identifier	Access	Reset	Name
31:0	e_error	r	0x0	—

e_error field Erase Error (Not Implemented)

reg_6_11_8 register

- Absolute Address: 0xB0C
- Base Offset: 0xB0C
- Size: 0x4

Bits	Identifier	Access	Reset	Name
30:0	reserved	r	0x0	—
31	supported	r	0x0	—

reserved field Reserved

supported field Variable number of dummy cycles volatile reg(Not Supported)

reg_6_11_9 register

- Absolute Address: 0xB10
- Base Offset: 0xB10
- Size: 0x4

Bits	Identifier	Access	Reset	Name
30:0	reserved	r	0x0	—
31	supported	r	0x0	—

reserved field Reserved

supported field Variable number of dummy cycles non-volatile reg(Not Supported)

**reg_6_11_10 register**

- Absolute Address: 0xB14
- Base Offset: 0xB14
- Size: 0x4

Bits	Identifier	Access	Reset	Name
30:0	reserved	r	0x0	—
31	supported	r	0x0	—

reserved field Reserved

supported field Variable number of dummy cycles Bit pattern(Not Supported)

reg_6_11_11 register

- Absolute Address: 0xB18
- Base Offset: 0xB18
- Size: 0x4

Bits	Identifier	Access	Reset	Name
30:0	reserved	r	0x0	—
31	supported	r	0x0	—

reserved field Reserved

supported field Variable number of dummy cycles Bit pattern(Not Supported)

reg_6_11_12 register

- Absolute Address: 0xB1C
- Base Offset: 0xB1C
- Size: 0x4



CONTENTS

Bits	Identifier	Access	Reset	Name
30:0	reserved	r	0x0	—
31	supported	r	0x0	—

reserved field Reserved

supported field Variable number of dummy cycles Bit pattern(Not Supported)

reg_6_11_13 register

- Absolute Address: 0xB20
- Base Offset: 0xB20
- Size: 0x4

Bits	Identifier	Access	Reset	Name
24:17	addr	r	0x0	—
28:25	msb_bit_location	r	0x0	—
29	addr_not_in_last_byte	r	0x0	—
31:30	drive_strength_num_bits	r	0x3	—

addr field Address of register TBD (typo in spec?)

msb_bit_location field Bit location of MSB Bit TBD

addr_not_in_last_byte field addr in last_byte TBD

drive_strength_num_bits field output drive strength num_bits (volatile)

reg_6_11_14 register

- Absolute Address: 0xB24
- Base Offset: 0xB24
- Size: 0x4



Bits Identifier Access Reset Name				
0	reserved	r	0x0	—

reserved field output drive strength nv (not implemented)

reg_6_11_15 register

- Absolute Address: 0xB28
- Base Offset: 0xB28
- Size: 0x4

Bits Identifier Access Reset Name				
0	tbd	r	0x0	—

tbd field output drive strength bit pattern TBD

reg_6_17_3 register

- Absolute Address: 0xB2C
- Base Offset: 0xB2C
- Size: 0x4

Bits	Identifier	Access Reset Name		
4	gram_spi	r	0x1	—
5	address_shift	r	0x0	—
11:8	md_address_format	r	0x4	—
19:16	md_address_write_access	r	0x0	—
23:20	md_address_read_access	r	0x0	—
27:24	md_active_die	r	0x0	—
31:28	md_address_offset	r	0x0	—

gram_spi field Gram SPI Mode TBD

address_shift field Address Shift TBD

md_address_format field Multi-Die Addressformat TBD



CONTENTS

md_address_write_access field Multi-Die Addressable reg read access
TBD

md_address_read_access field Multi-Die Addressable reg read access
TBD

md_active_die field Active-Die Selection for Multi Die Package TBD

md_address_offset field Multi-Die Address offset for Addressable Register TBD

reg_6_17_4 register

- Absolute Address: 0xB30
- Base Offset: 0xB30
- Size: 0x4

Bits	Identifier	Access	Reset	Name
7:0	write_opcode	r	0x71	—
15:8	read_opcode	r	0x65	—
20:16	dummy_cycles	r	0x8	—
23:21	dummy_cycles_override	r	0x0	—
27:24	write_enable	r	0x0	—
31:28	volatile_addr_offset	r	0x0	—

write_opcode field Write Opcode

read_opcode field Read Opcode

dummy_cycles field Dummy cycles

dummy_cycles_override field Dummy cycles override

write_enable field Write enable

volatile_addr_offset field Volatile address offset TBD

**reg_6_17_5 register**

- Absolute Address: 0xB34
- Base Offset: 0xB34
- Size: 0x4

Bits	Identifier	Access	Reset	Name
7:0	write_opcode	r	0x71	—
15:8	read_opcode	r	0x65	—
20:16	dummy_cycles	r	0x8	—
23:21	dummy_cycles_override	r	0x0	—
27:24	write_enable	r	0x0	—
31:28	volatile_addr_offset	r	0x0	—

write_opcode field Write Opcode

read_opcode field Read Opcode

dummy_cycles field Dummy cycles

dummy_cycles_override field Dummy cycles override

write_enable field Write enable

volatile_addr_offset field Volatile address offset TBD

reg_6_17_6 register

- Absolute Address: 0xB38
- Base Offset: 0xB38
- Size: 0x4

Bits	Identifier	Access	Reset	Name
3:0	statreg_bit0	r	0x1	—
11:8	statreg_access	r	0x1	—
14:12	dummy_cycles	r	0x2	—
23:16	aux1	r	0x0	—



CONTENTS

Bits	Identifier	Access	Reset	Name
31:24	aux2	r	0x5	—

statreg_bit0 field SR bit 0 is busy flag

statreg_access field SR Access

dummy_cycles field Dymmy Cycles

aux1 field Aux 1 TBD SR address

aux2 field Aux 2

SCCR_Reg address map

- Absolute Address: 0x0
- Base Offset: 0x0
- Size: 0x38

Status and control Registers

Offset	Identifier	Name
0x00	ID0	id0
0x08	ID1	ID1
0x10	CFG	—
0x18	xspi_status	—
0x20	xspi_control	—
0x28	xspi_rates	—
0x30	interrupt_status	—

ID0 register

- Absolute Address: 0x0
- Base Offset: 0x0
- Size: 0x8



Bits	Identifier	Access	Reset	Name
3:0	mgf_id	rw	0x2	—
15:4	devid0	rw	0x0	—

mgf_id field Manufacturer: TODO check how to obtain this number

devid0 field DEVID0 TODO Use as suitable

ID1 register

- Absolute Address: 0x8
- Base Offset: 0x8
- Size: 0x8

Bits	Identifier	Access	Reset	Name
3:0	dev_type	rw	0x0	—
15:4	devid1	rw	0x0	—

dev_type field Device Type set to hyperram

devid1 field TODO Use as appropriate

CFG register

- Absolute Address: 0x10
- Base Offset: 0x10
- Size: 0x8

Bits	Identifier	Access	Reset	Name
1:0	BurstLength	rw	0x2	—
2	HybridBurstEnable	r	0x0	—
3	FixedLatency	r	0x1	—
7:4	InitialLatency	rw	0x8	—
11:8	Reserved	r	0x1	—
14:12	DriveStrength	r	0x3	—
15	DeepPowerDown	rw	0x0	—



CONTENTS

Bits	Identifier	Access	Reset	Name
16	BurstEnable	rw	0x0	—
17	UltraDeepPowerDown	rw	0x0	—

BurstLength field Burst Length

HybridBurstEnable field Burst Enable

FixedLatency field Fixed Latency

InitialLatency field Initial Latency.

DriveStrength field Drive Strength.

DeepPowerDown field Deep Power down, Not too deep, not too shallow

BurstEnable field Enable bust access. Applicable only to memory access.
Register access are one at a time

UltraDeepPowerDown field Ultra Deep Power down.

xspi_status register

- Absolute Address: 0x18
- Base Offset: 0x18
- Size: 0x8

Bits	Identifier	Access	Reset	Name
0	wip	r	0x0	—

wip field write in progress; Valid only after a write mem transaction

**xspi_control register**

- Absolute Address: 0x20
- Base Offset: 0x20
- Size: 0x8

Bits Identifier		Access Reset Name		
0	use_xspi_clk	rw	0x0	—
1	interrupt_enable	rw	0x0	—

use_xspi_clk field Use xspi clock as system clock

interrupt_enable field Use xspi clock as system clock

xspi_rates register

- Absolute Address: 0x28
- Base Offset: 0x28
- Size: 0x8

Bits	Identifier	Access	Reset	Name
7:0	cmd_rate	rw	0x0	—
15:8	addr_rate	rw	0x0	—
23:16	data_rate	rw	0x0	—

cmd_rate field CMD Rate

addr_rate field CMD Rate

data_rate field CMD Rate

interrupt_status register

- Absolute Address: 0x30
- Base Offset: 0x30
- Size: 0x8



CONTENTS

Bits	Identifier	Access	Reset	Name
1:0	axi_resp	r, rclr	0x0	—
2	read_underflow	r, rclr	0x0	—
3	write_overflow	r, rclr	0x0	—

axi_resp field AXI Resp Error

read_underflow field Read FIFO Underflow Error

write_overflow field Write FIFO Overflow Error



Erbium CPU Subsystem: High Level Architecture and Programming Model

The CPU subsystem enables the Erbium platform to operate as an Edge AI device. In this mode, the CPU subsystem is the primary transaction initiator on the AXI4 interconnect to which it is connected.

The main CPU subsystem components are:

- **ET-Neighborhood** with 8 ET-Minion cores
 - **32KB L1 I-Cache data SRAM**
 - **Platform-Level Interrupt Controller (PLIC)**
 - **CPU Registers**
 - **Synchronization primitives:** Fast Local Barriers, Fast Credit Counters
 - **CLINT logic**
-

Table of Contents

- 1. CPU Subsystem hierarchy
 - 1.1 ET-Minion
 - 1.2 ET-Neighborhood
 - 1.3 CPU Subsystem
- 2. Programming Model
 - 2.1 ET System Registers (ESR)
 - ★ 2.1.1 Privilege Rules
 - ★ 2.1.2 Invalid or Non-Existent ESR Accesses
 - 2.2 Register File changes
 - 2.3 ET-Minion SIMD
 - 2.4 Floating-Point Denormal Handling
 - 2.5 Performance Monitoring Unit (PMU)
 - 2.6 Privileged Architecture Deviations
 - 2.7 Exceptions and Interrupts
 - 2.8 Synchronization primitives
 - ★ 2.8.1 Fast Local Barriers
 - ★ 2.8.2 Fast Credit Counters



- 2.9 Core Local Interrupt (CLINT)
- 3 ISA changes (Erbium)
- 4. Erbium Memory
 - 4.1 Memory Map
 - 4.2 Physical Memory Attributes
 - 4.3 Physical Memory Protection (PMP)

1. CPU Subsystem hierarchy

The hierarchy builds from the smallest unit upward starting from the ET-Minions.

1.1 ET-Minion

At the lowest level of the hierarchy there are the ET-Minion cores. Each ET-Minion core is a dual-threaded, in-order, single-issue, RISC-V core extended with a proprietary 8-lane SIMD extension. Specifically, the ET-Minion core implements the following RISC-V architecture:

Base	Version	Comments
RV64I	2.1	

Extension	Version	Comments
M	2.0	
F	2.2	Not fully compliant; see Programming Model
C	2.0	
Zicsr	2.0	
Zifencei	2.0	

Module	Version	Comments
Machine ISA	1.11	Not fully compliant; see Programming Model
Supervisor ISA	1.11	Not fully compliant; see Programming Model



Each ET-Minion core has its own 4KB private L1 Data Cache (that can also act as a scratchpad) but does not have a private Instruction Cache. The vector unit supports a custom ET-Minion SIMD extension that allows operating on eight floating point values in the same cycle. The vector unit supports transcendental instructions and tensor instructions that accelerate machine learning applications.

The ET-Minion is composed of a front-end (FE) unit for fetching instructions from an external, shared, Instruction Cache. Instructions are then fed to the Decoder unit (part of the FE) that will ship them either to the Integer/Dcache pipeline or to the Vector unit pipeline.

The Vector Processing Unit (VPU) is composed of eight identical lanes. Lanes always operate completely synchronously, so to the final programmer the VPU appears as a single entity that performs 8 operations per cycle. Each VPU Lane contains:

- A register file holding thirty-two 32b-wide registers. When viewed together, the eight lanes present to the programmer as a unified register file containing thirty-two 256b-wide registers.
- A Fused Multiply-Accumulate (FMA) unit capable of performing one 32b or two 16b floating point multiply-add operations
- A pair of Integer Multiply-Accumulate (IMA) units each capable of performing four 8b integer multiply-accumulate operations
- An Integer (Int) unit capable of performing 32b integer and logical operations
- A Transcendental (Trans) unit capable of performing 32b floating point transcendental instructions (`sin`, `exp`, `log`, `rcp`, `rsqrt`)

1.2 ET-Neighborhood

Eight ET-Minions are grouped together in a structure called ET-Neighborhood. The eight ET-Minions in a ET-Neighborhood share some infrastructure within the block: Instruction Caches and input/output buses.

The Instruction Caches are divided into 2 levels, L0 and L1. There are two L0 Instruction Caches, each one shared among a group of four ET-Minions (eight hardware threads). Each L0 Instruction cache has eight fully-associative entries (so, on average, there is one entry per hardware thread). Both L0 Instruction Caches are then serviced from a local 32KB shared L1 Instruction Cache.

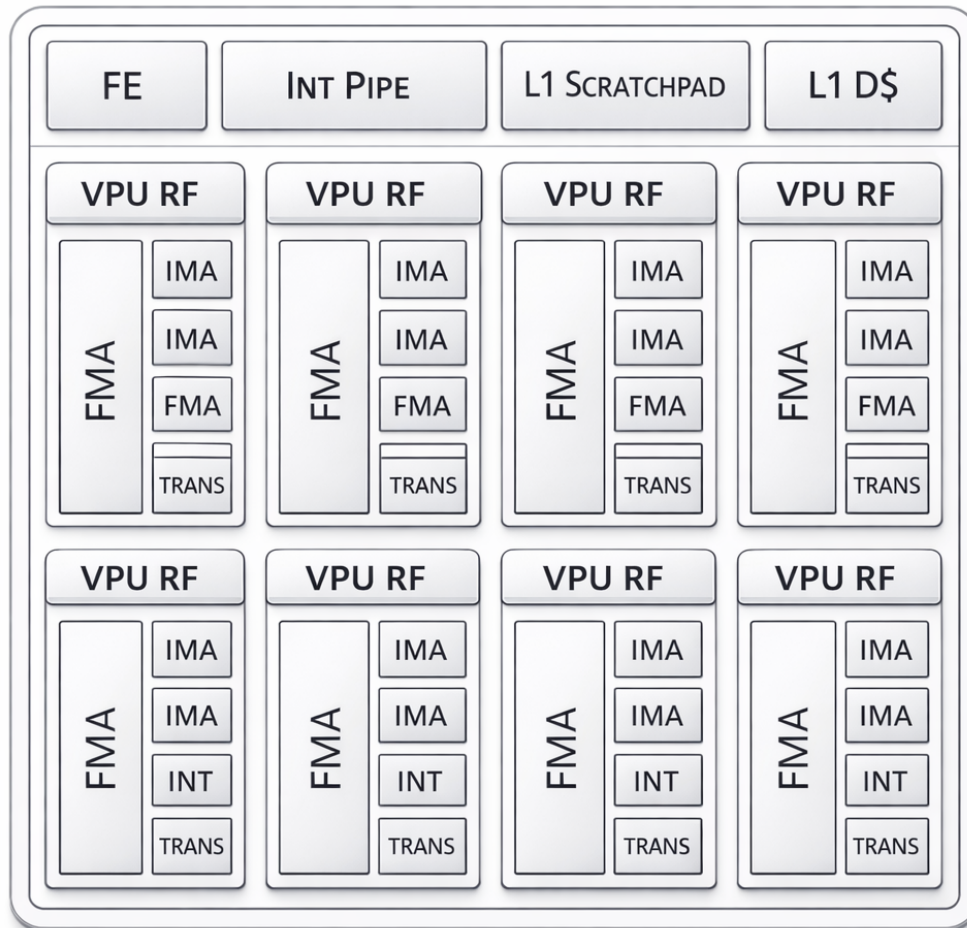


Figure 9: ET-Minion Diagram

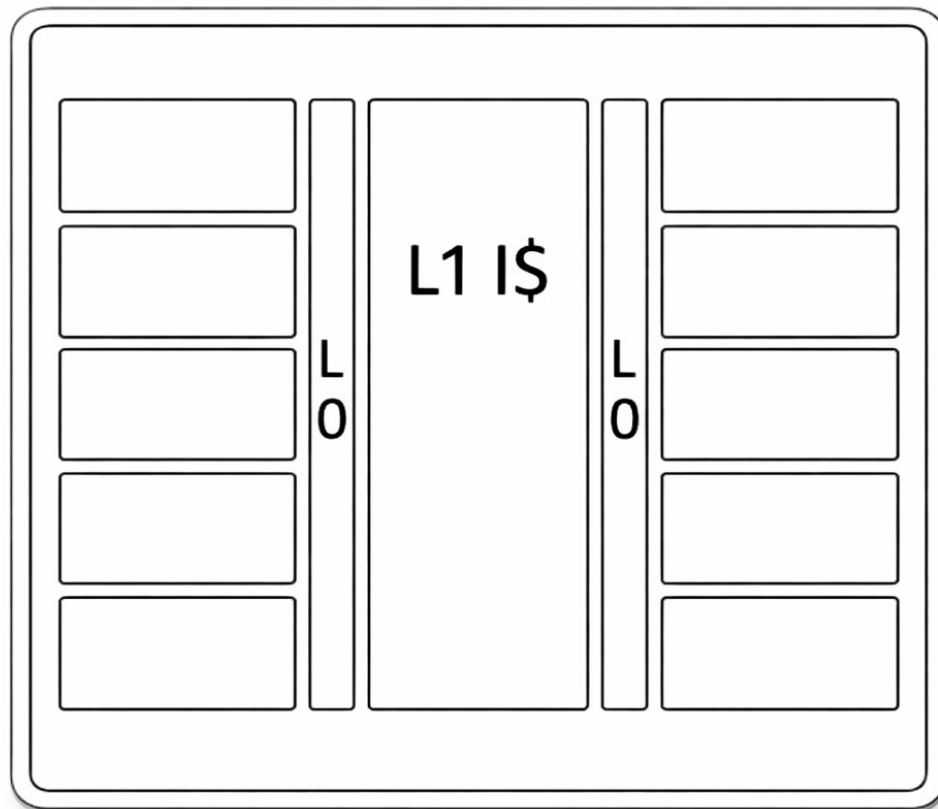


Figure 10: ET-Neighborhood Diagram



1.3 CPU Subsystem

Integrates the ET-Neighborhood with system-level infrastructure such as the Platform-Level Interrupt Controller (PLIC), CPU registers, synchronization primitives (Fast Local Barriers and Credit Counters), and CLINT logic.

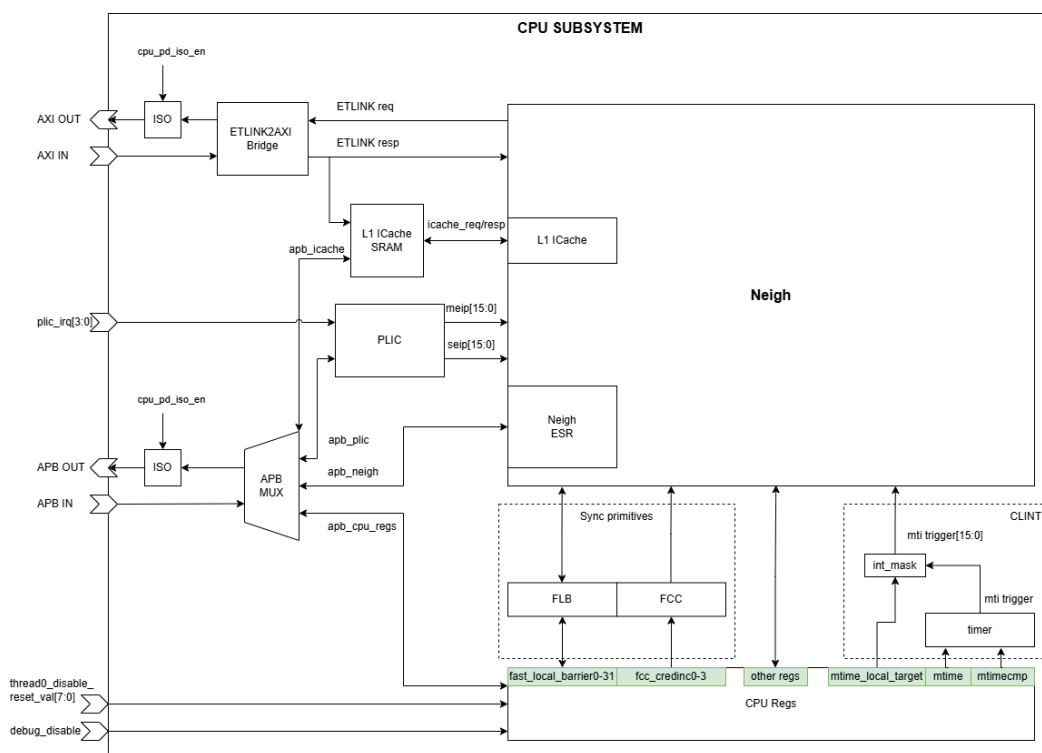


Figure 11: CPU Subsystem Diagram

2. Programming Model

2.1 ET System Registers (ESR)

ET System Registers (ESR) are **memory-mapped registers**: each register occupies a fixed address within this region, and software accesses them using standard load and store instructions. All ESRs are 64 bits in size and aligned to a 64-bit boundary.

This region is divided into four sub-regions based on address bits [23:22], referred to as the Protection Bits (PP bits).



The PP bits encode the minimum privilege level required to access a register within that sub-region:

- **00** — User mode
- **01** — Supervisor mode
- **11** — Machine mode
- **10** — Reserved for microarchitectural messages (e.g., reduce messages, debug accesses) and will always trap as an *access fault*

2.1.1 Privilege Rules The PP bits specify the *minimum* privilege required:

- **M-mode** software may access ESRs in PP = 00, 01, and 11 (but not PP = 10).
- **S-mode** software may access ESRs in PP = 00 and 01.
- **U-mode** software may access ESRs only in PP = 00.

Accesses from a privilege mode that does not meet the minimum PP requirement will raise an **Access Fault** exception.

2.1.2 Invalid or Non-Existent ESR Accesses If software with sufficient privilege accesses an address that does not correspond to an implemented ESR, the access will raise a **local Bus Error interrupt**.

Because the PP bits are part of the ESR address, each ESR exists in exactly one PP sub-region. Attempting to access the same register offset in a different PP sub-region is treated as an access to a non-existent ESR and will also raise a **Bus Error interrupt**.

2.2 Register File changes

The integer register file remains unchanged from RV64I. The floating-point registers (**f0–f31**) are widened to 256 bits (**FLEN = 256**). A new set of mask registers (**m0–m7**), each 8 bits in size, is added to support predicated SIMD execution.

Scalar floating-point operations (i.e., RV64F instructions) operate only on the low 32 bits of each floating-point register. Bits 32 through **FLEN-1** are forced to zero as part of the operation.



A new U-mode CSR, `GSC_PROGRESS`, is introduced to track the number of elements processed during gather or scatter operations.

2.3 ET-Minion SIMD

As mentioned above, the ET-Minion SIMD extension introduces a set of architectural enhancements aimed at accelerating packed floating-point and integer workloads. The extension provides eight mask registers (`m0–m7`), each 8 bits wide, together with basic mask-manipulation operations. The floating-point register file is widened to 256 bits, enabling each register to hold eight `float32` or eight `int32`/`uint32` elements.

A new class of packed-single (`ps`) instructions interprets the widened floating-point registers as vectors of eight `float32` values. All `ps` operations execute under the control of the `m0` mask.

Packed-integer (`pi`) instructions operate on the same widened registers but interpret them as eight `int32` or `uint32` elements. As with the `ps` subset, all `pi` instructions execute under the control of the `m0` mask.

Finally, the extension provides a group of tensor-oriented pseudo-instructions, implemented via CSR accesses, to efficiently multiply and accumulate `float16` and `int8` tensors. `TensorFMA` reads matrices A and B from the L1 scratch-pad or memory and accumulates into the floating-point or TenC register file, operating on 2D tiles in a fixed layout. `TensorLoad` and `TensorStore` provide compatible memory transfers for these layouts.

The extension also provides pseudo-instructions to control out-of-bounds handling and 2D convolution geometry, and to coordinate reductions across Minions. Cooperative `TensorLoad` and `TensorStore` variants are also available, which help coalesce accesses aiming for a better balance of performance and power efficiency.

2.4 Floating-Point Denormal Handling

Minion does not implement full IEEE-754 denormal support. All denormalized inputs are treated as zero, and any denormalized results are flushed to zero before being written back.

A sticky `InputDenorm` flag is added to the machine state. This flag is set whenever a denormalized input operand is flushed to zero during a floating-point operation. Software may read the flag through either `fcsr[31]` or `fflags[31]`, and may clear it by writing to either CSR.



2.5 Performance Monitoring Unit (PMU)

The standard `minstret` and `mcycle` CSRs, and by extension their user-mode aliases `instret` and `cycle`, **always return 0**.

Counters are implemented in a shared PMU unit covering 8 ET-Minion cores per ET-Neighborhood. The PMU must be explicitly enabled via an ESR write before it counts anything.

The PMU provides 6 programmable counters per HART group. Access is via `mhpmpcounter3`–`mhpmpcounter8` and event selectors `mhpmevent3`–`mhpmevent8`.

The hardware implements only twelve programmable counters: six shared by all even-numbered HARTs in a ET-Neighborhood and six shared by all odd-numbered HARTs, so each HART sees only `mhpmpcounter3`–`mhpmpcounter8`. Other registers like `mcycle`, `minstret`, and `mhpmpcounter9`–`mhpmpcounter31` are hard-wired to 0.

Counters `mhpmpcounter3`–`mhpmpcounter6` can count ET-Minion-related events: the same counter is shared among all HARTs that see it, and the counter accumulates the union of all events configured by any of those HARTs via `mhpmevent3`–`mhpmevent6`.

Regardless of whether a HART has configured an event or not, all even-numbered HARTs in a ET-Neighborhood will read the same value from `mhpmpcounter3`–`mhpmpcounter6`, and all odd-numbered HARTs in a ET-Neighborhood will read the same value, though different from the even-numbered HARTs in a ET-Neighborhood, when they read `mhpmpcounter3`–`mhpmpcounter6`. It is the responsibility of the programmer to configure appropriate events into `mhpmevent3`–`mhpmevent6` across the HARTs that share `mhpmpcounter3`–`mhpmpcounter6` such that the union of events makes sense.

NOTE: For example, if HART 0 and HART 2 both set `mhpmevent3` to `RETIRED_INST0`, then `mhpmpcounter3` of both HARTs will report the sum of the instructions retired in both HARTs. Similarly, if HART 1 sets `mhpmevent4` to `RETIRED_INST1` and HART 3 sets `mhpmevent4` to `CYCLES`, then `mhpmpcounter4` will be incremented on every cycle that HART 1 is executing and whenever HART 3 retires an instruction.

Counters `mhpmpcounter7`–`mhpmpcounter8` count ET-Neighborhood-related events; when multiple HARTs configure different events, the counter follows the event programmed by the HART with the lower `mhartid` (up to 32 ET-Neighborhood events). All even-numbered HARTs in a ET-Neighborhood read the same value for `mhpmpcounter3`–`mhpmpcounter6`, and all odd-numbered HARTs read matching but distinct values; the programmer must ensure the set of configured events per shared counter forms a sensible union.



2.6 Privileged Architecture Deviations

ET-Minion implements a variant of the **RISC-V Machine and Supervisor ISA v1.11** with the following Erbium-specific deviations that every programmer must be aware of.

satp CSR:

- The **satp** CSR is **shared between both HARTs** of a given ET-Minion core. Both threads always operate under the same address space configuration.

WFI Behavior:

- Executing **WFI** in U-mode **always causes an illegal instruction exception**.
- When **MSTATUS.TW = 1**, executing **WFI** in S-mode **also always causes an illegal instruction exception**.

2.7 Exceptions and Interrupts

The ET-Minion implements **mcause** and **scause** following the RISC-V privileged spec. The MSB indicates an interrupt; remaining bits indicate the cause code.

Standard Interrupts (Interrupt bit = 1):

Code	Name	Description
0	USI	User software interrupt
1	SSI	Supervisor software interrupt
3	MSI	Machine software interrupt
4	UTI	User timer interrupt
5	STI	Supervisor timer interrupt
7	MTI	Machine timer interrupt
8	UEI	User external interrupt
9	SEI	Supervisor external interrupt
11	MEI	Machine external interrupt
16	MBAD_RED	Bad IPI redirect interrupt (RW)
19	MIECO	I-cache ECC counter overflow (RO; clear via I-cache ESRs)
23	BUS_ERROR	Bus error interrupt (RW)

Standard Exceptions (Interrupt bit = 0):



Code Description	
0	Instruction address misaligned
1	Instruction access fault
2	Illegal instruction
3	Breakpoint
4	Load address misaligned
5	Load access fault
6	Store/AMO address misaligned
7	Store/AMO access fault
8	Environment call from U-mode
9	Environment call from S-mode
11	Environment call from M-mode
12	Instruction page fault
13	Load page fault
15	Store/AMO page fault
25	Instruction bus error (ET-specific)
26	Instruction ECC error (ET-specific)
27	Load split-page fault (ET-specific)
28	Store split-page fault (ET-specific)
30	M-code emulation (ET-specific)

If multiple exceptions and/or interrupts occur at the same time, the decreasing priority order of the following table indicates which trap is taken and reported in mcause/scause:

Priority	Exception Code	Description
1	—	Interrupts (see discussion below for detailed priority)
2	3	Instruction address breakpoint
3	12	Instruction page fault
4	1	Instruction access fault
4	25	Instruction bus error
4	26	Instruction ECC error
5	2	Illegal instruction (true illegal opcode)



Priority	Exception Code	Description
5	0	Instruction address misaligned
5	8, 9, 11	Environment call
5	3	Environment break
5	3	Load/Store/AMO address breakpoint
6	30	M-code emulation
7	2	Illegal instruction (unknown CSR, permission errors, disabled units, etc.)
8	6	Store/AMO address misaligned
8	4	Load address misaligned
9	28	Store split-page fault
9	27	Load split-page fault
10	15	Store/AMO page fault
10	13	Load page fault
11	7	Store/AMO access fault
11	5	Load access fault

2.8 Synchronization primitives

The Fast Synchronization extension provides mechanisms for coordination among operating ET-Minions, centered on two facilities: **Fast Local Barriers (FLB)** and **Fast Credit Counters (FCC)**. These are exposed via both CSRs and memory-mapped ESRs in the CPU register address space.

2.8.1 Fast Local Barriers The Fast Local Barrier (FLB) extension offers fast barrier synchronization across ET-Minions within an ET-Neighborhood. Multiple barrier counters are implemented as memory-mapped read-write registers accessible from user mode. The `flb` CSR atomically increments a barrier counter and compares its value against a threshold:

- To implement an N-HART barrier, one HART first clears the barrier counter via the ESR interface, then each HART uses `flb` to increment the counter and test whether it is the last to reach the threshold.



- The HART is not stalled; the mechanism only signals whether it is the last participant.

Access to these barrier counters does not block execution; they provide lightweight, software-visible barrier coordination.

2.8.2 Fast Credit Counters The Fast Credit Counter (FCC) extension provides a fast credit mechanism for coordinating ET-Minions. Each HART is extended with two local credit counters exposed via the `fcc` and `fccnb` CSRs (four counters per ET-Minion core). One HART can increment another HART's credit counter by writing to the corresponding `CREDINC0`–`CREDINC3` memory-mapped registers.

- The `fcc` CSR consumes credits and can stall the issuing HART until sufficient credits are available.
- The `fccnb` CSR non-blockingly reads the current value of the HART's own credit counters.

Credits can be used directly for producer/consumer synchronization or combined with FLB to implement barriers:

1. Initialize by clearing one credit counter on each participating HART and zeroing a fast local barrier.
2. When reaching the barrier, each HART uses `flb` to test whether it is the last to arrive.
3. If not last, the HART uses `fcc` to wait for a credit; if last, it distributes credits via the `CREDINCx` registers to all waiting HARTs.

2.9 Core Local Interrupt (CLINT)

The CPU subsystem provides a 64-bit timer (`mtime`) that increments every 100 ns (10 MHz) and overflows after ~64,000 years. It is driven from a 200 MHz reference clock via a programmable prescaler configured through `time_config`; alternatively, it can use an internal ring oscillator for approximate timing.

Timer interrupts are based on `mtimecmp`: when `mtimecmp > mtime`, a timer interrupt is asserted and de-asserted when the condition no longer holds. The timer interrupt is globally broadcast, and software selects which HARTs receive it via `mtime_local_target` and `mie`.

Local-HART interrupts can be generated or cleared directly using the `mipi_trigger` and `mipi_clear` registers.



3 ISA changes (Erbium)

Erbium changes the topology compared to ET-SoC-1, resulting in the following ISA-level modifications. All supported features remain available except where explicitly removed.

- **ET Atomic extension:** Global and local variants now affect the same level of the memory hierarchy, unlike ET-SoC-1.
- **TensorSend / TensorReceive:** These instructions will not be performed and will set `tensor_error[9]` if the Receiver/Sender ET-Minion ID is outside the range 0..7.
- **TensorReduce / TensorBroadcast:** These instructions will not be performed and will set `tensor_error[9]` if the tree depth is outside the range 0..2.
- **Removed instructions:**
 - `TensorLoadL2Scp` is not supported and will raise an illegal instruction exception.
- **Deprecated features from ET-SoC-1:**
 - The Messaging Extension has been removed.
 - The M-mode Virtual Memory extension has been removed.
 - The GFX extensions are still present, **except** that `frcp_fix.rast`, `fcvt.ps.rast`, and `fcvt.rast.ps` have been removed and will raise an illegal instruction exception.

4. Erbium Memory

4.1 Memory Map

The Erbium platform uses a **32-bit physical address** which defines a **4 GB address space** organized into different regions, as shown in the table below:

Region name	From	To	From (hex)	To (hex)	Size
System registers	32MB	32MB+4K	0x0200_0000	0x0200_0FFF	4K
MRAM registers	32MB+4K	32MB+8K	0x0200_1000	0x0200_1FFF	4K



Region name	From	To	From (hex)	To (hex)	Size
I2C registers	32MB+8K	32MB+12K	0x0200_2000	0x0200_2FFF	4K
QSPI registers	32MB+12K	32MB+16K	0x0200_3000	0x0200_3FFF	4K
UART registers	32MB+16K	32MB+20K	0x0200_4000	0x0200_4FFF	4K
Bootrom	32MB+32K	32MB+40K	0x0200_8000	0x0200_9FFF	8K
SRAM	32MB+48K	32MB+52K	0x0200_C000	0x0200_CFFF	4K
xSPI registers	32MB+60K	32MB+64K	0x0200_F000	0x0200_FFFF	4K
MRAM	1G	2G	0x4000_0000	0x7FFF_FFFF	8G
CPU subsystem	2G	2.5G	0x8000_0000	0x9FFF_FFF	0.5G
PLIC	2.5G	2.5G+64M	0xA000_0000	0xA3FF_FFF	64M
NIC config	3G+992M	3G+992M+80K	0xFE00_0000	0xFE01_5FF	88K

As per the RISC-V spec, if a HART attempts to access a region of memory where it does not have permissions for the type of access performed, or attempts to access a reserved region of memory, then an **Access fault exception** is generated. If attempts to access a region which is not reserved, but is vacant, a **Bus error interrupt** will occur. This interrupt is generally fatal/unrecoverable, as subsequent instructions may have been executed.

4.2 Physical Memory Attributes

Each ET-Minion core has a local PMA checker that validates all transactions. This PMA checker is controlled by the ET-Neighborhood **mprot** ESR, hence all ET-Minions in the ET-Neighborhood will see the same access checks.

Region name	Cached	Priv	Read	Write	Exec
System registers	No	M/S	Yes ³	Yes ³	No
MRAM registers	No	M	Yes ²	Yes ²	No
I2C registers	No	M/S	Yes ³	Yes ³	No
QSPI registers	No	M/S	Yes ³	Yes ³	No
UART registers	No	M/S	Yes ³	Yes ³	No
Bootrom	Yes	M/S/U	Yes	No	Yes
SRAM	Yes	M/S/U	Yes	Yes	Yes
xSPI registers	No	M	Yes ²	Yes ²	No
NIC config	No	M	Yes ⁴	Yes ⁴	No
MRAM	Yes	M/S/U	Yes ⁵	Yes ⁵	Yes ⁵
CPU subsystem	No	D/M/S/U	Yes ¹	Yes ¹	No
PLIC	No	M/S	Yes ⁴	Yes ⁴	No



¹ Access permitted only when all of the following conditions are met:

- Hart's execution mode $\geq$ ESR privilege mode
- Address maps to an existing ESR register (64-bit aligned)
- Read/write access is from a 64-wide load or store
- Instruction is not an AMO, TensorOp, or CacheOp

² Access permitted only when all of the following conditions are met:

- Read/write access is 64-bit aligned
- Read/write access is from a 64-wide load or store
- Instruction is not an AMO, TensorOp, or CacheOp

³ Access permitted only when all of the following conditions are met:

- Read/write access is 64-bit aligned
- Read/write access is from a 32-wide load or store
- Instruction is not an AMO, TensorOp, or CacheOp

⁴ Access permitted only when all of the following conditions are met:

- Read/write access is 32-bit aligned
- Read/write access is from a 32-wide load or store
- Instruction is not an AMO, TensorOp, or CacheOp

⁵ Accesses above the installed MRAM generate a Bus error interrupt.

If access is not permitted by the PMA, the access is not performed and an **Access fault exception** is generated. The PMAs enforce only the coarse-grain access rules defined in the preceding table. Passing a PMA check does not guarantee that the access is valid.

A load or store may still be illegal even if the PMA allows the address. This occurs when the target address:

- Does not correspond to any implemented device, device register, or memory location, or



- Corresponds to a device that rejects the access due to alignment, size, or access-type constraints.

In these cases, a **Bus Error interrupt** will be raised.

There are also restrictions regarding the type of operation that can be performed at every region as summarized in the table below:

Region name	RISC-V AMO TENSOR CACHEOP			
System registers	Yes	No	No	No
MRAM registers	Yes	No	No	No
I2C registers	Yes	No	No	No
QSPI registers	Yes	No	No	No
UART registers	Yes	No	No	No
Bootrom	Yes	No	No	No
SRAM	Yes	No	Yes	Yes
xSPI registers	Yes	No	No	No
NIC config	Yes	No	No	No
MRAM	Yes	Yes	Yes	Yes
CPU subsystem	Yes	No	No	No
PLIC	Yes	No	No	No

4.3 Physical Memory Protection (PMP)

The PMP mechanism provides a basic security functionality by defining Machine and Supervisor privilege level subregions inside the MRAM region.

Address Range	Privilege Access	Description
0x00_0000 → mmode_end	M-mode only	Machine-only region
mmode_end → smode_end	M/S-mode only	Machine + Supervisor region
smode_end → 0xFF_FFFF	M/S/U-mode	Full access region (Machine, Supervisor, User)

The mechanism can be enabled in the mprot ESR, and the size of the subregions is also configured in the same register.

If disabled, access to the entire MRAM region is allowed for all privilege levels.

When the PMP is enabled:



CONTENTS

- $\text{mmode_end} = 4\text{KB} * (2^{\text{mprot}[\text{mmode_size}]})$
- $\text{smode_end} = 4\text{KB} * (2^{\text{mprot}[\text{smode_size}]})$

Note that the S-mode region maximum size is limited by the size of the M-mode region. If the size of the S-mode region is 0 or less, it means that the S-mode region does not exist.

Access to the subregions with insufficient privilege level will cause an Access Fault Exception in the ET-Minion PMA.



Minion

ISA changes

Erbium presents a change in topology that affects some of the instructions that were available on ET-SoC1. All supported features are present with the following changes:

- ET Atomic extension instructions with global and local variants will affect the same level of memory hierarchy contrary to what happens in ET-SoC1.
- ET *TensorSend* and *TensorReceive* instructions will not be performed and set *tensor_error[9]* when executed with *Receiver/Sender ET-Minion ID* outside the range of 0..7
- ET *TensorReduce* and *TensorBroadcast* instructions will not be performed and set *tensor_error[9]* when executed with *Tree depth* outside the range of 0..2

The following instructions are not supported in Erbium and will raise an illegal exception

- *TensorLoadL2Scp*

ET-SoC1 had a set of deprecated features which suffered the following changes in Erbium:

- Messaging Extension has been removed
- M-mode Virtual Memory extension has been removed
- GFX extensions still present with the following changes
 - *frcp_fix.rast*, *fcvt.ps.rast* and *fcvt.rast.ps* instructions have been removed and will raise an illegal instruction exception

Errata fixes

Erbium fixes the following ET-SoC1 erratas.

- RTLMIN-5374: Gather/Scatter instruction is missing some load/store operations since gscprogress register is not updated
- RTLMIN-6136: Tensor Conv clashes with a write to *tensor_mask*
- RTLMIN-6221: Branch Taken events for performance counters are not correctly generated



- RTLMIN-6282: VPU not connected to Minion Shire warm reset
- RTLMIN-6488: Writes to PMU registers may happen twice
- RTLMIN-6520: Write to the Tensor mask does not wait for previous `tensor_conv_size`
- RTLMIN-6614: Intensive usage of UC operations in one Minion thread may produce starvation to the other thread
- RTLMIN-5985: HART ESR address is not correctly checked
- RTLMIN-6398: Part of a misaligned access may execute with the wrong privilege
- RTLMIN-6397: Exceptions on misaligned accesses may not release Minion resources
- RTLMIN-6455: Resume state machine only works for Minion 0
- RTLMIN-6452: Hart can execute unexpected instructions in debug mode
- RTLMIN-6496: Simultaneous reading from both Minion threads of the PMU registers may return incorrect data to the second thread doing the read
- RTLMIN-6509: Debug access to the L1 ICache tag memory incorrectly enables simultaneous access to the L0 microcache 0 memories

CSR Changes

CSRs are the same as Minion ET-SoC1 with the following changes:

- *mtvec/stvec* CSRs *BASE* field requires
 - In Direct mode, *BASE* field contains bits XLEN-1 through 2 of a 4-byte aligned address
 - In Vector mode, *BASE* field contains bits XLEN-1 though 7 of a 128-byte aligned address. Bits 6 to 2 become WARL(0)
- Field *coopneighmask* in *tensor_cooperation* becomes WARL(0..1)
- *matp* and *satp* are now WARL(0)
- *sum* field in *mstatus* is now RO(0)
- *marchid* CSR value changed to 0x4554000000000001
- *mimpid* CSR value changed to 0x10000
- The following CSRs were removed and will raise an illegal instruction exception
 - *portheadnb0*
 - *portheadnb1*
 - *portheadnb2*



- *portheadnb3*
- *porthead0*
- *porthead1*
- *porthead2*
- *porthead3*
- *portctl0*
- *portctl1*
- *portctl2*
- *portctl3*
- *tex_send*
- The following new CSRs are implemented
 - *tdata3* - implemented as WARL(0)



GPIO Datasheet

The GPIO module provides 11 general-purpose bidirectional digital I/O pins (GPIO[10:0]). Each pin can be individually configured as an input or output via software registers. Pins designated as inputs can optionally generate an interrupt on a rising edge transition. All GPIO pins are shared through a hardware multiplexer with the on-chip peripherals (SPI, QSPI, I2C, UART) and are reclaimed by the respective peripheral when that peripheral is enabled.

GPIO Registers

The GPIO registers are part of the System Registers block at base address **0x40000000**.

Register name	Offset address	Accessible Size	Description
GPIO_OE	0x78	32 bits	GPIO Output Enable Register (Read and Write)
GPIO_I	0x80	32 bits	GPIO Input Register (Read Only)
GPIO_O	0x88	32 bits	GPIO Output Register (Read and Write)
GPIO_Interru	0x90	32 bits	GPIO Interrupt Enable Register (Read and Write)

Register Descriptions

GPIO_OE

Address offset: 0x78 Reset value: 0x000

Bits	Identifier	Description
------	------------	-------------

10:0	gpio_oe	Output Enable: Write 1 to configure the corresponding pin as an output. Write 0 to configure as an input. One bit per GPIO pin.
------	---------	--

GPIO_I

Address offset: 0x80 Reset value: 0x000



Bit Identifier	Description
10:0 gpio_i	Input Value: For each pin configured as an input, reflects the current logic level on the pad. Read-only.

GPIO_O

Address offset: 0x88 Reset value: 0x000

Bit Identifier	Description
10:0 gpio_o	Output Value: For each pin configured as an output, the value written here is driven onto the pad.

GPIO_Interrupt_Enable

Address offset: 0x90 Reset value: 0x000

Bit Identifier	Description
10:0 gpio_inte	Interrupt Enable: For each input-mode pin, setting the corresponding bit to 1 enables an interrupt on a rising edge. The interrupt is aggregated into a single <code>gpio_interrupt</code> signal.

Pin Multiplexer

All 11 GPIO pins are shared with on-chip peripherals. When a peripheral is enabled in `SystemConfig`, the corresponding pins are claimed by the peripheral and are isolated from the GPIO registers (reads of `GPIO_I` will not capture values on those pins, and `GPIO_O` will not drive them).

GPIO Pin(s)	Peripheral	Enable bit in SystemConfig
GPIO[0]	OSC_CLK_OUT	TestMode HW pin (hardware-controlled)
GPIO[1:2]	I2C (SCL, SDA)	<code>i2c_enable</code> (bit 3)
GPIO[3:6]	SPI (CS, CLK, DQ[1:0])	<code>spi_enable</code> (bit 4)
GPIO[7:8]	QSPI (DQ[3:2])	<code>qspi_enable</code> (bit 5)
GPIO[9:10]	UART (TX, RX)	<code>uart_enable</code> (bit 6)

Note: SPI is enabled by default at reset (`spi_enable` = 1). To use GPIO[3:6] as general-purpose I/O, clear `spi_enable` in `SystemConfig`



first. Write `SystemConfig = 0x0` to disable all peripherals and gain full GPIO access to all 11 pins.

Program Flow

Configure GPIO as Output

1. **Disable Peripheral Mux** (if needed): Write to `SystemConfig` to clear any peripheral enable bits that claim the target pins.
2. **Set Output Enable**: Write 1 to the corresponding bit(s) in `GPIO_OE` to configure the pins as outputs.
3. **Drive Output Value**: Write the desired value to `GPIO_0`. The value is immediately reflected on the pad.
4. **Verify**: Read back `GPIO_0` to confirm the written value.

Configure GPIO as Input

1. **Disable Peripheral Mux** (if needed): Write to `SystemConfig` to clear any peripheral enable bits that claim the target pins.
2. **Set Input Mode**: Write 0 to the corresponding bit(s) in `GPIO_OE` to configure the pins as inputs (reset default).
3. **Read Input Value**: Read `GPIO_I`. The value at each bit reflects the current logic level on the corresponding pad.

Enable GPIO Interrupt

1. **Configure as Input**: Ensure the target pin(s) have their `GPIO_OE` bit cleared (input mode).
2. **Enable Interrupt**: Write 1 to the corresponding bit(s) in `GPIO_Interrupt_Enable`.
3. **Detect Interrupt**: Monitor the `gpio_interrupt` signal (aggregated from all enabled pins). A rising edge on any enabled input pin asserts `gpio_interrupt`.
4. **Clear / Disable**: Write 0 to the corresponding bit in `GPIO_Interrupt_Enable` to disable further interrupts for that pin.



I2C Controller Datasheet

- I2C Master-Only Controller
 - 7-bit slave addressing
 - Configurable clock via prescale register
- AXI-Stream FIFO interface (CMD / TX / RX FIFOs)
- Single byte and multi-byte (burst) write transfers
- Repeated START for combined write+read sequences
- NACK detection via `missed_ack` status bit
- Automatic STOP generation (`stop_on_idle` mode)

Memory Map

The I2C register block is accessible at base address **0x40002000**.

Offset	Identifier	Name	Access	Reset	Accessible Size
0x00	Commands	Command Registers	RW	0x0	32 bits
0x08	Status	Status Register	R	0x0	32 bits
0x10	Cfg	Configuration	RW	0x0	32 bits
0x18	Wdata	Write Data FIFO	W	0x0	32 bits
0x20	Rdata	Read Data FIFO	R	0x0	32 bits

Note: The I2C peripheral must be enabled

Registers

Commands Register (0x40002000)

Used to enqueue a command into the CMD FIFO. Write to this register to initiate or describe an I2C transaction. The `enq` bit [5] is a **singlepulse** — it self-clears one cycle after being written; hardware copies the command into the FIFO on the rising edge.

Bits	Identifier	Access	Reset	Description
0	start	RW	0x0	Issue START condition at beginning of transaction
1	read	RW	0x0	Perform a read transaction (slave → master)



Bits	Identifier	Access	Reset	Description
2	write	RW	0x0	Perform a single-byte write transaction
3	write_multi	RW	0x0	Perform a multi-byte write (uses wlast to end burst)
4	stop	RW	0x0	Issue STOP condition at end of transaction
5	enq	RW	0x0	Enqueue the command (single pulse — self-clears)
14:8	address	RW	0x0	7-bit slave address

Status Register (0x40002008)

Read-only register reflecting the current state of the I2C controller and FIFOs. All bits reset to 0 except FIFO not-full flags which are hardware-driven.

Bits	Identifier	Access	Reset	Description
0	busy	RO	0x0	High while a transaction is in progress
1	bus_control	RO	0x0	Master holds bus (has issued START, no STOP yet)
2	bus_active	RO	0x0	SCL/SDA bus activity detected
3	missed_ack	RO	0x0	Slave did not ACK the address (NACK detected)
4	cmd_ff_n_full	RO	—	1 = CMD FIFO has space; 0 = CMD FIFO full
5	tx_ff_n_full	RO	—	1 = TX FIFO has space; 0 = TX FIFO full
6	rx_ff_n_full	RO	—	1 = RX FIFO has data; 0 = RX FIFO empty
7	rx_overflow	RO	—	RX FIFO overflow — data was received when FIFO was full

Polling guidance:

Goal	Poll condition
Wait for transaction to end	<code>busy == 0</code>
Wait for TX space (write)	<code>tx_ff_n_full == 1</code>
Wait for RX data (read)	<code>rx_ff_n_full == 1</code>
Check for addressing error	<code>missed_ack == 1</code>
Check for CMD FIFO space	<code>cmd_ff_n_full == 1</code>



Cfg Register (0x40002010)

Configuration register for clock prescaling and bus-idle behaviour.

Bits	Identifier	Access	Reset	Description
15:0	prescale	RW	0x0	SCL clock divider: $\text{prescale} = \text{Fclk} / (\text{FI2C} \times 4)$
16	stop_on_idle	RW	0x0	When set, auto-issue STOP when CMD FIFO empties

Wdata Register (0x40002018)

Write-only register used to push one byte into the TX FIFO per write. Only valid bits are [8:0]; upper bits are ignored.

Bits	Identifier	Access	Reset	Description
7:0	wdata	WO	0x0	Data byte to transmit
8	wlast	WO	0x0	Set to 1 on the last byte of a burst write

Important: For `write_multiple` transfers, the controller waits in `STATE_WRITE_1` for each successive byte. Set `wlast = 1` on the final byte to signal end of burst and allow the STOP condition to be issued.

Rdata Register (0x40002020)

Read-only register used to pop one byte from the RX FIFO per read access. Poll `Status.rx_ff_n_full` before reading to confirm data is available.

Bits	Identifier	Access	Reset	Description
7:0	rdata	RO	0x0	Received data byte
8	rlast	RO	0x0	Set to 1 when this is the last byte in a read burst



Each read of this register pops one byte from the RX FIFO. Do **not** read more bytes than available — use `rx_ff_n_full` to gate reads.

Enabling the I2C Peripheral

The I2C controller is **disabled by default** at reset.

Full `SystemConfig` register at `0x40000008`:

Bit Field	Reset Description
3 <code>i2c_enable</code> 0	I2C peripheral enable

Program Flow

Write Transfer

1. **Enable Peripheral:** Write to the `SystemConfig` register in the System Control block to enable the I2C clock and interface (`i2c_enable = 1`).
 2. **Set Clock Frequency:** Write to the `CFG_REG` (CFGR) to configure the SCL clock frequency using the `prescale` field. `prescale = Fclk / (FI2C × 4)`.
 3. **Enqueue Command:** Write to the `COMMAND_REG` (CMD) to initiate the transaction. Set the appropriate flags for a write operation (e.g., `start`, `write`, `stop`, `enq`, `address`).
 4. **Provide Data:** Write the data to be transmitted into the `DATA_REG` (DR). For multiple words, ensure the FIFO has space by checking `FLEVEL` in `STATUS_REG`. For write transaction to start the fifo must be filled with minimum 4 bytes of data.
 5. **Wait for Completion:** Monitor `STATUS_REG[1]` (TCF - Transfer Complete Flag). Once set, the transaction is finished.
- For **write multiple bytes** set `wlast = 1` on the last byte to signal end of burst and allow the STOP condition to be issued. Set `Commands = start|write_multiple|stop|enq|(addr<<8)`

Read Transfer

1. **Enable Peripheral:** Write to the `SystemConfig` register in the System Control block to enable the I2C clock and interface (`i2c_enable = 1`).



2. **Set Clock Frequency:** Write to the `CFG_REG` (`CFGR`) to configure the SCL clock frequency using the `prescale` field. `prescale = Fclk / (FI2C × 4)`.
3. **Enqueue Command:** Write to the `COMMAND_REG` (`CMD`) to initiate the transaction. Set the appropriate flags for a read operation (e.g., `start`, `read`, `stop`, `enq`, `address`).
4. **Wait for Completion:** Monitor `STATUS_REG[1]` (TCF - Transfer Complete Flag). Once set, the transaction is finished.

Repeated START (Combined Write + Read)

Used for register-addressed reads (write register address, repeated START, read data).

1. Enable I2C and set prescale
2. Push register address byte: `Wdata = reg_addr | (1<<8)`
3. Enqueue WRITE without STOP:
`Commands = start|write|enq|(addr<<8)      # stop=0`
4. Poll: `while (Status.busy == 1): wait`
5. Enqueue READ with START (repeated START):
`Commands = start|read|stop|enq|(addr<<8)`
6. Collect RX bytes as in the Read Transfer flow above

Error Handling

NACK on Address

If the addressed slave does not ACK:

1. `Status.missed_ack` is set **for one clock cycle** during the ACK phase.
2. The controller terminates the transaction and returns to IDLE.
3. `Status.busy` clears normally.



MRAM

The MRAM subsystem provides the non-volatile memory target behind the SoC interconnect. At the IP level, the AXI-to-MRAM bridge accepts a 512-bit AXI4 slave interface and converts each transaction into commands for four ET MRAM banks. Each bank has its own controller path for startup, ECC, BIST, interrupt capture, and the physical or behavioral bank instance.

The bridge is a local-memory target. System address maps may place the MRAM at different external base addresses for CPU, xSPI, or chiplet views, but the bridge itself sees a zero-based local MRAM address.

Address Space

The normal MRAM data window is 16 MB at local address `0x0000_0000` through `0x00ff_ffff`.

The bridge also recognizes a 12 KB OTP aperture at local address `0x3fff_d000` through `0x3fff_ffff`. That aperture is translated into reserved MRAM rows and columns inside the bank address space. Software sees the OTP window as a contiguous byte range; the bridge performs the sparse physical mapping internally.

Any AXI transaction outside the supported local windows is rejected:

Access	Behavior
Read	Returns SLVERR on each beat, RDATA = 0, and normal RLAST cadence
Write	Drains all write-data beats, issues no MRAM command, and returns SLVERR

The bridge also returns **SLVERR** if the MRAM side is not ready, is not powered, or is in maintenance mode. These status conditions are aggregated across the four banks before the AXI transaction is accepted.

Top-Level Data Path

The current ET integration uses:



Item	Value
AXI data width	512 bits
AXI address width	32 bits
AXI ID width	9 bits
AXI burst length field	8 bits, so up to 256 beats
Bridge bank lanes	4
Per-bank read return	128 bits
Per-bank write payload	64 bits
Physical bank address	17 bits

The high-level path is:

AXI4 slave

- > mkAxi2Mram
- > four MramBankTranslator lanes
- > four ET bank wrappers
- > ET controller, ECC, BIST, startup, interrupt, and bank logic

Each 512-bit AXI beat is split into four 128-bit bank lanes. Inside each bank lane, writes are handled as 64-bit chunks, while reads return a 128-bit pair. The bridge therefore has to track both AXI beat-level ordering and the lower level bank instance and pair selection.

AXI Front End

The bridge accepts AXI read and write address channels independently, then uses an internal arbiter whenever both sides are ready at the same time. The arbitration mode is software configurable:

Mode	Policy
Write priority	Writes win every read/write conflict
Read priority	Reads win every read/write conflict
Round robin	Alternates after simultaneous conflicts
Oldest first	Chooses the channel that became pending first

Round-robin is the reset-default policy.

The bridge serializes active read and write work at the AXI sequencing level. This keeps the bank translators from seeing ambiguous ownership of a bank read or write sequence while still allowing the AXI channels to queue work through the AXI transactor.



Read Flow

For a read, the bridge builds two plans:

Plan	Purpose
MRAM issue plan	Which bank lanes must be read and at which translated bank address
AXI beat plan	Which bytes belong in each AXI RDATA beat and when bank responses can retire

This split is important for unaligned accesses and bursts whose bytes cross 64-byte AXI line boundaries. The bridge only issues a bank read when a new bank lane or physical group is needed, but it still emits AXI read beats in the requested order.

When all selected bank responses for an AXI beat are available, the bridge assembles the 512-bit RDATA value from the four 128-bit bank responses. Bytes outside the requested transfer size are masked to zero. The response code is normally OKAY, or EXOKAY for a successful exclusive read, unless the access was rejected.

Write Flow

For a write, the bridge drains the AXI W channel, masks WDATA byWSTRB, and accumulates data until either the current 64-byte line is complete or WLAST is observed. It then splits the accumulated value into per-bank commands.

Each bank lane contains two 64-bit write chunks per 128-bit lane:

Strobe shape	Bank command
Full 8-byte chunk	Direct write
Partial 8-byte chunk	Read-modify-write

Partial writes are handled by reading the old 64-bit word from the bank, merging only the strobed bytes, and then writing the merged word back. This keeps the stored ECC word coherent after byte-granular writes.

BRESP is returned after the bridge has drained the AXI write data and accepted or suppressed the corresponding bank commands. A rejected write drains normally but does not enqueue any bank command.



Exclusive Accesses

The bridge includes an AXI exclusive-access monitor. Exclusive reads create a reservation and return **EXOKAY**. Exclusive writes return **EXOKAY** only when a matching reservation is present; otherwise they return **OKAY** and suppress the MRAM write.

Normal writes snoop the monitor and invalidate overlapping reservations.

The monitor tracks up to 32 AXI IDs at a time, with up to four reservations per ID. Both levels use round-robin replacement when full.

Bank Translator

Each of the four bank lanes has a **MramBankTranslator**. This block converts bridge commands into MRAM-facing control signals:

Signal group	Purpose
ce_o[7:0]	Selects one write instance or one adjacent read pair
dout_en_o[7:0]	Captures a read result from the selected pair
we_o	Selects write versus read behavior
addr_o[16:0]	Physical bank word address
din_o[63:0]	64-bit write payload
bwe_o[63:0]	Bit write-enable mask for the 64-bit payload

Reads are pair-based: a read selects two adjacent instances and returns a 128-bit value. Writes are instance-based: a write selects one of the eight instances and writes a 64-bit value.

The translator also owns the bank timing hazards:

Hazard	Handling
Read/read pair conflict	Fixed read-pair busy pipeline
Read/write pair conflict	Writes wait through the read launch and dout_en window
Write/write instance conflict	Writes wait for local early-busy and sampled MRAM busy to clear
RMW atomicity	The selected pair stays reserved until the write-back phase completes

Write busy is tracked conservatively. The translator asserts local busy as soon as it launches a write, holds it while the command reaches the MRAM,



and then tracks the MRAM busy signal sampled on the bank clock edge. This avoids issuing a conflicting command in the gap before the memory macro reports busy.

ET Controller Path

Behind each bank translator is an ET bank wrapper. At a public architectural level, that wrapper contains:

Block	Role
Boot sequencer	Waits for power-good, releases MRAM reset, performs startup sequencing, and reports bank ready
Control mux	Chooses between normal AXI traffic and internal maintenance/BIST traffic
ECC wrapper	Encodes writes and corrects reads
BIST wrapper	Drives built-in memory test and reference-trim flows
Interrupt logic	Captures ECC error class and address for CPU interrupt reporting
Global configuration logic	Drives bank configuration and sleep behavior
Bank instance	Physical macro wrapper or behavioral model

The bridge observes each bank's ready, power-good, maintenance, busy, data, and ECC error signals through this controller path.

ECC

Normal data writes start as 64-bit payload words. The controller ECC path encodes each payload into the wider stored MRAM word. On reads, the lower and upper stored words from the bank are corrected and returned to the bridge as a 128-bit payload.

The ET path reports single-bit, double-bit, and triple-bit ECC events per read lane. The wrapper counts these events and can raise the MRAM interrupt. The interrupt path captures the first unmasked ECC event until software clears it through the system-level interrupt flow.



The controller also has a separate reference-word ECC path used by reference programming and trim behavior. That path is distinct from the normal payload BCH path.

Address Translation And Layout

The AXI side addresses memory as bytes. Internally, the bridge converts those byte addresses into bank, instance, and word locations.

At the bridge level, each 64-byte AXI data beat maps naturally onto the four bank lanes:

AXI byte range within a beat Bank lane	
Bytes 0-15	Bank 0
Bytes 16-31	Bank 1
Bytes 32-47	Bank 2
Bytes 48-63	Bank 3

Within each bank lane, the lower 8 bytes select one 64-bit instance write and the upper 8 bytes select the adjacent instance write. Reads select the pair and return both 64-bit halves as one 128-bit bank response.

The OTP aperture is not a separate physical memory. It is a software-visible window that the bridge translates into reserved MRAM locations. Only the OTP window receives this special translation; normal MRAM data addresses are passed through the regular bank-address mapping.

Functional Guarantees

The MRAM block behaves as a strongly ordered memory target at the bridge interface. It does not expose out-of-order read data or write responses to the requester. When read and write work arrive together, the selected arbitration policy chooses which channel enters the memory pipeline first.

Important software-visible guarantees are:

Topic	Guarantee
Rejected reads	Complete with SLVERR , zero data, and normal burst length
Rejected writes	Drain all write-data beats and return SLVERR without changing MRAM contents



Topic	Guarantee
Byte writes	Honor WSTRB ; unstrobbed bytes are preserved through RMW when needed
Exclusive writes	Modify MRAM only on a matching reservation
Normal writes	Invalidate overlapping exclusive reservations
MRAM not ready	Transactions are rejected rather than queued indefinitely
Clock gating	Does not change software-visible memory ordering

The bridge and bank translators also prevent same-bank timing hazards from leaking up to the AXI interface. Reads and writes may stall internally while a bank pair or instance is busy, but accepted transactions still complete through the normal AXI response channels.

BIST And Maintenance Traffic

The controller includes built-in memory test support for write, read, and reference-trim flows. BIST traffic is muxed onto the same bank command pins as normal AXI traffic, so the controller arbitrates the source before commands reach the bank macro.

BIST and maintenance flows can exercise ranges, loop over addresses, compare read data against expected patterns, stop on error, count replacement or compare failures, and run reference-trim search behavior. These flows are used for bring-up, manufacturing, and health checks rather than normal software data movement.

Clocking, Reset, And Power

The AXI bridge has one main clock and generates one gated MRAM-facing clock per bank lane. During startup, the bridge bypasses clock gating for an initial window. After that, each bank clock is enabled when any of the following is true:

Condition

Clock gating is disabled for that bank
AXI work is pending



Condition
The bank reports busy
The translator has tracked local busy
The per-bank command queue is active

The controller path has its own startup sequencing. It synchronizes power-good, holds MRAM reset until the bank is powered, optionally bypasses startup, and reports ready only after the startup sequence has completed.

Deep sleep and maintenance status are treated as transaction-blocking conditions by the bridge.

Register Visibility

The public bridge register block exposes control and status for:

Register class	Contents
Arbitration	Read/write conflict policy
Bridge status	AXI busy, command queue activity, per-bank ready
Error status	Sticky clear-on-read SLVERR causes
Control	Per-bank clock-gate disable and ECC interrupt masks
ECC counters	Saturating counts of observed ECC lane events

The register block is generated from SystemRDL as part of the integration collateral.



QSPI Datasheet

The Quad SPI is an improved version of SPI with multiple IO lines i.e. QSPI supports dual as well as Quad IO lines support. (Better performance compared to legacy SPI). The QSPI also comes with the Execute in Place option which avoids copying the code to the On chip memory and executing. The core can directly read from flash and execute. i.e. the QSPI flash chips can act like normal RAM to the core.

The QSPI implementation also has programmable address size, data size, dummy cycle, instruction size, etc. The IO lines as well as the QSPI clock are all software configurable.

Features

- Supports Single/dual/Quad SPI mode.
- Single / Double Data rate support.
- Supports 3 modes
 - Indirect mode (All operations are performed using QSPI registers).
 - Automatic status polling mode (flash status register is periodically read and interrupt is generated on flag setting).
 - Memory mapped mode (XiP mode): flash memory is mapped into device address space and seen as internal memory.
- Programmable for Single SPI (Legacy SPI), Dual IO mode & Quad IO mode.
- Programmable widths for Address, data, dummy cycles, instruction, alternative mode, etc.
- Common 32 byte FIFO is shared across both transmit & receive.
- Status register & interrupt enable registers.
- Programmable flash size.
- Mode 0 and mode 3 support.

Ports

Port Name	Description
CLK O	Serial clock for QSPI communication.
CS O	Chip select signal for QSPI applications.
IO[3:0] IO	Quad data lines. Output when configured to send commands and acts as input when configured to receive data (response).



QSPI registers

The original register IP sizes are variable for the Shakti use cases, and the data duplicates it across for the access size. Although logically 32-bit or less on the IP side, they are accessible as 64-bit values on the system bus.

Register name	Offset address	IP Side Size	Accessible Size	Description
CONTROL_REG0x00	0x00	32 bits	32 bits	Device Control Register (Read and Write)
DEVICE_CFG0x08	0x08	32 bits	32 bits	Device Configuration Register (Read and Write)
STATUS_REG0x10	0x10	16 bits	32 bits	Device Status Register (Read Only)
FLAG_CLEAR0x18	0x18	8 bits	32 bits	Flag Clear Register (Write and Read)
DATA_LENGTH_CFG0x20	0x20	32 bits	32 bits	Data Length Register (Read and Write)
COMMN_CFG0x28	0x28	32 bits	32 bits	Communication Configuration Register (Read and Write)
ADDRESS_REG0x30	0x30	32 bits	32 bits	QSPI Flash Address Register (Read and Write)
ALTERNATE0x38	0x38	32 bits	32 bits	Alternate Byte Register (Read and Write)
DATA_REG0x40	0x40	32 bits	32 bits	Data Register (Read and Write)
POLL_STS_0x48	0x48	32 bits	32 bits	Polling Status Mask Register (Read and Write)
POLL_STS_MATCH0x50	0x50	32 bits	32 bits	Polling Status Match Register (Read and Write)
POLL_INTF0x58	0x58	32 bits	32 bits	Polling Interval Register (Read and Write)
LOW_POWER_TIMEOUT_CFG0x60	0x60	32 bits	32 bits	Low Power Timeout Register (Read and Write)

Register Descriptions

CONTROL Register

Address offset: 0x00 Reset value: 0x0000 0x0000



Field	Description
31:0 FCLK divider	This bitfield defines the scalar factor for generating CLK based on the AHB clock (value+1).0: FCLK = FAHB, AHB clock used directly as QUADSPI CLK (prescaler bypassed)1: FCLK = FAHB/22: FCLK = FAHB/3...255: FCLK = FAHB/256Note: This bitfield can be modified only when BUSY = 0
2P Polling match mode	This bit indicates which method must be used for determining a “match” during automatic status-polling mode.0: AND match mode. SMF is set if all the unmasked bits received from the flash memory match the corresponding bits in the match register.1: OR match mode. SMF is set if any one of the unmasked bits received from the flash memory matches its corresponding bit in the match register.Note: This bit can be modified only when BUSY = 0.
2A Automatic status-polling mode stop	This bit determines if automatic status-polling is stopped after a match.0: Automatic status-polling mode is stopped only by abort or by disabling the QUADSPI.1: Automatic status-polling mode stops as soon as there is a match.Note: This bit can be modified only when BUSY = 0.
2R	Must be kept at reset value.
20 Timeout interrupt enable	This bit enables the timeout interrupt.
1S Status match interrupt enable	This bit enables the status match interrupt.
1F FIFO threshold interrupt enable	This bit enables the FIFO threshold interrupt.
1T Transfer complete interrupt enable	This bit enables the transfer complete interrupt.
1E Transfer error interrupt enable	This bit enables the transfer error interrupt.
1R	Must be kept at reset value.
12:0 FIFO threshold level	This bitfield defines, in indirect mode, the threshold number of bytes in the FIFO that causes the FIFO threshold flag (bit FTF in register STATUS_REG) to be set.
7F Flash memory selection	This bit selects the flash memory to be addressed in single-flash mode (when DFM = 0).0: FLASH 1 selected1: FLASH 2 selectedNote: This bit can be modified only when BUSY = 0. This bit is ignored when DFM = 1.



Bit	Description
6	Dual-flash mode: This bit activates dual-flash mode, where two external flash memories are used simultaneously to double throughput and capacity.0: Dual-flash mode disabled1: Dual-flash mode enabledNote: This bit can be modified only when $BUSY = 0$.
5	Must be kept at reset value.
4	Sample shift: By default, when $SSHIFT = 0$, the QUADSPI samples data on the following CLK edge after the data is driven by flash memory. When $SSHIFT = 1$, an additional data sampling delay is introduced. Note: This bitfield can only be modified when $BUSY = 0$.
3	Timeout counter enable: This bit is valid only when memory-mapped mode ($FMODE = 11$) is selected. Activating this bit causes the NCS to be released if there has not been an access after a certain amount of time.
2	DMA enable: This bit enables DMA for the corresponding channel.
1	Abort request: Writing 1 to this bit aborts the current transfer.
0	QSPI enable: This bit enables the QUADSPI.

DEVICE_CONFIG_REG (DCR)

Address offset: 0x08 Reset value: 0x0000 0x0000

Bit	Description
20	Flash memory size: This bitfield defines the size of the external memory. (Size in bytes = $2^{(FSIZE+1)}$).
10	Chip select high time: This bitfield defines the minimum number of clock cycles where the chip select (NCS) must remain high between two consecutive accesses.
0	Clock mode: Selects the clock mode.0: Mode 0 (CLK is 0 when idle)1: Mode 3 (CLK is 1 when idle)

STATUS_REG (SR)

Address offset: 0x10

Bit	Description
13	FIFO level: Indicates the number of bytes currently available in the FIFO.



Bits	Identifier	Description
5	BUS	Busy flag: This bit is set when the QUADSPI is performing an operation.
4	TOF	Timeout flag: Set when the timeout counter reaches 0 in memory-mapped mode.
3	SMF	Status match flag: Set in automatic status-polling mode when the received status matches the match register.
2	FTF	FIFO threshold flag: Set when the FIFO level crosses the threshold defined in FTHRES.
1	TCF	Transfer complete flag: Set when the programmed number of data has been transferred.
0	TEF	Transfer error flag: This bit is set in indirect mode when an invalid address is being accessed in indirect mode. It is cleared by writing 1 to CTEF.

FLAG_CLEAR_REG (FCR)

Address offset: 0x18 Reset value: 0x0000 0x0000

Bits	Identifier	Description
4	CTOF	Clear timeout flag: Writing 1 clears the TOF flag in STATUS_REG.
3	CSMF	Clear status match flag: Writing 1 clears the SMF flag in STATUS_REG.
1	CTCF	Clear transfer complete flag: Writing 1 clears the TCF flag in STATUS_REG.
0	CTEF	Clear transfer error flag: Writing 1 clears the TEF flag in STATUS_REG.

DATA_LENGTH_REG (DLR)

Address offset: 0x20 Reset value: 0x0000 0x0000

Bits	Identifier	Description
31:0	DL[31:0]	Data length: Number of bytes to transfer (programmed value + 1).

**COMMN_CONFIG_REG (CCR)**

Address offset: 0x28 Reset value: 0x0000 0x0000

Bit Description

- 31DDR Double data rate mode:** This bit sets the DDR mode for the address, alternate byte and data phase:0: DDR mode disabled1: DDR mode enabledNote: This bit can be written only when BUSY = 0.
- 30DH DDR hold:** This bit delays the data output by 1/4 of the QUADSPI output clock cycle in DDR mode:0: Data output delayed using analog delay1: Data output delayed by 1/4 of a QUADSPI output clock cycleThis feature is only active in DDR mode.Note: This bit can be written only when BUSY = 0. PRESCALER > 0 is mandatory when DHHC = 1.
- 28SIO Send instruction only once mode:** This bit has no effect when IMODE = 00.0: Instruction sent on every transaction1: Instruction sent only for the first commandNote: This bit can be written only when BUSY = 0.
- 27FM Functional mode:** This bitfield defines the QUADSPI functional mode of operation.00: Indirect-write mode01: Indirect-read mode10: Automatic status-polling mode11: Memory-mapped modeNote: This bitfield can be written only when BUSY = 0.
- 25DM Data mode:** This bitfield defines the data phase mode of operation:00: No data01: Data on a single line10: Data on two lines11: Data on four linesNote: This bitfield can be written only when BUSY = 0.
- 2'D Programming Dummy confirmation bit:** needed by Micron model to trigger XIP mode
- 22DSYN Number of dummy cycles enable:** This bitfield defines the duration of the dummy phase. In both SDR and DDR modes, it specifies a number of CLK cycles (0-31).Note: This bitfield can be written only when BUSY = 0.
- 1'AB Alternate-byte size:** This bit defines the size of alternate bytes.00: 8-bit alternate byte01: 16-bit alternate bytes10: 24-bit alternate bytes11: 32-bit alternate bytesNote: This bitfield can be written only when BUSY = 0.
- 15AB Alternate byte mode:** This bitfield defines the alternate-byte phase mode of operation.00: No alternate bytes01: Alternate bytes on a single line10: Alternate bytes on two lines11: Alternate bytes on four linesNote: This bitfield can be written only when BUSY = 0.



Bit Identification

1:AD	Address size: This bit defines address size:00: 8-bit address01: 16-bit address10: 24-bit address11: 32-bit addressNote: This bitfield can be written only when BUSY = 0.
11:AM	Address mode: This bitfield defines the address phase mode of operation.00: No address01: Address on a single line10: Address on two lines11: Address on four linesNote: This bitfield can be written only when BUSY = 0.
9:IM	Instruction mode: This bitfield defines the instruction phase mode of operation.00: No instruction01: Instruction on a single line10: Instruction on two lines11: Instruction on four linesNote: This bitfield can be written only when BUSY = 0.
7:INS	Instruction: Instruction opcode to be sent to the external SPI device.Note: This bitfield can be written only when BUSY = 0.

ADDRESS_REG (AR)

Address offset: 0x30 Reset value: 0x0000 0x0000

Bit Identification

31:ADR	Address: This bitfield contains the address to be sent to the external flash memory.
--------	---

ALTERNATE_BYTE_REG (ABR)

Address offset: 0x38 Reset value: 0x0000 0x0000

Bit Identification

31:ALT	Alternate bytes: Optional data to be sent to the external SPI device right after the address.Note: This bitfield can be written only when BUSY = 0.
--------	--

DATA_REG (DR)

Address offset: 0x40 Reset value: 0x0000 0x0000



Bitfield Description

DATA[31:0] Data to be sent/received to/from the external SPI device. In indirect write mode, data written to this register is stored on the FIFO before it is sent to the flash memory during the data phase. If the FIFO is too full, a write operation is stalled until the FIFO has enough space to accept the amount of data being written. In indirect read mode, reading this register gives (via the FIFO) the data which was received from the flash memory. If the FIFO does not have as many bytes as requested by the read operation and if BUSY=1, the read operation is stalled until enough data is present or until the transfer is complete, whichever happens first. In automatic status-polling mode, this register contains the last data read from the flash memory (without masking). Word, halfword, and byte accesses to this register are supported. In indirect write mode, a byte write adds 1 byte to the FIFO, a halfword write 2, and a word write 4. Similarly, in indirect read mode, a byte read removes 1 byte from the FIFO, a halfword read 2, and a word read 4. Accesses in indirect mode must be aligned to the bottom of this register: a byte read must read DATA[7:0] and a halfword read must read DATA[15:0].

POLL_STS_MASK_REG (PSMKR)

Address offset: 0x48 Reset value: 0x0000 0x0000

Bitfield Description

STATUS_MASK[31:0] Mask to be applied to the status bytes received in automatic status-polling mode. Note: This bitfield can be written only when BUSY = 0.

POLL_STS_MATCH_REG (PSMAR)

Address offset: 0x50 Reset value: 0x0000 0x0000

Bitfield Description

STATUS_MATCH[31:0] Value to be compared with the masked status register to get a match. Note: This bitfield can be written only when BUSY = 0.



POLL_INTERVAL_REG (PIR)

Address offset: 0x58 Reset value: 0x0000 0x0000

Bit	Description
-----	-------------

31:10	Polling interval: Number of CLK cycles between two reads during automatic status-polling phases. Note: This bitfield can be written only when BUSY = 0.
-------	--

LOW_POWER_TIMEOUT_REG (LPTR)

Address offset: 0x60 Reset value: 0x0000 0x0000

Bit	Description
-----	-------------

31:10	Timeout period: Number of CLK cycles the QUADSPI waits after the FIFO becomes full until it raises NCS in memory-mapped mode. Note: This bitfield can be written only when BUSY = 0.
-------	---

Features Tested

- Single/Quad SPI mode.
- indirect mode (All operations are performed using QSPI registers).

Program Flow

Initialization

1. **Enable Peripheral:** Write to the **SystemConfig** register in the System Control block to enable the QSPI clock and interface (**qspi_enable** = 1).
2. **Configure Flash Parameters (optional):** Write to the **DEVICE_CONFIG_REG** (DCR) to set the flash memory size (**FSIZE**) and clock mode (**CKMODE**).
3. **Enable Controller:** Write to **CONTROL_REG** (CR) to set the clock prescaler and enable the controller (**QSPI_ENABLE** = 1).

Indirect Write Transaction

1. **Set Data Length:** Write the number of bytes to be transferred to the **DATA_LENGTH_REG** (DLR).



2. **Configure Transaction:** Write to the `COMMN_CONFIG_REG` (CCR) to specify the instruction, address mode, data mode, and dummy cycles. Set `FMODE = 00` (Indirect Write).
3. **Set Address:** Write the target flash address to the `ADDRESS_REG` (AR). This prepares the controller for the start of the transaction.
4. **Provide Data:** Write the data to be transmitted into the `DATA_REG` (DR). For multiple words, ensure the FIFO has space by checking `FLEVEL` in `STATUS_REG`. For write transaction to start the fifo must be filled with minimum of 4 bytes of data.
5. **Wait for Completion:** Monitor `STATUS_REG[1]` (TCF - Transfer Complete Flag). Once set, the transaction is finished.
6. **Clear Flags:** Write 1 to `FLAG_CLEAR_REG[1]` (CTCF) to clear the transfer complete flag for the next operation.

Example: Quad Page Program (Instruction 0x32)

- Target: Write 4 bytes to address 0x000000.
- DLR = 0x00000003 (4 bytes).
- CCR = 0x03002532 (imode=1, admode=1, adsize=2, dmode=3, fmode=0, instr=0x32).
- AR = 0x00000000.
- DR = 0xDEADBEEF.

Indirect Read Transaction

1. **Set Data Length:** Write the number of bytes to be read to the `DATA_LENGTH_REG` (DLR).
2. **Configure Transaction:** Write to the `COMMN_CONFIG_REG` (CCR) with `FMODE = 01` (Indirect Read) and relevant phase modes.
3. **Trigger Transaction:** Write the source flash address to the `ADDRESS_REG` (AR). The controller will automatically start fetching data.
4. **Read Data:** Monitor `STATUS_REG[13:8]` (FLEVEL) or `STATUS_REG[2]` (FTF). Read the received data from the `DATA_REG` (DR).
5. **Wrap Up:** Wait for TCF to be set, then clear the flag via `FLAG_CLEAR_REG`.

Example: Quad Fast Read (Instruction 0xEB)

- Target: Read 4 bytes from address 0x001000.
- DLR = 0x00000003 (4 bytes).
- CCR = 0x07182CEB (imode=1, admode=3, adsize=2, dmode=3, fmode=1, dcyc=6, instr=0xEB).



CONTENTS

- AR = 0x00001000.
- Read DR once FTF or TCF is set.



UART Datasheet

The UART module provides a two-wire asynchronous serial non-return-to-zero (NRZ) communication with RS-232 (RS-422/485) interface. Each UART module has transmit and receive buffers that can hold up to 16 entries. Data transfer rate can be modified by providing appropriate value to the UART-BAUD register.

Required data rate = Internal clock frequency / (UARTBAUD * 16)

The data transmission and reception starts with a start bit followed by data words whose length can be varied through UARTCHARSIZE in UART-CONTROL register which is followed by optional parity bit and stop bits. The parity can be Odd or Even and the parity can also be disabled through UARTPARITY bits in the UARTCONTROL register. The number of stop bits can also be varied through UARTSTOPBITS in the UARTCONTROL register. Delayed Transmit control is done by providing the required delay in the UARCTXDELAY register.

The value to be transmitted is written into the UARCTXDATA register and the value received is read from the UARTRXDATA register. UARTSTATUS register holds the UART status & UARTINTERRUPTEN register holds the interrupt enable for the various interrupt events. UARTRXTH register holds the receiver FIFO threshold value, when the RX FIFO level increases beyond the threshold, corresponding status bit will be set and when interrupt is enabled, interrupt will be raised. UART can be operated in full-duplex or half-duplex mode. The inputs to UART receiver and transmitter are double-buffered and each has its own separate enable and interrupt bits.

UART has three external pins - Transmitter data pin, Receiver data pin and Transmitter output enable pin.

UART Registers

The original register IP sizes are variable for the Shakti use cases, and the data duplicates it across for the access size. Although logically 32-bit or less on the IP side, they are accessible as 64-bit values on the system bus.

Register name	Offset address	IP Side Size	Accessible Size	Description
BaudReg	0x00	16 bits	32 bits	UART Baud Rate Register (Read and write)



Register name	Offset address	IP Side Size	Accessible Size	Description
TxReg	0x08	32 bits	32 bits	UART Transmitter Data Register (Write only)
RxReg	0x10	32 bits	32 bits	UART Receiver Data Register (Read Only)
StatusReg	0x18	16 bits	32 bits	UART Status Register (Read only)
DelayReg	0x20	16 bits	32 bits	UART Transmitter Delay Register (Read and write)
ControlReg	0x28	16 bits	32 bits	UART Control Register (Read and write)
IQC	0x30	8 bits	32 bits	Input Qualification Control Register (Read and write)
Rx_Thre	0x38	8 bits	32 bits	UART Receiver Threshold Register (Read and write)
InterruptRaw	0x48	16 bits	32 bits	Raw Interrupt Status
InterruptEn	0x50	16 bits	32 bits	Interrupt Enable
InterruptMask	0x58	16 bits	32 bits	The post-masked value of interrupt.

Register Descriptions

BaudReg

Address offset: 0x00 Reset value: 0x05

Bits	Identifier	Description
15:0	baud_value	Baud Rate: Calculated by $\text{Baud_value} = (\text{Clock Frequency}) / (16 * \text{Baudrate})$

TxReg

Address offset: 0x08

Bits	Identifier	Description
31:0	data	TX Data: The value to be transmitted is written into this register.



RxReg

Address offset: 0x10

Bits	Identifier	Description
------	------------	-------------

31:0	data	RX Data: The received value can be read from this register.
------	------	--

StatusReg

Address offset: 0x18

Bit	Identifier	Comments
-----	------------	----------

8	rx_fifo_threshold	RX FIFO Threshold: Set if RX FIFO reaches the value set in the Rx_Threshold register.
7	break_error	Break Error: Set when the data and stop bits are both zero.
6	frame_error	Frame Error: Set when the stop bit is zero.
5	overrun_error	Overrun Error: Set when additional data arrives while the RX FIFO is full.
4	parity_error	Parity Error: Set when the receive character has incorrect parity.
3	rx_full	Receiver Full: Set when the Receive Buffer is Full.
2	rx_notEmpty	Receiver Not Empty: Set when there is some data in the Receive Buffer.
1	tx_full	Transmitter Full: Set when the transmit Buffer is full.
0	tx_empty	Transmitter Empty: Set when the Transmit Buffer is empty.

DelayReg

Address offset: 0x20 Reset value: 0x0

Bits	Identifier	Description
------	------------	-------------

15:0	delay_count	Transmit Delay Count: Delayed Transmit control delay cycles.
------	-------------	---

ControlReg

Address offset: 0x28



InterruptRaw

10:5charsize	Character size: Character size of data (max 32). Default is 8.
4:3 parity	Parity selection: 0 -> no parity, 1 -> odd parity, 2 -> even parity
2:1 stopbits	Stop bits: 00 -> 1 stop bit, 01 -> 1.5 stop bits, 10 -> 2 stop bits

InterruptRaw

Address offset: 0x48

Bits	Identifier	Comments
8	rx_fifo_threshold	RX FIFO ~80% Full Interrupt
7	break_error	Break Error Interrupt
6	frame_error	Frame Error Interrupt
5	overrun_error	Overrun Interrupt
4	parity_error	Parity Error Interrupt
3	rx_not_empty	Receiver Not Empty Interrupt
2	rx_not_full	Receiver Not Full Interrupt
1	tx_not_full	Transmitter Not Full Interrupt
0	tx_done	Transmission Done Interrupt

InterruptEnable

Address offset: 0x50

Bits	Identifier	Comments
8	rx_fifo_threshold_en	RX FIFO ~80% Full Interrupt Enable
7	break_error_en	Break Error Interrupt Enable
6	frame_error_en	Frame Error Interrupt Enable
5	overrun_error_en	Overrun Interrupt Enable
4	parity_error_en	Parity Error Interrupt Enable
3	rx_not_empty_en	Receiver Not Empty Interrupt Enable
2	rx_not_full_en	Receiver Not Full Interrupt Enable
1	tx_not_full_en	Transmitter Not Full Interrupt Enable
0	tx_done_en	Transmission Done Interrupt Enable

InterruptStatus

Address offset: 0x58



Bits	Identifier	Comments
8	rx_fifo_threshold	RX FIFO ~80% Full Interrupt
7	break_error	Break Error Interrupt
6	frame_error	Frame Error Interrupt
5	overrun_error	Overrun Interrupt
4	parity_error	Parity Error Interrupt
3	rx_not_empty	Receiver Not Empty Interrupt
2	rx_not_full	Receiver Not Full Interrupt
1	tx_not_full	Transmitter Not Full Interrupt
0	tx_done	Transmission Done Interrupt

IQC

Address offset: 0x30

Bits	Identifier	Description
7:0	qual_cycles	Input Qualification Cycles: Filter cycles for IQC.

Rx_Threshold

Address offset: 0x38

Bits	Identifier	Description
7:0	rx_level	RX Threshold Level: Holds the receiver FIFO threshold value. When the RX FIFO level increases beyond the threshold, the corresponding status bit will be set.

Program Flow

Initialization

1. **Global Enable:** Enable the UART peripheral in the `SystemConfig` register (set `uart_enable = 1`).
2. **Baud Rate:** Set the `BaudReg` with the calculated divisor for the desired baud rate.
3. **Line Coding:** Configure `ControlReg` for the required character size (e.g., 8-bit), parity (None/Odd/Even), and stop bits (1, 1.5, or 2).



Data Transmission

1. **Check Status:** Read `StatusReg` and verify that `tx_full` (bit 1) is 0 or `tx_empty` (bit 0) is 1.
2. **Write Data:** Write the 8-bit or 32-bit data word to the `TxReg`.
3. **Wait for Done:** (Optional) Poll `StatusReg[0]` to ensure the transmitter has finished sending all bits if consecutive high-speed writes are not being used.

Data Reception

1. **Monitor Status:** Poll `StatusReg[2]` (`rx_notEmpty`) to check if data has been received and stored in the RX FIFO.
2. **Read Data:** Read the received word from the `RxReg`.
3. **Error Checking:** After reading, check `StatusReg` bits [7:4] for any `break_error`, `frame_error`, `overrun_error`, or `parity_error` that might have occurred during the reception.

Interrupt

The interrupt mechanism consists of 3 registers, Interrupt Raw, Interrupt Mask and Interrupt Status,

- `InterruptStatus` is a readonly register and contains the result of anding the Raw and Mask registers. The interrupt generated from uart is teh OR reduce of all bits of `interruptStatus`.
- `InterruptStatus` is a stick bit set when the corresponding interrupt condition occurs and stays set until it is cleared by the software. Clearing of interrupt is done by writing the current value of the corresponding status bit back to that bit. i.e.
- Writing a 1 to a cleared status bit sets the interrupt, this can be used to test the interrupt behavior during S/W development/debug.

Current Value	Write Value	Result
0	0	cleared
1	1	set
1	0	set
0	1	set



Steps for running verification

- `git clone git@github.com:openhwgroup/core-et-erbium.git`
- `cd hdl-et/`
- `make -C tb setup`
- `uv sync`
- `source .venv/bin/activate`
- `make <MODULE=testname>`



CONTENTS

Note

Arrow direction is from Initiator to Target.

Do we want the client to use the chiplet interface to preload a program in the IMEM of cpu subsystem.

Power Domains

1. Power switch is on VDD.
2. Isolation cells are on output ports
3. Logic outside the powerdomain blocks is

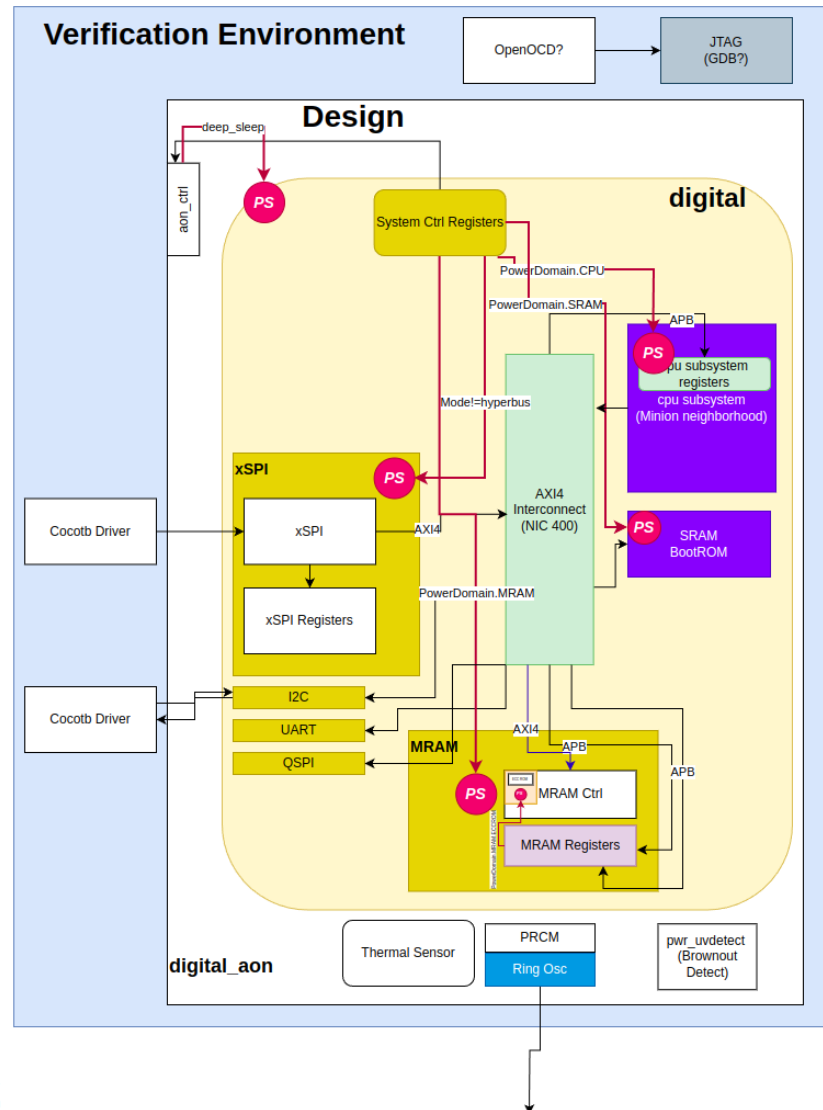


Figure 12: Verification Environment



Simulation Environment

- The Simulation environment is a mix of Cocotb + SystemVerilog.
- Existing unit level environments in SV/SV-UVM are retained as is.
- New environments are created using the cocotb framework.
- The simulation env for each module is in the testbench folder. This is a Makefile based cocotb environment.
- The Makefile is customized for VCS, for other simulators replace COMPILE_ARGS, PLUS_ARGS and SIM with the appropriate options.
- The code may contain language artifacts that cannot be processed by opensource simulators like Verilator/Icarus verilog.
- Python version equal to or higher than 3.10 is required to run the simulation.
- Use test_digital.py as a template for new tests.
- Most of the SOC tests are in `erbium_digital/testbench` folder

To know more about cocotb ref

- [cocotb documentation](#),
- [cocotb tutorial](#)



DV Master Plan

(for ET Erbium)

Architecture Freeze - DV Plan

Mission

Verify functional correctness, robustness, and compliance before tape-out

Verification Strategy

- Methodology:
 - Directed testing
 - Constrained randoms to uncover corner cases, unexpected interactions and boost confidence in the design
- Testbench Architecture:
 - Re-use and improve
 - ★ Erbium and ET-SoC-1 testbenches
 - ★ Copper SOC testbench
 - ★ MRAM testbench
 - Develop
 - ★ ET-Link to AXI4 bridge testbench
 - ★ Periph testbench (QSPI, xSPI, I2C, UART)
- Regression and automation:
 - B4C (Github CI) to validate new changes (subset of tests)
 - Weekly regression to measure overall progress and readiness (full test suite)

DV Testbenches

- CORE-ET Verification Library (EVL):
 - Based on SystemVerilog and Universal Verification Methodology (UVM)
 - Covers only Neigh and Minion verification
 - Source code: [nekkoai/et-soc](#)



- ET-Link to AXI4 (ETL2AXI):
 - Based on PyUVM, written in Python
 - Covers only the ET-Link to AXI4 Bridge
 - Source code: [nekkoi/etl2axi-tb](#)
- CocoTB:
 - Based on Python
 - Covers Full Chip verification
 - Source code: [Erbium/HDL](#)
- MRAM:
 - Based on SystemVerilog and Universal Verification Methodology (UVM)
 - Covers AXI Bridge, BIST, and target MRAM functionality.
 - Source code: [Erbium/Netlist Testbenches](#)

DV Testbenches - EVL

- EVL includes many checking mechanisms:
 - L1 Data Cache scoreboard
 - Neighborhood scoreboard
 - Bus (ET-Link) verification components
 - SystemVerilog Assertions
- Co-simulation:
 - Covers the gap that EVL checkers leave (e.g. memory consistency checking, ISA checking)
- Stimulus:
 - Current number of C and Asm tests: 1529
 - Will require porting shire build tests to the neigh build
 - Re-use ET-SoC-1 coverage to measure how thoroughly the design has been tested against its specifications.

DV Testbenches - ETL2AXI

- Why a new testbench?
 - **New development:** Bridge will be implemented largely from scratch
→ initial verification confidence is low



- **Critical component:** Any malfunction could stall cores, block transactions, corrupt data, and isolate the Minions from the outside the Neigh world
- **Verification approach:** Build a lightweight yet comprehensive testbench to ensure robustness and reliability while enabling faster turnaround
- Checking mechanisms:
 - Bus protocol (ET-Link and AXI4) scoreboards
 - ET-Link to AXI4 translation scoreboards
 - Assertion based protocol checks
- Stimulus:
 - Random bus requests injected directly into the RTL bridge bus interface
 - Error injection and back pressure knobs

DV Testbenches - CocoTB

- Full Chip verification scope:
 - Verify data flow and connectivity between Minions, MRAM, interconnects and peripherals
 - Verify system-level interactions like interrupts and IO devices
 - Test the real bootrom in the RTL
 - Verify top level pad connections
- Co-simulation:
 - Integrate sysemu co-simulation in CocoTB (very nice to have)
- Stimulus:
 - TODO - Tests? [Verification_Plan.xlsx](#)
 - TODO - Coverage?

DV Testbenches - MRAM

- Focused Functional Verification:
 - **Multi-level testbench:** Reduces test time by running different hierarchies of the MRAM with directed testing of different features



- **AXI protocol checking:** checks the AXI2MRAM bridge for adherence to the underlying protocol.
- **Debug interface checking:** ensures that all test registers are accessible and have their desired functionality
- Checking Mechanism:
 - ???
- Stimulus:
 - Directed Testing
 - Directed Random

DV Testbenches - xSPI

- Requirements
 - Synopsys VIP: For functional verification using UVM.
 - Synaptics-Lab Controller: For verification on FPGA.
 - Various Opensource SPI/QSPI/xSPI(?) Controllers: For compatibility testing
 - cocotbext-spi: For SPI Mode verification
 - cocotbext-xspi: Inhouse developed for all mode verification.
- Environments
 - Multiple Testbenches:
 - Cocotb xspi Env: Developed inhouse. for testing all features.
 - UVM Env with Synopsys VIP: For mitigating the risks of same person developing RTL and cocotb VIP.
 - Env with QSPI RTL: For testing compatibility with available controller.
 - FPGA with QSPI controller, for testing compatibility.
- Checking Mechanism:
 - Assertion/Scoreboard.
 - Waveform Analysis (WAL?)
- Stimulus:
 - Directed Testing(TOML file Based)
 - Directed Random(Cross of Rates, Bursts, Transaction types,XIP,Error cases)



- Coverage:
 - 100% line, close to 100% for branch,toggle, fsm.
 - 100% Functional with waivers

DV Testbenches - I2C, QSPI, UART

- Opensource IP's either FPGA or Silicon Proven. with their own testbenches.
- Tasks:
 - Check testbenches for completeness, code +functional coverages.
 - Assume testbench suffers from “same developer error” Reimplement in cocotb and reverify.
- Environments
 - Unit level testbenches for each IP:
 - Cocotb xspi Env: Developed inhouse. for testing all features.
 - UVM Env with Synopsys VIP: For mitigating the risks of same person developing RTL and cocotb VIP.
 - cocotbext-spi,cocotbext-i2c,cocotbext-uart based unit testing.
- Checking Mechanism:
 - Assertion/Scoreboard.
- Stimulus:
 - Directed Testing(TOML file Based)
 - Directed Random(Cross of Rates, Bursts,Transaction types,XIP,Error cases)
- Coverage:
 - 100% line, close to 100% for branch,toggle, fsm.
 - 100% Functional with waivers

DV Testbenches - SOC

- Plan
 - Existing Cocotb based TB with mix of SV/UVM code.
 - Add new tests for QSPI, UART, I2C
 - ★ Use cocotbext-[SPI,UART,I2C] VIP's



- ★ Test QSPI using Synopsys xSPI VIP in SPI and QSPI Mode.
- Modify C code from ARM to RISC-V
- Add New tests for Changes in Registers.
- Add CPU-Subsystem integration tests.
- Goal:
 - Assume all features are unit tested. Focus on integration and functionality tests.
 - Check every unit is able to communicate with every other unit in every mode of operation.
 - Verify Memory Map.
 - Check robustness of communication with outside world.
 - Run at least one realworld usecase and measure performance.
- Coverage methodology.
 - Coverage will not be considered for IP's which are unit tested.
 - ★ Pin toggle covered will be considered for integration verification completeness
 - RAL Tests will not contribute to coverage for any environment (unit, system,soc)

DV Testbenches - Validation

Validation on Alveo U250

DV Co-Simulation

- Use *sysemu* to handle the co-simulation in RTL
 - Source code: [aifoundry-org/et-platform](https://github.com/aifoundry-org/et-platform)
- How co-simulation works?
 - Every time a Minion commits an instruction, its architectural state is captured
 - During test execution, the captured architectural state is continuously compared against the golden model (sysemu)
- Example of co-simulation checks :
 - Ensure that RTL does not violate software program order
 - Check memory consistency order
 - Verify memory data correctness



Constrained Randoms

- Corner-case bugs are often difficult to anticipate and even more challenging to reproduce reliably.
- Some ET-SoC-1 show-stopper bugs that would have been missed by directed tests:
 - Thread 1 starves Thread 0 to complete a Tensor Store
 - DCACHE replay_queue and buffer_array entry counters may not discount allocated entries (credit/counter leaking) when gsc and regular operations overlap
 - TensorReduce stalls both threads 0 and 1 from executing VPU instructions ends up in deadlock
- For ET-SoC-1 we used STING (commercial, Synopsys), but for post-silicon we developed an in-house random test generator named MTG.
- **Proposal:** revive MTG to fill the gap and boost confidence in the design
- MTG is written in Python, does not require a license and is highly configurable
- Source code: [nekkoai/dv](https://github.com/nekkoai/dv)

Verification Flows

RTL Simulation: Validates functionality at a high level with zero-delay models.

Gate-Level Simulation: Runs on the synthesized netlist, supporting either zero-delay (for faster simulation) or real timing delays (closer to silicon behavior)

X-Propagation: Model unknown values (X) realistically, ensuring simulation reflects real silicon behavior

Power-Aware verification (UPF): validates power intent across domains

Emulation (FPGA): Run the design on hardware to accelerate verification and enable early software testing

DV tools

- **Primary toolset:** Synopsys tools (commercial)
 - Verilator is currently not supported but it is something we should explore in the future



- Key functions:
 - Simulation (VCS) for block-level and full-chip tests
 - Coverage analysis and closure tracking
 - Debug environment for waveform and log analysis
- Licenses requirement:
 - For CI (B4C): TBD
 - ★ # of ET Tests: 134
 - ★ Total time assuming one license
 - For Regressions: TBD
 - ★ # of Erbium tests: (?)
 - ★ # of ET tests: 1529
 - ★ Total time assuming one license

Verification Goals

Achieve 100% of planned functional coverage
Reach $\geq 95\%$ code coverage on reachable logic
All directed test passing in regression
All random tests passing and stable in regression
Enable co-simulation with sysemu
Run successfully the bootrom in simulation
Run MRAM wellness algorithm in simulation
...

DV Master Plan

(for ET Erbium)

Architecture Freeze - DV Plan



ErbiumxSPI_MemoryMap address map

- Absolute Address: 0x0
- Base Offset: 0x0
- Size: 0xFE016000

Offset	Identifier	Name
0x00000000	mram	MRAM
0x40000000	system_registers	System Registers
0x40001000	mram_registers	AXI-to-MRAM Bridge Register Map
0x40002000	i2c_registers	—
0x40003000	qspi_registers	QSPI Registers
0x40004000	uart_registers	UART Registers
0x40008000	ROMRAM	—
0x80000000	cpu_registers	ESR Map of Erbium
0xA0000000	plic	—
0xFE000000	nic_config	—

mram memory

- Absolute Address: 0x0
- Base Offset: 0x0
- Size: 0x1000000

No supported members.

system_registers address map

- Absolute Address: 0x40000000
- Base Offset: 0x40000000
- Size: 0xC4

Register Controlling System Behavior

Offset	Identifier	Name
0x00	Version	—
0x08	SystemConfig	—
0x10	watchdog_count	—



Offset	Identifier	Name
0x18	Watchdog	—
0x20	SysInterrupt	—
0x28	SoftReset	—
0x30	ResetCause	—
0x38	PowerDomainReq	—
0x40	PowerDomainAck	—
0x48	PowerGood	—
0x50	PowerStatus	—
0x58	SpinLock	—
0x60	ChipMode	—
0x68	Mailbox0	—
0x70	Mailbox1	—
0x78	GPIO_OE	—
0x80	GPIO_I	—
0x88	GPIO_O	—
0x90	GPIO_Interrupt_Enable	—
0x98	ring_osc	Ring Oscillator Control
0xA0	cpu_divider	—
0xA8	system_divider	—
0xB0	periph_divider	—
0xB8	tsense_ctrl	Temperature Sensor Control
0xC0	tsense_status	Temperature Sensor Status

Version register

- Absolute Address: 0x40000000
- Base Offset: 0x0
- Size: 0x4

Device identifier, used by software to identify the device family (chipid) variant, and bugfix version

Bits	Identifier	Access	Reset	Name
7:0	respin	r	0x0	—
15:8	variation	r	0x0	—
31:16	chipid	r	0xEB68	—



SystemConfig register

- Absolute Address: 0x40000008
- Base Offset: 0x8
- Size: 0x4

System configuration fields. use to enable/disable various features

Bits Identifier		Access Reset Name		
0	sys_interrupt_enable	rw	0x0	—
1	mram_startup_bypass	rw	0x0	—
2	wdog_disable	rw	0x1	—
3	i2c_enable	rw	0x0	—
4	spi_enable	rw	0x1	—
5	qspi_enable	rw	0x0	—
6	uart_enable	rw	0x0	—
7	osc_out_enable	rw	0x0	—

sys_interrupt_enable field reg interrupt: Writing to this bit generates an interrupt.

mram_startup_bypass field connected to mram_startup_bypass of mram_wrapper

wdog_disable field Watchdog Disable

i2c_enable field I2C Enable

spi_enable field SPI Enable

qspi_enable field QSPI Enable

uart_enable field UART Enable

osc_out_enable field OSC_Out Enable

**watchdog_count register**

- Absolute Address: 0x40000010
- Base Offset: 0x10
- Size: 0x4

The watchdog detects ‘hang’ conditions and resets the system. This feature is disabled by default. Clear `cpu_config.wdog_disable` to enable this. Once enabled, S/W should write to `Watchdog.kick` to reset the `WATCHDOG_COUNT.count` field. If the count reaches 0, it triggers a system reset.

Bits Identifier	Access	Reset	Name
31:0	watchdog_count	rw	0xFFFF —

watchdog_count field When the watchdog timer is enabled it counts down from this value

Watchdog register

- Absolute Address: 0x40000018
- Base Offset: 0x18
- Size: 0x4

Bits	Identifier	Access	Reset	Name
7	kick	rw	0x0	—

kick field Resets the watchdog timer, cpu needs to regularly write to this bit to avoid a watchdog timeout based reset of the device

SysInterrupt register

- Absolute Address: 0x40000020
- Base Offset: 0x20
- Size: 0x4



Bits	Identifier	Access	Reset	Name
0	interrupt	rw, rclr	0x0	—
1	gpio_interrupt	r, rclr	0x0	—

interrupt field Write to this bit to generate an interrupt. Clear on read

gpio_interrupt field GPIO In Interrupt. Clear on read

SoftReset register

- Absolute Address: 0x40000028
- Base Offset: 0x28
- Size: 0x4

Bits	Identifier	Access	Reset	Name
0	soft_reset	rw	0x0	—
1	cpu_warm_reset	rw	0x0	—
2	mram_rst_b	rw	0x1	—
3	cpu_soft_reset	rw	0x0	—

soft_reset field System Soft Reset, Active High

cpu_warm_reset field CPU Warm Reset, Active High

mram_rst_b field MRAM Resetn, Active Low

cpu_soft_reset field CPU cold Reset, Self clearing Active High

ResetCause register

- Absolute Address: 0x40000030
- Base Offset: 0x30
- Size: 0x4

This register reports the cause of reset. There are multiple reset sources.

- Power on Reset,



- Brownout Reset, and
 - Various S/W reset requests. These clear on read bits capture the reset cause since the last read Note if the por bit is set ignore the other cause registers, they will have random values until the first read.

Bits Identifier		Access Reset Name		
0	por	r	—	—
1	watchdog_timeout	r	—	—
2	sysreset_req	r	—	—
3	brownout	r	—	—
4	softreset	r	—	—
5	cpu_warm_reset	r	—	—

por field System POR was toggled

watchdog_timeout field Watchdog was enabled and CPU failed to clear the watchdog timer

sysreset_req field NA. CPU detected an architecture level lockup and requested a system reset

brownout field The brownout detector triggered a reset

softreset field The soft reset bit was written to

cpu_warm_reset field NA. The cpu reset bit was written to

PowerDomainReq register

- Absolute Address: 0x40000038
- Base Offset: 0x38
- Size: 0x4

Writing to PD fields initiates the shutdown of the corresponding power domain. A shutdown request is sent to the domain, This domain waits until all outstanding transactions are completed and then generates a corresponding



CONTENTS

ack signal. Once the ack signal is generated the power controller will turn off power to that domain.

Bits Identifier		Access Reset Name		
0	cpu_pd	rw	0x0	—
1	cpu_rom_powerdown	rw	0x0	—
2	cpu_ram_powerdown	rw	0x0	—
3	chiplet_pd	rw	0x0	—
4	mram_pd	rw	0x0	—
5	system_poweroff	rw	0x0	—
6	xspi_pd	rw	0x0	—
15:8	minion_pd	rw	0x0	—
16	mram_dsleep_en	rw	0x0	—
17	cpu_sleep_en	rw	0x0	—

cpu_pd field Poweroff the CPU Domain

cpu_rom_powerdown field Powerdown the boot ROM

cpu_ram_powerdown field Powerdown the CPU ram. this puts the ram in deepsleep mode

chiplet_pd field Not used; chiplet power domain is controlled via the mode bits

mram_pd field Power down for MRAM digital logic and MRAM domain. When set to 1, this powers off the domain; setting to 0 powers it on.

system_poweroff field Power off the chip. Only wakeup logic is powered on

xspi_pd field Not used; hyperbus power domain is controlled via the mode bits.

minion_pd field Minion PowerDown Req



mram_dsleep_en field Connects directly to the MRAM's main power switch, bypassing the digital domain's power switch. When set to 1, the MRAM is in deep sleep; setting to 0 powers the MRAM on.

cpu_sleep_en field currently not used. In future this will be used for CPU sleep management.

PowerDomainAck register

- Absolute Address: 0x40000040
- Base Offset: 0x40
- Size: 0x4

This mirrors the Ack signal generated in response to power down request.

Bits Identifier		Access Reset Name		
0	cpu_pd_ack	r	0x0	—
1	sram_pd_ack	r	0x0	—
3	chiplet_pd_ack	r	0x0	—
4	mram_pd_ack	r	0x0	—
5	system_pd_ack	r	0x0	—
6	hyperbus_pd_ack	r	0x0	—
15:8	minion_pd_ack	r	0x0	—

cpu_pd_ack field pd ack for cpu

sram_pd_ack field Not used: pd ack for TCM

chiplet_pd_ack field Not used: pd ack for chiplet

mram_pd_ack field pd ack for mram

system_pd_ack field pd ack for system

hyperbus_pd_ack field Not used: pd ack for hyperbus

minion_pd_ack field Minion PowerDown Req



PowerGood register

- Absolute Address: 0x40000048
- Base Offset: 0x48
- Size: 0x4

When a power domain is switched on, it takes time for the voltage to stabilize, this time is process dependent. The default value is sufficiently large to account for all variations.

Bits	Identifier	Access	Reset	Name
20:0	counter	rw	0xFFFFF	—

counter field Counter for powerGood

PowerStatus register

- Absolute Address: 0x40000050
- Base Offset: 0x50
- Size: 0x4

Status

Bits	Identifier	Access	Reset	Name
3:0	reserved	r	0x0	—

reserved field Asserted when MRAM is powered on/reset until internal setup is done.

SpinLock register

- Absolute Address: 0x40000058
- Base Offset: 0x58
- Size: 0x4

initially locked=0, A read on this register will set the lock bit. Usage:

1. Read this field. If you get a value of zero you got the lock. Else a different processing element acquired the lock. Poll at fixed intervals(dont spam) until you get the lock.



2. If you acquired the lock, Once you have finished interacting with the locked resource write 0 to this register to release the lock.

Bits	Identifier	Access	Reset	Name
0	lock	rw, rset	0x0	—

ChipMode register

- Absolute Address: 0x40000060
- Base Offset: 0x60
- Size: 0x4

Bits	Identifier	Access	Reset	Name
1:0	chip_mode	r	—	—
2	ifc_width	r	—	—
4:3	bootload	rw	0x0	—
6:5	load_external	rw	0x0	—

chip_mode field The mode in which chip is working, hyperbus, axi,ahb,gc

ifc_width field If chip is axi/ahb mode datawidth

bootload field Jump to 00:no Jump, 01:TCM,10:MRAM

load_external field Jump to 00:no load, 01:Load TCM via Chiplet,10:Load MRAM via Chiplet

Mailbox0 register

- Absolute Address: 0x40000068
- Base Offset: 0x68
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	mbox0	rw	0x0	—



mbox0 field Mailbox0

Mailbox1 register

- Absolute Address: 0x40000070
- Base Offset: 0x70
- Size: 0x4

Bits Identifier Access Reset Name			
31:0	mbox1	rw	0x0 —

mbox1 field Mailbox1

GPIO_OE register

- Absolute Address: 0x40000078
- Base Offset: 0x78
- Size: 0x4

Gpio Output enable. Write 1 to this bit to set the GPIO register in output mode.

Bits Identifier Access Reset Name			
10:0	gpio_oe	rw	0x0 —

gpio_oe field Gpio output enable

GPIO_I register

- Absolute Address: 0x40000080
- Base Offset: 0x80
- Size: 0x4

Gpio Input For each bit in input mode this register captures the input value.



Bits	Identifier	Access	Reset	Name
10:0	gpio_i	r	0x0	—

gpio_i field Gpio input

GPIO_O register

- Absolute Address: 0x40000088
- Base Offset: 0x88
- Size: 0x4

Gpio Output For each bit in output mode the content of the corresponding bit in this register is reflected on the GPIO Pin

Bits	Identifier	Access	Reset	Name
10:0	gpio_o	rw	0x0	—

gpio_o field Gpio input

GPIO_Interrupt_Enable register

- Absolute Address: 0x40000090
- Base Offset: 0x90
- Size: 0x4

Gpio Interrupt Enable For each bit in input mode if the input signal toggles and the content of the corresponding bit in this register is 1, then a GPIO interrupt is raised.

Bits	Identifier	Access	Reset	Name
10:0	gpio_interrupt_en	rw	0x0	—

gpio_interrupt_en field Gpio input

**ring_osc register**

- Absolute Address: 0x40000098
- Base Offset: 0x98
- Size: 0x4

Controls the ring oscillator (ring_osc) input pins. In normal mode (dbg_en=0), only en, divby2_sel, and trm are active. In debug mode (dbg_en=1), the dbg_* fields independently gate the analog bias block (anachip) and the oscillator core (rochip).

Bits Identifier		Access Reset Name	
0	en	rw	0x1 —
1	divby2_sel	rw	0x1 —
6:2	trm	rw	0x1F —
7	dbg_en	rw	0x0 —
8	dbg_anachip_en	rw	0x0 —
9	dbg_rohchip_en	rw	0x0 —
10	dbg_sah_en_b	rw	0x0 —

en field Master oscillator enable. Drive high to power up both the analog bias block (anachip) and the ring oscillator core (rochip). Has no effect when dbg_en is asserted.

divby2_sel field Divide-by-2 frequency select. 0: Full-speed range (899 MHz to 1.608 GHz, set by trm). 1: Half-speed range (449 MHz to 794 MHz, set by trm).

trm field Frequency trim code. Selects the oscillator output frequency within the range determined by divby2_sel. 0x00 = fastest (1.608 GHz or 794 MHz). 0x1F = slowest (899 MHz or 449 MHz). Frequency decreases linearly with increasing trim value.

dbg_en field Debug override enable. When asserted, the en input is ignored and dbg_anachip_en and dbg_rohchip_en independently control the analog bias block and oscillator core respectively. dbg_sah_en_b also becomes active. 0: Normal mode – en controls both blocks. 1: Debug mode – dbg_* fields control each block separately.



dbg_anachip_en field Debug enable for the analog bias block (anachip). Only active when dbg_en=1. When asserted with dbg_en=1, powers up the anachip and drives h1_mnvdd09_g high, which is required for the oscillator core to run. When deasserted, h1_mnvdd09_g is held low and the oscillator core cannot oscillate regardless of dbg_rohcip_en.

dbg_rohcip_en field Debug enable for the ring oscillator core (rochip). Only active when dbg_en=1. When asserted with dbg_en=1 and dbg_anachip_en=1, the oscillator core is enabled and clk will toggle. When deasserted, the oscillator core is disabled and clk is held low regardless of dbg_anachip_en.

dbg_sah_en_b field Debug SAH enable (active low). Only active when dbg_en=1. Selects the anachip operating mode: 0: Normal SAH operation (anachip mode 3'b100, h1=1). 1: Test mode, high power (anachip mode 3'b101, h1=1). In both cases h1_mnvdd09_g remains high when dbg_anachip_en=1; this bit selects the internal anachip bias configuration only.

cpu_divider register

- Absolute Address: 0x400000A0
- Base Offset: 0xA0
- Size: 0x4

Bits Identifier		Access	Reset	Name
3:0	count	rw	0xF	—
4	div_enable	rw	0x1	—

count field clock divider. output clk is 2 x count

div_enable field Enable clk divider

system_divider register

- Absolute Address: 0x400000A8
- Base Offset: 0xA8
- Size: 0x4



Bits Identifier		Access	Reset	Name
3:0	count	rw	0xF	—
4	div_enable	rw	0x1	—

count field clock divider. output clk is 2 x count

div_enable field Enable clk divider

periph_divider register

- Absolute Address: 0x400000B0
- Base Offset: 0xB0
- Size: 0x4

Bits Identifier		Access	Reset	Name
3:0	count	rw	0xF	—
4	div_enable	rw	0x1	—

count field clock divider. output clk is 2 x count

div_enable field Enable clk divider

tsense_ctrl register

- Absolute Address: 0x400000B8
- Base Offset: 0xB8
- Size: 0x4

Controls the temperature sensor and ADC conversion. Write 1 to conv to trigger a single conversion. The bit auto-clears after one cycle. Select which sensor to sample with sen_sel, and set the ADC clock divider with clk_div.

Bits Identifier		Access	Reset	Name
0	conv	rw	0x0	—
1	sen_en	rw	0x0	—
3:2	sen_sel	rw	0x0	—
11:4	clk_div	rw	0x1	—



Bits Identifier Access Reset Name

conv field Start conversion. Writing 1 triggers a single ADC sample. Auto-clears to 0 after one cycle. Clears the valid bit in `tsense_status` when written.

sen_en field Sensor enable. When set, the sensor selected by `sel` is powered on. When clear, all sensors are off.

sen_sel field Sensor select. Selects which of the 4 temperature sensors (0-3) is connected to the ADC.

clk_div field ADC clock divider. The ADC clock frequency is $\text{sys_clk} / (2 * (\text{clk_div} + 1))$. 0x00 = $\text{sys_clk}/2$ (fastest). 0xFF = $\text{sys_clk}/512$ (slowest).

tsense_status register

- Absolute Address: 0x400000C0
- Base Offset: 0xC0
- Size: 0x4

Read-only status from the most recent ADC conversion.

Bits Identifier Access Reset Name

3:0	data	r	0x0	—
4	valid	r	0x0	—
5	conv_b	r	0x1	—

data field ADC output data from the last completed conversion.

valid field Data valid. Set by hardware when a conversion completes. Cleared when a new conversion is started (`conv` written).

conv_b field ADC conversion status. Active-low while the ADC conversion is in progress and high when idle / conversion complete.



mram__registers address map

- Absolute Address: 0x40001000
- Base Offset: 0x40001000
- Size: 0x4A0

Unified register map for the AXI-to-MRAM bridge. Contains bridge configuration registers and per-bank MRAM controller test registers.

Offset Identifier	Name
0x000	bridge__regs AXI-to-MRAM Bridge Control and Status Registers
0x100	bank0__tregs MRAM Control and Status Test Registers
0x200	bank1__tregs MRAM Control and Status Test Registers
0x300	bank2__tregs MRAM Control and Status Test Registers
0x400	bank3__tregs MRAM Control and Status Test Registers

bridge__regs register file

- Absolute Address: 0x40001000
- Base Offset: 0x0
- Size: 0x38

Register space for configuration and status of the AXI-to-MRAM bridge.

Offset Identifier	Name
0x00	arbiter__mode__reg Arbiter Mode
0x08	bridge__status__reg Bridge Status
0x10	slverr__status__reg AXI Slave Error Status
0x18	control__reg AXI2MRAM Control
0x20	ecc_1bit_error_count__reg ECC 1-bit Error Counter
0x28	ecc_2bit_error_count__reg ECC 2-bit Error Counter
0x30	ecc_3bit_error_count__reg ECC 3-bit Error Counter

arbiter__mode__reg register

- Absolute Address: 0x40001000
- Base Offset: 0x0
- Size: 0x8



Controls the read/write arbitration policy when both channels have simultaneous pending requests.

Bits Identifier		Access Reset Name	
1:0	arbiter_mode	rw	0x2 arbiter_mode

arbiter_mode field Arbitration mode selection. 0 = Write Priority (writes always win) 1 = Read Priority (reads always win) 2 = Round Robin (alternate after each conflict) 3 = Oldest First (whichever request arrived first wins)

bridge_status_reg register

- Absolute Address: 0x40001008
- Base Offset: 0x8
- Size: 0x8

Read-only status indicators for the AXI-to-MRAM bridge.

Bits Identifier		Access Reset Name	
0	axi_busy	r	0x0 axi_busy
7:4	cmd_queue_active	r	0x0 cmd_queue_active
11:8	mram_ready	r	0x0 mram_ready

axi_busy field High when the bridge has outstanding AXI transactions.

cmd_queue_active field Per-bank command queue active flags. Bit N = bank N has pending write/RMW commands.

mram_ready field Per-bank ready signals for bringing MRAM out of deep sleep. When the bit is 1, that bank is ready for operations.

slverr_status_reg register

- Absolute Address: 0x40001010
- Base Offset: 0x10
- Size: 0x8



Sticky status bits capturing the reason(s) why the bridge returned SLVERR on the AXI bus. Bits are set by hardware and cleared automatically when software reads this register (clear-on-read). Multiple causes may be set simultaneously.

Bits Identifier		Access Reset Name	
0	oor_read	r, rclr 0x0	oor_read
1	oor_write	r, rclr 0x0	oor_write
2	mram_not_ready	r, rclr 0x0	mram_not_ready
3	mram_unpowered	r, rclr 0x0	mram_unpowered
4	maintenance	r, rclr 0x0	maintenance
5	unrecoverable_error	r, rclr 0x0	unrecoverable_error

oor_read field Set when a read transaction was rejected with SLVERR because ARADDR falls outside the valid MRAM window (ARADDR[31:24] != 0, i.e. ARADDR >= 0x100_0000).

oor_write field Set when a write transaction was rejected with SLVERR because AWADDR falls outside the valid MRAM window (AWADDR[31:24] != 0, i.e. AWADDR >= 0x100_0000).

mram_not_ready field Set when a transaction was rejected with SLVERR because the targeted MRAM bank reports not-ready.

mram_unpowered field Set when a transaction was rejected with SLVERR because the targeted MRAM bank reports power-not-ok.

maintenance field Set when a transaction was rejected with SLVERR because the targeted MRAM bank is in maintenance mode.

unrecoverable_error field Set when read data was returned with an unrecoverable ECC condition (triple-bit error) and the bridge responded with SLVERR.

control_reg register

- Absolute Address: 0x40001018
- Base Offset: 0x18



- Size: 0x8

Software control bits for the AXI-to-MRAM bridge.

Bits Identifier		Access	Reset	Name
3:0	disable_clock_gate	rw	0x0	disable_clock_gate
4	ecc_1bit_intr_mask	rw	0x1	ecc_1bit_intr_mask
5	ecc_2bit_intr_mask	rw	0x0	ecc_2bit_intr_mask
6	ecc_3bit_intr_mask	rw	0x0	ecc_3bit_intr_mask

disable_clock_gate field Per-bank clock-gate disable control. Bit N disables the clock gating structure driving MRAM bank N so that a continuous clock is driven into the bank interface.

ecc_1bit_intr_mask field Masks CPU interrupt generation for 1-bit ECC events in the ET interrupt logic. 1 = masked/suppressed, 0 = unmasked/enabled. Reset default is 1 (masked).

ecc_2bit_intr_mask field Masks CPU interrupt generation for 2-bit ECC events in the ET interrupt logic. 1 = masked/suppressed, 0 = unmasked/enabled. Reset default is 0 (enabled).

ecc_3bit_intr_mask field Masks CPU interrupt generation for 3-bit ECC events in the ET interrupt logic. 1 = masked/suppressed, 0 = unmasked/enabled. Reset default is 0 (enabled).

ecc_1bit_error_count_reg register

- Absolute Address: 0x40001020
- Base Offset: 0x20
- Size: 0x8

Counts observed 1-bit ECC events across all banks/lanes. The counter increments by the number of asserted lane bits per cycle (0..8).

Bits Identifier		Access	Reset	Name
31:0	count	r	0x0	count



count field Accumulated 1-bit ECC event count.

ecc_2bit_error_count_reg register

- Absolute Address: 0x40001028
- Base Offset: 0x28
- Size: 0x8

Counts observed 2-bit ECC events across all banks/lanes. The counter increments by the number of asserted lane bits per cycle (0..8).

Bits Identifier Access Reset Name			
31:0 count	r	0x0	count

count field Accumulated 2-bit ECC event count.

ecc_3bit_error_count_reg register

- Absolute Address: 0x40001030
- Base Offset: 0x30
- Size: 0x8

Counts observed 3-bit ECC events across all banks/lanes. The counter increments by the number of asserted lane bits per cycle (0..8).

Bits Identifier Access Reset Name			
31:0 count	r	0x0	count

count field Accumulated 3-bit ECC event count.

bank0_tregs register file

- Absolute Address: 0x40001100
- Base Offset: 0x100
- Size: 0xA0

Register space that contains the test registers for test functionality and debug of the MRAM.



Offset	Identifier	Name
0x00	mram_control	—
0x20	mram_control_pulse	—
0x28	gbl_cfg_ovr_0	—
0x30	gbl_cfg_0	—
0x38	man_control_0	—
0x40	man_control_1	—
0x48	mram_status_0	—
0x50	mram_status_1	—
0x58	mram_dout_even_lower	—
0x60	mram_dout_odd_lower	—
0x68	mram_dout_uppers	—
0x70	bist_status_0	—
0x78	bist_control	—
0x88	ecc_correction	—
0x98	bist_status_1	—

mram_control register

- Absolute Address: 0x40001100
- Base Offset: 0x0
- Size: 0x20

Bits	Identifier	Access	Reset	Name
16:0	addr_in	rw	0x0	addr_in
24:17	ce	rw	0x0	ce
25	we	rw	0x0	we
26	rd_pulse_meas_en	rw	0x0	rd_pulse_meas_en
33:27	rca_ovr	rw	0x0	rca_ovr
34	rca_ovr_en	rw	0x0	rca_ovr_en
35	gbl_cfg_ovr_en	rw	0x0	gbl_cfg_ovr_en
36	rd_en_ovr	rw	0x0	rd_en_ovr
37	ref_prg_en	rw	0x0	ref_prg_en
38	dsleep_mram_en	rw	0x0	dsleep_mram_en
39	reg_logic_sup_sleep_ovr	rw	0x1	reg_logic_sup_sleep_ovr
40	prg_rd1_byp	rw	0x0	prg_rd1_byp
41	wr_en_ovr	rw	0x0	wr_en_ovr
42	dma_en	rw	0x0	dma_en



CONTENTS

Bits	Identifier	Access	Reset	Name
43	vbbslx_gain_mode_ovr	rw	0x0	vbbslx_gain_mode_ovr
47:44	powerup_trim_load_ovr	rw	0x0	powerup_trim_load_ovr
52	test_cal_en	rw	0x0	test_cal_en
55:53	anatest0_sel	rw	0x0	anatest0_sel
58:56	anatest1_sel	rw	0x0	anatest1_sel
59	eccrom_deep_sleep	rw	0x1	eccrom_deep_sleep
60	ref_ecc_sel	rw	0x0	ref_ecc_sel
61	disable_cpu_intr	rw	0x0	disable_cpu_intr
62	disable_ted	rw	0x0	disable_ted
63	ecc_bypass_en	rw	0x0	ecc_bypass_en
142:64	bwe	rw	0x0	bwe
143	mram_clk_en	rw	0x1	mram_clk_en
222:144	din	rw	0x0	din
223	test_reg_ovr_en	rw	0x0	test_reg_ovr_en
224	otp_wr_en	rw	0x0	otp_wr_en
225	rst_cpu_intr	rw	0x0	rst_cpu_intr
233:226	dout_en	rw	0x0	dout_en
236:234	ecc_en	rw	0x0	ecc_en
240:237	even_man_stripe_sel	rw	0x0	even_man_stripe_sel
241	even_man_wr_0	rw	0x0	even_man_wr<0>
242	even_man_wr_1	rw	0x0	even_man_wr<1>
243	even_man_wr_2	rw	0x0	even_man_wr<2>
244	even_man_wr_3	rw	0x0	even_man_wr<3>
248:245	odd_man_stripe_sel	rw	0x0	odd_man_stripe_sel
249	odd_man_wr_0	rw	0x0	odd_man_wr<0>
250	odd_man_wr_1	rw	0x0	odd_man_wr<1>
251	odd_man_wr_2	rw	0x0	odd_man_wr<2>
252	odd_man_wr_3	rw	0x0	odd_man_wr<3>
253	sah_en	rw	0x1	sah_en
254	scc_otp_en	rw	0x0	scc_otp_en
255	maintenance_mode	rw	0x0	maintenance_mode

mram_control_pulse register

- Absolute Address: 0x40001120
- Base Offset: 0x20
- Size: 0x8



Bits	Identifier	Access	Reset	Name
0	powerup_trim_load_ovr_single_pulse<0>	rw	0x0	powerup_trim_load_ovr_single_pulse<0>
1	powerup_trim_load_ovr_single_pulse<1>	rw	0x0	powerup_trim_load_ovr_single_pulse<1>
2	powerup_trim_load_ovr_single_pulse<2>	rw	0x0	powerup_trim_load_ovr_single_pulse<2>
3	powerup_trim_load_ovr_single_pulse<3>	rw	0x0	powerup_trim_load_ovr_single_pulse<3>
4	mram_clk_single_pulse	rw	0x0	mram_clk_single_pulse

gbl_cfg_ovr_0 register

- Absolute Address: 0x40001128
- Base Offset: 0x28
- Size: 0x8

Bits	Identifier	Access	Reset	Name
1:0	sa_equal_trim	rw	0x1	sa_equal_trim<1:0>
4:2	vblslx_boost_trim	rw	0x4	vblslx_boost_trim<2:0>
8:5	wr_en_msb_trim	rw	0x9	wr_en_msb_trim<3:0>
11:9	wr_en_lsb_trim	rw	0x6	wr_en_lsb_trim<2:0>
12	vblslx_gain_mode	rw	0x0	vblslx_gain_mode
16:13	repulse_trim	rw	0x6	repulse_trim<3:0>
17	repulse_en	rw	0x1	repulse_en
20:18	rd_en_trim	rw	0x5	rd_en_trim<2:0>
25:22	osc_wr_div_trim	rw	0x3	osc_wr_div_trim<3:0>
29:26	vblsl_trim	rw	0x4	vblsl_trim<3:0>
33:30	tcsel_trim	rw	0x3	tcsel_trim<3:0>
37:34	vwlrwr_trim	rw	0x7	vwlrwr_trim<3:0>
41:38	vcr_gate_trim	rw	0xE	vcr_gate_trim<3:0>

gbl_cfg_0 register

- Absolute Address: 0x40001130
- Base Offset: 0x30
- Size: 0x8

Bits	Identifier	Access	Reset	Name
1:0	sa_equal_trim	r	—	sa_equal_trim<1:0>



Bits	Identifier	Access	Reset	Name
4:2	vbbslx_boost_trim	r	—	vbbslx_boost_trim<2:0>
8:5	wr_en_msb_trim	r	—	wr_en_msb_trim<3:0>
11:9	wr_en_lsb_trim	r	—	wr_en_lsb_trim<2:0>
12	vbbslx_gain_mode	r	—	vbbslx_gain_mode
16:13	repulse_trim	r	—	repulse_trim<3:0>
17	repulse_en	r	—	repulse_en
20:18	rd_en_trim	r	—	rd_en_trim<2:0>
25:22	osc_wr_div_trim	r	—	osc_wr_div_trim<3:0>
29:26	vbbsl_trim	r	—	vbbsl_trim<3:0>
33:30	tcsel_trim	r	—	tcsel_trim<3:0>
37:34	vwlrwr_trim	r	—	vwlrwr_trim<3:0>
41:38	vcr_gate_trim	r	—	vcr_gate_trim<3:0>

man_control_0 register

- Absolute Address: 0x40001138
- Base Offset: 0x38
- Size: 0x8

Bits	Identifier	Access	Reset	Name
3:0	blk0_man_ccnt	r	—	blk0_man_ccnt<3:0>
7:4	blk0_man_cnfg	rw	0x0	blk0_man_cnfg<3:0>
9:8	blk0_man_fcnt	r	—	blk0_man_fcnt<1:0>
19:16	blk1_man_ccnt	r	—	blk1_man_ccnt<3:0>
23:20	blk1_man_cnfg	rw	0x0	blk1_man_cnfg<3:0>
25:24	blk1_man_fcnt	r	—	blk1_man_fcnt<1:0>
35:32	blk2_man_ccnt	r	—	blk2_man_ccnt<3:0>
39:36	blk2_man_cnfg	rw	0x0	blk2_man_cnfg<3:0>
41:40	blk2_man_fcnt	r	—	blk2_man_fcnt<1:0>
51:48	blk3_man_ccnt	r	—	blk3_man_ccnt<3:0>
55:52	blk3_man_cnfg	rw	0x0	blk3_man_cnfg<3:0>
57:56	blk3_man_fcnt	r	—	blk3_man_fcnt<1:0>

man_control_1 register

- Absolute Address: 0x40001140
- Base Offset: 0x40



- Size: 0x8

Bits	Identifier	Access	Reset	Name
3:0	blk4_man_ccnt	r	—	blk4_man_ccnt<3:0>
7:4	blk4_man_cnfg	rw	0x0	blk4_man_cnfg<3:0>
9:8	blk4_man_fcnt	r	—	blk4_man_fcnt<1:0>
19:16	blk5_man_ccnt	r	—	blk5_man_ccnt<3:0>
23:20	blk5_man_cnfg	rw	0x0	blk5_man_cnfg<3:0>
25:24	blk5_man_fcnt	r	—	blk5_man_fcnt<1:0>
35:32	blk6_man_ccnt	r	—	blk6_man_ccnt<3:0>
39:36	blk6_man_cnfg	rw	0x0	blk6_man_cnfg<3:0>
41:40	blk6_man_fcnt	r	—	blk6_man_fcnt<1:0>
51:48	blk7_man_ccnt	r	—	blk7_man_ccnt<3:0>
55:52	blk7_man_cnfg	rw	0x0	blk7_man_cnfg<3:0>
57:56	blk7_man_fcnt	r	—	blk7_man_fcnt<1:0>

mram_status_0 register

- Absolute Address: 0x40001148
- Base Offset: 0x48
- Size: 0x8

Bits	Identifier	Access	Reset	Name
15:0	bist_error_loop	r	—	bist_error_loop<15:0>
33:17	bist_error_count	r	—	bist_error_count<16:0>
40:34	bist_rh0	r	—	bist_rh0<6:0>
47:41	bist_rh1	r	—	bist_rh1<6:0>
54:48	bist_rh2	r	—	bist_rh2<6:0>
56:55	cpu_intr_flag	r	—	cpu_intr_flag
58:57	ecc_1bit_flag	r	—	ecc_1bit_flag<1:0>
60:59	ecc_2bit_flag	r	—	ecc_2bit_flag<1:0>
62:61	ecc_3bit_flag	r	—	ecc_3bit_flag<1:0>

mram_status_1 register

- Absolute Address: 0x40001150
- Base Offset: 0x50



CONTENTS

- Size: 0x8

Bits	Identifier	Access	Reset	Name
1:0	temp	r	—	temp<1:0>
4:3	ecc_1bit	r	—	ecc_1bit<1:0>
6:5	ecc_2bit	r	—	ecc_2bit<1:0>
8:7	ecc_3bit	r	—	ecc_3bit<1:0>
9	pwr_ok	r	—	pwr_ok
10	eccrom_pwr_ok	r	—	eccrom_pwr_ok
29:11	intr_error_lane0_addr	r	—	intr_error_lane0_addr<18:0>
48:30	intr_error_lane1_addr	r	—	intr_error_lane1_addr<18:0>
57:50	busy	r	—	busy<7:0>

mram_dout_even_lower register

- Absolute Address: 0x40001158
- Base Offset: 0x58
- Size: 0x8

Bits	Identifier	Access	Reset	Name
63:0	dout	r	—	dout<63:0>

mram_dout_odd_lower register

- Absolute Address: 0x40001160
- Base Offset: 0x60
- Size: 0x8

Bits	Identifier	Access	Reset	Name
63:0	dout	r	—	dout<142:79>

mram_dout_uppers register

- Absolute Address: 0x40001168
- Base Offset: 0x68
- Size: 0x8



Bits	Identifier	Access	Reset	Name
14:0	dout_even_msb r	—		dout<78:64>
46:32	dout_odd_msb r	—		dout<157:143>

bist_status_0 register

- Absolute Address: 0x40001170
- Base Offset: 0x70
- Size: 0x8

Bits	Identifier	Access	Reset	Name
63:0	bist_error_value r	—		bist_error_value<63:0>

bist_control register

- Absolute Address: 0x40001178
- Base Offset: 0x78
- Size: 0x10

Bits	Identifier	Access	Reset	Name
0	bist_rte_en	rw	0x0	bist_rte_en
1	bist_data_inv	rw	0x0	bist_data_inv
4:2	bist_add_inc	rw	0x0	bist_add_inc<2:0>
5	bist_stop_on_error	rw	0x0	bist_stop_on_error
25:6	bist_start_add	rw	0x0	bist_start_add<19:0>
45:26	bist_stop_add	rw	0x0	bist_stop_add<19:0>
50:46	RH4margin	rw	0xA	RH4margin<4:0>
55:51	rh2_offset	rw	0x0	rh2_offset<4:0>
78:64	bist_error_value	r	—	bist_error_value<78:64>
94:79	bist_loop_count	rw	0x0	bist_loop_count<15:0>
95	bist_start	rw	0x0	bist_start
96	bist_trim_mode	rw	0x0	bist_trim_mode
97	bist_stop_on_repl_of	rw	0x0	bist_stop_on_repl_of
98	bist_rst_b	rw	0x1	bist_rst_b
99	bist_reset	rw	0x0	bist_reset
100	bist_rd_en	rw	0x0	bist_rd_en



Bits	Identifier	Access	Reset	Name
101	bist_wr_en	rw	0x0	bist_wr_en

ecc_correction register

- Absolute Address: 0x40001188
- Base Offset: 0x88
- Size: 0x10

Bits	Identifier	Access	Reset	Name
127:0	dout	r	—	dout<127:0>

bist_status_1 register

- Absolute Address: 0x40001198
- Base Offset: 0x98
- Size: 0x8

Bits	Identifier	Access	Reset	Name
19:0	bist_err_add r	—		bist_err_add<19:0>
20	bist_error	r	—	bist_error
21	bist_busy	r	—	bist_busy

bank1_tregs register file

- Absolute Address: 0x40001200
- Base Offset: 0x200
- Size: 0xA0

Register space that contains the test registers for test functionality and debug of the MRAM.

Offset	Identifier	Name
0x00	mram_control	—
0x20	mram_control_pulse	—
0x28	gbl_cfg_ovr_0	—



Offset	Identifier	Name
0x30	gbl_cfg_0	—
0x38	man_control_0	—
0x40	man_control_1	—
0x48	mram_status_0	—
0x50	mram_status_1	—
0x58	mram_dout_even_lower	—
0x60	mram_dout_odd_lower	—
0x68	mram_dout_uppers	—
0x70	bist_status_0	—
0x78	bist_control	—
0x88	ecc_correction	—
0x98	bist_status_1	—

mram_control register

- Absolute Address: 0x40001200
- Base Offset: 0x0
- Size: 0x20

Bits	Identifier	Access	Reset	Name
16:0	addr_in	rw	0x0	addr_in
24:17	ce	rw	0x0	ce
25	we	rw	0x0	we
26	rd_pulse_meas_en	rw	0x0	rd_pulse_meas_en
33:27	rca_ovr	rw	0x0	rca_ovr
34	rca_ovr_en	rw	0x0	rca_ovr_en
35	gbl_cfg_ovr_en	rw	0x0	gbl_cfg_ovr_en
36	rd_en_ovr	rw	0x0	rd_en_ovr
37	ref_prg_en	rw	0x0	ref_prg_en
38	dsleep_mram_en	rw	0x0	dsleep_mram_en
39	reg_logic_sup_sleep_ovr	rw	0x1	reg_logic_sup_sleep_ovr
40	prg_rd1_byp	rw	0x0	prg_rd1_byp
41	wr_en_ovr	rw	0x0	wr_en_ovr
42	dma_en	rw	0x0	dma_en
43	vbslx_gain_mode_ovr	rw	0x0	vbslx_gain_mode_ovr
47:44	powerup_trim_load_ovr	rw	0x0	powerup_trim_load_ovr
52	test_cal_en	rw	0x0	test_cal_en



CONTENTS

Bits	Identifier	Access	Reset	Name
55:53	anatest0_sel	rw	0x0	anatest0_sel
58:56	anatest1_sel	rw	0x0	anatest1_sel
59	eccrom_deep_sleep	rw	0x1	eccrom_deep_sleep
60	ref_ecc_sel	rw	0x0	ref_ecc_sel
61	disable_cpu_intr	rw	0x0	disable_cpu_intr
62	disable_ted	rw	0x0	disable_ted
63	ecc_bypass_en	rw	0x0	ecc_bypass_en
142:64	bwe	rw	0x0	bwe
143	mram_clk_en	rw	0x1	mram_clk_en
222:144	din	rw	0x0	din
223	test_reg_ovr_en	rw	0x0	test_reg_ovr_en
224	otp_wr_en	rw	0x0	otp_wr_en
225	rst_cpu_intr	rw	0x0	rst_cpu_intr
233:226	dout_en	rw	0x0	dout_en
236:234	ecc_en	rw	0x0	ecc_en
240:237	even_man_stripe_sel	rw	0x0	even_man_stripe_sel
241	even_man_wr_0	rw	0x0	even_man_wr<0>
242	even_man_wr_1	rw	0x0	even_man_wr<1>
243	even_man_wr_2	rw	0x0	even_man_wr<2>
244	even_man_wr_3	rw	0x0	even_man_wr<3>
248:245	odd_man_stripe_sel	rw	0x0	odd_man_stripe_sel
249	odd_man_wr_0	rw	0x0	odd_man_wr<0>
250	odd_man_wr_1	rw	0x0	odd_man_wr<1>
251	odd_man_wr_2	rw	0x0	odd_man_wr<2>
252	odd_man_wr_3	rw	0x0	odd_man_wr<3>
253	sah_en	rw	0x1	sah_en
254	scc_otp_en	rw	0x0	scc_otp_en
255	maintenance_mode	rw	0x0	maintenance_mode

mram_control_pulse register

- Absolute Address: 0x40001220
- Base Offset: 0x20
- Size: 0x8



Bits	Identifier	Access	Reset	Name
0	powerup_trim_load_ovr_single_pulse<0>	rw	0x0	powerup_trim_load_ovr_single_pulse<0>
1	powerup_trim_load_ovr_single_pulse<1>	rw	0x0	powerup_trim_load_ovr_single_pulse<1>
2	powerup_trim_load_ovr_single_pulse<2>	rw	0x0	powerup_trim_load_ovr_single_pulse<2>
3	powerup_trim_load_ovr_single_pulse<3>	rw	0x0	powerup_trim_load_ovr_single_pulse<3>
4	mram_clk_single_pulse	rw	0x0	mram_clk_single_pulse

gbl_cfg_ovr_0 register

- Absolute Address: 0x40001228
- Base Offset: 0x28
- Size: 0x8

Bits	Identifier	Access	Reset	Name
1:0	sa_equal_trim	rw	0x1	sa_equal_trim<1:0>
4:2	vblslx_boost_trim	rw	0x4	vblslx_boost_trim<2:0>
8:5	wr_en_msb_trim	rw	0x9	wr_en_msb_trim<3:0>
11:9	wr_en_lsb_trim	rw	0x6	wr_en_lsb_trim<2:0>
12	vblslx_gain_mode	rw	0x0	vblslx_gain_mode
16:13	repulse_trim	rw	0x6	repulse_trim<3:0>
17	repulse_en	rw	0x1	repulse_en
20:18	rd_en_trim	rw	0x5	rd_en_trim<2:0>
25:22	osc_wr_div_trim	rw	0x3	osc_wr_div_trim<3:0>
29:26	vblsl_trim	rw	0x4	vblsl_trim<3:0>
33:30	tcsel_trim	rw	0x3	tcsel_trim<3:0>
37:34	vwlrwr_trim	rw	0x7	vwlrwr_trim<3:0>
41:38	vcr_gate_trim	rw	0xE	vcr_gate_trim<3:0>

gbl_cfg_0 register

- Absolute Address: 0x40001230
- Base Offset: 0x30
- Size: 0x8

Bits	Identifier	Access	Reset	Name
1:0	sa_equal_trim	r	—	sa_equal_trim<1:0>



CONTENTS

Bits	Identifier	Access	Reset	Name
4:2	vbbslx_boost_trim	r	—	vbbslx_boost_trim<2:0>
8:5	wr_en_msb_trim	r	—	wr_en_msb_trim<3:0>
11:9	wr_en_lsb_trim	r	—	wr_en_lsb_trim<2:0>
12	vbbslx_gain_mode	r	—	vbbslx_gain_mode
16:13	repulse_trim	r	—	repulse_trim<3:0>
17	repulse_en	r	—	repulse_en
20:18	rd_en_trim	r	—	rd_en_trim<2:0>
25:22	osc_wr_div_trim	r	—	osc_wr_div_trim<3:0>
29:26	vbbsl_trim	r	—	vbbsl_trim<3:0>
33:30	tcsel_trim	r	—	tcsel_trim<3:0>
37:34	vwlrwr_trim	r	—	vwlrwr_trim<3:0>
41:38	vcr_gate_trim	r	—	vcr_gate_trim<3:0>

man_control_0 register

- Absolute Address: 0x40001238
- Base Offset: 0x38
- Size: 0x8

Bits	Identifier	Access	Reset	Name
3:0	blk0_man_ccnt	r	—	blk0_man_ccnt<3:0>
7:4	blk0_man_cnfg	rw	0x0	blk0_man_cnfg<3:0>
9:8	blk0_man_fcnt	r	—	blk0_man_fcnt<1:0>
19:16	blk1_man_ccnt	r	—	blk1_man_ccnt<3:0>
23:20	blk1_man_cnfg	rw	0x0	blk1_man_cnfg<3:0>
25:24	blk1_man_fcnt	r	—	blk1_man_fcnt<1:0>
35:32	blk2_man_ccnt	r	—	blk2_man_ccnt<3:0>
39:36	blk2_man_cnfg	rw	0x0	blk2_man_cnfg<3:0>
41:40	blk2_man_fcnt	r	—	blk2_man_fcnt<1:0>
51:48	blk3_man_ccnt	r	—	blk3_man_ccnt<3:0>
55:52	blk3_man_cnfg	rw	0x0	blk3_man_cnfg<3:0>
57:56	blk3_man_fcnt	r	—	blk3_man_fcnt<1:0>

man_control_1 register

- Absolute Address: 0x40001240
- Base Offset: 0x40



- Size: 0x8

Bits	Identifier	Access	Reset	Name
3:0	blk4_man_ccnt	r	—	blk4_man_ccnt<3:0>
7:4	blk4_man_cnfg	rw	0x0	blk4_man_cnfg<3:0>
9:8	blk4_man_fcnt	r	—	blk4_man_fcnt<1:0>
19:16	blk5_man_ccnt	r	—	blk5_man_ccnt<3:0>
23:20	blk5_man_cnfg	rw	0x0	blk5_man_cnfg<3:0>
25:24	blk5_man_fcnt	r	—	blk5_man_fcnt<1:0>
35:32	blk6_man_ccnt	r	—	blk6_man_ccnt<3:0>
39:36	blk6_man_cnfg	rw	0x0	blk6_man_cnfg<3:0>
41:40	blk6_man_fcnt	r	—	blk6_man_fcnt<1:0>
51:48	blk7_man_ccnt	r	—	blk7_man_ccnt<3:0>
55:52	blk7_man_cnfg	rw	0x0	blk7_man_cnfg<3:0>
57:56	blk7_man_fcnt	r	—	blk7_man_fcnt<1:0>

mram_status_0 register

- Absolute Address: 0x40001248
- Base Offset: 0x48
- Size: 0x8

Bits	Identifier	Access	Reset	Name
15:0	bist_error_loop	r	—	bist_error_loop<15:0>
33:17	bist_error_count	r	—	bist_error_count<16:0>
40:34	bist_rh0	r	—	bist_rh0<6:0>
47:41	bist_rh1	r	—	bist_rh1<6:0>
54:48	bist_rh2	r	—	bist_rh2<6:0>
56:55	cpu_intr_flag	r	—	cpu_intr_flag
58:57	ecc_1bit_flag	r	—	ecc_1bit_flag<1:0>
60:59	ecc_2bit_flag	r	—	ecc_2bit_flag<1:0>
62:61	ecc_3bit_flag	r	—	ecc_3bit_flag<1:0>

mram_status_1 register

- Absolute Address: 0x40001250
- Base Offset: 0x50



CONTENTS

- Size: 0x8

Bits	Identifier	Access	Reset	Name
1:0	temp	r	—	temp<1:0>
4:3	ecc_1bit	r	—	ecc_1bit<1:0>
6:5	ecc_2bit	r	—	ecc_2bit<1:0>
8:7	ecc_3bit	r	—	ecc_3bit<1:0>
9	pwr_ok	r	—	pwr_ok
10	eccrom_pwr_ok	r	—	eccrom_pwr_ok
29:11	intr_error_lane0_addr	r	—	intr_error_lane0_addr<18:0>
48:30	intr_error_lane1_addr	r	—	intr_error_lane1_addr<18:0>
57:50	busy	r	—	busy<7:0>

mram_dout_even_lower register

- Absolute Address: 0x40001258
- Base Offset: 0x58
- Size: 0x8

Bits	Identifier	Access	Reset	Name
63:0	dout	r	—	dout<63:0>

mram_dout_odd_lower register

- Absolute Address: 0x40001260
- Base Offset: 0x60
- Size: 0x8

Bits	Identifier	Access	Reset	Name
63:0	dout	r	—	dout<142:79>

mram_dout_uppers register

- Absolute Address: 0x40001268
- Base Offset: 0x68
- Size: 0x8



Bits	Identifier	Access	Reset	Name
14:0	dout_even_msb r	—		dout<78:64>
46:32	dout_odd_msb r	—		dout<157:143>

bist_status_0 register

- Absolute Address: 0x40001270
- Base Offset: 0x70
- Size: 0x8

Bits	Identifier	Access	Reset	Name
63:0	bist_error_value r	—		bist_error_value<63:0>

bist_control register

- Absolute Address: 0x40001278
- Base Offset: 0x78
- Size: 0x10

Bits	Identifier	Access	Reset	Name
0	bist_rte_en	rw	0x0	bist_rte_en
1	bist_data_inv	rw	0x0	bist_data_inv
4:2	bist_add_inc	rw	0x0	bist_add_inc<2:0>
5	bist_stop_on_error	rw	0x0	bist_stop_on_error
25:6	bist_start_add	rw	0x0	bist_start_add<19:0>
45:26	bist_stop_add	rw	0x0	bist_stop_add<19:0>
50:46	RH4margin	rw	0xA	RH4margin<4:0>
55:51	rh2_offset	rw	0x0	rh2_offset<4:0>
78:64	bist_error_value	r	—	bist_error_value<78:64>
94:79	bist_loop_count	rw	0x0	bist_loop_count<15:0>
95	bist_start	rw	0x0	bist_start
96	bist_trim_mode	rw	0x0	bist_trim_mode
97	bist_stop_on_repl_of	rw	0x0	bist_stop_on_repl_of
98	bist_rst_b	rw	0x1	bist_rst_b
99	bist_reset	rw	0x0	bist_reset
100	bist_rd_en	rw	0x0	bist_rd_en



Bits	Identifier	Access	Reset	Name
101	bist_wr_en	rw	0x0	bist_wr_en

ecc_correction register

- Absolute Address: 0x40001288
- Base Offset: 0x88
- Size: 0x10

Bits	Identifier	Access	Reset	Name
127:0	dout	r	—	dout<127:0>

bist_status_1 register

- Absolute Address: 0x40001298
- Base Offset: 0x98
- Size: 0x8

Bits	Identifier	Access	Reset	Name
19:0	bist_err_add	r	—	bist_err_add<19:0>
20	bist_error	r	—	bist_error
21	bist_busy	r	—	bist_busy

bank2_tregs register file

- Absolute Address: 0x40001300
- Base Offset: 0x300
- Size: 0xA0

Register space that contains the test registers for test functionality and debug of the MRAM.

Offset	Identifier	Name
0x00	mram_control	—
0x20	mram_control_pulse	—
0x28	gbl_cfg_ovr_0	—



Offset	Identifier	Name
0x30	gbl_cfg_0	—
0x38	man_control_0	—
0x40	man_control_1	—
0x48	mram_status_0	—
0x50	mram_status_1	—
0x58	mram_dout_even_lower	—
0x60	mram_dout_odd_lower	—
0x68	mram_dout_uppers	—
0x70	bist_status_0	—
0x78	bist_control	—
0x88	ecc_correction	—
0x98	bist_status_1	—

mram_control register

- Absolute Address: 0x40001300
- Base Offset: 0x0
- Size: 0x20

Bits	Identifier	Access	Reset	Name
16:0	addr_in	rw	0x0	addr_in
24:17	ce	rw	0x0	ce
25	we	rw	0x0	we
26	rd_pulse_meas_en	rw	0x0	rd_pulse_meas_en
33:27	rca_ovr	rw	0x0	rca_ovr
34	rca_ovr_en	rw	0x0	rca_ovr_en
35	gbl_cfg_ovr_en	rw	0x0	gbl_cfg_ovr_en
36	rd_en_ovr	rw	0x0	rd_en_ovr
37	ref_prg_en	rw	0x0	ref_prg_en
38	dsleep_mram_en	rw	0x0	dsleep_mram_en
39	reg_logic_sup_sleep_ovr	rw	0x1	reg_logic_sup_sleep_ovr
40	prg_rd1_byp	rw	0x0	prg_rd1_byp
41	wr_en_ovr	rw	0x0	wr_en_ovr
42	dma_en	rw	0x0	dma_en
43	vbslx_gain_mode_ovr	rw	0x0	vbslx_gain_mode_ovr
47:44	powerup_trim_load_ovr	rw	0x0	powerup_trim_load_ovr
52	test_cal_en	rw	0x0	test_cal_en



CONTENTS

Bits	Identifier	Access	Reset	Name
55:53	anatest0_sel	rw	0x0	anatest0_sel
58:56	anatest1_sel	rw	0x0	anatest1_sel
59	eccrom_deep_sleep	rw	0x1	eccrom_deep_sleep
60	ref_ecc_sel	rw	0x0	ref_ecc_sel
61	disable_cpu_intr	rw	0x0	disable_cpu_intr
62	disable_ted	rw	0x0	disable_ted
63	ecc_bypass_en	rw	0x0	ecc_bypass_en
142:64	bwe	rw	0x0	bwe
143	mram_clk_en	rw	0x1	mram_clk_en
222:144	din	rw	0x0	din
223	test_reg_ovr_en	rw	0x0	test_reg_ovr_en
224	otp_wr_en	rw	0x0	otp_wr_en
225	rst_cpu_intr	rw	0x0	rst_cpu_intr
233:226	dout_en	rw	0x0	dout_en
236:234	ecc_en	rw	0x0	ecc_en
240:237	even_man_stripe_sel	rw	0x0	even_man_stripe_sel
241	even_man_wr_0	rw	0x0	even_man_wr<0>
242	even_man_wr_1	rw	0x0	even_man_wr<1>
243	even_man_wr_2	rw	0x0	even_man_wr<2>
244	even_man_wr_3	rw	0x0	even_man_wr<3>
248:245	odd_man_stripe_sel	rw	0x0	odd_man_stripe_sel
249	odd_man_wr_0	rw	0x0	odd_man_wr<0>
250	odd_man_wr_1	rw	0x0	odd_man_wr<1>
251	odd_man_wr_2	rw	0x0	odd_man_wr<2>
252	odd_man_wr_3	rw	0x0	odd_man_wr<3>
253	sah_en	rw	0x1	sah_en
254	scc_otp_en	rw	0x0	scc_otp_en
255	maintenance_mode	rw	0x0	maintenance_mode

mram_control_pulse register

- Absolute Address: 0x40001320
- Base Offset: 0x20
- Size: 0x8



Bits	Identifier	Access	Reset	Name
0	powerup_trim_load_ovr_single_pulse<0>	rw	0x0	powerup_trim_load_ovr_single_pulse<0>
1	powerup_trim_load_ovr_single_pulse<1>	rw	0x0	powerup_trim_load_ovr_single_pulse<1>
2	powerup_trim_load_ovr_single_pulse<2>	rw	0x0	powerup_trim_load_ovr_single_pulse<2>
3	powerup_trim_load_ovr_single_pulse<3>	rw	0x0	powerup_trim_load_ovr_single_pulse<3>
4	mram_clk_single_pulse	rw	0x0	mram_clk_single_pulse

gbl_cfg_ovr_0 register

- Absolute Address: 0x40001328
- Base Offset: 0x28
- Size: 0x8

Bits	Identifier	Access	Reset	Name
1:0	sa_equal_trim	rw	0x1	sa_equal_trim<1:0>
4:2	vblslx_boost_trim	rw	0x4	vblslx_boost_trim<2:0>
8:5	wr_en_msb_trim	rw	0x9	wr_en_msb_trim<3:0>
11:9	wr_en_lsb_trim	rw	0x6	wr_en_lsb_trim<2:0>
12	vblslx_gain_mode	rw	0x0	vblslx_gain_mode
16:13	repulse_trim	rw	0x6	repulse_trim<3:0>
17	repulse_en	rw	0x1	repulse_en
20:18	rd_en_trim	rw	0x5	rd_en_trim<2:0>
25:22	osc_wr_div_trim	rw	0x3	osc_wr_div_trim<3:0>
29:26	vblsl_trim	rw	0x4	vblsl_trim<3:0>
33:30	tcsel_trim	rw	0x3	tcsel_trim<3:0>
37:34	vwlrwr_trim	rw	0x7	vwlrwr_trim<3:0>
41:38	vcr_gate_trim	rw	0xE	vcr_gate_trim<3:0>

gbl_cfg_0 register

- Absolute Address: 0x40001330
- Base Offset: 0x30
- Size: 0x8

Bits	Identifier	Access	Reset	Name
1:0	sa_equal_trim	r	—	sa_equal_trim<1:0>



Bits	Identifier	Access	Reset	Name
4:2	vbbslx_boost_trim	r	—	vbbslx_boost_trim<2:0>
8:5	wr_en_msb_trim	r	—	wr_en_msb_trim<3:0>
11:9	wr_en_lsb_trim	r	—	wr_en_lsb_trim<2:0>
12	vbbslx_gain_mode	r	—	vbbslx_gain_mode
16:13	repulse_trim	r	—	repulse_trim<3:0>
17	repulse_en	r	—	repulse_en
20:18	rd_en_trim	r	—	rd_en_trim<2:0>
25:22	osc_wr_div_trim	r	—	osc_wr_div_trim<3:0>
29:26	vbbsl_trim	r	—	vbbsl_trim<3:0>
33:30	tcsel_trim	r	—	tcsel_trim<3:0>
37:34	vwlrwr_trim	r	—	vwlrwr_trim<3:0>
41:38	vcr_gate_trim	r	—	vcr_gate_trim<3:0>

man_control_0 register

- Absolute Address: 0x40001338
- Base Offset: 0x38
- Size: 0x8

Bits	Identifier	Access	Reset	Name
3:0	blk0_man_ccnt	r	—	blk0_man_ccnt<3:0>
7:4	blk0_man_cnfg	rw	0x0	blk0_man_cnfg<3:0>
9:8	blk0_man_fcnt	r	—	blk0_man_fcnt<1:0>
19:16	blk1_man_ccnt	r	—	blk1_man_ccnt<3:0>
23:20	blk1_man_cnfg	rw	0x0	blk1_man_cnfg<3:0>
25:24	blk1_man_fcnt	r	—	blk1_man_fcnt<1:0>
35:32	blk2_man_ccnt	r	—	blk2_man_ccnt<3:0>
39:36	blk2_man_cnfg	rw	0x0	blk2_man_cnfg<3:0>
41:40	blk2_man_fcnt	r	—	blk2_man_fcnt<1:0>
51:48	blk3_man_ccnt	r	—	blk3_man_ccnt<3:0>
55:52	blk3_man_cnfg	rw	0x0	blk3_man_cnfg<3:0>
57:56	blk3_man_fcnt	r	—	blk3_man_fcnt<1:0>

man_control_1 register

- Absolute Address: 0x40001340
- Base Offset: 0x40



- Size: 0x8

Bits	Identifier	Access	Reset	Name
3:0	blk4_man_ccnt	r	—	blk4_man_ccnt<3:0>
7:4	blk4_man_cnfg	rw	0x0	blk4_man_cnfg<3:0>
9:8	blk4_man_fcnt	r	—	blk4_man_fcnt<1:0>
19:16	blk5_man_ccnt	r	—	blk5_man_ccnt<3:0>
23:20	blk5_man_cnfg	rw	0x0	blk5_man_cnfg<3:0>
25:24	blk5_man_fcnt	r	—	blk5_man_fcnt<1:0>
35:32	blk6_man_ccnt	r	—	blk6_man_ccnt<3:0>
39:36	blk6_man_cnfg	rw	0x0	blk6_man_cnfg<3:0>
41:40	blk6_man_fcnt	r	—	blk6_man_fcnt<1:0>
51:48	blk7_man_ccnt	r	—	blk7_man_ccnt<3:0>
55:52	blk7_man_cnfg	rw	0x0	blk7_man_cnfg<3:0>
57:56	blk7_man_fcnt	r	—	blk7_man_fcnt<1:0>

mram_status_0 register

- Absolute Address: 0x40001348
- Base Offset: 0x48
- Size: 0x8

Bits	Identifier	Access	Reset	Name
15:0	bist_error_loop	r	—	bist_error_loop<15:0>
33:17	bist_error_count	r	—	bist_error_count<16:0>
40:34	bist_rh0	r	—	bist_rh0<6:0>
47:41	bist_rh1	r	—	bist_rh1<6:0>
54:48	bist_rh2	r	—	bist_rh2<6:0>
56:55	cpu_intr_flag	r	—	cpu_intr_flag
58:57	ecc_1bit_flag	r	—	ecc_1bit_flag<1:0>
60:59	ecc_2bit_flag	r	—	ecc_2bit_flag<1:0>
62:61	ecc_3bit_flag	r	—	ecc_3bit_flag<1:0>

mram_status_1 register

- Absolute Address: 0x40001350
- Base Offset: 0x50



- Size: 0x8

Bits	Identifier	Access	Reset	Name
1:0	temp	r	—	temp<1:0>
4:3	ecc_1bit	r	—	ecc_1bit<1:0>
6:5	ecc_2bit	r	—	ecc_2bit<1:0>
8:7	ecc_3bit	r	—	ecc_3bit<1:0>
9	pwr_ok	r	—	pwr_ok
10	eccrom_pwr_ok	r	—	eccrom_pwr_ok
29:11	intr_error_lane0_addr	r	—	intr_error_lane0_addr<18:0>
48:30	intr_error_lane1_addr	r	—	intr_error_lane1_addr<18:0>
57:50	busy	r	—	busy<7:0>

mram_dout_even_lower register

- Absolute Address: 0x40001358
- Base Offset: 0x58
- Size: 0x8

Bits	Identifier	Access	Reset	Name
63:0	dout	r	—	dout<63:0>

mram_dout_odd_lower register

- Absolute Address: 0x40001360
- Base Offset: 0x60
- Size: 0x8

Bits	Identifier	Access	Reset	Name
63:0	dout	r	—	dout<142:79>

mram_dout_uppers register

- Absolute Address: 0x40001368
- Base Offset: 0x68
- Size: 0x8



Bits	Identifier	Access	Reset	Name
14:0	dout_even_msb r	—		dout<78:64>
46:32	dout_odd_msb r	—		dout<157:143>

bist_status_0 register

- Absolute Address: 0x40001370
- Base Offset: 0x70
- Size: 0x8

Bits	Identifier	Access	Reset	Name
63:0	bist_error_value r	—		bist_error_value<63:0>

bist_control register

- Absolute Address: 0x40001378
- Base Offset: 0x78
- Size: 0x10

Bits	Identifier	Access	Reset	Name
0	bist_rte_en	rw	0x0	bist_rte_en
1	bist_data_inv	rw	0x0	bist_data_inv
4:2	bist_add_inc	rw	0x0	bist_add_inc<2:0>
5	bist_stop_on_error	rw	0x0	bist_stop_on_error
25:6	bist_start_add	rw	0x0	bist_start_add<19:0>
45:26	bist_stop_add	rw	0x0	bist_stop_add<19:0>
50:46	RH4margin	rw	0xA	RH4margin<4:0>
55:51	rh2_offset	rw	0x0	rh2_offset<4:0>
78:64	bist_error_value	r	—	bist_error_value<78:64>
94:79	bist_loop_count	rw	0x0	bist_loop_count<15:0>
95	bist_start	rw	0x0	bist_start
96	bist_trim_mode	rw	0x0	bist_trim_mode
97	bist_stop_on_repl_of	rw	0x0	bist_stop_on_repl_of
98	bist_rst_b	rw	0x1	bist_rst_b
99	bist_reset	rw	0x0	bist_reset
100	bist_rd_en	rw	0x0	bist_rd_en



Bits	Identifier	Access	Reset	Name
101	bist_wr_en	rw	0x0	bist_wr_en

ecc_correction register

- Absolute Address: 0x40001388
- Base Offset: 0x88
- Size: 0x10

Bits	Identifier	Access	Reset	Name
127:0	dout	r	—	dout<127:0>

bist_status_1 register

- Absolute Address: 0x40001398
- Base Offset: 0x98
- Size: 0x8

Bits	Identifier	Access	Reset	Name
19:0	bist_err_add	r	—	bist_err_add<19:0>
20	bist_error	r	—	bist_error
21	bist_busy	r	—	bist_busy

bank3_tregs register file

- Absolute Address: 0x40001400
- Base Offset: 0x400
- Size: 0xA0

Register space that contains the test registers for test functionality and debug of the MRAM.

Offset	Identifier	Name
0x00	mram_control	—
0x20	mram_control_pulse	—
0x28	gbl_cfg_ovr_0	—



Offset	Identifier	Name
0x30	gbl_cfg_0	—
0x38	man_control_0	—
0x40	man_control_1	—
0x48	mram_status_0	—
0x50	mram_status_1	—
0x58	mram_dout_even_lower	—
0x60	mram_dout_odd_lower	—
0x68	mram_dout_uppers	—
0x70	bist_status_0	—
0x78	bist_control	—
0x88	ecc_correction	—
0x98	bist_status_1	—

mram_control register

- Absolute Address: 0x40001400
- Base Offset: 0x0
- Size: 0x20

Bits	Identifier	Access	Reset	Name
16:0	addr_in	rw	0x0	addr_in
24:17	ce	rw	0x0	ce
25	we	rw	0x0	we
26	rd_pulse_meas_en	rw	0x0	rd_pulse_meas_en
33:27	rca_ovr	rw	0x0	rca_ovr
34	rca_ovr_en	rw	0x0	rca_ovr_en
35	gbl_cfg_ovr_en	rw	0x0	gbl_cfg_ovr_en
36	rd_en_ovr	rw	0x0	rd_en_ovr
37	ref_prg_en	rw	0x0	ref_prg_en
38	dsleep_mram_en	rw	0x0	dsleep_mram_en
39	reg_logic_sup_sleep_ovr	rw	0x1	reg_logic_sup_sleep_ovr
40	prg_rd1_byp	rw	0x0	prg_rd1_byp
41	wr_en_ovr	rw	0x0	wr_en_ovr
42	dma_en	rw	0x0	dma_en
43	vbslx_gain_mode_ovr	rw	0x0	vbslx_gain_mode_ovr
47:44	powerup_trim_load_ovr	rw	0x0	powerup_trim_load_ovr
52	test_cal_en	rw	0x0	test_cal_en



CONTENTS

Bits	Identifier	Access	Reset	Name
55:53	anatest0_sel	rw	0x0	anatest0_sel
58:56	anatest1_sel	rw	0x0	anatest1_sel
59	eccrom_deep_sleep	rw	0x1	eccrom_deep_sleep
60	ref_ecc_sel	rw	0x0	ref_ecc_sel
61	disable_cpu_intr	rw	0x0	disable_cpu_intr
62	disable_ted	rw	0x0	disable_ted
63	ecc_bypass_en	rw	0x0	ecc_bypass_en
142:64	bwe	rw	0x0	bwe
143	mram_clk_en	rw	0x1	mram_clk_en
222:144	din	rw	0x0	din
223	test_reg_ovr_en	rw	0x0	test_reg_ovr_en
224	otp_wr_en	rw	0x0	otp_wr_en
225	rst_cpu_intr	rw	0x0	rst_cpu_intr
233:226	dout_en	rw	0x0	dout_en
236:234	ecc_en	rw	0x0	ecc_en
240:237	even_man_stripe_sel	rw	0x0	even_man_stripe_sel
241	even_man_wr_0	rw	0x0	even_man_wr<0>
242	even_man_wr_1	rw	0x0	even_man_wr<1>
243	even_man_wr_2	rw	0x0	even_man_wr<2>
244	even_man_wr_3	rw	0x0	even_man_wr<3>
248:245	odd_man_stripe_sel	rw	0x0	odd_man_stripe_sel
249	odd_man_wr_0	rw	0x0	odd_man_wr<0>
250	odd_man_wr_1	rw	0x0	odd_man_wr<1>
251	odd_man_wr_2	rw	0x0	odd_man_wr<2>
252	odd_man_wr_3	rw	0x0	odd_man_wr<3>
253	sah_en	rw	0x1	sah_en
254	scc_otp_en	rw	0x0	scc_otp_en
255	maintenance_mode	rw	0x0	maintenance_mode

mram_control_pulse register

- Absolute Address: 0x40001420
- Base Offset: 0x20
- Size: 0x8



Bits	Identifier	Access	Reset	Name
0	powerup_trim_load_ovr_single_pulse<0>	rw	0x0	powerup_trim_load_ovr_single_pulse<0>
1	powerup_trim_load_ovr_single_pulse<1>	rw	0x0	powerup_trim_load_ovr_single_pulse<1>
2	powerup_trim_load_ovr_single_pulse<2>	rw	0x0	powerup_trim_load_ovr_single_pulse<2>
3	powerup_trim_load_ovr_single_pulse<3>	rw	0x0	powerup_trim_load_ovr_single_pulse<3>
4	mram_clk_single_pulse	rw	0x0	mram_clk_single_pulse

gbl_cfg_ovr_0 register

- Absolute Address: 0x40001428
- Base Offset: 0x28
- Size: 0x8

Bits	Identifier	Access	Reset	Name
1:0	sa_equal_trim	rw	0x1	sa_equal_trim<1:0>
4:2	vblslx_boost_trim	rw	0x4	vblslx_boost_trim<2:0>
8:5	wr_en_msb_trim	rw	0x9	wr_en_msb_trim<3:0>
11:9	wr_en_lsb_trim	rw	0x6	wr_en_lsb_trim<2:0>
12	vblslx_gain_mode	rw	0x0	vblslx_gain_mode
16:13	repulse_trim	rw	0x6	repulse_trim<3:0>
17	repulse_en	rw	0x1	repulse_en
20:18	rd_en_trim	rw	0x5	rd_en_trim<2:0>
25:22	osc_wr_div_trim	rw	0x3	osc_wr_div_trim<3:0>
29:26	vblsl_trim	rw	0x4	vblsl_trim<3:0>
33:30	tcsel_trim	rw	0x3	tcsel_trim<3:0>
37:34	vwlrwr_trim	rw	0x7	vwlrwr_trim<3:0>
41:38	vcr_gate_trim	rw	0xE	vcr_gate_trim<3:0>

gbl_cfg_0 register

- Absolute Address: 0x40001430
- Base Offset: 0x30
- Size: 0x8

Bits	Identifier	Access	Reset	Name
1:0	sa_equal_trim	r	—	sa_equal_trim<1:0>



CONTENTS

Bits	Identifier	Access	Reset	Name
4:2	vbbslx_boost_trim	r	—	vbbslx_boost_trim<2:0>
8:5	wr_en_msb_trim	r	—	wr_en_msb_trim<3:0>
11:9	wr_en_lsb_trim	r	—	wr_en_lsb_trim<2:0>
12	vbbslx_gain_mode	r	—	vbbslx_gain_mode
16:13	repulse_trim	r	—	repulse_trim<3:0>
17	repulse_en	r	—	repulse_en
20:18	rd_en_trim	r	—	rd_en_trim<2:0>
25:22	osc_wr_div_trim	r	—	osc_wr_div_trim<3:0>
29:26	vbbsl_trim	r	—	vbbsl_trim<3:0>
33:30	tcsel_trim	r	—	tcsel_trim<3:0>
37:34	vwlrwr_trim	r	—	vwlrwr_trim<3:0>
41:38	vcr_gate_trim	r	—	vcr_gate_trim<3:0>

man_control_0 register

- Absolute Address: 0x40001438
- Base Offset: 0x38
- Size: 0x8

Bits	Identifier	Access	Reset	Name
3:0	blk0_man_ccnt	r	—	blk0_man_ccnt<3:0>
7:4	blk0_man_cnfg	rw	0x0	blk0_man_cnfg<3:0>
9:8	blk0_man_fcnt	r	—	blk0_man_fcnt<1:0>
19:16	blk1_man_ccnt	r	—	blk1_man_ccnt<3:0>
23:20	blk1_man_cnfg	rw	0x0	blk1_man_cnfg<3:0>
25:24	blk1_man_fcnt	r	—	blk1_man_fcnt<1:0>
35:32	blk2_man_ccnt	r	—	blk2_man_ccnt<3:0>
39:36	blk2_man_cnfg	rw	0x0	blk2_man_cnfg<3:0>
41:40	blk2_man_fcnt	r	—	blk2_man_fcnt<1:0>
51:48	blk3_man_ccnt	r	—	blk3_man_ccnt<3:0>
55:52	blk3_man_cnfg	rw	0x0	blk3_man_cnfg<3:0>
57:56	blk3_man_fcnt	r	—	blk3_man_fcnt<1:0>

man_control_1 register

- Absolute Address: 0x40001440
- Base Offset: 0x40



- Size: 0x8

Bits	Identifier	Access	Reset	Name
3:0	blk4_man_ccnt	r	—	blk4_man_ccnt<3:0>
7:4	blk4_man_cnfg	rw	0x0	blk4_man_cnfg<3:0>
9:8	blk4_man_fcnt	r	—	blk4_man_fcnt<1:0>
19:16	blk5_man_ccnt	r	—	blk5_man_ccnt<3:0>
23:20	blk5_man_cnfg	rw	0x0	blk5_man_cnfg<3:0>
25:24	blk5_man_fcnt	r	—	blk5_man_fcnt<1:0>
35:32	blk6_man_ccnt	r	—	blk6_man_ccnt<3:0>
39:36	blk6_man_cnfg	rw	0x0	blk6_man_cnfg<3:0>
41:40	blk6_man_fcnt	r	—	blk6_man_fcnt<1:0>
51:48	blk7_man_ccnt	r	—	blk7_man_ccnt<3:0>
55:52	blk7_man_cnfg	rw	0x0	blk7_man_cnfg<3:0>
57:56	blk7_man_fcnt	r	—	blk7_man_fcnt<1:0>

mram_status_0 register

- Absolute Address: 0x40001448
- Base Offset: 0x48
- Size: 0x8

Bits	Identifier	Access	Reset	Name
15:0	bist_error_loop	r	—	bist_error_loop<15:0>
33:17	bist_error_count	r	—	bist_error_count<16:0>
40:34	bist_rh0	r	—	bist_rh0<6:0>
47:41	bist_rh1	r	—	bist_rh1<6:0>
54:48	bist_rh2	r	—	bist_rh2<6:0>
56:55	cpu_intr_flag	r	—	cpu_intr_flag
58:57	ecc_1bit_flag	r	—	ecc_1bit_flag<1:0>
60:59	ecc_2bit_flag	r	—	ecc_2bit_flag<1:0>
62:61	ecc_3bit_flag	r	—	ecc_3bit_flag<1:0>

mram_status_1 register

- Absolute Address: 0x40001450
- Base Offset: 0x50



CONTENTS

- Size: 0x8

Bits	Identifier	Access	Reset	Name
1:0	temp	r	—	temp<1:0>
4:3	ecc_1bit	r	—	ecc_1bit<1:0>
6:5	ecc_2bit	r	—	ecc_2bit<1:0>
8:7	ecc_3bit	r	—	ecc_3bit<1:0>
9	pwr_ok	r	—	pwr_ok
10	eccrom_pwr_ok	r	—	eccrom_pwr_ok
29:11	intr_error_lane0_addr	r	—	intr_error_lane0_addr<18:0>
48:30	intr_error_lane1_addr	r	—	intr_error_lane1_addr<18:0>
57:50	busy	r	—	busy<7:0>

mram_dout_even_lower register

- Absolute Address: 0x40001458
- Base Offset: 0x58
- Size: 0x8

Bits	Identifier	Access	Reset	Name
63:0	dout	r	—	dout<63:0>

mram_dout_odd_lower register

- Absolute Address: 0x40001460
- Base Offset: 0x60
- Size: 0x8

Bits	Identifier	Access	Reset	Name
63:0	dout	r	—	dout<142:79>

mram_dout_uppers register

- Absolute Address: 0x40001468
- Base Offset: 0x68
- Size: 0x8



Bits	Identifier	Access	Reset	Name
14:0	dout_even_msb r	—		dout<78:64>
46:32	dout_odd_msb r	—		dout<157:143>

bist_status_0 register

- Absolute Address: 0x40001470
- Base Offset: 0x70
- Size: 0x8

Bits	Identifier	Access	Reset	Name
63:0	bist_error_value r	—		bist_error_value<63:0>

bist_control register

- Absolute Address: 0x40001478
- Base Offset: 0x78
- Size: 0x10

Bits	Identifier	Access	Reset	Name
0	bist_rte_en	rw	0x0	bist_rte_en
1	bist_data_inv	rw	0x0	bist_data_inv
4:2	bist_add_inc	rw	0x0	bist_add_inc<2:0>
5	bist_stop_on_error	rw	0x0	bist_stop_on_error
25:6	bist_start_add	rw	0x0	bist_start_add<19:0>
45:26	bist_stop_add	rw	0x0	bist_stop_add<19:0>
50:46	RH4margin	rw	0xA	RH4margin<4:0>
55:51	rh2_offset	rw	0x0	rh2_offset<4:0>
78:64	bist_error_value	r	—	bist_error_value<78:64>
94:79	bist_loop_count	rw	0x0	bist_loop_count<15:0>
95	bist_start	rw	0x0	bist_start
96	bist_trim_mode	rw	0x0	bist_trim_mode
97	bist_stop_on_repl_of	rw	0x0	bist_stop_on_repl_of
98	bist_rst_b	rw	0x1	bist_rst_b
99	bist_reset	rw	0x0	bist_reset
100	bist_rd_en	rw	0x0	bist_rd_en



CONTENTS

Bits	Identifier	Access	Reset	Name
101	bist_wr_en	rw	0x0	bist_wr_en

ecc_correction register

- Absolute Address: 0x40001488
- Base Offset: 0x88
- Size: 0x10

Bits	Identifier	Access	Reset	Name
127:0	dout	r	—	dout<127:0>

bist_status_1 register

- Absolute Address: 0x40001498
- Base Offset: 0x98
- Size: 0x8

Bits	Identifier	Access	Reset	Name
19:0	bist_err_add	r	—	bist_err_add<19:0>
20	bist_error	r	—	bist_error
21	bist_busy	r	—	bist_busy

i2c_registers address map

- Absolute Address: 0x40002000
- Base Offset: 0x40002000
- Size: 0x24

Offset	Identifier	Name
0x00	Commands	Command Registers
0x08	Status	—
0x10	Cfg	—
0x18	Wdata	—
0x20	Rdata	—



Commands register

- Absolute Address: 0x40002000
- Base Offset: 0x0
- Size: 0x4

I2C Command Registers

Bits Identifier		Access Reset Name		
0	start	rw	0x0	—
1	read	rw	0x0	—
2	write	rw	0x0	—
3	write_multiple	rw	0x0	—
4	stop	rw	0x0	—
5	enq	rw	0x0	—
14:8	address	rw	0x0	—

start field Start

read field TODO

write field TODO

write_multiple field TODO

stop field TODO

enq field TODO

address field Address

Status register

- Absolute Address: 0x40002008
- Base Offset: 0x8
- Size: 0x4



CONTENTS

Bits Identifier		Access Reset Name	
0	busy	r	0x0 —
1	bus_control	r	0x0 —
2	bus_active	r	0x0 —
3	missed_ack	r, rclr	0x0 —
4	cmd_ff_n_full	r	— —
5	tx_ff_n_full	r	— —
6	rx_ff_n_full	r	— —
7	rx_overflow	r	— —

busy field Transaction ongoing

bus_control field Transaction ongoing

bus_active field Transaction ongoing

missed_ack field Transaction ongoing

cmd_ff_n_full field cmd_ff not Full

tx_ff_n_full field tx_ff not Full

rx_ff_n_full field rx_ff not Full

rx_overflow field rx overflow

Cfg register

- Absolute Address: 0x40002010
- Base Offset: 0x10
- Size: 0x4

Bits Identifier		Access Reset Name	
15:0	prescale	rw	0x0 —
16	stop_on_idle	rw	0x0 —



prescale field Prescale

stop_on_idle field Stop on Idle

Wdata register

- Absolute Address: 0x40002018
- Base Offset: 0x18
- Size: 0x4

Bits Identifier Access Reset Name				
7:0	wdata	w	0x0	—
8	wlast	w	0x0	—

wdata field TxData

wlast field Write Last

Rdata register

- Absolute Address: 0x40002020
- Base Offset: 0x20
- Size: 0x4

Bits Identifier Access Reset Name				
7:0	rdata	r	0x0	—
8	rlast	r	0x0	—

rdata field RxData

rlast field Read Last



qspi_registers address map

- Absolute Address: 0x40003000
- Base Offset: 0x40003000
- Size: 0x64

Shakti QSPI Registers

Offset Identifier Name		
0x00	CR	—
0x08	DCR	—
0x10	SR	—
0x18	FCR	—
0x20	DLR	—
0x28	CCR	—
0x30	AR	—
0x38	ABR	—
0x40	DR	—
0x48	PSMKR	—
0x50	PSMAR	—
0x58	PIR	—
0x60	LPTR	—

CR register

- Absolute Address: 0x40003000
- Base Offset: 0x0
- Size: 0x4

Bits	Identifier	Access	Reset	Name
0	qspi_enable	rw	0x0	—
1	abort	rw	0x0	—
2	dmaen	rw	0x0	—
3	tcen	rw	0x0	—
4	sshift	rw	0x0	—
6	dfm	rw	0x0	—
7	fsel	rw	0x0	—
12:8	ftlhres	rw	0x0	—



Bits	Identifier	Access	Reset	Name
16	teie	rw	0x0	—
17	tcie	rw	0x0	—
18	ftie	rw	0x0	—
19	smie	rw	0x0	—
20	toie	rw	0x0	—
22	apms	rw	0x0	—
23	pmm	rw	0x0	—
31:24	prescaler	rw	0x0	—

qspi_enable field QSPI enable

abort field Abort request

dmaen field DMA enable

tcen field Timeout counter enable

sshift field Sample shift

dfm field Dual flash mode enable

fsel field Flash select

ftbres field FIFO threshold level

teie field Transfer error interrupt enable

tcie field Transfer complete interrupt enable

ftie field FIFO threshold interrupt enable

smie field Status match interrupt enable

toie field Timeout interrupt enable

apms field Automatic polling mode stop



CONTENTS

pmm field Polling match mode (0=AND,1=OR)

prescaler field Clock prescaler (PRESCALER[7:0])

DCR register

- Absolute Address: 0x40003008
- Base Offset: 0x8
- Size: 0x4

Bits	Identifier	Access	Reset	Name
0	ckmode	rw	0x0	—
10:8	csht	rw	0x0	—
20:16	fsize	rw	0x0	—

ckmode field Clock mode (0=Mode0,1=Mode3)

csht field Chip select high time

fsize field Flash memory size

SR register

- Absolute Address: 0x40003010
- Base Offset: 0x10
- Size: 0x4

Bits	Identifier	Access	Reset	Name
0	tef	r	—	—
1	tcf	r	—	—
2	ftf	r	—	—
3	smf	r	—	—
4	tof	r	—	—
5	busy	r	—	—
13:8	flevel	r	—	—

tef field Transfer error flag



tcf field Transfer complete flag

ftf field FIFO threshold flag

smf field Status match flag

tof field Timeout flag

busy field Busy flag

flevel field FIFO level

FCR register

- Absolute Address: 0x40003018
- Base Offset: 0x18
- Size: 0x4

Bits Identifier Access Reset Name				
0	ctef	rw	0x0	—
1	ctcf	rw	0x0	—
3	csmf	rw	0x0	—
4	ctof	rw	0x0	—

ctef field Clear transfer error flag

ctcf field Clear transfer complete flag

csmf field Clear status match flag

ctof field Clear timeout flag

**DLR register**

- Absolute Address: 0x40003020
- Base Offset: 0x20
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	dl	rw	0x0	—

dl field Number of bytes to transfer (value+1)

CCR register

- Absolute Address: 0x40003028
- Base Offset: 0x28
- Size: 0x4

Bits	Identifier	Access	Reset	Name
7:0	instr	rw	0x0	—
9:8	imode	rw	0x0	—
11:10	admode	rw	0x0	—
13:12	adsize	rw	0x0	—
15:14	abmode	rw	0x0	—
17:16	absize	rw	0x0	—
22:18	dcyc	rw	0x0	—
23	d_conf	rw	0x0	—
25:24	dmode	rw	0x0	—
27:26	fmode	rw	0x0	—
28	sioo	rw	0x0	—
30	dhhc	rw	0x0	—
31	ddrm	rw	0x0	—

instr field Instruction opcode

imode field Instruction mode

admode field Address mode



adsize field Address size

abmode field Alternate byte mode

absize field Alternate byte size

dcyc field Dummy cycles

d_conf field Dummy confirmation

dmode field Data mode

fmode field Functional mode

sioo field Send instruction only once

dhhc field DDR hold quarter cycle

ddrm field DDR mode enable

AR register

- Absolute Address: 0x40003030
- Base Offset: 0x30
- Size: 0x4

Bits	Identifier	Access	Reset	Name
------	------------	--------	-------	------

31:0	address	rw	0x0	—
------	---------	----	-----	---

address field Flash address

**ABR register**

- Absolute Address: 0x40003038
- Base Offset: 0x38
- Size: 0x4

Bits Identifier	Access	Reset	Name
31:0 alternate_bytes	rw	0x0	—

alternate_bytes field Alternate bytes

DR register

- Absolute Address: 0x40003040
- Base Offset: 0x40
- Size: 0x4

Bits Identifier	Access	Reset	Name
31:0 data	rw	0x0	—

data field Data register (FIFO access)

PSMKR register

- Absolute Address: 0x40003048
- Base Offset: 0x48
- Size: 0x4

Bits Identifier	Access	Reset	Name
31:0 mask	rw	0x0	—

mask field Polling status mask

**PSMAR register**

- Absolute Address: 0x40003050
- Base Offset: 0x50
- Size: 0x4

Bits Identifier Access Reset Name			
31:0 match	rw	0x0	—

match field Polling status match value

PIR register

- Absolute Address: 0x40003058
- Base Offset: 0x58
- Size: 0x4

Bits Identifier Access Reset Name			
31:0 interval	rw	0x0	—

interval field Polling interval in clock cycles

LPTR register

- Absolute Address: 0x40003060
- Base Offset: 0x60
- Size: 0x4

Bits Identifier Access Reset Name			
31:0 timeout	rw	0x0	—

timeout field Low power timeout value



uart__registers address map

- Absolute Address: 0x40004000
- Base Offset: 0x40004000
- Size: 0x5C

UART Registers Refer shakti GC doc for details

Offset Identifier		Name
0x00	BaudReg	—
0x08	TxReg	—
0x10	RxReg	—
0x18	StatusReg	—
0x20	DelayReg	—
0x28	ControlReg	—
0x30	IQC	—
0x38	Rx_Threshold	—
0x40	NotUsed	—
0x48	InterruptRaw	—
0x50	InterruptMask	—
0x58	InterruptStatus	—

BaudReg register

- Absolute Address: 0x40004000
- Base Offset: 0x0
- Size: 0x4

Bits Identifier	Access	Reset	Name
15:0 baud_value	rw	0x5	—

baud_value field $\text{Baud} = \text{Clk_Freq} / (16 * \text{baud_value})$

TxReg register

- Absolute Address: 0x40004008
- Base Offset: 0x8
- Size: 0x4



Bits Identifier Access Reset Name			
31:0 data	w	0x0	—

data field

RxReg register

- Absolute Address: 0x40004010
- Base Offset: 0x10
- Size: 0x4

Bits Identifier Access Reset Name			
31:0 data	r	0x0	—

data field

StatusReg register

- Absolute Address: 0x40004018
- Base Offset: 0x18
- Size: 0x4

Bits Identifier		Access	Reset Name
0	tx_empty	rw, woclr	0x0 —
1	tx_full	rw, woclr	0x0 —
2	rx_notEmpty	rw, woclr	0x0 —
3	rx_full	rw, woclr	0x0 —
4	parity_error	rw, woclr	0x0 —
5	overrun_error	rw, woclr	0x0 —
6	frame_error	rw, woclr	0x0 —
7	break_error	rw, woclr	0x0 —
8	rx_fifo_threshold	rw, woclr	0x0 —

tx_empty field Tx FIFO is Empty

tx_full field Tx FIFO is Full



rx_notEmpty field Rx FIFO is Not Empty

rx_full field Rx FIFO is Full

parity_error field Rx Parity error

overrun_error field Rx FIFO overflow

frame_error field Rx stop bit is zero

break_error field Rx Data and stop bits are zero

rx_fifo_threshold field Threshold level crossed on Rx.

DelayReg register

- Absolute Address: 0x40004020
- Base Offset: 0x20
- Size: 0x4

Bits Identifier	Access	Reset	Name
15:0	delay_control	rw	0x0 —

delay_control field Transmit delay count

ControlReg register

- Absolute Address: 0x40004028
- Base Offset: 0x28
- Size: 0x4

Bits Identifier	Access	Reset	Name
2:1	stopbits	rw	0x0 —
4:3	parity	rw	0x0 —
10:5	charsize	rw	0x8 —



stopbits field 00 -> 1 stop bit; 01 -> 1.5 stop bits ; 10 -> 2 stop bits

parity field 0-> no parity, 1 -> odd parity; 2 -> even parity

charsize field char size of data (max 32b)

IQC register

- Absolute Address: 0x40004030
- Base Offset: 0x30
- Size: 0x4

Bits Identifier	Access	Reset	Name
7:0	qual_cycles	rw	0x0 —

qual_cycles field

Rx_Threshold register

- Absolute Address: 0x40004038
- Base Offset: 0x38
- Size: 0x4

Bits Identifier	Access	Reset	Name
4:0	rx_level	rw	0x5 —

rx_level field

NotUsed register

- Absolute Address: 0x40004040
- Base Offset: 0x40
- Size: 0x4



Bits	Identifier	Access	Reset	Name
7:0	notused	r	0x0	—

notused field notused

InterruptRaw register

- Absolute Address: 0x40004048
- Base Offset: 0x48
- Size: 0x4

RAW interrupt register, the bit is sticky and set when the condition for interrupt occurs and remains set until a value of 1 is written to it.

Bits	Identifier	Access	Reset	Name
0	tx_done	rw, wocl	0x0	—
1	tx_not_full	rw, wocl	0x0	—
2	rx_not_empty	rw, wocl	0x0	—
3	rx_full	rw, wocl	0x0	—
4	parity_error	rw, wocl	0x0	—
5	overrun_error	rw, wocl	0x0	—
6	frame_error	rw, wocl	0x0	—
7	break_error	rw, wocl	0x0	—
8	rx_fifo_threshold	rw, wocl	0x0	—

tx_done field Transmission Done Interrupt

tx_not_full field Transmitter Not Full Interrupt

rx_not_empty field Receiver Not Empty Interrupt

rx_full field Receiver Full Interrupt

parity_error field Parity Error Interrupt

overrun_error field Overrun Interrupt



frame_error field Frame Error Interrupt

break_error field Break Error Interrupt

rx_fifo_threshold field RX FIFO ~80% Full Interrupt

InterruptMask register

- Absolute Address: 0x40004050
- Base Offset: 0x50
- Size: 0x4

interrupt Mask Register, this is anded with the raw interrupt bit. if the result is 1 an interrupt is generated.

Bits Identifier		Access Reset Name		
0	tx_done_en	rw	0x0	—
1	tx_not_full_en	rw	0x0	—
2	rx_not_empty_en	rw	0x0	—
3	rx_full_en	rw	0x0	—
4	parity_error_en	rw	0x0	—
5	overrun_error_en	rw	0x0	—
6	frame_error_en	rw	0x0	—
7	break_error_en	rw	0x0	—
8	rx_fifo_threshold_en	rw	0x0	—

tx_done_en field Transmission Done Interrupt Mask

tx_not_full_en field Transmitter Not Full Interrupt Mask

rx_not_empty_en field Receiver Not Empty Interrupt Mask

rx_full_en field Receiver Full Interrupt Mask

parity_error_en field Parity Error Interrupt Mask

overrun_error_en field Overrun Interrupt Mask



frame__error__en field Frame Error Interrupt Mask

break__error__en field Break Error Interrupt Mask

rx_fifo_threshold__en field RX FIFO ~80% Full Interrupt Mask

InterruptStatus register

- Absolute Address: 0x40004058
- Base Offset: 0x58
- Size: 0x4

Bits Identifier		Access Reset Name		
0	tx_done	r	0x0	—
1	tx_not_full	r	0x0	—
2	rx_not_empty	r	0x0	—
3	rx_full	r	0x0	—
4	parity_error	r	0x0	—
5	overrun_error	r	0x0	—
6	frame_error	r	0x0	—
7	break_error	r	0x0	—
8	rx_fifo_threshold	r	0x0	—

tx_done field Transmission Done Interrupt

tx_not_full field Transmitter Not Full Interrupt

rx_not_empty field Receiver Not Empty Interrupt

rx_full field Receiver Full Interrupt

parity_error field Parity Error Interrupt

overrun_error field Overrun Interrupt

frame_error field Frame Error Interrupt



break_error field Break Error Interrupt

rx_fifo_threshold field RX FIFO ~80% Full Interrupt

ROMRAM address map

- Absolute Address: 0x40008000
- Base Offset: 0x40008000
- Size: 0x5000

Offset	Identifier Name
0x0000 ROM	BootROM
0x4000 SRAM	SRAM

ROM memory

- Absolute Address: 0x40008000
- Base Offset: 0x0
- Size: 0x2000

No supported members.

SRAM memory

- Absolute Address: 0x4000C000
- Base Offset: 0x4000
- Size: 0x1000

No supported members.

cpu_registers address map

- Absolute Address: 0x80000000
- Base Offset: 0x80000000
- Size: 0xF40318



Offset	Identifier	Name
0x100000	User_neigh	U_neigh
0x340000	User_cpu	U_cpu
0x740000	Supervisor_cpu	S_cpu
0x800000	D_hart_esr	D_hart_esr
0x900000	D_neigh	D_neigh
0xB5F000	D_cpu	D_cpu
0xD00000	Machine_neigh	M_neigh
0xF40000	Machine_cpu	M_cpu

User_neigh address map

- Absolute Address: 0x80100000
- Base Offset: 0x100000
- Size: 0x48

Offset	Identifier	Name
0x40	ipi_redirect_pc	ipi_redirect_pc

ipi_redirect_pc register

- Absolute Address: 0x80100040
- Base Offset: 0x40
- Size: 0x8

Bits	Identifier	Access	Reset	Name
47:0	ipi_redirect_pc	rw	0x0	—

ipi_redirect_pc field In bits 47:0 of this register, user-level software writes the 48 least significant bits of a target virtual PC of a redirect IPI for all the harts in a given neighborhood. Also, bit 0 of the PC is always 0

User_cpu address map

- Absolute Address: 0x80340000
- Base Offset: 0x340000



- Size: 0x300

Offset	Identifier	Name
0x080	ipi_redirect_trigger	ipi_redirect_trigger
0x0C0	CREDINC0	CREDINC0
0x0C8	CREDINC1	CREDINC1
0x0D0	CREDINC2	CREDINC2
0x0D8	CREDINC3	CREDINC3
0x100	fast_local_barrier0	fast_local_barrier0
0x108	fast_local_barrier1	fast_local_barrier1
0x110	fast_local_barrier2	fast_local_barrier2
0x118	fast_local_barrier3	fast_local_barrier3
0x120	fast_local_barrier4	fast_local_barrier4
0x128	fast_local_barrier5	fast_local_barrier5
0x130	fast_local_barrier6	fast_local_barrier6
0x138	fast_local_barrier7	fast_local_barrier7
0x140	fast_local_barrier8	fast_local_barrier8
0x148	fast_local_barrier9	fast_local_barrier9
0x150	fast_local_barrier10	fast_local_barrier10
0x158	fast_local_barrier11	fast_local_barrier11
0x160	fast_local_barrier12	fast_local_barrier12
0x168	fast_local_barrier13	fast_local_barrier13
0x170	fast_local_barrier14	fast_local_barrier14
0x178	fast_local_barrier15	fast_local_barrier15
0x180	fast_local_barrier16	fast_local_barrier16
0x188	fast_local_barrier17	fast_local_barrier17
0x190	fast_local_barrier18	fast_local_barrier18
0x198	fast_local_barrier19	fast_local_barrier19
0x1A0	fast_local_barrier20	fast_local_barrier20
0x1A8	fast_local_barrier21	fast_local_barrier21
0x1B0	fast_local_barrier22	fast_local_barrier22
0x1B8	fast_local_barrier23	fast_local_barrier23
0x1C0	fast_local_barrier24	fast_local_barrier24
0x1C8	fast_local_barrier25	fast_local_barrier25
0x1D0	fast_local_barrier26	fast_local_barrier26
0x1D8	fast_local_barrier27	fast_local_barrier27
0x1E0	fast_local_barrier28	fast_local_barrier28
0x1E8	fast_local_barrier29	fast_local_barrier29



Offset	Identifier	Name
0x1F0	fast_local_barrier30	fast_local_barrier30
0x1F8	fast_local_barrier31	fast_local_barrier31
0x2F8	icache_uprefetch	icache_uprefetch

ipi_redirect_trigger register

- Absolute Address: 0x80340080
- Base Offset: 0x80
- Size: 0x8

Bits	Identifier	Access	Reset	Name
15:0	ipi_redirect_trigger	rw	—	—

ipi_redirect_trigger field User-level software writes a bitmask to this register to request an IPI redirect to all harts indicated by the mask. Before delivery, the bitmask is unconditionally ANDed with the value in IPI_REDIRECT_FILTER. The register is encoded as follows: Bit 0: Minion 0, hart 0 Bit 1: Minion 0, hart 1 Bit 2: Minion 1, hart 0 Bit 3: Minion 1, hart 1 ... Bit 12: Minion 6, hart 0 Bit 13: Minion 6, hart 1 Bit 14: Minion 7, hart 0 Bit 15: Minion 7, hart 1 When a redirect IPI is delivered to a hart, that hart must satisfy two conditions, It must be executing in user mode. It must be capable of blocking on a stall CSR read instruction.If either condition is violated, the hart must take a trap into M-mode with, mcause set to Bad IPI redirect interrupt (5'h10), mtval set to the redirection PC. Reading this register returns a 0x0 value

CREDINC0 register

- Absolute Address: 0x803400C0
- Base Offset: 0xC0
- Size: 0x8

Bits	Identifier	Access	Reset	Name
7:0	mask0	r	—	—



mask0 field Writing this register increments the COUNTER0 credit counter for the first hart of all the cores selected in MASK0. Reading this register returns a value of 0.

CREDINC1 register

- Absolute Address: 0x803400C8
- Base Offset: 0xC8
- Size: 0x8

Bits Identifier Access Reset Name				
7:0	mask1	r	—	—

mask1 field Writing this register increments the COUNTER1 credit counter for the first hart of all the cores selected in MASK1.

CREDINC2 register

- Absolute Address: 0x803400D0
- Base Offset: 0xD0
- Size: 0x8

Bits Identifier Access Reset Name				
7:0	mask2	r	—	—

mask2 field Writing this register increments the COUNTER2 credit counter for the first hart of all the cores selected in MASK2. Reading this register returns a value of 0.

CREDINC3 register

- Absolute Address: 0x803400D8
- Base Offset: 0xD8
- Size: 0x8



Bits	Identifier	Access	Reset	Name
7:0	mask3	r	—	—

mask3 field Writing this register increments the COUNTER3 credit counter for the first hart of all the cores selected in MASK3. Reading this register returns a value of 0.

fast_local_barrier0 register

- Absolute Address: 0x80340100
- Base Offset: 0x100
- Size: 0x8

Bits	Identifier	Access	Reset	Name
7:0	fast_local_barrier0	rw	0x0	—

fast_local_barrier0 field The CORE-ET Fast Local Barrier (FLB) extension registers provide fast barrier capabilities across the Minions within a Shire. Multiple barrier counters allow a subset of threads in a Shire to atomically modify a barrier counter and determine whether all participating threads have reached the barrier. The FLB extension provides 32 barrier counters, each 8 bits wide, supporting an all-thread (64 threads) barrier in the Shire. Barrier counter *i* should be initialized by software before starting the barrier operation using a regular RISC-V store instruction to address (address + *i* * 8). Software typically initializes barrier counters to zero, but any value from 0 to 255 is legal. For maximum performance, FLB counters are directly accessible by threads in the Shire using CSRRW instructions. Threads participating in a barrier use CSRRW to atomically increment the barrier counter. If a thread is the last to join the barrier, the CSRRW returns 0x1; otherwise, it returns 0x0.

fast_local_barrier1 register

- Absolute Address: 0x80340108
- Base Offset: 0x108
- Size: 0x8



Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier1	rw	0x0	—

fast_local_barrier1 field see description for fast_local_barrier_0

fast_local_barrier2 register

- Absolute Address: 0x80340110
- Base Offset: 0x110
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier2	rw	0x0	—

fast_local_barrier2 field see description for fast_local_barrier_0

fast_local_barrier3 register

- Absolute Address: 0x80340118
- Base Offset: 0x118
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier3	rw	0x0	—

fast_local_barrier3 field see description for fast_local_barrier_0

fast_local_barrier4 register

- Absolute Address: 0x80340120
- Base Offset: 0x120
- Size: 0x8



CONTENTS

Bits Identifier	Access	Reset	Name
7:0	fast_local_barrier4	rw	0x0 —

fast_local_barrier4 field see description for fast_local_barrier_0

fast_local_barrier5 register

- Absolute Address: 0x80340128
- Base Offset: 0x128
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0	fast_local_barrier5	rw	0x0 —

fast_local_barrier5 field see description for fast_local_barrier_0

fast_local_barrier6 register

- Absolute Address: 0x80340130
- Base Offset: 0x130
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0	fast_local_barrier6	rw	0x0 —

fast_local_barrier6 field see description for fast_local_barrier_0

fast_local_barrier7 register

- Absolute Address: 0x80340138
- Base Offset: 0x138
- Size: 0x8



Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier7	rw	0x0	—

fast_local_barrier7 field see description for fast_local_barrier_0

fast_local_barrier8 register

- Absolute Address: 0x80340140
- Base Offset: 0x140
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier8	rw	0x0	—

fast_local_barrier8 field see description for fast_local_barrier_0

fast_local_barrier9 register

- Absolute Address: 0x80340148
- Base Offset: 0x148
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier9	rw	0x0	—

fast_local_barrier9 field see description for fast_local_barrier_0

fast_local_barrier10 register

- Absolute Address: 0x80340150
- Base Offset: 0x150
- Size: 0x8



Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier10	rw	0x0	—

fast_local_barrier10 field see description for fast_local_barrier_0

fast_local_barrier11 register

- Absolute Address: 0x80340158
- Base Offset: 0x158
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier11	rw	0x0	—

fast_local_barrier11 field see description for fast_local_barrier_0

fast_local_barrier12 register

- Absolute Address: 0x80340160
- Base Offset: 0x160
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier12	rw	0x0	—

fast_local_barrier12 field see description for fast_local_barrier_0

fast_local_barrier13 register

- Absolute Address: 0x80340168
- Base Offset: 0x168
- Size: 0x8



Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier13	rw	0x0	—

fast_local_barrier13 field see description for fast_local_barrier_0

fast_local_barrier14 register

- Absolute Address: 0x80340170
- Base Offset: 0x170
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier14	rw	0x0	—

fast_local_barrier14 field see description for fast_local_barrier_0

fast_local_barrier15 register

- Absolute Address: 0x80340178
- Base Offset: 0x178
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier15	rw	0x0	—

fast_local_barrier15 field see description for fast_local_barrier_0

fast_local_barrier16 register

- Absolute Address: 0x80340180
- Base Offset: 0x180
- Size: 0x8



CONTENTS

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier16	rw	0x0	—

fast_local_barrier16 field see description for fast_local_barrier_0

fast_local_barrier17 register

- Absolute Address: 0x80340188
- Base Offset: 0x188
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier17	rw	0x0	—

fast_local_barrier17 field see description for fast_local_barrier_0

fast_local_barrier18 register

- Absolute Address: 0x80340190
- Base Offset: 0x190
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier18	rw	0x0	—

fast_local_barrier18 field see description for fast_local_barrier_0

fast_local_barrier19 register

- Absolute Address: 0x80340198
- Base Offset: 0x198
- Size: 0x8



Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier19	rw	0x0	—

fast_local_barrier19 field see description for fast_local_barrier_0

fast_local_barrier20 register

- Absolute Address: 0x803401A0
- Base Offset: 0x1A0
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier20	rw	0x0	—

fast_local_barrier20 field see description for fast_local_barrier_0

fast_local_barrier21 register

- Absolute Address: 0x803401A8
- Base Offset: 0x1A8
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier21	rw	0x0	—

fast_local_barrier21 field see description for fast_local_barrier_0

fast_local_barrier22 register

- Absolute Address: 0x803401B0
- Base Offset: 0x1B0
- Size: 0x8



Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier22	rw	0x0	—

fast_local_barrier22 field see description for fast_local_barrier_0

fast_local_barrier23 register

- Absolute Address: 0x803401B8
- Base Offset: 0x1B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier23	rw	0x0	—

fast_local_barrier23 field see description for fast_local_barrier_0

fast_local_barrier24 register

- Absolute Address: 0x803401C0
- Base Offset: 0x1C0
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier24	rw	0x0	—

fast_local_barrier24 field see description for fast_local_barrier_0

fast_local_barrier25 register

- Absolute Address: 0x803401C8
- Base Offset: 0x1C8
- Size: 0x8



Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier25	rw	0x0	—

fast_local_barrier25 field see description for fast_local_barrier_0

fast_local_barrier26 register

- Absolute Address: 0x803401D0
- Base Offset: 0x1D0
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier26	rw	0x0	—

fast_local_barrier26 field see description for fast_local_barrier_0

fast_local_barrier27 register

- Absolute Address: 0x803401D8
- Base Offset: 0x1D8
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier27	rw	0x0	—

fast_local_barrier27 field see description for fast_local_barrier_0

fast_local_barrier28 register

- Absolute Address: 0x803401E0
- Base Offset: 0x1E0
- Size: 0x8



Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier28	rw	0x0	—

fast_local_barrier28 field see description for fast_local_barrier_0

fast_local_barrier29 register

- Absolute Address: 0x803401E8
- Base Offset: 0x1E8
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier29	rw	0x0	—

fast_local_barrier29 field see description for fast_local_barrier_0

fast_local_barrier30 register

- Absolute Address: 0x803401F0
- Base Offset: 0x1F0
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier30	rw	0x0	—

fast_local_barrier30 field see description for fast_local_barrier_0

fast_local_barrier31 register

- Absolute Address: 0x803401F8
- Base Offset: 0x1F8
- Size: 0x8



Bits Identifier		Access Reset Name	
7:0	fast_local_barrier31	rw	0x0 —

fast_local_barrier31 field see description for fast_local_barrier_0

icache_uprefetch register

- Absolute Address: 0x803402F8
- Base Offset: 0x2F8
- Size: 0x8

If no prefetch with a higher privilege mode associated with it is in progress, a write in this ESR prefetches Count+1 consecutive lines starting at address Address to the instruction cache. The privilege mode associated with this operation is User-mode, and all memory accesses are performed with User-mode privileges. If any of the memory requests generated by the prefetch engine generates an exception, the line is not prefetched and the operation continues with the next line. Reading this register returns the idle status of the prefetch engine.

Bits Identifier		Access Reset Name	
5:0	count	rw	0x0 —
47:6	address	rw	— —

count field Number of lines to be prefetched (minus 1) starting at the given Address.

address field Bits [47:6] of the address where prefetching should start from (aligned to a cacheline).

Supervisor_cpu address map

- Absolute Address: 0x80740000
- Base Offset: 0x740000
- Size: 0x308



Offset Identifier	Name
0x290 shire_coop_mode	shire_coop_mode
0x300 icache_sprefetch	icache_sprefetch

shire_coop_mode register

- Absolute Address: 0x80740290
- Base Offset: 0x290
- Size: 0x8

Bits Identifier	Access	Reset	Name
0 coop_mode	rw	0x0	—

coop_mode field Software can set this bit to enable cooperative prefetches, loads, and stores.

icache_sprefetch register

- Absolute Address: 0x80740300
- Base Offset: 0x300
- Size: 0x8

If no prefetch with a higher privilege mode associated with it is in progress, a write in this ESR prefetches Count+1 consecutive lines starting at address Address to the instruction caches. The privilege mode associated with this operation is Supervisor-mode, and all memory accesses are performed with Supervisor-mode privileges. If any of the memory requests generated by the prefetch engine generates an exception, the line is not prefetched and the operation continues with the next line. Reading this register returns the idle status of the prefetch engine.

Bits Identifier	Access	Reset	Name
5:0 count	rw	0x0	—
47:6 address	rw	—	—

count field Number of lines to be prefetched (minus 1) starting at the given Address.



address field Bits [47:6] of the address where prefetching should start from (aligned to a cacheline).

D_hart__esr address map

- Absolute Address: 0x80800000
- Base Offset: 0x800000
- Size: 0x10000

These registers are per hart. ([15:12] bits) in the address indicates hart_id. bits[15:13] minion id and bit [12] is thread id.

Offset	Identifier	Name
0x0000	dbg__hart[0]	—
0x1000	dbg__hart[1]	—
0x2000	dbg__hart[2]	—
0x3000	dbg__hart[3]	—
0x4000	dbg__hart[4]	—
0x5000	dbg__hart[5]	—
0x6000	dbg__hart[6]	—
0x7000	dbg__hart[7]	—
0x8000	dbg__hart[8]	—
0x9000	dbg__hart[9]	—
0xA000	dbg__hart[10]	—
0xB000	dbg__hart[11]	—
0xC000	dbg__hart[12]	—
0xD000	dbg__hart[13]	—
0xE000	dbg__hart[14]	—
0xF000	dbg__hart[15]	—

dbg__hart register file

- Absolute Address: 0x80800000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x80800780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x80800788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x80800790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x80800798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x808007A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuff0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x808007A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuff1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x808007B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x808007B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x808007C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x80801000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x80801780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x80801788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x80801790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x80801798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x808017A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuf0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x808017A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuf1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x808017B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x808017B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x808017C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x80802000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x80802780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x80802788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0

**NXDATA1_f1 field** WIRI**AXDATA0 register**

- Absolute Address: 0x80802790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x80802798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x808027A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuf0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x808027A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuf1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x808027B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x808027B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x808027C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x80803000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



CONTENTS

Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x80803780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x80803788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x80803790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x80803798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x808037A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuf0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x808037A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuf1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x808037B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x808037B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x808037C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x80804000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x80804780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x80804788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x80804790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x80804798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x808047A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuf0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x808047A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuf1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x808047B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x808047B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x808047C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x80805000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x80805780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x80805788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x80805790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x80805798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x808057A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuf0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x808057A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuf1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x808057B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x808057B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x808057C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x80806000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x80806780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x80806788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x80806790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x80806798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x808067A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuf0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x808067A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuf1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x808067B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x808067B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x808067C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x80807000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x80807780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x80807788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x80807790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x80807798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x808077A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuf0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x808077A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuf1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x808077B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x808077B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x808077C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x80808000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x80808780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x80808788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x80808790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x80808798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x808087A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuf0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x808087A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuf1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x808087B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x808087B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x808087C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x80809000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x80809780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x80809788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x80809790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x80809798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x808097A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuf0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x808097A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuf1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x808097B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x808097B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x808097C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x8080A000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x8080A780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x8080A788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x8080A790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x8080A798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x8080A7A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuff0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x8080A7A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuff1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x8080A7B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x8080A7B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x8080A7C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x8080B000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x8080B780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x8080B788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x8080B790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x8080B798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x8080B7A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuff0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x8080B7A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuff1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x8080B7B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x8080B7B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x8080B7C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x8080C000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x8080C780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x8080C788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x8080C790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x8080C798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x8080C7A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuf0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x8080C7A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuf1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x8080C7B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x8080C7B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x8080C7C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x8080D000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x8080D780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x8080D788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x8080D790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x8080D798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x8080D7A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuf0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x8080D7A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuf1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x8080D7B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x8080D7B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x8080D7C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x8080E000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x8080E780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x8080E788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x8080E790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x8080E798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x8080E7A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset	Name
31:0 AXPROGBUFF0	rw	—	—

AXPROGBUFF0 field This register shadows the nxprogbuf0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x8080E7A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 AXPROGBUFF1	rw	—	—

AXPROGBUFF1 field This register shadows the nxprogbuf1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x8080E7B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF0	rw	—	—

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x8080E7B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x8080E7C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x8080F000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



CONTENTS

Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x8080F780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x8080F788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x8080F790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x8080F798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x8080F7A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuf0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x8080F7A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuf1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x8080F7B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x8080F7B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x8080F7C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

D_neigh address map

- Absolute Address: 0x80900000
- Base Offset: 0x900000
- Size: 0xFFA0



Offset	Identifier	Name
0xFF80	hactrl	hactrl
0xFF88	hastatus0	hastatus0
0xFF90	hastatus1	hastatus1
0xFF98	and_or_tree_IO	and_or_tree_IO

hactrl register

- Absolute Address: 0x8090FF80
- Base Offset: 0xFF80
- Size: 0x8

Bits	Identifier	Access	Reset	Name
15:0	hawindow	rw	0x0	—
31:16	hart_mask	rw	0x0	—
47:32	resethalt	r	0x0	—

hawindow field The n-th bit selects the n-th hart in the neighborhood, if hasel is 1. Non-selected harts ignore the reset (except ndmreset), halt and resume requests from the DM.

hart_mask field The n-th bit selects the n-th hart in the neighborhood (independent of the value of hasel). Non-selected harts ignore the reset (except ndmreset), halt and resume requests from the DM.

resethalt field Each bit in this field corresponds to one hart in the neighborhood. While a given resethalt bit is asserted, the corresponding hart will halt upon the next deassertion of its reset. These bits are set for all selected harts when the setresethaltreq bit in the RISC-V Debug Module Control register (DMCTRL) is written. This field is cleared for all selected harts when the clrresethaltreq bit in the DMCTRL register is written.

hastatus0 register

- Absolute Address: 0x8090FF88
- Base Offset: 0xFF88
- Size: 0x8



Bits	Identifier	Access	Reset	Name
15:0	halted	r	0x0	—
31:16	running	r	0x0	—
47:32	resumeack	r	0x0	—
63:48	havereset	r	0x0	—

halted field Each bit in this field corresponds to one of the 16 harts in the neighborhood. The sticky halted bit for a hart is asserted when the hart is in Debug Mode, and hardware clears this bit when the hart receives a reset or resume request.

running field Each bit in this field corresponds to one of the 16 harts in the neighborhood. The sticky running bit for each hart is asserted when a hart finishes resuming execution, and hardware clears this bit when a hart receives a reset or halt request.

resumeack field Each bit in this field corresponds to one of the 16 harts in the neighborhood. The sticky resumeack bit for a hart is asserted when a hart finishes resuming execution, and hardware clears this bit whenever the resumereq bit in the RISC-V Debug Module Control register (DMCTRL) is deasserted.

havereset field Each bit in this field corresponds to one of the 16 harts in the neighborhood. The sticky havereset bit for a hart is asserted when a hart exits reset, and hardware clears this bit whenever the ackhavereset bit in the RISC-V Debug Module Control register (DMCTRL) is written. Each ET-Minion core contains two harts; however, it is not possible to reset an individual hart within an ET-Minion core. Therefore, the havereset field can be implemented as eight physical bits, with bits $2m$ and $2m+1$ corresponding to the m -th Minion hardwired to the same value. For example, bits 0 and 1 correspond to the two harts in Minion core 0, but only one of these bits is required to reset both threads. Similarly, bits 2 and 3 correspond to ET-Minion core 1, and so on

hastatus1 register

- Absolute Address: 0x8090FF90
- Base Offset: 0xFF90



- Size: 0x8

Bits	Identifier	Access	Reset	Name
15:0	busy	r	0x0	—
31:16	exception	rw	0x0	—
47:32	error	rw	0x0	—

busy field Each bit in this field corresponds to one of the 16 harts in the neighborhood, and a bit is set to 1 while the corresponding hart executes an Abstract Command or a Program Buffer program.

exception field Each bit in this field corresponds to one of the 16 harts in the neighborhood. A bit is set to 1 if the execution of an Abstract Command or a Program Buffer program by the corresponding hart generates an exception. These bits are sticky and must be cleared using an explicit write to the register.

error field Each bit in this field corresponds to one of the 16 harts in the neighborhood. A bit is set to 1 if the execution of an Abstract Command or a Program Buffer program by the corresponding hart cannot complete, but no exception is generated. Examples of this include writes to ET-Minion core registers such as `abscmd`, `axprogbuf0`, `axprogbuf1`, `nxprogbuf0`, `nxprogbuf1`, and so on. These bits are sticky and must be cleared using an explicit write to the register.

and_or_tree_IO register

- Absolute Address: 0x8090FF98
- Base Offset: 0xFF98
- Size: 0x8

Bits	Identifier	Access	Reset	Name
0	anyhalted	r	—	—
1	allhalted	r	—	—
2	anyrunning	r	—	—
3	allrunning	r	—	—
4	anyresumeack	r	—	—
5	allresumeack	r	—	—



Bits Identifier		Access Reset Name		
6	anyhavereset	r	—	—
7	allhavereset	r	—	—
8	anyunavailable	r	—	—
9	anyselected	r	—	—

anyhalted field Hardware sets this bit whenever any of the 16 bits of the halted field in the CORE-ET Neighborhood Hart Status 0 register for the selected harts is set, and the hasel bit in the RISC-V Debug Module Control register (DMCTRL) is 1.

allhalted field Hardware sets this bit when all 16 bits of the halted field in the CORE-ET Neighborhood Hart Status 0 register for the selected harts are set, and the hasel bit in the RISC-V Debug Module Control register (DMCTRL) is 1.

anyrunning field Hardware sets this bit whenever any of the 16 bits of the running field in the CORE-ET Neighborhood Hart Status 0 register for the selected harts is set, and the hasel bit in the RISC-V Debug Module Control register (DMCTRL) is 1.

allrunning field Hardware sets this bit whenever all 16 bits of the running field in the CORE-ET Neighborhood Hart Status 0 register for the selected harts are set, and the hasel bit in the RISC-V Debug Module Control register (DMCTRL) is 1.

anyresumeack field Hardware sets this bit whenever any of the 16 bits of the resumeack field in the CORE-ET Neighborhood Hart Status 0 register for the selected harts is set, and the hasel bit in the RISC-V Debug Module Control register (DMCTRL) is 1.

allresumeack field Hardware sets this bit whenever any of the 16 bits of the resumeack field in the CORE-ET Neighborhood Hart Status 0 register for the selected harts is set, and the hasel bit in the RISC-V Debug Module Control register (DMCTRL) is 1.



anyhavereset field Hardware sets this bit whenever any 16 bits of the havereset field in the CORE-ET Neighborhood Hart Status 0 register for the selected harts are set, and the hasel bit in the RISC-V Debug Module Control register (DMCTRL) is 1.

allhavereset field Hardware sets this bit whenever all of the 16 bits of the havereset field in the CORE-ET Neighborhood Hart Status 0 register for the selected harts is set, and the hasel bit in the RISC-V Debug Module Control register (DMCTRL) is 1.

anyunavailable field Hardware sets this bit when any of the selected harts in the neighborhood are unavailable, and the hasel bit in the RISC-V Debug Module Control register (DMCTRL) is 1. A selected hart is considered unavailable if both HASTATUS0.halted[n] and HASTATUS0.running[n] are 0.

anyselected field Hardware sets this bit when any of the harts in the neighborhood are selected, and the hasel bit in the RISC-V Debug Module Control register (DMCTRL) is 1.

D_cpu address map

- Absolute Address: 0x80B5F000
- Base Offset: 0xB5F000
- Size: 0xFD8

Offset	Identifier	Name
0xF88	dmctrl	dmctrl
0xF90	sm_config	sm_config
0xF98	sm_trigger	sm_trigger
0xFA8	sm_match	sm_match
0xFB0	sm_filter0	sm_filter0
0xFB8	sm_filter1	sm_filter1
0xFC0	sm_filter2	sm_filter2
0xFC8	sm_data0	sm_data0
0xFD0	sm_data1	sm_data1

**dmctrl register**

- Absolute Address: 0x80B5FF88
- Base Offset: 0xF88
- Size: 0x8

Bits	Identifier	Access	Reset	Name
0	dmactive	rw	0x0	—
1	ndmreset	rw	—	—
2	clrresethaltreq	rw	—	—
3	setresethaltreq	rw	—	—
25:4	reserved25_4	r	—	—
26	hasel	rw	—	—
27	reserved27	r	—	—
28	ackhavereset	rw	—	—
29	hartreset	rw	—	—
30	resumereq	rw	—	—
31	haltreq	rw	—	—

dmactive field This bit controls the dmactive signal from the DM; it is broadcast to all Minion shires. When the signal is low it should hold at reset the debug logic of the receiving harts, and of the DM. After writing 1, the debugger should poll DMCTRL until dmactive is high. Hardware may take an arbitrarily long time to initialize and will indicate completion by setting dmactive to 1

ndmreset field This bit controls the ndmreset signal from the DM; it is broadcast to all Minion shires. The signal should reset every part of the receiving shire, except for any logic required for debug.

clrresethaltreq field This bit controls the clrresethaltreq signal from the DM; it is broadcast to all Minion shires. Writing 1 to this bit clears the HACTRL.resethalt bits of all currently selected harts. This bit always reads as 0.

setresethaltreq field This bit controls the setresethaltreq signal from the DM; it is broadcast to all Minion shires. Writing 1 to this bit sets the HACTRL.resethalt bits of all currently selected harts, unless clrresethaltreq is simultaneously set to 1. This bit always reads as 0.



reserved25_4 field WARL(0)

hasel field This bit controls the hasel signal from the DM; it is broadcast to all Minion shires. If set to 0, then DM requests, such as halt and resume apply only to the harts that are selected by HACTRL.hartmask. If set to 1, then DM requests apply to all harts selected by the union of HACTRL.hartmask and HACTRL.hawindow.

reserved27 field WARL(0)

ackhavereset field This bit controls the ackhavereset signal from the DM; it is broadcast to all Minion shires. Writing 1 to this bit clears HASTATUS0.havereset for all currently selected harts. This bit always reads as 0.

hartreset field This bit controls the hartreset signal from the DM; it is broadcast to all Minion shires.

resumereq field This bit controls the resumereq signal from the DM; it is broadcast to all Minion shires. Writing 1 to this bit causes the currently selected harts to resume and clears the HASTATUS0.resumeack bit for those harts. This bit always reads as 0.

haltreq field Bits [31:31]: This bit controls the haltreq signal from the DM; it is broadcast to all Minion shires. Writing 1 to this bit sets the halt request bit for all currently selected harts. Writing 0 to this bit clears the halt request bit for all currently selected harts. This may cancel outstanding halt requests for those harts. This bit always reads as 0.

sm_config register

- Absolute Address: 0x80B5FF90
- Base Offset: 0xF90
- Size: 0x8

Bits Identifier		Access		Reset Name	
6:0	sm_data_sel	rw	0x0	—	—
10:7	sm_target_sel	rw	—	—	—
11	sm_enable	rw	—	—	—



sm_data_sel field Data selector within each target

sm_target_sel field Target selector:

sm_enable field Status Monitor logic enable.

sm_trigger register

- Absolute Address: 0x80B5FF98
- Base Offset: 0xF98
- Size: 0x8

Bits	Identifier	Access	Reset	Name
0	sm_trigger	r	—	—

sm_trigger field Writing this bit triggers a simultaneous snapshot captured in all the SM ESRs

sm_match register

- Absolute Address: 0x80B5FFA8
- Base Offset: 0xFA8
- Size: 0x8

Bits	Identifier	Access	Reset	Name
63:0	sm_match	r	—	—

sm_match field Match bus

sm_filter0 register

- Absolute Address: 0x80B5FFB0
- Base Offset: 0xFB0
- Size: 0x8



CONTENTS

Bits	Identifier	Access	Reset	Name
63:0	sm_filter0	r	—	—

sm_filter0 field Filter bus [63:0]

sm_filter1 register

- Absolute Address: 0x80B5FFB8
- Base Offset: 0xFB8
- Size: 0x8

Bits	Identifier	Access	Reset	Name
63:0	sm_filter1	r	—	—

sm_filter1 field Filter bus [127:64]

sm_filter2 register

- Absolute Address: 0x80B5FFC0
- Base Offset: 0xFC0
- Size: 0x8

Bits	Identifier	Access	Reset	Name
63:0	sm_filter2	r	—	—

sm_filter2 field Filter bus [191:128]

sm_data0 register

- Absolute Address: 0x80B5FFC8
- Base Offset: 0xFC8
- Size: 0x8



Bits	Identifier	Access	Reset	Name
63:0	sm_data0	r	—	—

sm_data0 field Data bus [63:0]

sm_data1 register

- Absolute Address: 0x80B5FFD0
- Base Offset: 0xFD0
- Size: 0x8

Bits	Identifier	Access	Reset	Name
63:0	sm_data1	r	—	—

sm_data1 field Data bus [127:64]

Machine_neigh address map

- Absolute Address: 0x80D00000
- Base Offset: 0xD00000
- Size: 0x98

Offset	Identifier	Name
0x18	minion_boot	minion_boot
0x38	mprot	mprot
0x68	pmu_ctrl	pmu_ctrl
0x70	neigh_chicken	neigh_chicken
0x78	icache_err_log_ctl	icache_err_log_ctl
0x80	icache_err_log_info	icache_err_log_info
0x88	icache_err_log_address	icache_err_log_address
0x90	icache_sbe_dbe_counts	icache_sbe_dbe_counts

minion_boot register

- Absolute Address: 0x80D00018
- Base Offset: 0x18



- Size: 0x8

Bits Identifier	Access	Reset	Name
47:0	minion_boot	rw	0x2008000 —

minion_boot field Each neighborhood contains a MINION_BOOT ESR that determines the initial PC of the Minions in the neighborhood after reset. This register holds the 48 least significant bits of the reset PC, bits 63:48 of the PC are ignored on writes. Software must make sure that fetching from the PC value written in this register will not generate any faults.

mprot register

- Absolute Address: 0x80D00038
- Base Offset: 0x38
- Size: 0x8

Bits Identifier	Access	Reset	Name
3:0	smode_size	rw	0x0 —
7:4	mmode_size	rw	0x0 —
8	mprot_en	rw	0x0 —

smode_size field Size of the MRAM region which can be only accessed by Smode. The size is $4KB(2^{\text{SMODE_SIZE}}) - 4KB(2^{\text{MMODE_SIZE}})$ starting at offset $4KB \cdot (2^{\text{MMODE_SIZE}})$ in the MRAM.

mmode_size field Size of the MRAM region which can be only accessed by Mmode. The size is $4KB \cdot (2^{\text{MMODE_SIZE}})$ starting at offset 0 in the MRAM.

mprot_en field Enable privilege based memory protection.

pmu_ctrl register

- Absolute Address: 0x80D00068
- Base Offset: 0x68
- Size: 0x8



Bits Identifier		Access		Reset	Name
0	disable_clock	rw	0x0	—	

disable_clock field Set this bit to disable the clock

neigh_chicken register

- Absolute Address: 0x80D00070
- Base Offset: 0x70
- Size: 0x8

Bits Identifier		Access		Reset	Name
0	bypass_icache	rw	0x0	—	
1	bypass_dcache	rw	0x0	—	
2	force_dest_fifo	rw	0x0	—	
4:3	dest_fifo	rw	0x0	—	
5	force_all_agents	rw	0x0	—	
6	agent_forced	rw	0x0	—	

bypass_icache field Setting this bit bypasses the instruction cache.

bypass_dcache field Setting this bit bypasses the data cache.

force_dest_fifo field If enabled, all traffic from agents specified by the force_all_agents and agent_forced fields in the neighborhood will be routed to the destination specified in the dest_fifo field.

dest_fifo field Destination to which requests are routed when the force_dest_fifo bit of this register is set:

force_all_agents field When the force_dest_fifo bit and this bit are set, requests from all agents in the neighborhood go to the FIFO selected by the dest_fifo field.



agent_forced field When the force_dest_fifo bit is set and the force_all_agents bit is clear, this field selects agents from which requests will be forced to go to the FIFO selected by the dest_fifo field. 0: Requests are routed to the corresponding bank FIFO, according to the destination address. 1: Requests are routed to the UC FIFO. 2-3: Reserved

icache_err_log_ctl register

- Absolute Address: 0x80D00078
- Base Offset: 0x78
- Size: 0x8

Bits Identifier		Access Reset Name	
2:0	err_interrupt_enable	rw	0x6 —

err_interrupt_enable field This 3-bit field is used to enable or prevent certain categories of logged errors from generating interrupts or sending error responses. In general, single-bit ECC errors should not generate an interrupt. Software may choose whether the ECC error counter saturation should generate an interrupt. Each bit has a specific meaning: Bit 0 enables single-bit ECC errors, Bit 1 enables double-bit ECC errors, and Bit 2 enables ECC counter saturation. Error codes that are not enabled are still logged but have lower priority than errors that generate interrupts. Masked interrupts do not prevent subsequent unmasked errors from being recorded. When an enabled interrupt error overwrites a masked error, the multiple error bit is not set, and multiple masked errors also do not cause the multiple error bit to be set. The multiple error bit is intended to indicate that a fatal error was missed.

icache_err_log_info register

- Absolute Address: 0x80D00080
- Base Offset: 0x80
- Size: 0x8

Bits Identifier		Access Reset Name	
0	valid	rw	0x0 —
1	multi	rw	0x0 —



Bits	Identifier	Access	Reset	Name
2	enabled	rw	0x0	—
3	reserved	r	—	—
7:4	err_code	rw	0x0	—
63:8	info	rw	—	—

valid field When set, this bit indicates an error has occurred and that the register contents are valid. This can be cleared by writing a 1 to this field and writing the matching code to `err_code`.

multi field When set, this bit indicates that multiple errors have occurred. When an error is detected, the `esr_icache_err_log_info` register records the details of the first error encountered. Any subsequent errors set this bit to indicate that another fatal error occurred, but their details are not recorded.

enabled field The error detected was enabled by `esr_icache_err_log_ctl.err_interrupt_enable`. Error codes that are not enabled are still logged, but they have lower priority than errors that generate interrupts.

err_code field This 4-bit field indicates the type of error. This field is used to determine how to decode the info bits in positions 63:8. 0: ECC single-bit error 1: ECC double-bit error 2: ECC error counter saturation

info field Additional information about the error. The format of this field is dependent upon the type of error. SBE_DBE format: - Bits 14:8: set. This field indicates the way that contained the error - Bits 16:15: way. This field indicates the way that contained the error - Bits 47:40: error_bits. Indicates which doubleword contained the error. The error_bits field is multi-hot. ECC_Saturation format: - Bit 52: double. This bit indicates the type of error counter that saturated and is encoded as follows: 0: The single-bit error counter saturated. 1: The double-bit error counter saturated.

icache_err_log_address register

- Absolute Address: 0x80D00088
- Base Offset: 0x88
- Size: 0x8



Bits Identifier	Access	Reset	Name
33:0 icache_err_log_address_f0 r	0x0	—	

icache_err_log_address_f0 field This register contains the physical address (PA) associated with the error if it is available.

icache_sbe_dbe_counts register

- Absolute Address: 0x80D00090
- Base Offset: 0x90
- Size: 0x8

Single-bit errors (SBEs) are corrected so normal processing can continue, and no error response is sent to the requesting core because the request was processed correctly. SBEs should be logged locally, and no action should be taken. Double-bit errors (DBEs) cannot be corrected and are considered fatal, requiring immediate reporting. SBE and DBE responses occur once per ECC protection, meaning one error bit per dword in the cache line (8 bits). The SBE counter is 8 bits and saturates at 255, while the DBE counter is 3 bits and saturates at 7. All fields of this register are read-only. Writing all ones to a field clears its respective counter. If a counter saturates, it can be error-logged as described in the icache_err_log_info register.

Bits Identifier	Access	Reset	Name
7:0 sbe_counts rw	0x0	—	
10:8 db_e_counts rw	—	—	

sbe_counts field This field stores the data RAM single-bit error count.

db_e_counts field This field stores the data RAM double-bit error count.

Machine_cpu address map

- Absolute Address: 0x80F40000
- Base Offset: 0xF40000
- Size: 0x318



Offset Identifier	Name
0x000 minion_feature	minion_feature
0x010 thread1_disable	thread1_disable
0x088 ipi_redirect_filter	ipi_redirect_filter
0x090 ipi_trigger	ipi_trigger
0x098 ipi_trigger_clear	ipi_trigger_clear
0x200 mtime	mtime
0x208 mtime_cmp	mtime_cmp
0x210 time_config	time_config
0x218 mtime_local_target	mtime_local_target
0x240 thread0_disable	thread0_disable
0x308 icache_mprefetch	icache_mprefetch
0x310 clk_gate_ctrl	clk_gate_ctrl

minion_feature register

- Absolute Address: 0x80F40000
- Base Offset: 0x0
- Size: 0x8

Bits Identifier	Access	Reset	Name
0 trap_on_gfx	rw	0x1	—
1 trap_on_ml	rw	0x0	—
2 trap_on_u_cacheops	rw	0x0	—
3 trap_on_u_scp	rw	0x0	—
4 disable_multithreading	rw	0x0	—
5 disable_lock_unlock	rw	0x0	—

trap_on_gfx field Set this bit to trap on GFX. When this bit is enabled, GFX instructions generate an illegal instruction exception.

trap_on_ml field Set this bit to trap on ML. When this bit is set, tensor pseudo-instructions generate an illegal instruction exception.

trap_on_u_cacheops field Setting this bit disables user cacheops. If set, user mode will not have access to the cache op pseudo-instructions.



trap_on_u_scp field Setting this bit disables the user scratchpad. When set, user mode cannot enable or disable the scratchpad.

disable_multithreading field Set this bit to disable multithreading, making only thread 0 available. The hart IDs for all thread 0s in a Shire must be contiguous.

disable_lock_unlock field Set this bit to disable lock/unlock. When set, user mode cannot use the LockVA and UnlockVA operations.

thread1_disable register

- Absolute Address: 0x80F40010
- Base Offset: 0x10
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0	thread1_disable_f0	rw	0xFF —

thread1_disable_f0 field Each bit of this register corresponds to thread 1 of each Minion core in the Neighborhood. When a bit is set, thread 1 of the corresponding Minion is disabled. The field is encoded as follows: Bit 0: Minion 0, thread 1 Bit 1: Minion 1, thread 1 Bit 2: Minion 2, thread 1 ... Bit 7: Minion 7, thread 1

ipi_redirect_filter register

- Absolute Address: 0x80F40088
- Base Offset: 0x88
- Size: 0x8

Bits Identifier	Access	Reset	Name
15:0	ipi_redirect_filter	rw	0xFFFF —



ipi_redirect_filter field ipi_redirect_filter Writing a 16-bit value to this register sets the hart ID filter mask for subsequent writes to the IPI_REDIRECT_TRIGGER register. This register is used by privileged software (M-Mode) to prevent accidental user-level redirections. Its value is ANDed with the IPI_REDIRECT_TRIGGER ESR before sending any user-level redirections; setting this register to 0x0 disables all user-level redirections. The encoding of this register is as follows: Bit 0: Minion 0, hart 0 Bit 1: Minion 0, hart 1 Bit 2: Minion 1, hart 0 Bit 3: Minion 1, hart 1 ... Bit 12: Minion 6, hart 0 Bit 13: Minion 6, hart 1 Bit 14: Minion 7, hart 0 Bit 15: Minion 7, hart 1

ipi_trigger register

- Absolute Address: 0x80F40090
- Base Offset: 0x90
- Size: 0x8

Bits Identifier	Access	Reset	Name
15:0 ipi_trigger	rw	0x0	—

ipi_trigger field Machine-level software writes a a bit-mask into this register to set the MSIP bit of one or more harts in the Neighborhood. The encoding of this register is as follows: Bit 0: Minion 0, hart 0 Bit 1: Minion 0, hart 1 Bit 2: Minion 1, hart 0 Bit 3: Minion 1, hart 1 ... Bit 12: Minion 6, hart 0 Bit 13: Minion 6, hart 1 Bit 14: Minion 7, hart 0 Bit 15: Minion 7, hart 1 Reading this register reads the value of the MSIP bits of the harts in the Neighborhood. Writing a value with the i-th bit set to 0, will have no effect on the value of the MSIP bit of the corresponding hart.

ipi_trigger_clear register

- Absolute Address: 0x80F40098
- Base Offset: 0x98
- Size: 0x8

Bits Identifier	Access	Reset	Name
15:0 ipi_trigger_clear	r	0x0	—



ipi_trigger_clear field Machine-level software writes a bit-mask into this register to clear the MSIP bit of one or more harts in the Neighborhood. The encoding of this register is as follows: Bit 0: Minion 0, hart 0 Bit 1: Minion 0, hart 1 Bit 2: Minion 1, hart 0 Bit 3: Minion 1, hart 1 ... Bit 12: Minion 6, hart 0 Bit 13: Minion 6, hart 1 Bit 14: Minion 7, hart 0 Bit 15: Minion 7, hart 1 Reading this register returns a 0x0 value.

mtime register

- Absolute Address: 0x80F40200
- Base Offset: 0x200
- Size: 0x8

Bits	Identifier	Access	Reset	Name
63:0	mtime	rw	0x0	—

mtime field This register matches the definition on the RISC-V Privilege v1.1 specification. 64-bit timer counter. This timer increments in 100ns intervals. The mtimecmp register is compared to the value in this register. When the two are equal, a timer interrupt is generated.

mtime_cmp register

- Absolute Address: 0x80F40208
- Base Offset: 0x208
- Size: 0x8

Bits	Identifier	Access	Reset	Name
63:0	mtime_cmp	rw	0x0	—

mtime_cmp field This register matches the definition on the RISC-V Privilege v1.1 specification. 64-bit timer counter compare. Software sets the desired timer interval using this register. The value is then compared to the value in the mtime register. When the two are equal, a timer interrupt is generated.

**time_config register**

- Absolute Address: 0x80F40210
- Base Offset: 0x210
- Size: 0x8

Bits Identifier		Access	Reset	Name
0:6	prescalar_threshold	rw	0x28	—
7	ref_clock_mux	rw	0x0	—

prescalar_threshold field The value in this field will be used as the timer prescalar threshold.

ref_clock_mux field When this field holds 0 the timer is driven by the external reference clock. When this field holds 1 the timer is driven by the same clock that drives the cpu subsystem (external clock or ring oscillator)

mtime_local_target register

- Absolute Address: 0x80F40218
- Base Offset: 0x218
- Size: 0x8

Bits Identifier		Access	Reset	Name
15:0	mtime_local_target	rw	0xFFFF	—

mtime_local_target field This register controls which harts receive a timer interrupt. If bit N in mtime_local_target register is 1, hart N would be able to observe a timer interrupt in mip CSR if the mtimecmp is greater than mtime. If bit N is 0, the interrupt will be masked and not observable on hart N mip CSR.

thread0_disable register

- Absolute Address: 0x80F40240
- Base Offset: 0x240
- Size: 0x8



Bits Identifier	Access	Reset	Name
7:0	thread0_disable	rw	0xFE —

thread0_disable field Each bit of this register corresponds to thread 0 of each Minion core in the Neighborhood. When a bit is set, thread 0 of the corresponding Minion is disabled. The field is encoded as follows: Bit 0: Minion 0, thread 0 Bit 1: Minion 1, thread 0 Bit 2: Minion 2, thread 0 Bit 7: Minion 7, thread 0

icache__mprefetch register

- Absolute Address: 0x80F40308
- Base Offset: 0x308
- Size: 0x8

A write in this ESR prefetches Count+1 consecutive lines starting at address Address to the instruction cache. The privilege mode associated with this operation is Machine-mode, and all memory accesses are performed with Machine-mode privileges. If any of the memory requests generated by the prefetch engine generates an exception, the line is not prefetched and the operation continues with the next line. Reading this register returns the idle status of the prefetch engine.

Bits Identifier	Access	Reset	Name
5:0	count	rw	0x0 —
47:6	address	rw	0x0 —

count field Number of lines to be prefetched (minus 1) starting at the given Address.

address field Bits [47:6] of the address where prefetching should start from (aligned to a cacheline).

clk_gate_ctrl register

- Absolute Address: 0x80F40310
- Base Offset: 0x310
- Size: 0x8



Bits Identifier		Access Reset Name		
0	min_intpipe_clock_gate_disable	rw	0x0	—
1	min_vpulane_clock_gate_disable	rw	0x0	—
2	min_vputima_clock_gate_disable	rw	0x0	—
3	min_vputrans_clock_gate_disable	rw	0x0	—
4	min_dcache_clock_gate_disable	rw	—	—
5	reserved0	r	—	—
6	min_frontend_clock_gate_disable	rw	0x0	—
10:7	reserved1	r	—	—

min_intpipe_clock_gate_disable field .Setting this bit disables integer pipeline clock gating

min_vpulane_clock_gate_disable field Setting this bit disables VPU lane clock gating.

min_vputima_clock_gate_disable field Setting this bit disables VPUtima clock gating

min_vputrans_clock_gate_disable field Setting this bit disables VPU-trans clock gating

min_dcache_clock_gate_disable field Setting this bit disables data cache clock gating

reserved0 field Write as zero. Returns 0 on read.

min_frontend_clock_gate_disable field Setting this bit disables front end clock gating.

reserved1 field Write as zero. Returns 0 on read.

plic address map

- Absolute Address: 0xA0000000
- Base Offset: 0xA0000000
- Size: 0x21F008



RISC-V Platform-Level Interrupt Controller register map.

Offset	Identifier	Name
0x000004	priority_src01	—
0x000008	priority_src02	—
0x00000C	priority_src03	—
0x000010	priority_src04	—
0x000014	priority_src05	—
0x000018	priority_src06	—
0x001000	pending	—
0x002000	enable_ctx00	Interrupt Enable – Hart 0, M-mode
0x002080	enable_ctx01	Interrupt Enable – Hart 0, S-mode
0x002100	enable_ctx02	Interrupt Enable – Hart 1, M-mode
0x002180	enable_ctx03	Interrupt Enable – Hart 1, S-mode
0x002200	enable_ctx04	Interrupt Enable – Hart 2, M-mode
0x002280	enable_ctx05	Interrupt Enable – Hart 2, S-mode
0x002300	enable_ctx06	Interrupt Enable – Hart 3, M-mode
0x002380	enable_ctx07	Interrupt Enable – Hart 3, S-mode
0x002400	enable_ctx08	Interrupt Enable – Hart 4, M-mode
0x002480	enable_ctx09	Interrupt Enable – Hart 4, S-mode
0x002500	enable_ctx10	Interrupt Enable – Hart 5, M-mode
0x002580	enable_ctx11	Interrupt Enable – Hart 5, S-mode
0x002600	enable_ctx12	Interrupt Enable – Hart 6, M-mode
0x002680	enable_ctx13	Interrupt Enable – Hart 6, S-mode
0x002700	enable_ctx14	Interrupt Enable – Hart 7, M-mode
0x002780	enable_ctx15	Interrupt Enable – Hart 7, S-mode
0x002800	enable_ctx16	Interrupt Enable – Hart 8, M-mode
0x002880	enable_ctx17	Interrupt Enable – Hart 8, S-mode
0x002900	enable_ctx18	Interrupt Enable – Hart 9, M-mode
0x002980	enable_ctx19	Interrupt Enable – Hart 9, S-mode
0x002A00	enable_ctx20	Interrupt Enable – Hart 10, M-mode
0x002A80	enable_ctx21	Interrupt Enable – Hart 10, S-mode
0x002B00	enable_ctx22	Interrupt Enable – Hart 11, M-mode
0x002B80	enable_ctx23	Interrupt Enable – Hart 11, S-mode
0x002C00	enable_ctx24	Interrupt Enable – Hart 12, M-mode
0x002C80	enable_ctx25	Interrupt Enable – Hart 12, S-mode
0x002D00	enable_ctx26	Interrupt Enable – Hart 13, M-mode
0x002D80	enable_ctx27	Interrupt Enable – Hart 13, S-mode
0x002E00	enable_ctx28	Interrupt Enable – Hart 14, M-mode



Offset	Identifier	Name
0x002E80	enable_ctx29	Interrupt Enable – Hart 14, S-mode
0x002F00	enable_ctx30	Interrupt Enable – Hart 15, M-mode
0x002F80	enable_ctx31	Interrupt Enable – Hart 15, S-mode
0x200000	threshold_ctx00	Priority Threshold – Hart 0, M-mode
0x200004	claim_complete_ctx00	Claim/Complete – Hart 0, M-mode
0x201000	threshold_ctx01	Priority Threshold – Hart 0, S-mode
0x201004	claim_complete_ctx01	Claim/Complete – Hart 0, S-mode
0x202000	threshold_ctx02	Priority Threshold – Hart 1, M-mode
0x202004	claim_complete_ctx02	Claim/Complete – Hart 1, M-mode
0x203000	threshold_ctx03	Priority Threshold – Hart 1, S-mode
0x203004	claim_complete_ctx03	Claim/Complete – Hart 1, S-mode
0x204000	threshold_ctx04	Priority Threshold – Hart 2, M-mode
0x204004	claim_complete_ctx04	Claim/Complete – Hart 2, M-mode
0x205000	threshold_ctx05	Priority Threshold – Hart 2, S-mode
0x205004	claim_complete_ctx05	Claim/Complete – Hart 2, S-mode
0x206000	threshold_ctx06	Priority Threshold – Hart 3, M-mode
0x206004	claim_complete_ctx06	Claim/Complete – Hart 3, M-mode
0x207000	threshold_ctx07	Priority Threshold – Hart 3, S-mode
0x207004	claim_complete_ctx07	Claim/Complete – Hart 3, S-mode
0x208000	threshold_ctx08	Priority Threshold – Hart 4, M-mode
0x208004	claim_complete_ctx08	Claim/Complete – Hart 4, M-mode
0x209000	threshold_ctx09	Priority Threshold – Hart 4, S-mode
0x209004	claim_complete_ctx09	Claim/Complete – Hart 4, S-mode
0x20A000	threshold_ctx10	Priority Threshold – Hart 5, M-mode
0x20A004	claim_complete_ctx10	Claim/Complete – Hart 5, M-mode
0x20B000	threshold_ctx11	Priority Threshold – Hart 5, S-mode
0x20B004	claim_complete_ctx11	Claim/Complete – Hart 5, S-mode
0x20C000	threshold_ctx12	Priority Threshold – Hart 6, M-mode
0x20C004	claim_complete_ctx12	Claim/Complete – Hart 6, M-mode
0x20D000	threshold_ctx13	Priority Threshold – Hart 6, S-mode
0x20D004	claim_complete_ctx13	Claim/Complete – Hart 6, S-mode
0x20E000	threshold_ctx14	Priority Threshold – Hart 7, M-mode
0x20E004	claim_complete_ctx14	Claim/Complete – Hart 7, M-mode
0x20F000	threshold_ctx15	Priority Threshold – Hart 7, S-mode
0x20F004	claim_complete_ctx15	Claim/Complete – Hart 7, S-mode
0x210000	threshold_ctx16	Priority Threshold – Hart 8, M-mode
0x210004	claim_complete_ctx16	Claim/Complete – Hart 8, M-mode
0x211000	threshold_ctx17	Priority Threshold – Hart 8, S-mode



Offset	Identifier	Name
0x211004	claim_complete_ctx17	Claim/Complete – Hart 8, S-mode
0x212000	threshold_ctx18	Priority Threshold – Hart 9, M-mode
0x212004	claim_complete_ctx18	Claim/Complete – Hart 9, M-mode
0x213000	threshold_ctx19	Priority Threshold – Hart 9, S-mode
0x213004	claim_complete_ctx19	Claim/Complete – Hart 9, S-mode
0x214000	threshold_ctx20	Priority Threshold – Hart 10, M-mode
0x214004	claim_complete_ctx20	Claim/Complete – Hart 10, M-mode
0x215000	threshold_ctx21	Priority Threshold – Hart 10, S-mode
0x215004	claim_complete_ctx21	Claim/Complete – Hart 10, S-mode
0x216000	threshold_ctx22	Priority Threshold – Hart 11, M-mode
0x216004	claim_complete_ctx22	Claim/Complete – Hart 11, M-mode
0x217000	threshold_ctx23	Priority Threshold – Hart 11, S-mode
0x217004	claim_complete_ctx23	Claim/Complete – Hart 11, S-mode
0x218000	threshold_ctx24	Priority Threshold – Hart 12, M-mode
0x218004	claim_complete_ctx24	Claim/Complete – Hart 12, M-mode
0x219000	threshold_ctx25	Priority Threshold – Hart 12, S-mode
0x219004	claim_complete_ctx25	Claim/Complete – Hart 12, S-mode
0x21A000	threshold_ctx26	Priority Threshold – Hart 13, M-mode
0x21A004	claim_complete_ctx26	Claim/Complete – Hart 13, M-mode
0x21B000	threshold_ctx27	Priority Threshold – Hart 13, S-mode
0x21B004	claim_complete_ctx27	Claim/Complete – Hart 13, S-mode
0x21C000	threshold_ctx28	Priority Threshold – Hart 14, M-mode
0x21C004	claim_complete_ctx28	Claim/Complete – Hart 14, M-mode
0x21D000	threshold_ctx29	Priority Threshold – Hart 14, S-mode
0x21D004	claim_complete_ctx29	Claim/Complete – Hart 14, S-mode
0x21E000	threshold_ctx30	Priority Threshold – Hart 15, M-mode
0x21E004	claim_complete_ctx30	Claim/Complete – Hart 15, M-mode
0x21F000	threshold_ctx31	Priority Threshold – Hart 15, S-mode
0x21F004	claim_complete_ctx31	Claim/Complete – Hart 15, S-mode

priority_src01 register

- Absolute Address: 0xA0000004
- Base Offset: 0x4
- Size: 0x4

Interrupt source priority. Hardwired to 1 — no priority levels are implemented.



Bits Identifier		Access Reset Name	
0	PRIORITY r	0x1	—

PRIORITY field Priority level (always 1)

priority_src02 register

- Absolute Address: 0xA0000008
- Base Offset: 0x8
- Size: 0x4

Interrupt source priority. Hardwired to 1 — no priority levels are implemented.

Bits Identifier		Access Reset Name	
0	PRIORITY r	0x1	—

PRIORITY field Priority level (always 1)

priority_src03 register

- Absolute Address: 0xA000000C
- Base Offset: 0xC
- Size: 0x4

Interrupt source priority. Hardwired to 1 — no priority levels are implemented.

Bits Identifier		Access Reset Name	
0	PRIORITY r	0x1	—

PRIORITY field Priority level (always 1)

priority_src04 register

- Absolute Address: 0xA0000010
- Base Offset: 0x10
- Size: 0x4



Interrupt source priority. Hardwired to 1 — no priority levels are implemented.

Bits Identifier		Access Reset Name	
0	PRIORITY r	0x1	—

PRIORITY field Priority level (always 1)

priority_src05 register

- Absolute Address: 0xA0000014
- Base Offset: 0x14
- Size: 0x4

Interrupt source priority. Hardwired to 1 — no priority levels are implemented.

Bits Identifier		Access Reset Name	
0	PRIORITY r	0x1	—

PRIORITY field Priority level (always 1)

priority_src06 register

- Absolute Address: 0xA0000018
- Base Offset: 0x18
- Size: 0x4

Interrupt source priority. Hardwired to 1 — no priority levels are implemented.

Bits Identifier		Access Reset Name	
0	PRIORITY r	0x1	—

PRIORITY field Priority level (always 1)

**pending register**

- Absolute Address: 0xA0001000
- Base Offset: 0x1000
- Size: 0x4

Interrupt pending bits. Hardware-managed; read-only for software.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	PENDING	r	0x0	—

RESERVED field Reserved — source 0 does not exist

PENDING field Pending flags for interrupt sources 1..6

enable_ctx00 register

- Absolute Address: 0xA0002000
- Base Offset: 0x2000
- Size: 0x4

Interrupt enable bits for context 0 (hart 0, Machine mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx01 register

- Absolute Address: 0xA0002080
- Base Offset: 0x2080
- Size: 0x4



CONTENTS

Interrupt enable bits for context 1 (hart 0, Supervisor mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx02 register

- Absolute Address: 0xA0002100
- Base Offset: 0x2100
- Size: 0x4

Interrupt enable bits for context 2 (hart 1, Machine mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx03 register

- Absolute Address: 0xA0002180
- Base Offset: 0x2180
- Size: 0x4

Interrupt enable bits for context 3 (hart 1, Supervisor mode). Bit N enables source N for this context.



Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable__ctx04 register

- Absolute Address: 0xA0002200
- Base Offset: 0x2200
- Size: 0x4

Interrupt enable bits for context 4 (hart 2, Machine mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable__ctx05 register

- Absolute Address: 0xA0002280
- Base Offset: 0x2280
- Size: 0x4

Interrupt enable bits for context 5 (hart 2, Supervisor mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—



RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx06 register

- Absolute Address: 0xA0002300
- Base Offset: 0x2300
- Size: 0x4

Interrupt enable bits for context 6 (hart 3, Machine mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx07 register

- Absolute Address: 0xA0002380
- Base Offset: 0x2380
- Size: 0x4

Interrupt enable bits for context 7 (hart 3, Supervisor mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

**enable_ctx08 register**

- Absolute Address: 0xA0002400
- Base Offset: 0x2400
- Size: 0x4

Interrupt enable bits for context 8 (hart 4, Machine mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx09 register

- Absolute Address: 0xA0002480
- Base Offset: 0x2480
- Size: 0x4

Interrupt enable bits for context 9 (hart 4, Supervisor mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx10 register

- Absolute Address: 0xA0002500
- Base Offset: 0x2500
- Size: 0x4



CONTENTS

Interrupt enable bits for context 10 (hart 5, Machine mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx11 register

- Absolute Address: 0xA0002580
- Base Offset: 0x2580
- Size: 0x4

Interrupt enable bits for context 11 (hart 5, Supervisor mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx12 register

- Absolute Address: 0xA0002600
- Base Offset: 0x2600
- Size: 0x4

Interrupt enable bits for context 12 (hart 6, Machine mode). Bit N enables source N for this context.



Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable__ctx13 register

- Absolute Address: 0xA0002680
- Base Offset: 0x2680
- Size: 0x4

Interrupt enable bits for context 13 (hart 6, Supervisor mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable__ctx14 register

- Absolute Address: 0xA0002700
- Base Offset: 0x2700
- Size: 0x4

Interrupt enable bits for context 14 (hart 7, Machine mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—



RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx15 register

- Absolute Address: 0xA0002780
- Base Offset: 0x2780
- Size: 0x4

Interrupt enable bits for context 15 (hart 7, Supervisor mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx16 register

- Absolute Address: 0xA0002800
- Base Offset: 0x2800
- Size: 0x4

Interrupt enable bits for context 16 (hart 8, Machine mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

**enable_ctx17 register**

- Absolute Address: 0xA0002880
- Base Offset: 0x2880
- Size: 0x4

Interrupt enable bits for context 17 (hart 8, Supervisor mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx18 register

- Absolute Address: 0xA0002900
- Base Offset: 0x2900
- Size: 0x4

Interrupt enable bits for context 18 (hart 9, Machine mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx19 register

- Absolute Address: 0xA0002980
- Base Offset: 0x2980
- Size: 0x4



Interrupt enable bits for context 19 (hart 9, Supervisor mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx20 register

- Absolute Address: 0xA0002A00
- Base Offset: 0x2A00
- Size: 0x4

Interrupt enable bits for context 20 (hart 10, Machine mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx21 register

- Absolute Address: 0xA0002A80
- Base Offset: 0x2A80
- Size: 0x4

Interrupt enable bits for context 21 (hart 10, Supervisor mode). Bit N enables source N for this context.



Bits Identifier		Access Reset Name		
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable__ctx22 register

- Absolute Address: 0xA0002B00
- Base Offset: 0x2B00
- Size: 0x4

Interrupt enable bits for context 22 (hart 11, Machine mode). Bit N enables source N for this context.

Bits Identifier		Access Reset Name		
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable__ctx23 register

- Absolute Address: 0xA0002B80
- Base Offset: 0x2B80
- Size: 0x4

Interrupt enable bits for context 23 (hart 11, Supervisor mode). Bit N enables source N for this context.

Bits Identifier		Access Reset Name		
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—



RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx24 register

- Absolute Address: 0xA0002C00
- Base Offset: 0x2C00
- Size: 0x4

Interrupt enable bits for context 24 (hart 12, Machine mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx25 register

- Absolute Address: 0xA0002C80
- Base Offset: 0x2C80
- Size: 0x4

Interrupt enable bits for context 25 (hart 12, Supervisor mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

**enable_ctx26 register**

- Absolute Address: 0xA0002D00
- Base Offset: 0x2D00
- Size: 0x4

Interrupt enable bits for context 26 (hart 13, Machine mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx27 register

- Absolute Address: 0xA0002D80
- Base Offset: 0x2D80
- Size: 0x4

Interrupt enable bits for context 27 (hart 13, Supervisor mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx28 register

- Absolute Address: 0xA0002E00
- Base Offset: 0x2E00
- Size: 0x4



CONTENTS

Interrupt enable bits for context 28 (hart 14, Machine mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx29 register

- Absolute Address: 0xA0002E80
- Base Offset: 0x2E80
- Size: 0x4

Interrupt enable bits for context 29 (hart 14, Supervisor mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx30 register

- Absolute Address: 0xA0002F00
- Base Offset: 0x2F00
- Size: 0x4

Interrupt enable bits for context 30 (hart 15, Machine mode). Bit N enables source N for this context.



Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx31 register

- Absolute Address: 0xA0002F80
- Base Offset: 0x2F80
- Size: 0x4

Interrupt enable bits for context 31 (hart 15, Supervisor mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

threshold_ctx00 register

- Absolute Address: 0xA0200000
- Base Offset: 0x200000
- Size: 0x4

Priority threshold for context 0 (hart 0, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access	Reset	Name
0	THRESHOLD	r	0x0	—

THRESHOLD field Threshold (always 0)

**claim_complete_ctx00 register**

- Absolute Address: 0xA0200004
- Base Offset: 0x200004
- Size: 0x4

Claim/Complete for context 0 (hart 0, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx01 register

- Absolute Address: 0xA0201000
- Base Offset: 0x201000
- Size: 0x4

Priority threshold for context 1 (hart 0, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD	r	0x0 —

THRESHOLD field Threshold (always 0)

claim_complete_ctx01 register

- Absolute Address: 0xA0201004
- Base Offset: 0x201004
- Size: 0x4

Claim/Complete for context 1 (hart 0, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.



Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx02 register

- Absolute Address: 0xA0202000
- Base Offset: 0x202000
- Size: 0x4

Priority threshold for context 2 (hart 1, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD	r	0x0 —

THRESHOLD field Threshold (always 0)

claim_complete_ctx02 register

- Absolute Address: 0xA0202004
- Base Offset: 0x202004
- Size: 0x4

Claim/Complete for context 2 (hart 1, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

**threshold_ctx03 register**

- Absolute Address: 0xA0203000
- Base Offset: 0x203000
- Size: 0x4

Priority threshold for context 3 (hart 1, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx03 register

- Absolute Address: 0xA0203004
- Base Offset: 0x203004
- Size: 0x4

Claim/Complete for context 3 (hart 1, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier		Access Reset Name	
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx04 register

- Absolute Address: 0xA0204000
- Base Offset: 0x204000
- Size: 0x4

Priority threshold for context 4 (hart 2, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.



Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx04 register

- Absolute Address: 0xA0204004
- Base Offset: 0x204004
- Size: 0x4

Claim/Complete for context 4 (hart 2, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier		Access Reset Name	
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx05 register

- Absolute Address: 0xA0205000
- Base Offset: 0x205000
- Size: 0x4

Priority threshold for context 5 (hart 2, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

**claim_complete_ctx05 register**

- Absolute Address: 0xA0205004
- Base Offset: 0x205004
- Size: 0x4

Claim/Complete for context 5 (hart 2, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx06 register

- Absolute Address: 0xA0206000
- Base Offset: 0x206000
- Size: 0x4

Priority threshold for context 6 (hart 3, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD	r	0x0 —

THRESHOLD field Threshold (always 0)

claim_complete_ctx06 register

- Absolute Address: 0xA0206004
- Base Offset: 0x206004
- Size: 0x4

Claim/Complete for context 6 (hart 3, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.



Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx07 register

- Absolute Address: 0xA0207000
- Base Offset: 0x207000
- Size: 0x4

Priority threshold for context 7 (hart 3, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx07 register

- Absolute Address: 0xA0207004
- Base Offset: 0x207004
- Size: 0x4

Claim/Complete for context 7 (hart 3, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

**threshold_ctx08 register**

- Absolute Address: 0xA0208000
- Base Offset: 0x208000
- Size: 0x4

Priority threshold for context 8 (hart 4, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx08 register

- Absolute Address: 0xA0208004
- Base Offset: 0x208004
- Size: 0x4

Claim/Complete for context 8 (hart 4, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier		Access Reset Name	
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx09 register

- Absolute Address: 0xA0209000
- Base Offset: 0x209000
- Size: 0x4

Priority threshold for context 9 (hart 4, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.



Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx09 register

- Absolute Address: 0xA0209004
- Base Offset: 0x209004
- Size: 0x4

Claim/Complete for context 9 (hart 4, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier		Access Reset Name	
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx10 register

- Absolute Address: 0xA020A000
- Base Offset: 0x20A000
- Size: 0x4

Priority threshold for context 10 (hart 5, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

**claim_complete_ctx10 register**

- Absolute Address: 0xA020A004
- Base Offset: 0x20A004
- Size: 0x4

Claim/Complete for context 10 (hart 5, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx11 register

- Absolute Address: 0xA020B000
- Base Offset: 0x20B000
- Size: 0x4

Priority threshold for context 11 (hart 5, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD	r	0x0 —

THRESHOLD field Threshold (always 0)

claim_complete_ctx11 register

- Absolute Address: 0xA020B004
- Base Offset: 0x20B004
- Size: 0x4

Claim/Complete for context 11 (hart 5, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.



Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx12 register

- Absolute Address: 0xA020C000
- Base Offset: 0x20C000
- Size: 0x4

Priority threshold for context 12 (hart 6, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD	r	0x0 —

THRESHOLD field Threshold (always 0)

claim_complete_ctx12 register

- Absolute Address: 0xA020C004
- Base Offset: 0x20C004
- Size: 0x4

Claim/Complete for context 12 (hart 6, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

**threshold_ctx13 register**

- Absolute Address: 0xA020D000
- Base Offset: 0x20D000
- Size: 0x4

Priority threshold for context 13 (hart 6, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx13 register

- Absolute Address: 0xA020D004
- Base Offset: 0x20D004
- Size: 0x4

Claim/Complete for context 13 (hart 6, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier		Access Reset Name	
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx14 register

- Absolute Address: 0xA020E000
- Base Offset: 0x20E000
- Size: 0x4

Priority threshold for context 14 (hart 7, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.



Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx14 register

- Absolute Address: 0xA020E004
- Base Offset: 0x20E004
- Size: 0x4

Claim/Complete for context 14 (hart 7, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier		Access Reset Name	
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx15 register

- Absolute Address: 0xA020F000
- Base Offset: 0x20F000
- Size: 0x4

Priority threshold for context 15 (hart 7, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

**claim_complete_ctx15 register**

- Absolute Address: 0xA020F004
- Base Offset: 0x20F004
- Size: 0x4

Claim/Complete for context 15 (hart 7, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx16 register

- Absolute Address: 0xA0210000
- Base Offset: 0x210000
- Size: 0x4

Priority threshold for context 16 (hart 8, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD	r	0x0 —

THRESHOLD field Threshold (always 0)

claim_complete_ctx16 register

- Absolute Address: 0xA0210004
- Base Offset: 0x210004
- Size: 0x4

Claim/Complete for context 16 (hart 8, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.



Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx17 register

- Absolute Address: 0xA0211000
- Base Offset: 0x211000
- Size: 0x4

Priority threshold for context 17 (hart 8, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx17 register

- Absolute Address: 0xA0211004
- Base Offset: 0x211004
- Size: 0x4

Claim/Complete for context 17 (hart 8, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

**threshold_ctx18 register**

- Absolute Address: 0xA0212000
- Base Offset: 0x212000
- Size: 0x4

Priority threshold for context 18 (hart 9, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx18 register

- Absolute Address: 0xA0212004
- Base Offset: 0x212004
- Size: 0x4

Claim/Complete for context 18 (hart 9, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier		Access Reset Name	
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx19 register

- Absolute Address: 0xA0213000
- Base Offset: 0x213000
- Size: 0x4

Priority threshold for context 19 (hart 9, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.



Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx19 register

- Absolute Address: 0xA0213004
- Base Offset: 0x213004
- Size: 0x4

Claim/Complete for context 19 (hart 9, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier		Access Reset Name	
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx20 register

- Absolute Address: 0xA0214000
- Base Offset: 0x214000
- Size: 0x4

Priority threshold for context 20 (hart 10, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

**claim_complete_ctx20 register**

- Absolute Address: 0xA0214004
- Base Offset: 0x214004
- Size: 0x4

Claim/Complete for context 20 (hart 10, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx21 register

- Absolute Address: 0xA0215000
- Base Offset: 0x215000
- Size: 0x4

Priority threshold for context 21 (hart 10, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD	r	0x0 —

THRESHOLD field Threshold (always 0)

claim_complete_ctx21 register

- Absolute Address: 0xA0215004
- Base Offset: 0x215004
- Size: 0x4

Claim/Complete for context 21 (hart 10, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.



Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx22 register

- Absolute Address: 0xA0216000
- Base Offset: 0x216000
- Size: 0x4

Priority threshold for context 22 (hart 11, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD	r	0x0 —

THRESHOLD field Threshold (always 0)

claim_complete_ctx22 register

- Absolute Address: 0xA0216004
- Base Offset: 0x216004
- Size: 0x4

Claim/Complete for context 22 (hart 11, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

**threshold_ctx23 register**

- Absolute Address: 0xA0217000
- Base Offset: 0x217000
- Size: 0x4

Priority threshold for context 23 (hart 11, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx23 register

- Absolute Address: 0xA0217004
- Base Offset: 0x217004
- Size: 0x4

Claim/Complete for context 23 (hart 11, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier		Access Reset Name	
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx24 register

- Absolute Address: 0xA0218000
- Base Offset: 0x218000
- Size: 0x4

Priority threshold for context 24 (hart 12, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.



Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx24 register

- Absolute Address: 0xA0218004
- Base Offset: 0x218004
- Size: 0x4

Claim/Complete for context 24 (hart 12, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier		Access Reset Name	
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx25 register

- Absolute Address: 0xA0219000
- Base Offset: 0x219000
- Size: 0x4

Priority threshold for context 25 (hart 12, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

**claim__complete__ctx25 register**

- Absolute Address: 0xA0219004
- Base Offset: 0x219004
- Size: 0x4

Claim/Complete for context 25 (hart 12, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold__ctx26 register

- Absolute Address: 0xA021A000
- Base Offset: 0x21A000
- Size: 0x4

Priority threshold for context 26 (hart 13, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD	r	0x0 —

THRESHOLD field Threshold (always 0)

claim__complete__ctx26 register

- Absolute Address: 0xA021A004
- Base Offset: 0x21A004
- Size: 0x4

Claim/Complete for context 26 (hart 13, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.



Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx27 register

- Absolute Address: 0xA021B000
- Base Offset: 0x21B000
- Size: 0x4

Priority threshold for context 27 (hart 13, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx27 register

- Absolute Address: 0xA021B004
- Base Offset: 0x21B004
- Size: 0x4

Claim/Complete for context 27 (hart 13, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

**threshold_ctx28 register**

- Absolute Address: 0xA021C000
- Base Offset: 0x21C000
- Size: 0x4

Priority threshold for context 28 (hart 14, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx28 register

- Absolute Address: 0xA021C004
- Base Offset: 0x21C004
- Size: 0x4

Claim/Complete for context 28 (hart 14, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier		Access Reset Name	
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx29 register

- Absolute Address: 0xA021D000
- Base Offset: 0x21D000
- Size: 0x4

Priority threshold for context 29 (hart 14, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.



Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx29 register

- Absolute Address: 0xA021D004
- Base Offset: 0x21D004
- Size: 0x4

Claim/Complete for context 29 (hart 14, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier		Access Reset Name	
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx30 register

- Absolute Address: 0xA021E000
- Base Offset: 0x21E000
- Size: 0x4

Priority threshold for context 30 (hart 15, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

**claim__complete__ctx30 register**

- Absolute Address: 0xA021E004
- Base Offset: 0x21E004
- Size: 0x4

Claim/Complete for context 30 (hart 15, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold__ctx31 register

- Absolute Address: 0xA021F000
- Base Offset: 0x21F000
- Size: 0x4

Priority threshold for context 31 (hart 15, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD	r	0x0 —

THRESHOLD field Threshold (always 0)

claim__complete__ctx31 register

- Absolute Address: 0xA021F004
- Base Offset: 0x21F004
- Size: 0x4

Claim/Complete for context 31 (hart 15, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.



Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

nic_config address map

- Absolute Address: 0xFE000000
- Base Offset: 0xFE000000
- Size: 0x16000

Offset Identifier		Name
0x0	ni700_registers	—

ni700_registers memory

- Absolute Address: 0xFE000000
- Base Offset: 0x0
- Size: 0x16000

Refer to [arm_corelink_ni_700_network_on_chip_trm_101566_0201_09_en-3.pdf](#) for details

No supported members.



ErbiumCPU_MemoryMap address map

- Absolute Address: 0x0
- Base Offset: 0x0
- Size: 0xFE016000

Offset	Identifier	Name
0x02000000	system_registers	System Registers
0x02001000	mram_registers	AXI-to-MRAM Bridge Register Map
0x02002000	i2c_registers	—
0x02003000	qspi_registers	QSPI Registers
0x02004000	uart_registers	UART Registers
0x02008000	ROMRAM	—
0x0200F000	xspi_registers	sccr
0x40000000	mram	MRAM
0x80000000	cpu_registers	ESR Map of Erbium
0xA0000000	plic	—
0xFE000000	nic_config	—

system_registers address map

- Absolute Address: 0x2000000
- Base Offset: 0x2000000
- Size: 0xC4

Register Controlling System Behavior

Offset	Identifier	Name
0x00	Version	—
0x08	SystemConfig	—
0x10	watchdog_count	—
0x18	Watchdog	—
0x20	SysInterrupt	—
0x28	SoftReset	—
0x30	ResetCause	—
0x38	PowerDomainReq	—
0x40	PowerDomainAck	—
0x48	PowerGood	—



Offset	Identifier	Name
0x50	PowerStatus	—
0x58	SpinLock	—
0x60	ChipMode	—
0x68	Mailbox0	—
0x70	Mailbox1	—
0x78	GPIO_OE	—
0x80	GPIO_I	—
0x88	GPIO_O	—
0x90	GPIO_Interrupt_Enable	—
0x98	ring_osc	Ring Oscillator Control
0xA0	cpu_divider	—
0xA8	system_divider	—
0xB0	periph_divider	—
0xB8	tsense_ctrl	Temperature Sensor Control
0xC0	tsense_status	Temperature Sensor Status

Version register

- Absolute Address: 0x2000000
- Base Offset: 0x0
- Size: 0x4

Device identifier, used by software to identify the device family (chipid) variant, and bugfix version

Bits	Identifier	Access	Reset	Name
7:0	respin	r	0x0	—
15:8	variation	r	0x0	—
31:16	chipid	r	0xEB68	—

SystemConfig register

- Absolute Address: 0x20000008
- Base Offset: 0x8
- Size: 0x4

System configuration fields. use to enable/disable various features



CONTENTS

Bits Identifier		Access Reset Name		
0	sys_interrupt_enable	rw	0x0	—
1	mram_startup_bypass	rw	0x0	—
2	wdog_disable	rw	0x1	—
3	i2c_enable	rw	0x0	—
4	spi_enable	rw	0x1	—
5	qspi_enable	rw	0x0	—
6	uart_enable	rw	0x0	—
7	osc_out_enable	rw	0x0	—

sys_interrupt_enable field reg interrupt: Writing to this bit generates an interrupt.

mram_startup_bypass field connected to mram_startup_bypass of mram_wrapper

wdog_disable field Watchdog Disable

i2c_enable field I2C Enable

spi_enable field SPI Enable

qspi_enable field QSPI Enable

uart_enable field UART Enable

osc_out_enable field OSC_Out Enable

watchdog_count register

- Absolute Address: 0x2000010
- Base Offset: 0x10
- Size: 0x4

The watchdog detects ‘hang’ conditions and resets the system. This feature is disabled by default. Clear `cpu_config.wdog_disable` to enable this. Once enabled, S/W should write to `Watchdog.kick` to reset the `WATCHDOG_COUNT.count` field. If the count reaches 0, it triggers a system reset.



Bits	Identifier	Access	Reset	Name
31:0	watchdog_count	rw	0xFFFF	—

watchdog_count field When the watchdog timer is enabled it counts down from this value

Watchdog register

- Absolute Address: 0x2000018
- Base Offset: 0x18
- Size: 0x4

Bits	Identifier	Access	Reset	Name
7	kick	rw	0x0	—

kick field Resets the watchdog timer, cpu needs to regularly write to this bit to avoid a watchdog timeout based reset of the device

SysInterrupt register

- Absolute Address: 0x2000020
- Base Offset: 0x20
- Size: 0x4

Bits	Identifier	Access	Reset	Name
0	interrupt	rw, rclr	0x0	—
1	gpio_interrupt	r, rclr	0x0	—

interrupt field Write to this bit to generate an interrupt. Clear on read

gpio_interrupt field GPIO In Interrupt. Clear on read

**SoftReset register**

- Absolute Address: 0x2000028
- Base Offset: 0x28
- Size: 0x4

Bits Identifier		Access Reset Name		
0	soft_reset	rw	0x0	—
1	cpu_warm_reset	rw	0x0	—
2	mram_rst_b	rw	0x1	—
3	cpu_soft_reset	rw	0x0	—

soft_reset field System Soft Reset, Active High

cpu_warm_reset field CPU Warm Reset, Active High

mram_rst_b field MRAM Resetn, Active Low

cpu_soft_reset field CPU cold Reset, Self clearing Active High

ResetCause register

- Absolute Address: 0x2000030
- Base Offset: 0x30
- Size: 0x4

This register reports the cause of reset. There are multiple reset sources.

- Power on Reset,
- Brownout Reset, and
 - Various S/W reset requests. These clear on read bits capture the reset cause since the last read Note if the por bit is set ignore the other cause registers, they will have random values until the first read.



Bits Identifier		Access Reset Name		
0	por	r	—	—
1	watchdog_timeout	r	—	—
2	sysreset_req	r	—	—
3	brownout	r	—	—
4	softreset	r	—	—
5	cpu_warm_reset	r	—	—

por field System POR was toggled

watchdog_timeout field Watchdog was enabled and CPU failed to clear the watchdog timer

sysreset_req field NA. CPU detected an architecture level lockup and requested a system reset

brownout field The brownout detector triggered a reset

softreset field The soft reset bit was written to

cpu_warm_reset field NA. The cpu reset bit was written to

PowerDomainReq register

- Absolute Address: 0x2000038
- Base Offset: 0x38
- Size: 0x4

Writing to PD fields initiates the shutdown of the corresponding power domain. A shutdown request is sent to the domain, This domain waits until all outstanding transactions are completed and then generates a corresponding ack signal. Once the ack signal is generated the power controller will turn off power to that domain.

Bits Identifier		Access Reset Name		
0	cpu_pd	rw	0x0	—
1	cpu_rom_powerdown	rw	0x0	—
2	cpu_ram_powerdown	rw	0x0	—



CONTENTS

Bits Identifier		Access Reset Name		
3	chiplet_pd	rw	0x0	—
4	mram_pd	rw	0x0	—
5	system_poweroff	rw	0x0	—
6	xspi_pd	rw	0x0	—
15:8	minion_pd	rw	0x0	—
16	mram_dsleep_en	rw	0x0	—
17	cpu_sleep_en	rw	0x0	—

cpu_pd field Poweroff the CPU Domain

cpu_rom_powerdown field Powerdown the boot ROM

cpu_ram_powerdown field Powerdown the CPU ram. this puts the ram in deepsleep mode

chiplet_pd field Not used; chiplet power domain is controlled via the mode bits

mram_pd field Power down for MRAM digital logic and MRAM domain. When set to 1, this powers off the domain; setting to 0 powers it on.

system_poweroff field Power off the chip. Only wakeup logic is powered on

xspi_pd field Not used; hyperbus power domain is controlled via the mode bits.

minion_pd field Minion PowerDown Req

mram_dsleep_en field Connects directly to the MRAM's main power switch, bypassing the digital domain's power switch. When set to 1, the MRAM is in deep sleep; setting to 0 powers the MRAM on.

cpu_sleep_en field currently not used. In future this will be used for CPU sleep management.

**PowerDomainAck register**

- Absolute Address: 0x2000040
- Base Offset: 0x40
- Size: 0x4

This mirrors the Ack signal generated in response to power down request.

Bits Identifier		Access Reset Name		
0	cpu_pd_ack	r	0x0	—
1	sram_pd_ack	r	0x0	—
3	chiplet_pd_ack	r	0x0	—
4	mram_pd_ack	r	0x0	—
5	system_pd_ack	r	0x0	—
6	hyperbus_pd_ack	r	0x0	—
15:8	minion_pd_ack	r	0x0	—

cpu_pd_ack field pd ack for cpu

sram_pd_ack field Not used: pd ack for TCM

chiplet_pd_ack field Not used: pd ack for chiplet

mram_pd_ack field pd ack for mram

system_pd_ack field pd ack for system

hyperbus_pd_ack field Not used: pd ack for hyperbus

minion_pd_ack field Minion PowerDown Req

PowerGood register

- Absolute Address: 0x2000048
- Base Offset: 0x48
- Size: 0x4



CONTENTS

When a power domain is switched on, it takes time for the voltage to stabilize, this time is process dependent. The default value is sufficiently large to account for all variations.

Bits	Identifier	Access	Reset	Name
20:0	counter	rw	0xFFFF	—

counter field Counter for powerGood

PowerStatus register

- Absolute Address: 0x2000050
- Base Offset: 0x50
- Size: 0x4

Status

Bits	Identifier	Access	Reset	Name
3:0	reserved	r	0x0	—

reserved field Asserted when MRAM is powered on/reset until internal setup is done.

SpinLock register

- Absolute Address: 0x2000058
- Base Offset: 0x58
- Size: 0x4

initially locked=0, A read on this register will set the lock bit. Usage:

1. Read this field. If you get a value of zero you got the lock. Else a different processing element acquired the lock. Poll at fixed intervals(dont spam) until you get the lock.
2. If you acquired the lock, Once you have finished interacting with the locked resource write 0 to this register to release the lock.



Bits	Identifier	Access	Reset	Name
0	lock	rw	rset 0x0	—

ChipMode register

- Absolute Address: 0x2000060
- Base Offset: 0x60
- Size: 0x4

Bits	Identifier	Access	Reset	Name
1:0	chip_mode	r	—	—
2	ifc_width	r	—	—
4:3	bootload	rw	0x0	—
6:5	load_external	rw	0x0	—

chip_mode field The mode in which chip is working, hyperbus, axi,ahb,gc

ifc_width field If chip is axi/ahb mode datawidth

bootload field Jump to 00:no Jump, 01:TCM,10:MRAM

load_external field Jump to 00:no load, 01:Load TCM via Chiplet,10:Load MRAM via Chiplet

Mailbox0 register

- Absolute Address: 0x2000068
- Base Offset: 0x68
- Size: 0x4

Bits	Identifier	Access	Reset	Name
31:0	mbox0	rw	0x0	—

mbox0 field Mailbox0

**Mailbox1 register**

- Absolute Address: 0x2000070
- Base Offset: 0x70
- Size: 0x4

Bits Identifier Access Reset Name			
31:0	mbox1	rw	0x0 —

mbox1 field Mailbox1**GPIO_OE register**

- Absolute Address: 0x2000078
- Base Offset: 0x78
- Size: 0x4

Gpio Output enable. Write 1 to this bit to set the GPIO register in output mode.

Bits Identifier Access Reset Name			
10:0	gpio_oe	rw	0x0 —

gpio_oe field Gpio output enable**GPIO_I register**

- Absolute Address: 0x2000080
- Base Offset: 0x80
- Size: 0x4

Gpio Input For each bit in input mode this register captures the input value.

Bits Identifier Access Reset Name			
10:0	gpio_i	r	0x0 —

gpio_i field Gpio input

**GPIO_O register**

- Absolute Address: 0x2000088
- Base Offset: 0x88
- Size: 0x4

Gpio Output For each bit in output mode the content of the corresponding bit in this register is reflected on the GPIO Pin

Bits Identifier Access Reset Name			
10:0	gpio_o	rw	0x0 —

gpio_o field Gpio input

GPIO_Interrupt_Enable register

- Absolute Address: 0x2000090
- Base Offset: 0x90
- Size: 0x4

Gpio Interrupt Enable For each bit in input mode if the input signal toggles and the content of the corresponding bit in this register is 1, then a GPIO interrupt is raised.

Bits Identifier		Access Reset Name	
10:0	gpio_interrupt_en	rw	0x0 —

gpio_interrupt_en field Gpio input

ring_osc register

- Absolute Address: 0x2000098
- Base Offset: 0x98
- Size: 0x4

Controls the ring oscillator (ring_osc) input pins. In normal mode (dbg_en=0), only en, divby2_sel, and trm are active. In debug mode (dbg_en=1), the dbg_* fields independently gate the analog bias block (anachip) and the oscillator core (rochip).



Bits Identifier		Access Reset Name		
0	en	rw	0x1	—
1	divby2_sel	rw	0x1	—
6:2	trm	rw	0x1F	—
7	dbg_en	rw	0x0	—
8	dbg_anachip_en	rw	0x0	—
9	dbg_rohcip_en	rw	0x0	—
10	dbg_sah_en_b	rw	0x0	—

en field Master oscillator enable. Drive high to power up both the analog bias block (anachip) and the ring oscillator core (rochip). Has no effect when dbg_en is asserted.

divby2_sel field Divide-by-2 frequency select. 0: Full-speed range (899 MHz to 1.608 GHz, set by trm). 1: Half-speed range (449 MHz to 794 MHz, set by trm).

trm field Frequency trim code. Selects the oscillator output frequency within the range determined by divby2_sel. 0x00 = fastest (1.608 GHz or 794 MHz). 0x1F = slowest (899 MHz or 449 MHz). Frequency decreases linearly with increasing trim value.

dbg_en field Debug override enable. When asserted, the en input is ignored and dbg_anachip_en and dbg_rohcip_en independently control the analog bias block and oscillator core respectively. dbg_sah_en_b also becomes active. 0: Normal mode – en controls both blocks. 1: Debug mode – dbg_* fields control each block separately.

dbg_anachip_en field Debug enable for the analog bias block (anachip). Only active when dbg_en=1. When asserted with dbg_en=1, powers up the anachip and drives h1_mnvdd09_g high, which is required for the oscillator core to run. When deasserted, h1_mnvdd09_g is held low and the oscillator core cannot oscillate regardless of dbg_rohcip_en.

dbg_rohcip_en field Debug enable for the ring oscillator core (rochip). Only active when dbg_en=1. When asserted with dbg_en=1 and dbg_anachip_en=1, the oscillator core is enabled and clk will toggle. When deasserted, the oscillator core is disabled and clk is held low regardless of dbg_anachip_en.



dbg_sah_en_b field Debug SAH enable (active low). Only active when dbg_en=1. Selects the anachip operating mode: 0: Normal SAH operation (anachip mode 3'b100, h1=1). 1: Test mode, high power (anachip mode 3'b101, h1=1). In both cases h1_mnvdd09_g remains high when dbg_anachip_en=1; this bit selects the internal anachip bias configuration only.

cpu_divider register

- Absolute Address: 0x20000A0
- Base Offset: 0xA0
- Size: 0x4

Bits Identifier		Access	Reset	Name
3:0	count	rw	0xF	—
4	div_enable	rw	0x1	—

count field clock divider. output clk is 2 x count

div_enable field Enable clk divider

system_divider register

- Absolute Address: 0x20000A8
- Base Offset: 0xA8
- Size: 0x4

Bits Identifier		Access	Reset	Name
3:0	count	rw	0xF	—
4	div_enable	rw	0x1	—

count field clock divider. output clk is 2 x count

div_enable field Enable clk divider

**periph_divider register**

- Absolute Address: 0x20000B0
- Base Offset: 0xB0
- Size: 0x4

Bits Identifier		Access	Reset	Name
3:0	count	rw	0xF	—
4	div_enable	rw	0x1	—

count field clock divider. output clk is 2 x count

div_enable field Enable clk divider

tsense_ctrl register

- Absolute Address: 0x20000B8
- Base Offset: 0xB8
- Size: 0x4

Controls the temperature sensor and ADC conversion. Write 1 to conv to trigger a single conversion. The bit auto-clears after one cycle. Select which sensor to sample with sen_sel, and set the ADC clock divider with clk_div.

Bits Identifier		Access	Reset	Name
0	conv	rw	0x0	—
1	sen_en	rw	0x0	—
3:2	sen_sel	rw	0x0	—
11:4	clk_div	rw	0x1	—

conv field Start conversion. Writing 1 triggers a single ADC sample. Auto-clears to 0 after one cycle. Clears the valid bit in tsense_status when written.

sen_en field Sensor enable. When set, the sensor selected by sel is powered on. When clear, all sensors are off.

sen_sel field Sensor select. Selects which of the 4 temperature sensors (0-3) is connected to the ADC.



clk_div field ADC clock divider. The ADC clock frequency is $\text{sys_clk} / (2 * (\text{clk_div} + 1))$. 0x00 = $\text{sys_clk}/2$ (fastest). 0xFF = $\text{sys_clk}/512$ (slowest).

tsense_status register

- Absolute Address: 0x20000C0
- Base Offset: 0xC0
- Size: 0x4

Read-only status from the most recent ADC conversion.

Bits Identifier Access Reset Name				
3:0	data	r	0x0	—
4	valid	r	0x0	—
5	conv_b	r	0x1	—

data field ADC output data from the last completed conversion.

valid field Data valid. Set by hardware when a conversion completes. Cleared when a new conversion is started (conv written).

conv_b field ADC conversion status. Active-low while the ADC conversion is in progress and high when idle / conversion complete.

mram_registers address map

- Absolute Address: 0x2001000
- Base Offset: 0x2001000
- Size: 0x4A0

Unified register map for the AXI-to-MRAM bridge. Contains bridge configuration registers and per-bank MRAM controller test registers.

Offset Identifier	Name
0x000 bridge_regs	AXI-to-MRAM Bridge Control and Status Registers
0x100 bank0_tregs	MRAM Control and Status Test Registers
0x200 bank1_tregs	MRAM Control and Status Test Registers
0x300 bank2_tregs	MRAM Control and Status Test Registers



Offset Identifier	Name
0x400 bank3_tregs	MRAM Control and Status Test Registers

bridge_regs register file

- Absolute Address: 0x2001000
- Base Offset: 0x0
- Size: 0x38

Register space for configuration and status of the AXI-to-MRAM bridge.

Offset Identifier	Name
0x00 arbiter_mode_reg	Arbiter Mode
0x08 bridge_status_reg	Bridge Status
0x10 slverr_status_reg	AXI Slave Error Status
0x18 control_reg	AXI2MRAM Control
0x20 ecc_1bit_error_count_reg	ECC 1-bit Error Counter
0x28 ecc_2bit_error_count_reg	ECC 2-bit Error Counter
0x30 ecc_3bit_error_count_reg	ECC 3-bit Error Counter

arbiter_mode_reg register

- Absolute Address: 0x2001000
- Base Offset: 0x0
- Size: 0x8

Controls the read/write arbitration policy when both channels have simultaneous pending requests.

Bits Identifier	Access	Reset	Name
1:0 arbiter_mode	rw	0x2	arbiter_mode

arbiter_mode field Arbitration mode selection. 0 = Write Priority (writes always win) 1 = Read Priority (reads always win) 2 = Round Robin (alternate after each conflict) 3 = Oldest First (whichever request arrived first wins)

**bridge_status_reg register**

- Absolute Address: 0x2001008
- Base Offset: 0x8
- Size: 0x8

Read-only status indicators for the AXI-to-MRAM bridge.

Bits Identifier		Access	Reset	Name
0	axi_busy	r	0x0	axi_busy
7:4	cmd_queue_active	r	0x0	cmd_queue_active
11:8	mram_ready	r	0x0	mram_ready

axi_busy field High when the bridge has outstanding AXI transactions.

cmd_queue_active field Per-bank command queue active flags. Bit N = bank N has pending write/RMW commands.

mram_ready field Per-bank ready signals for bringing MRAM out of deep sleep. When the bit is 1, that bank is ready for operations.

slverr_status_reg register

- Absolute Address: 0x2001010
- Base Offset: 0x10
- Size: 0x8

Sticky status bits capturing the reason(s) why the bridge returned SLVERR on the AXI bus. Bits are set by hardware and cleared automatically when software reads this register (clear-on-read). Multiple causes may be set simultaneously.

Bits Identifier		Access	Reset	Name
0	oor_read	r, rclr	0x0	oor_read
1	oor_write	r, rclr	0x0	oor_write
2	mram_not_ready	r, rclr	0x0	mram_not_ready
3	mram_unpowered	r, rclr	0x0	mram_unpowered
4	maintenance	r, rclr	0x0	maintenance
5	unrecoverable_error	r, rclr	0x0	unrecoverable_error



oor_read field Set when a read transaction was rejected with SLVERR because ARADDR falls outside the valid MRAM window (ARADDR[31:24] != 0, i.e. ARADDR >= 0x100_0000).

oor_write field Set when a write transaction was rejected with SLVERR because AWADDR falls outside the valid MRAM window (AWADDR[31:24] != 0, i.e. AWADDR >= 0x100_0000).

mram_not_ready field Set when a transaction was rejected with SLVERR because the targeted MRAM bank reports not-ready.

mram_unpowered field Set when a transaction was rejected with SLVERR because the targeted MRAM bank reports power-not-ok.

maintenance field Set when a transaction was rejected with SLVERR because the targeted MRAM bank is in maintenance mode.

unrecoverable_error field Set when read data was returned with an unrecoverable ECC condition (triple-bit error) and the bridge responded with SLVERR.

control_reg register

- Absolute Address: 0x2001018
- Base Offset: 0x18
- Size: 0x8

Software control bits for the AXI-to-MRAM bridge.

Bits Identifier		Access Reset Name	
3:0	disable_clock_gate	rw 0x0	disable_clock_gate
4	ecc_1bit_intr_mask	rw 0x1	ecc_1bit_intr_mask
5	ecc_2bit_intr_mask	rw 0x0	ecc_2bit_intr_mask
6	ecc_3bit_intr_mask	rw 0x0	ecc_3bit_intr_mask

disable_clock_gate field Per-bank clock-gate disable control. Bit N disables the clock gating structure driving MRAM bank N so that a continuous clock is driven into the bank interface.



ecc_1bit_intr_mask field Masks CPU interrupt generation for 1-bit ECC events in the ET interrupt logic. 1 = masked/suppressed, 0 = unmasked/enabled. Reset default is 1 (masked).

ecc_2bit_intr_mask field Masks CPU interrupt generation for 2-bit ECC events in the ET interrupt logic. 1 = masked/suppressed, 0 = unmasked/enabled. Reset default is 0 (enabled).

ecc_3bit_intr_mask field Masks CPU interrupt generation for 3-bit ECC events in the ET interrupt logic. 1 = masked/suppressed, 0 = unmasked/enabled. Reset default is 0 (enabled).

ecc_1bit_error_count_reg register

- Absolute Address: 0x2001020
- Base Offset: 0x20
- Size: 0x8

Counts observed 1-bit ECC events across all banks/lanes. The counter increments by the number of asserted lane bits per cycle (0..8).

Bits Identifier Access Reset Name			
31:0 count	r	0x0	count

count field Accumulated 1-bit ECC event count.

ecc_2bit_error_count_reg register

- Absolute Address: 0x2001028
- Base Offset: 0x28
- Size: 0x8

Counts observed 2-bit ECC events across all banks/lanes. The counter increments by the number of asserted lane bits per cycle (0..8).

Bits Identifier Access Reset Name			
31:0 count	r	0x0	count

count field Accumulated 2-bit ECC event count.

**ecc_3bit_error_count_reg register**

- Absolute Address: 0x2001030
- Base Offset: 0x30
- Size: 0x8

Counts observed 3-bit ECC events across all banks/lanes. The counter increments by the number of asserted lane bits per cycle (0..8).

Bits Identifier Access Reset Name			
31:0 count	r	0x0	count

count field Accumulated 3-bit ECC event count.

bank0_tregs register file

- Absolute Address: 0x2001100
- Base Offset: 0x100
- Size: 0xA0

Register space that contains the test registers for test functionality and debug of the MRAM.

Offset Identifier		Name
0x00	mram_control	—
0x20	mram_control_pulse	—
0x28	gbl_cfg_ovr_0	—
0x30	gbl_cfg_0	—
0x38	man_control_0	—
0x40	man_control_1	—
0x48	mram_status_0	—
0x50	mram_status_1	—
0x58	mram_dout_even_lower	—
0x60	mram_dout_odd_lower	—
0x68	mram_dout_uppers	—
0x70	bist_status_0	—
0x78	bist_control	—
0x88	ecc_correction	—
0x98	bist_status_1	—

**mram_control register**

- Absolute Address: 0x2001100
- Base Offset: 0x0
- Size: 0x20

Bits	Identifier	Access	Reset	Name
16:0	addr_in	rw	0x0	addr_in
24:17	ce	rw	0x0	ce
25	we	rw	0x0	we
26	rd_pulse_meas_en	rw	0x0	rd_pulse_meas_en
33:27	rca_ovr	rw	0x0	rca_ovr
34	rca_ovr_en	rw	0x0	rca_ovr_en
35	gbl_cfg_ovr_en	rw	0x0	gbl_cfg_ovr_en
36	rd_en_ovr	rw	0x0	rd_en_ovr
37	ref_prg_en	rw	0x0	ref_prg_en
38	dsleep_mram_en	rw	0x0	dsleep_mram_en
39	reg_logic_sup_sleep_ovr	rw	0x1	reg_logic_sup_sleep_ovr
40	prg_rd1_byp	rw	0x0	prg_rd1_byp
41	wr_en_ovr	rw	0x0	wr_en_ovr
42	dma_en	rw	0x0	dma_en
43	vbslx_gain_mode_ovr	rw	0x0	vbslx_gain_mode_ovr
47:44	powerup_trim_load_ovr	rw	0x0	powerup_trim_load_ovr
52	test_cal_en	rw	0x0	test_cal_en
55:53	anatest0_sel	rw	0x0	anatest0_sel
58:56	anatest1_sel	rw	0x0	anatest1_sel
59	eccrom_deep_sleep	rw	0x1	eccrom_deep_sleep
60	ref_ecc_sel	rw	0x0	ref_ecc_sel
61	disable_cpu_intr	rw	0x0	disable_cpu_intr
62	disable_ted	rw	0x0	disable_ted
63	ecc_bypass_en	rw	0x0	ecc_bypass_en
142:64	bwe	rw	0x0	bwe
143	mram_clk_en	rw	0x1	mram_clk_en
222:144	din	rw	0x0	din
223	test_reg_ovr_en	rw	0x0	test_reg_ovr_en
224	otp_wr_en	rw	0x0	otp_wr_en
225	rst_cpu_intr	rw	0x0	rst_cpu_intr
233:226	dout_en	rw	0x0	dout_en
236:234	ecc_en	rw	0x0	ecc_en



CONTENTS

Bits	Identifier	Access	Reset	Name
240:237	even_man_stripe_sel	rw	0x0	even_man_stripe_sel
241	even_man_wr_0	rw	0x0	even_man_wr<0>
242	even_man_wr_1	rw	0x0	even_man_wr<1>
243	even_man_wr_2	rw	0x0	even_man_wr<2>
244	even_man_wr_3	rw	0x0	even_man_wr<3>
248:245	odd_man_stripe_sel	rw	0x0	odd_man_stripe_sel
249	odd_man_wr_0	rw	0x0	odd_man_wr<0>
250	odd_man_wr_1	rw	0x0	odd_man_wr<1>
251	odd_man_wr_2	rw	0x0	odd_man_wr<2>
252	odd_man_wr_3	rw	0x0	odd_man_wr<3>
253	sah_en	rw	0x1	sah_en
254	scc_otp_en	rw	0x0	scc_otp_en
255	maintenance_mode	rw	0x0	maintenance_mode

mram_control_pulse register

- Absolute Address: 0x2001120
- Base Offset: 0x20
- Size: 0x8

Bits	Identifier	Access	Reset	Name
0	powerup_trim_load_ovr_single_pulse<0>	rw	0x0	powerup_trim_load_ovr_single_pulse<0>
1	powerup_trim_load_ovr_single_pulse<1>	rw	0x0	powerup_trim_load_ovr_single_pulse<1>
2	powerup_trim_load_ovr_single_pulse<2>	rw	0x0	powerup_trim_load_ovr_single_pulse<2>
3	powerup_trim_load_ovr_single_pulse<3>	rw	0x0	powerup_trim_load_ovr_single_pulse<3>
4	mram_clk_single_pulse	rw	0x0	mram_clk_single_pulse

gbl_cfg_ovr_0 register

- Absolute Address: 0x2001128
- Base Offset: 0x28
- Size: 0x8

Bits	Identifier	Access	Reset	Name
1:0	sa_equal_trim	rw	0x1	sa_equal_trim<1:0>



Bits	Identifier	Access	Reset	Name
4:2	vbbslx_boost_trim	rw	0x4	vbbslx_boost_trim<2:0>
8:5	wr_en_msb_trim	rw	0x9	wr_en_msb_trim<3:0>
11:9	wr_en_lsb_trim	rw	0x6	wr_en_lsb_trim<2:0>
12	vbbslx_gain_mode	rw	0x0	vbbslx_gain_mode
16:13	repulse_trim	rw	0x6	repulse_trim<3:0>
17	repulse_en	rw	0x1	repulse_en
20:18	rd_en_trim	rw	0x5	rd_en_trim<2:0>
25:22	osc_wr_div_trim	rw	0x3	osc_wr_div_trim<3:0>
29:26	vbbsl_trim	rw	0x4	vbbsl_trim<3:0>
33:30	tcsel_trim	rw	0x3	tcsel_trim<3:0>
37:34	vwlwr_trim	rw	0x7	vwlwr_trim<3:0>
41:38	vcr_gate_trim	rw	0xE	vcr_gate_trim<3:0>

gbl_cfg_0 register

- Absolute Address: 0x2001130
- Base Offset: 0x30
- Size: 0x8

Bits	Identifier	Access	Reset	Name
1:0	sa_equal_trim	r	—	sa_equal_trim<1:0>
4:2	vbbslx_boost_trim	r	—	vbbslx_boost_trim<2:0>
8:5	wr_en_msb_trim	r	—	wr_en_msb_trim<3:0>
11:9	wr_en_lsb_trim	r	—	wr_en_lsb_trim<2:0>
12	vbbslx_gain_mode	r	—	vbbslx_gain_mode
16:13	repulse_trim	r	—	repulse_trim<3:0>
17	repulse_en	r	—	repulse_en
20:18	rd_en_trim	r	—	rd_en_trim<2:0>
25:22	osc_wr_div_trim	r	—	osc_wr_div_trim<3:0>
29:26	vbbsl_trim	r	—	vbbsl_trim<3:0>
33:30	tcsel_trim	r	—	tcsel_trim<3:0>
37:34	vwlwr_trim	r	—	vwlwr_trim<3:0>
41:38	vcr_gate_trim	r	—	vcr_gate_trim<3:0>

man_control_0 register

- Absolute Address: 0x2001138



CONTENTS

- Base Offset: 0x38
- Size: 0x8

Bits	Identifier	Access	Reset	Name
3:0	blk0_man_ccnt	r	—	blk0_man_ccnt<3:0>
7:4	blk0_man_cnfg	rw	0x0	blk0_man_cnfg<3:0>
9:8	blk0_man_fcnt	r	—	blk0_man_fcnt<1:0>
19:16	blk1_man_ccnt	r	—	blk1_man_ccnt<3:0>
23:20	blk1_man_cnfg	rw	0x0	blk1_man_cnfg<3:0>
25:24	blk1_man_fcnt	r	—	blk1_man_fcnt<1:0>
35:32	blk2_man_ccnt	r	—	blk2_man_ccnt<3:0>
39:36	blk2_man_cnfg	rw	0x0	blk2_man_cnfg<3:0>
41:40	blk2_man_fcnt	r	—	blk2_man_fcnt<1:0>
51:48	blk3_man_ccnt	r	—	blk3_man_ccnt<3:0>
55:52	blk3_man_cnfg	rw	0x0	blk3_man_cnfg<3:0>
57:56	blk3_man_fcnt	r	—	blk3_man_fcnt<1:0>

man_control_1 register

- Absolute Address: 0x2001140
- Base Offset: 0x40
- Size: 0x8

Bits	Identifier	Access	Reset	Name
3:0	blk4_man_ccnt	r	—	blk4_man_ccnt<3:0>
7:4	blk4_man_cnfg	rw	0x0	blk4_man_cnfg<3:0>
9:8	blk4_man_fcnt	r	—	blk4_man_fcnt<1:0>
19:16	blk5_man_ccnt	r	—	blk5_man_ccnt<3:0>
23:20	blk5_man_cnfg	rw	0x0	blk5_man_cnfg<3:0>
25:24	blk5_man_fcnt	r	—	blk5_man_fcnt<1:0>
35:32	blk6_man_ccnt	r	—	blk6_man_ccnt<3:0>
39:36	blk6_man_cnfg	rw	0x0	blk6_man_cnfg<3:0>
41:40	blk6_man_fcnt	r	—	blk6_man_fcnt<1:0>
51:48	blk7_man_ccnt	r	—	blk7_man_ccnt<3:0>
55:52	blk7_man_cnfg	rw	0x0	blk7_man_cnfg<3:0>
57:56	blk7_man_fcnt	r	—	blk7_man_fcnt<1:0>

**mram_status_0 register**

- Absolute Address: 0x2001148
- Base Offset: 0x48
- Size: 0x8

Bits	Identifier	Access	Reset	Name
15:0	bist_error_loop	r	—	bist_error_loop<15:0>
33:17	bist_error_count	r	—	bist_error_count<16:0>
40:34	bist_rh0	r	—	bist_rh0<6:0>
47:41	bist_rh1	r	—	bist_rh1<6:0>
54:48	bist_rh2	r	—	bist_rh2<6:0>
56:55	cpu_intr_flag	r	—	cpu_intr_flag
58:57	ecc_1bit_flag	r	—	ecc_1bit_flag<1:0>
60:59	ecc_2bit_flag	r	—	ecc_2bit_flag<1:0>
62:61	ecc_3bit_flag	r	—	ecc_3bit_flag<1:0>

mram_status_1 register

- Absolute Address: 0x2001150
- Base Offset: 0x50
- Size: 0x8

Bits	Identifier	Access	Reset	Name
1:0	temp	r	—	temp<1:0>
4:3	ecc_1bit	r	—	ecc_1bit<1:0>
6:5	ecc_2bit	r	—	ecc_2bit<1:0>
8:7	ecc_3bit	r	—	ecc_3bit<1:0>
9	pwr_ok	r	—	pwr_ok
10	eccrom_pwr_ok	r	—	eccrom_pwr_ok
29:11	intr_error_lane0_addr	r	—	intr_error_lane0_addr<18:0>
48:30	intr_error_lane1_addr	r	—	intr_error_lane1_addr<18:0>
57:50	busy	r	—	busy<7:0>

mram_dout_even_lower register

- Absolute Address: 0x2001158
- Base Offset: 0x58



- Size: 0x8

Bits Identifier Access Reset Name			
63:0	dout	r	dout<63:0>

mram_dout_odd_lower register

- Absolute Address: 0x2001160
- Base Offset: 0x60
- Size: 0x8

Bits Identifier Access Reset Name			
63:0	dout	r	dout<142:79>

mram_dout_uppers register

- Absolute Address: 0x2001168
- Base Offset: 0x68
- Size: 0x8

Bits	Identifier	Access	Reset Name
14:0	dout_even_msb	r	dout<78:64>
46:32	dout_odd_msb	r	dout<157:143>

bist_status_0 register

- Absolute Address: 0x2001170
- Base Offset: 0x70
- Size: 0x8

Bits	Identifier	Access	Reset Name
63:0	bist_error_value	r	bist_error_value<63:0>

**bist_control register**

- Absolute Address: 0x2001178
- Base Offset: 0x78
- Size: 0x10

Bits	Identifier	Access	Reset	Name
0	bist_rte_en	rw	0x0	bist_rte_en
1	bist_data_inv	rw	0x0	bist_data_inv
4:2	bist_add_inc	rw	0x0	bist_add_inc<2:0>
5	bist_stop_on_error	rw	0x0	bist_stop_on_error
25:6	bist_start_add	rw	0x0	bist_start_add<19:0>
45:26	bist_stop_add	rw	0x0	bist_stop_add<19:0>
50:46	RH4margin	rw	0xA	RH4margin<4:0>
55:51	rh2_offset	rw	0x0	rh2_offset<4:0>
78:64	bist_error_value	r	—	bist_error_value<78:64>
94:79	bist_loop_count	rw	0x0	bist_loop_count<15:0>
95	bist_start	rw	0x0	bist_start
96	bist_trim_mode	rw	0x0	bist_trim_mode
97	bist_stop_on_repl_of	rw	0x0	bist_stop_on_repl_of
98	bist_rst_b	rw	0x1	bist_rst_b
99	bist_reset	rw	0x0	bist_reset
100	bist_rd_en	rw	0x0	bist_rd_en
101	bist_wr_en	rw	0x0	bist_wr_en

ecc_correction register

- Absolute Address: 0x2001188
- Base Offset: 0x88
- Size: 0x10

Bits	Identifier	Access	Reset	Name
127:0	dout	r	—	dout<127:0>

bist_status_1 register

- Absolute Address: 0x2001198
- Base Offset: 0x98



- Size: 0x8

Bits Identifier		Access		Reset	Name
19:0	bist_err_add r	—			bist_err_add<19:0>
20	bist_error	r	—		bist_error
21	bist_busy	r	—		bist_busy

bank1_tregs register file

- Absolute Address: 0x2001200
- Base Offset: 0x200
- Size: 0xA0

Register space that contains the test registers for test functionality and debug of the MRAM.

Offset Identifier		Name
0x00	mram_control	—
0x20	mram_control_pulse	—
0x28	gbl_cfg_ovr_0	—
0x30	gbl_cfg_0	—
0x38	man_control_0	—
0x40	man_control_1	—
0x48	mram_status_0	—
0x50	mram_status_1	—
0x58	mram_dout_even_lower	—
0x60	mram_dout_odd_lower	—
0x68	mram_dout_uppers	—
0x70	bist_status_0	—
0x78	bist_control	—
0x88	ecc_correction	—
0x98	bist_status_1	—

mram_control register

- Absolute Address: 0x2001200
- Base Offset: 0x0
- Size: 0x20



Bits	Identifier	Access	Reset	Name
16:0	addr_in	rw	0x0	addr_in
24:17	ce	rw	0x0	ce
25	we	rw	0x0	we
26	rd_pulse_meas_en	rw	0x0	rd_pulse_meas_en
33:27	rca_ovr	rw	0x0	rca_ovr
34	rca_ovr_en	rw	0x0	rca_ovr_en
35	gbl_cfg_ovr_en	rw	0x0	gbl_cfg_ovr_en
36	rd_en_ovr	rw	0x0	rd_en_ovr
37	ref_prg_en	rw	0x0	ref_prg_en
38	dsleep_mram_en	rw	0x0	dsleep_mram_en
39	reg_logic_sup_sleep_ovr	rw	0x1	reg_logic_sup_sleep_ovr
40	prg_rd1_byp	rw	0x0	prg_rd1_byp
41	wr_en_ovr	rw	0x0	wr_en_ovr
42	dma_en	rw	0x0	dma_en
43	vbslx_gain_mode_ovr	rw	0x0	vbslx_gain_mode_ovr
47:44	powerup_trim_load_ovr	rw	0x0	powerup_trim_load_ovr
52	test_cal_en	rw	0x0	test_cal_en
55:53	anatest0_sel	rw	0x0	anatest0_sel
58:56	anatest1_sel	rw	0x0	anatest1_sel
59	eccrom_deep_sleep	rw	0x1	eccrom_deep_sleep
60	ref_ecc_sel	rw	0x0	ref_ecc_sel
61	disable_cpu_intr	rw	0x0	disable_cpu_intr
62	disable_ted	rw	0x0	disable_ted
63	ecc_bypass_en	rw	0x0	ecc_bypass_en
142:64	bwe	rw	0x0	bwe
143	mram_clk_en	rw	0x1	mram_clk_en
222:144	din	rw	0x0	din
223	test_reg_ovr_en	rw	0x0	test_reg_ovr_en
224	otp_wr_en	rw	0x0	otp_wr_en
225	rst_cpu_intr	rw	0x0	rst_cpu_intr
233:226	dout_en	rw	0x0	dout_en
236:234	ecc_en	rw	0x0	ecc_en
240:237	even_man_stripe_sel	rw	0x0	even_man_stripe_sel
241	even_man_wr_0	rw	0x0	even_man_wr<0>
242	even_man_wr_1	rw	0x0	even_man_wr<1>
243	even_man_wr_2	rw	0x0	even_man_wr<2>
244	even_man_wr_3	rw	0x0	even_man_wr<3>
248:245	odd_man_stripe_sel	rw	0x0	odd_man_stripe_sel



CONTENTS

Bits	Identifier	Access	Reset	Name
249	odd_man_wr_0	rw	0x0	odd_man_wr<0>
250	odd_man_wr_1	rw	0x0	odd_man_wr<1>
251	odd_man_wr_2	rw	0x0	odd_man_wr<2>
252	odd_man_wr_3	rw	0x0	odd_man_wr<3>
253	sah_en	rw	0x1	sah_en
254	scc_otp_en	rw	0x0	scc_otp_en
255	maintenance_mode	rw	0x0	maintenance_mode

mram_control_pulse register

- Absolute Address: 0x2001220
- Base Offset: 0x20
- Size: 0x8

Bits	Identifier	Access	Reset	Name
0	powerup_trim_load_ovr_single_pulse	rw	0x0	powerup_trim_load_ovr_single_pulse<0>
1	powerup_trim_load_ovr_single_pulse	rw	0x0	powerup_trim_load_ovr_single_pulse<1>
2	powerup_trim_load_ovr_single_pulse	rw	0x0	powerup_trim_load_ovr_single_pulse<2>
3	powerup_trim_load_ovr_single_pulse	rw	0x0	powerup_trim_load_ovr_single_pulse<3>
4	mram_clk_single_pulse	rw	0x0	mram_clk_single_pulse

gbl_cfg_ovr_0 register

- Absolute Address: 0x2001228
- Base Offset: 0x28
- Size: 0x8

Bits	Identifier	Access	Reset	Name
1:0	sa_equal_trim	rw	0x1	sa_equal_trim<1:0>
4:2	vblslx_boost_trim	rw	0x4	vblslx_boost_trim<2:0>
8:5	wr_en_msb_trim	rw	0x9	wr_en_msb_trim<3:0>
11:9	wr_en_lsb_trim	rw	0x6	wr_en_lsb_trim<2:0>
12	vblslx_gain_mode	rw	0x0	vblslx_gain_mode
16:13	repulse_trim	rw	0x6	repulse_trim<3:0>
17	repulse_en	rw	0x1	repulse_en



Bits	Identifier	Access	Reset	Name
20:18	rd_en_trim	rw	0x5	rd_en_trim<2:0>
25:22	osc_wr_div_trim	rw	0x3	osc_wr_div_trim<3:0>
29:26	vblsl_trim	rw	0x4	vblsl_trim<3:0>
33:30	tcsel_trim	rw	0x3	tcsel_trim<3:0>
37:34	vwlrwr_trim	rw	0x7	vwlrwr_trim<3:0>
41:38	vcr_gate_trim	rw	0xE	vcr_gate_trim<3:0>

gbl_cfg_0 register

- Absolute Address: 0x2001230
- Base Offset: 0x30
- Size: 0x8

Bits	Identifier	Access	Reset	Name
1:0	sa_equal_trim	r	—	sa_equal_trim<1:0>
4:2	vblslx_boost_trim	r	—	vblslx_boost_trim<2:0>
8:5	wr_en_msb_trim	r	—	wr_en_msb_trim<3:0>
11:9	wr_en_lsb_trim	r	—	wr_en_lsb_trim<2:0>
12	vblslx_gain_mode	r	—	vblslx_gain_mode
16:13	repulse_trim	r	—	repulse_trim<3:0>
17	repulse_en	r	—	repulse_en
20:18	rd_en_trim	r	—	rd_en_trim<2:0>
25:22	osc_wr_div_trim	r	—	osc_wr_div_trim<3:0>
29:26	vblsl_trim	r	—	vblsl_trim<3:0>
33:30	tcsel_trim	r	—	tcsel_trim<3:0>
37:34	vwlrwr_trim	r	—	vwlrwr_trim<3:0>
41:38	vcr_gate_trim	r	—	vcr_gate_trim<3:0>

man_control_0 register

- Absolute Address: 0x2001238
- Base Offset: 0x38
- Size: 0x8



Bits	Identifier	Access	Reset	Name
3:0	blk0_man_ccnt	r	—	blk0_man_ccnt<3:0>
7:4	blk0_man_cnfg	rw	0x0	blk0_man_cnfg<3:0>
9:8	blk0_man_fcnt	r	—	blk0_man_fcnt<1:0>
19:16	blk1_man_ccnt	r	—	blk1_man_ccnt<3:0>
23:20	blk1_man_cnfg	rw	0x0	blk1_man_cnfg<3:0>
25:24	blk1_man_fcnt	r	—	blk1_man_fcnt<1:0>
35:32	blk2_man_ccnt	r	—	blk2_man_ccnt<3:0>
39:36	blk2_man_cnfg	rw	0x0	blk2_man_cnfg<3:0>
41:40	blk2_man_fcnt	r	—	blk2_man_fcnt<1:0>
51:48	blk3_man_ccnt	r	—	blk3_man_ccnt<3:0>
55:52	blk3_man_cnfg	rw	0x0	blk3_man_cnfg<3:0>
57:56	blk3_man_fcnt	r	—	blk3_man_fcnt<1:0>

man_control_1 register

- Absolute Address: 0x2001240
- Base Offset: 0x40
- Size: 0x8

Bits	Identifier	Access	Reset	Name
3:0	blk4_man_ccnt	r	—	blk4_man_ccnt<3:0>
7:4	blk4_man_cnfg	rw	0x0	blk4_man_cnfg<3:0>
9:8	blk4_man_fcnt	r	—	blk4_man_fcnt<1:0>
19:16	blk5_man_ccnt	r	—	blk5_man_ccnt<3:0>
23:20	blk5_man_cnfg	rw	0x0	blk5_man_cnfg<3:0>
25:24	blk5_man_fcnt	r	—	blk5_man_fcnt<1:0>
35:32	blk6_man_ccnt	r	—	blk6_man_ccnt<3:0>
39:36	blk6_man_cnfg	rw	0x0	blk6_man_cnfg<3:0>
41:40	blk6_man_fcnt	r	—	blk6_man_fcnt<1:0>
51:48	blk7_man_ccnt	r	—	blk7_man_ccnt<3:0>
55:52	blk7_man_cnfg	rw	0x0	blk7_man_cnfg<3:0>
57:56	blk7_man_fcnt	r	—	blk7_man_fcnt<1:0>

mram_status_0 register

- Absolute Address: 0x2001248
- Base Offset: 0x48



- Size: 0x8

Bits	Identifier	Access	Reset	Name
15:0	bist_error_loop	r	—	bist_error_loop<15:0>
33:17	bist_error_count	r	—	bist_error_count<16:0>
40:34	bist_rh0	r	—	bist_rh0<6:0>
47:41	bist_rh1	r	—	bist_rh1<6:0>
54:48	bist_rh2	r	—	bist_rh2<6:0>
56:55	cpu_intr_flag	r	—	cpu_intr_flag
58:57	ecc_1bit_flag	r	—	ecc_1bit_flag<1:0>
60:59	ecc_2bit_flag	r	—	ecc_2bit_flag<1:0>
62:61	ecc_3bit_flag	r	—	ecc_3bit_flag<1:0>

mram_status_1 register

- Absolute Address: 0x2001250
- Base Offset: 0x50
- Size: 0x8

Bits	Identifier	Access	Reset	Name
1:0	temp	r	—	temp<1:0>
4:3	ecc_1bit	r	—	ecc_1bit<1:0>
6:5	ecc_2bit	r	—	ecc_2bit<1:0>
8:7	ecc_3bit	r	—	ecc_3bit<1:0>
9	pwr_ok	r	—	pwr_ok
10	eccrom_pwr_ok	r	—	eccrom_pwr_ok
29:11	intr_error_lane0_addr	r	—	intr_error_lane0_addr<18:0>
48:30	intr_error_lane1_addr	r	—	intr_error_lane1_addr<18:0>
57:50	busy	r	—	busy<7:0>

mram_dout_even_lower register

- Absolute Address: 0x2001258
- Base Offset: 0x58
- Size: 0x8



Bits Identifier Access Reset Name			
63:0	dout	r	dout<63:0>

mram_dout_odd_lower register

- Absolute Address: 0x2001260
- Base Offset: 0x60
- Size: 0x8

Bits Identifier Access Reset Name			
63:0	dout	r	dout<142:79>

mram_dout_uppers register

- Absolute Address: 0x2001268
- Base Offset: 0x68
- Size: 0x8

Bits	Identifier	Access Reset Name	
14:0	dout_even_msb	r	dout<78:64>
46:32	dout_odd_msb	r	dout<157:143>

bist_status_0 register

- Absolute Address: 0x2001270
- Base Offset: 0x70
- Size: 0x8

Bits	Identifier	Access Reset Name	
63:0	bist_error_value	r	bist_error_value<63:0>

**bist_control register**

- Absolute Address: 0x2001278
- Base Offset: 0x78
- Size: 0x10

Bits	Identifier	Access	Reset	Name
0	bist_rte_en	rw	0x0	bist_rte_en
1	bist_data_inv	rw	0x0	bist_data_inv
4:2	bist_add_inc	rw	0x0	bist_add_inc<2:0>
5	bist_stop_on_error	rw	0x0	bist_stop_on_error
25:6	bist_start_add	rw	0x0	bist_start_add<19:0>
45:26	bist_stop_add	rw	0x0	bist_stop_add<19:0>
50:46	RH4margin	rw	0xA	RH4margin<4:0>
55:51	rh2_offset	rw	0x0	rh2_offset<4:0>
78:64	bist_error_value	r	—	bist_error_value<78:64>
94:79	bist_loop_count	rw	0x0	bist_loop_count<15:0>
95	bist_start	rw	0x0	bist_start
96	bist_trim_mode	rw	0x0	bist_trim_mode
97	bist_stop_on_repl_of	rw	0x0	bist_stop_on_repl_of
98	bist_rst_b	rw	0x1	bist_rst_b
99	bist_reset	rw	0x0	bist_reset
100	bist_rd_en	rw	0x0	bist_rd_en
101	bist_wr_en	rw	0x0	bist_wr_en

ecc_correction register

- Absolute Address: 0x2001288
- Base Offset: 0x88
- Size: 0x10

Bits	Identifier	Access	Reset	Name
127:0	dout	r	—	dout<127:0>

bist_status_1 register

- Absolute Address: 0x2001298
- Base Offset: 0x98



- Size: 0x8

Bits Identifier		Access		Reset	Name
19:0	bist_err_add	r	—		bist_err_add<19:0>
20	bist_error	r	—		bist_error
21	bist_busy	r	—		bist_busy

bank2_tregs register file

- Absolute Address: 0x2001300
- Base Offset: 0x300
- Size: 0xA0

Register space that contains the test registers for test functionality and debug of the MRAM.

Offset Identifier		Name
0x00	mram_control	—
0x20	mram_control_pulse	—
0x28	gbl_cfg_ovr_0	—
0x30	gbl_cfg_0	—
0x38	man_control_0	—
0x40	man_control_1	—
0x48	mram_status_0	—
0x50	mram_status_1	—
0x58	mram_dout_even_lower	—
0x60	mram_dout_odd_lower	—
0x68	mram_dout_uppers	—
0x70	bist_status_0	—
0x78	bist_control	—
0x88	ecc_correction	—
0x98	bist_status_1	—

mram_control register

- Absolute Address: 0x2001300
- Base Offset: 0x0
- Size: 0x20



Bits	Identifier	Access	Reset	Name
16:0	addr_in	rw	0x0	addr_in
24:17	ce	rw	0x0	ce
25	we	rw	0x0	we
26	rd_pulse_meas_en	rw	0x0	rd_pulse_meas_en
33:27	rca_ovr	rw	0x0	rca_ovr
34	rca_ovr_en	rw	0x0	rca_ovr_en
35	gbl_cfg_ovr_en	rw	0x0	gbl_cfg_ovr_en
36	rd_en_ovr	rw	0x0	rd_en_ovr
37	ref_prg_en	rw	0x0	ref_prg_en
38	dsleep_mram_en	rw	0x0	dsleep_mram_en
39	reg_logic_sup_sleep_ovr	rw	0x1	reg_logic_sup_sleep_ovr
40	prg_rd1_byp	rw	0x0	prg_rd1_byp
41	wr_en_ovr	rw	0x0	wr_en_ovr
42	dma_en	rw	0x0	dma_en
43	vbslx_gain_mode_ovr	rw	0x0	vbslx_gain_mode_ovr
47:44	powerup_trim_load_ovr	rw	0x0	powerup_trim_load_ovr
52	test_cal_en	rw	0x0	test_cal_en
55:53	anatest0_sel	rw	0x0	anatest0_sel
58:56	anatest1_sel	rw	0x0	anatest1_sel
59	eccrom_deep_sleep	rw	0x1	eccrom_deep_sleep
60	ref_ecc_sel	rw	0x0	ref_ecc_sel
61	disable_cpu_intr	rw	0x0	disable_cpu_intr
62	disable_ted	rw	0x0	disable_ted
63	ecc_bypass_en	rw	0x0	ecc_bypass_en
142:64	bwe	rw	0x0	bwe
143	mram_clk_en	rw	0x1	mram_clk_en
222:144	din	rw	0x0	din
223	test_reg_ovr_en	rw	0x0	test_reg_ovr_en
224	otp_wr_en	rw	0x0	otp_wr_en
225	rst_cpu_intr	rw	0x0	rst_cpu_intr
233:226	dout_en	rw	0x0	dout_en
236:234	ecc_en	rw	0x0	ecc_en
240:237	even_man_stripe_sel	rw	0x0	even_man_stripe_sel
241	even_man_wr_0	rw	0x0	even_man_wr<0>
242	even_man_wr_1	rw	0x0	even_man_wr<1>
243	even_man_wr_2	rw	0x0	even_man_wr<2>
244	even_man_wr_3	rw	0x0	even_man_wr<3>
248:245	odd_man_stripe_sel	rw	0x0	odd_man_stripe_sel



CONTENTS

Bits	Identifier	Access	Reset	Name
249	odd_man_wr_0	rw	0x0	odd_man_wr<0>
250	odd_man_wr_1	rw	0x0	odd_man_wr<1>
251	odd_man_wr_2	rw	0x0	odd_man_wr<2>
252	odd_man_wr_3	rw	0x0	odd_man_wr<3>
253	sah_en	rw	0x1	sah_en
254	scc_otp_en	rw	0x0	scc_otp_en
255	maintenance_mode	rw	0x0	maintenance_mode

mram_control_pulse register

- Absolute Address: 0x2001320
- Base Offset: 0x20
- Size: 0x8

Bits	Identifier	Access	Reset	Name
0	powerup_trim_load_ovr_single_pulse	rw	0x0	powerup_trim_load_ovr_single_pulse<0>
1	powerup_trim_load_ovr_single_pulse	rw	0x0	powerup_trim_load_ovr_single_pulse<1>
2	powerup_trim_load_ovr_single_pulse	rw	0x0	powerup_trim_load_ovr_single_pulse<2>
3	powerup_trim_load_ovr_single_pulse	rw	0x0	powerup_trim_load_ovr_single_pulse<3>
4	mram_clk_single_pulse	rw	0x0	mram_clk_single_pulse

gbl_cfg_ovr_0 register

- Absolute Address: 0x2001328
- Base Offset: 0x28
- Size: 0x8

Bits	Identifier	Access	Reset	Name
1:0	sa_equal_trim	rw	0x1	sa_equal_trim<1:0>
4:2	vblslx_boost_trim	rw	0x4	vblslx_boost_trim<2:0>
8:5	wr_en_msb_trim	rw	0x9	wr_en_msb_trim<3:0>
11:9	wr_en_lsb_trim	rw	0x6	wr_en_lsb_trim<2:0>
12	vblslx_gain_mode	rw	0x0	vblslx_gain_mode
16:13	repulse_trim	rw	0x6	repulse_trim<3:0>
17	repulse_en	rw	0x1	repulse_en



Bits	Identifier	Access	Reset	Name
20:18	rd_en_trim	rw	0x5	rd_en_trim<2:0>
25:22	osc_wr_div_trim	rw	0x3	osc_wr_div_trim<3:0>
29:26	vblsl_trim	rw	0x4	vblsl_trim<3:0>
33:30	tcsel_trim	rw	0x3	tcsel_trim<3:0>
37:34	vwlrwr_trim	rw	0x7	vwlrwr_trim<3:0>
41:38	vcr_gate_trim	rw	0xE	vcr_gate_trim<3:0>

gbl_cfg_0 register

- Absolute Address: 0x2001330
- Base Offset: 0x30
- Size: 0x8

Bits	Identifier	Access	Reset	Name
1:0	sa_equal_trim	r	—	sa_equal_trim<1:0>
4:2	vblslx_boost_trim	r	—	vblslx_boost_trim<2:0>
8:5	wr_en_msb_trim	r	—	wr_en_msb_trim<3:0>
11:9	wr_en_lsb_trim	r	—	wr_en_lsb_trim<2:0>
12	vblslx_gain_mode	r	—	vblslx_gain_mode
16:13	repulse_trim	r	—	repulse_trim<3:0>
17	repulse_en	r	—	repulse_en
20:18	rd_en_trim	r	—	rd_en_trim<2:0>
25:22	osc_wr_div_trim	r	—	osc_wr_div_trim<3:0>
29:26	vblsl_trim	r	—	vblsl_trim<3:0>
33:30	tcsel_trim	r	—	tcsel_trim<3:0>
37:34	vwlrwr_trim	r	—	vwlrwr_trim<3:0>
41:38	vcr_gate_trim	r	—	vcr_gate_trim<3:0>

man_control_0 register

- Absolute Address: 0x2001338
- Base Offset: 0x38
- Size: 0x8



Bits	Identifier	Access	Reset	Name
3:0	blk0_man_ccnt	r	—	blk0_man_ccnt<3:0>
7:4	blk0_man_cnfg	rw	0x0	blk0_man_cnfg<3:0>
9:8	blk0_man_fcnt	r	—	blk0_man_fcnt<1:0>
19:16	blk1_man_ccnt	r	—	blk1_man_ccnt<3:0>
23:20	blk1_man_cnfg	rw	0x0	blk1_man_cnfg<3:0>
25:24	blk1_man_fcnt	r	—	blk1_man_fcnt<1:0>
35:32	blk2_man_ccnt	r	—	blk2_man_ccnt<3:0>
39:36	blk2_man_cnfg	rw	0x0	blk2_man_cnfg<3:0>
41:40	blk2_man_fcnt	r	—	blk2_man_fcnt<1:0>
51:48	blk3_man_ccnt	r	—	blk3_man_ccnt<3:0>
55:52	blk3_man_cnfg	rw	0x0	blk3_man_cnfg<3:0>
57:56	blk3_man_fcnt	r	—	blk3_man_fcnt<1:0>

man_control_1 register

- Absolute Address: 0x2001340
- Base Offset: 0x40
- Size: 0x8

Bits	Identifier	Access	Reset	Name
3:0	blk4_man_ccnt	r	—	blk4_man_ccnt<3:0>
7:4	blk4_man_cnfg	rw	0x0	blk4_man_cnfg<3:0>
9:8	blk4_man_fcnt	r	—	blk4_man_fcnt<1:0>
19:16	blk5_man_ccnt	r	—	blk5_man_ccnt<3:0>
23:20	blk5_man_cnfg	rw	0x0	blk5_man_cnfg<3:0>
25:24	blk5_man_fcnt	r	—	blk5_man_fcnt<1:0>
35:32	blk6_man_ccnt	r	—	blk6_man_ccnt<3:0>
39:36	blk6_man_cnfg	rw	0x0	blk6_man_cnfg<3:0>
41:40	blk6_man_fcnt	r	—	blk6_man_fcnt<1:0>
51:48	blk7_man_ccnt	r	—	blk7_man_ccnt<3:0>
55:52	blk7_man_cnfg	rw	0x0	blk7_man_cnfg<3:0>
57:56	blk7_man_fcnt	r	—	blk7_man_fcnt<1:0>

mram_status_0 register

- Absolute Address: 0x2001348
- Base Offset: 0x48



- Size: 0x8

Bits	Identifier	Access	Reset	Name
15:0	bist_error_loop	r	—	bist_error_loop<15:0>
33:17	bist_error_count	r	—	bist_error_count<16:0>
40:34	bist_rh0	r	—	bist_rh0<6:0>
47:41	bist_rh1	r	—	bist_rh1<6:0>
54:48	bist_rh2	r	—	bist_rh2<6:0>
56:55	cpu_intr_flag	r	—	cpu_intr_flag
58:57	ecc_1bit_flag	r	—	ecc_1bit_flag<1:0>
60:59	ecc_2bit_flag	r	—	ecc_2bit_flag<1:0>
62:61	ecc_3bit_flag	r	—	ecc_3bit_flag<1:0>

mram_status_1 register

- Absolute Address: 0x2001350
- Base Offset: 0x50
- Size: 0x8

Bits	Identifier	Access	Reset	Name
1:0	temp	r	—	temp<1:0>
4:3	ecc_1bit	r	—	ecc_1bit<1:0>
6:5	ecc_2bit	r	—	ecc_2bit<1:0>
8:7	ecc_3bit	r	—	ecc_3bit<1:0>
9	pwr_ok	r	—	pwr_ok
10	eccrom_pwr_ok	r	—	eccrom_pwr_ok
29:11	intr_error_lane0_addr	r	—	intr_error_lane0_addr<18:0>
48:30	intr_error_lane1_addr	r	—	intr_error_lane1_addr<18:0>
57:50	busy	r	—	busy<7:0>

mram_dout_even_lower register

- Absolute Address: 0x2001358
- Base Offset: 0x58
- Size: 0x8



Bits Identifier Access Reset Name			
63:0	dout	r	dout<63:0>

mram_dout_odd_lower register

- Absolute Address: 0x2001360
- Base Offset: 0x60
- Size: 0x8

Bits Identifier Access Reset Name			
63:0	dout	r	dout<142:79>

mram_dout_uppers register

- Absolute Address: 0x2001368
- Base Offset: 0x68
- Size: 0x8

Bits	Identifier	Access Reset Name	
14:0	dout_even_msb	r	dout<78:64>
46:32	dout_odd_msb	r	dout<157:143>

bist_status_0 register

- Absolute Address: 0x2001370
- Base Offset: 0x70
- Size: 0x8

Bits	Identifier	Access Reset Name	
63:0	bist_error_value	r	bist_error_value<63:0>

**bist_control register**

- Absolute Address: 0x2001378
- Base Offset: 0x78
- Size: 0x10

Bits	Identifier	Access	Reset	Name
0	bist_rte_en	rw	0x0	bist_rte_en
1	bist_data_inv	rw	0x0	bist_data_inv
4:2	bist_add_inc	rw	0x0	bist_add_inc<2:0>
5	bist_stop_on_error	rw	0x0	bist_stop_on_error
25:6	bist_start_add	rw	0x0	bist_start_add<19:0>
45:26	bist_stop_add	rw	0x0	bist_stop_add<19:0>
50:46	RH4margin	rw	0xA	RH4margin<4:0>
55:51	rh2_offset	rw	0x0	rh2_offset<4:0>
78:64	bist_error_value	r	—	bist_error_value<78:64>
94:79	bist_loop_count	rw	0x0	bist_loop_count<15:0>
95	bist_start	rw	0x0	bist_start
96	bist_trim_mode	rw	0x0	bist_trim_mode
97	bist_stop_on_repl_of	rw	0x0	bist_stop_on_repl_of
98	bist_rst_b	rw	0x1	bist_rst_b
99	bist_reset	rw	0x0	bist_reset
100	bist_rd_en	rw	0x0	bist_rd_en
101	bist_wr_en	rw	0x0	bist_wr_en

ecc_correction register

- Absolute Address: 0x2001388
- Base Offset: 0x88
- Size: 0x10

Bits	Identifier	Access	Reset	Name
127:0	dout	r	—	dout<127:0>

bist_status_1 register

- Absolute Address: 0x2001398
- Base Offset: 0x98



- Size: 0x8

Bits Identifier		Access Reset Name	
19:0	bist_err_add r	—	bist_err_add<19:0>
20	bist_error	r	bist_error
21	bist_busy	r	bist_busy

bank3_tregs register file

- Absolute Address: 0x2001400
- Base Offset: 0x400
- Size: 0xA0

Register space that contains the test registers for test functionality and debug of the MRAM.

Offset Identifier		Name
0x00	mram_control	—
0x20	mram_control_pulse	—
0x28	gbl_cfg_ovr_0	—
0x30	gbl_cfg_0	—
0x38	man_control_0	—
0x40	man_control_1	—
0x48	mram_status_0	—
0x50	mram_status_1	—
0x58	mram_dout_even_lower	—
0x60	mram_dout_odd_lower	—
0x68	mram_dout_uppers	—
0x70	bist_status_0	—
0x78	bist_control	—
0x88	ecc_correction	—
0x98	bist_status_1	—

mram_control register

- Absolute Address: 0x2001400
- Base Offset: 0x0
- Size: 0x20



Bits	Identifier	Access	Reset	Name
16:0	addr_in	rw	0x0	addr_in
24:17	ce	rw	0x0	ce
25	we	rw	0x0	we
26	rd_pulse_meas_en	rw	0x0	rd_pulse_meas_en
33:27	rca_ovr	rw	0x0	rca_ovr
34	rca_ovr_en	rw	0x0	rca_ovr_en
35	gbl_cfg_ovr_en	rw	0x0	gbl_cfg_ovr_en
36	rd_en_ovr	rw	0x0	rd_en_ovr
37	ref_prg_en	rw	0x0	ref_prg_en
38	dsleep_mram_en	rw	0x0	dsleep_mram_en
39	reg_logic_sup_sleep_ovr	rw	0x1	reg_logic_sup_sleep_ovr
40	prg_rd1_byp	rw	0x0	prg_rd1_byp
41	wr_en_ovr	rw	0x0	wr_en_ovr
42	dma_en	rw	0x0	dma_en
43	vblslx_gain_mode_ovr	rw	0x0	vblslx_gain_mode_ovr
47:44	powerup_trim_load_ovr	rw	0x0	powerup_trim_load_ovr
52	test_cal_en	rw	0x0	test_cal_en
55:53	anatest0_sel	rw	0x0	anatest0_sel
58:56	anatest1_sel	rw	0x0	anatest1_sel
59	eccrom_deep_sleep	rw	0x1	eccrom_deep_sleep
60	ref_ecc_sel	rw	0x0	ref_ecc_sel
61	disable_cpu_intr	rw	0x0	disable_cpu_intr
62	disable_ted	rw	0x0	disable_ted
63	ecc_bypass_en	rw	0x0	ecc_bypass_en
142:64	bwe	rw	0x0	bwe
143	mram_clk_en	rw	0x1	mram_clk_en
222:144	din	rw	0x0	din
223	test_reg_ovr_en	rw	0x0	test_reg_ovr_en
224	otp_wr_en	rw	0x0	otp_wr_en
225	rst_cpu_intr	rw	0x0	rst_cpu_intr
233:226	dout_en	rw	0x0	dout_en
236:234	ecc_en	rw	0x0	ecc_en
240:237	even_man_stripe_sel	rw	0x0	even_man_stripe_sel
241	even_man_wr_0	rw	0x0	even_man_wr<0>
242	even_man_wr_1	rw	0x0	even_man_wr<1>
243	even_man_wr_2	rw	0x0	even_man_wr<2>
244	even_man_wr_3	rw	0x0	even_man_wr<3>
248:245	odd_man_stripe_sel	rw	0x0	odd_man_stripe_sel



CONTENTS

Bits	Identifier	Access	Reset	Name
249	odd_man_wr_0	rw	0x0	odd_man_wr<0>
250	odd_man_wr_1	rw	0x0	odd_man_wr<1>
251	odd_man_wr_2	rw	0x0	odd_man_wr<2>
252	odd_man_wr_3	rw	0x0	odd_man_wr<3>
253	sah_en	rw	0x1	sah_en
254	scc_otp_en	rw	0x0	scc_otp_en
255	maintenance_mode	rw	0x0	maintenance_mode

mram_control_pulse register

- Absolute Address: 0x2001420
- Base Offset: 0x20
- Size: 0x8

Bits	Identifier	Access	Reset	Name
0	powerup_trim_load_ovr_single_pulse	rw	0x0	powerup_trim_load_ovr_single_pulse<0>
1	powerup_trim_load_ovr_single_pulse	rw	0x0	powerup_trim_load_ovr_single_pulse<1>
2	powerup_trim_load_ovr_single_pulse	rw	0x0	powerup_trim_load_ovr_single_pulse<2>
3	powerup_trim_load_ovr_single_pulse	rw	0x0	powerup_trim_load_ovr_single_pulse<3>
4	mram_clk_single_pulse	rw	0x0	mram_clk_single_pulse

gbl_cfg_ovr_0 register

- Absolute Address: 0x2001428
- Base Offset: 0x28
- Size: 0x8

Bits	Identifier	Access	Reset	Name
1:0	sa_equal_trim	rw	0x1	sa_equal_trim<1:0>
4:2	vblslx_boost_trim	rw	0x4	vblslx_boost_trim<2:0>
8:5	wr_en_msb_trim	rw	0x9	wr_en_msb_trim<3:0>
11:9	wr_en_lsb_trim	rw	0x6	wr_en_lsb_trim<2:0>
12	vblslx_gain_mode	rw	0x0	vblslx_gain_mode
16:13	repulse_trim	rw	0x6	repulse_trim<3:0>
17	repulse_en	rw	0x1	repulse_en



Bits	Identifier	Access	Reset	Name
20:18	rd_en_trim	rw	0x5	rd_en_trim<2:0>
25:22	osc_wr_div_trim	rw	0x3	osc_wr_div_trim<3:0>
29:26	vblsl_trim	rw	0x4	vblsl_trim<3:0>
33:30	tcsel_trim	rw	0x3	tcsel_trim<3:0>
37:34	vwlrwr_trim	rw	0x7	vwlrwr_trim<3:0>
41:38	vcr_gate_trim	rw	0xE	vcr_gate_trim<3:0>

gbl_cfg_0 register

- Absolute Address: 0x2001430
- Base Offset: 0x30
- Size: 0x8

Bits	Identifier	Access	Reset	Name
1:0	sa_equal_trim	r	—	sa_equal_trim<1:0>
4:2	vblslx_boost_trim	r	—	vblslx_boost_trim<2:0>
8:5	wr_en_msb_trim	r	—	wr_en_msb_trim<3:0>
11:9	wr_en_lsb_trim	r	—	wr_en_lsb_trim<2:0>
12	vblslx_gain_mode	r	—	vblslx_gain_mode
16:13	repulse_trim	r	—	repulse_trim<3:0>
17	repulse_en	r	—	repulse_en
20:18	rd_en_trim	r	—	rd_en_trim<2:0>
25:22	osc_wr_div_trim	r	—	osc_wr_div_trim<3:0>
29:26	vblsl_trim	r	—	vblsl_trim<3:0>
33:30	tcsel_trim	r	—	tcsel_trim<3:0>
37:34	vwlrwr_trim	r	—	vwlrwr_trim<3:0>
41:38	vcr_gate_trim	r	—	vcr_gate_trim<3:0>

man_control_0 register

- Absolute Address: 0x2001438
- Base Offset: 0x38
- Size: 0x8



Bits	Identifier	Access	Reset	Name
3:0	blk0_man_ccnt	r	—	blk0_man_ccnt<3:0>
7:4	blk0_man_cnfg	rw	0x0	blk0_man_cnfg<3:0>
9:8	blk0_man_fcnt	r	—	blk0_man_fcnt<1:0>
19:16	blk1_man_ccnt	r	—	blk1_man_ccnt<3:0>
23:20	blk1_man_cnfg	rw	0x0	blk1_man_cnfg<3:0>
25:24	blk1_man_fcnt	r	—	blk1_man_fcnt<1:0>
35:32	blk2_man_ccnt	r	—	blk2_man_ccnt<3:0>
39:36	blk2_man_cnfg	rw	0x0	blk2_man_cnfg<3:0>
41:40	blk2_man_fcnt	r	—	blk2_man_fcnt<1:0>
51:48	blk3_man_ccnt	r	—	blk3_man_ccnt<3:0>
55:52	blk3_man_cnfg	rw	0x0	blk3_man_cnfg<3:0>
57:56	blk3_man_fcnt	r	—	blk3_man_fcnt<1:0>

man_control_1 register

- Absolute Address: 0x2001440
- Base Offset: 0x40
- Size: 0x8

Bits	Identifier	Access	Reset	Name
3:0	blk4_man_ccnt	r	—	blk4_man_ccnt<3:0>
7:4	blk4_man_cnfg	rw	0x0	blk4_man_cnfg<3:0>
9:8	blk4_man_fcnt	r	—	blk4_man_fcnt<1:0>
19:16	blk5_man_ccnt	r	—	blk5_man_ccnt<3:0>
23:20	blk5_man_cnfg	rw	0x0	blk5_man_cnfg<3:0>
25:24	blk5_man_fcnt	r	—	blk5_man_fcnt<1:0>
35:32	blk6_man_ccnt	r	—	blk6_man_ccnt<3:0>
39:36	blk6_man_cnfg	rw	0x0	blk6_man_cnfg<3:0>
41:40	blk6_man_fcnt	r	—	blk6_man_fcnt<1:0>
51:48	blk7_man_ccnt	r	—	blk7_man_ccnt<3:0>
55:52	blk7_man_cnfg	rw	0x0	blk7_man_cnfg<3:0>
57:56	blk7_man_fcnt	r	—	blk7_man_fcnt<1:0>

mram_status_0 register

- Absolute Address: 0x2001448
- Base Offset: 0x48



- Size: 0x8

Bits	Identifier	Access	Reset	Name
15:0	bist_error_loop	r	—	bist_error_loop<15:0>
33:17	bist_error_count	r	—	bist_error_count<16:0>
40:34	bist_rh0	r	—	bist_rh0<6:0>
47:41	bist_rh1	r	—	bist_rh1<6:0>
54:48	bist_rh2	r	—	bist_rh2<6:0>
56:55	cpu_intr_flag	r	—	cpu_intr_flag
58:57	ecc_1bit_flag	r	—	ecc_1bit_flag<1:0>
60:59	ecc_2bit_flag	r	—	ecc_2bit_flag<1:0>
62:61	ecc_3bit_flag	r	—	ecc_3bit_flag<1:0>

mram_status_1 register

- Absolute Address: 0x2001450
- Base Offset: 0x50
- Size: 0x8

Bits	Identifier	Access	Reset	Name
1:0	temp	r	—	temp<1:0>
4:3	ecc_1bit	r	—	ecc_1bit<1:0>
6:5	ecc_2bit	r	—	ecc_2bit<1:0>
8:7	ecc_3bit	r	—	ecc_3bit<1:0>
9	pwr_ok	r	—	pwr_ok
10	eccrom_pwr_ok	r	—	eccrom_pwr_ok
29:11	intr_error_lane0_addr	r	—	intr_error_lane0_addr<18:0>
48:30	intr_error_lane1_addr	r	—	intr_error_lane1_addr<18:0>
57:50	busy	r	—	busy<7:0>

mram_dout_even_lower register

- Absolute Address: 0x2001458
- Base Offset: 0x58
- Size: 0x8



Bits Identifier Access Reset Name			
63:0	dout	r	dout<63:0>

mram_dout_odd_lower register

- Absolute Address: 0x2001460
- Base Offset: 0x60
- Size: 0x8

Bits Identifier Access Reset Name			
63:0	dout	r	dout<142:79>

mram_dout_uppers register

- Absolute Address: 0x2001468
- Base Offset: 0x68
- Size: 0x8

Bits	Identifier	Access Reset Name	
14:0	dout_even_msb	r	dout<78:64>
46:32	dout_odd_msb	r	dout<157:143>

bist_status_0 register

- Absolute Address: 0x2001470
- Base Offset: 0x70
- Size: 0x8

Bits	Identifier	Access Reset Name	
63:0	bist_error_value	r	bist_error_value<63:0>

**bist_control register**

- Absolute Address: 0x2001478
- Base Offset: 0x78
- Size: 0x10

Bits	Identifier	Access	Reset	Name
0	bist_rte_en	rw	0x0	bist_rte_en
1	bist_data_inv	rw	0x0	bist_data_inv
4:2	bist_add_inc	rw	0x0	bist_add_inc<2:0>
5	bist_stop_on_error	rw	0x0	bist_stop_on_error
25:6	bist_start_add	rw	0x0	bist_start_add<19:0>
45:26	bist_stop_add	rw	0x0	bist_stop_add<19:0>
50:46	RH4margin	rw	0xA	RH4margin<4:0>
55:51	rh2_offset	rw	0x0	rh2_offset<4:0>
78:64	bist_error_value	r	—	bist_error_value<78:64>
94:79	bist_loop_count	rw	0x0	bist_loop_count<15:0>
95	bist_start	rw	0x0	bist_start
96	bist_trim_mode	rw	0x0	bist_trim_mode
97	bist_stop_on_repl_of	rw	0x0	bist_stop_on_repl_of
98	bist_rst_b	rw	0x1	bist_rst_b
99	bist_reset	rw	0x0	bist_reset
100	bist_rd_en	rw	0x0	bist_rd_en
101	bist_wr_en	rw	0x0	bist_wr_en

ecc_correction register

- Absolute Address: 0x2001488
- Base Offset: 0x88
- Size: 0x10

Bits	Identifier	Access	Reset	Name
127:0	dout	r	—	dout<127:0>

bist_status_1 register

- Absolute Address: 0x2001498
- Base Offset: 0x98



- Size: 0x8

Bits Identifier		Access		Reset	Name
19:0	bist_err_add	r	—		bist_err_add<19:0>
20	bist_error	r	—		bist_error
21	bist_busy	r	—		bist_busy

i2c_registers address map

- Absolute Address: 0x2002000
- Base Offset: 0x2002000
- Size: 0x24

Offset	Identifier	Name
0x00	Commands	Command Registers
0x08	Status	—
0x10	Cfg	—
0x18	Wdata	—
0x20	Rdata	—

Commands register

- Absolute Address: 0x2002000
- Base Offset: 0x0
- Size: 0x4

I2C Command Registers

Bits Identifier		Access		Reset	Name
0	start	rw	0x0	—	
1	read	rw	0x0	—	
2	write	rw	0x0	—	
3	write_multiple	rw	0x0	—	
4	stop	rw	0x0	—	
5	enq	rw	0x0	—	
14:8	address	rw	0x0	—	



start field Start

read field TODO

write field TODO

write_multiple field TODO

stop field TODO

enq field TODO

address field Address

Status register

- Absolute Address: 0x2002008
- Base Offset: 0x8
- Size: 0x4

Bits Identifier		Access Reset Name	
0	busy	r	0x0 —
1	bus_control	r	0x0 —
2	bus_active	r	0x0 —
3	missed_ack	r, rclr	0x0 —
4	cmd_ff_n_full	r	— —
5	tx_ff_n_full	r	— —
6	rx_ff_n_full	r	— —
7	rx_overflow	r	— —

busy field Transaction ongoing

bus_control field Transaction ongoing

bus_active field Transaction ongoing

missed_ack field Transaction ongoing



CONTENTS

cmd_ff_n_full field cmd_ff not Full

tx_ff_n_full field tx_ff not Full

rx_ff_n_full field rx_ff not Full

rx_overflow field rx overflow

Cfg register

- Absolute Address: 0x2002010
- Base Offset: 0x10
- Size: 0x4

Bits Identifier		Access Reset Name		
15:0	prescale	rw	0x0	—
16	stop_on_idle	rw	0x0	—

prescale field Prescale

stop_on_idle field Stop on Idle

Wdata register

- Absolute Address: 0x2002018
- Base Offset: 0x18
- Size: 0x4

Bits Identifier		Access Reset Name		
7:0	wdata	w	0x0	—
8	wlast	w	0x0	—

wdata field TxData

wlast field Write Last



Rdata register

- Absolute Address: 0x2002020
- Base Offset: 0x20
- Size: 0x4

Bits Identifier Access Reset Name				
7:0	rdata	r	0x0	—
8	rlast	r	0x0	—

rdata field RxData

rlast field Read Last

qspi_registers address map

- Absolute Address: 0x2003000
- Base Offset: 0x2003000
- Size: 0x64

Shakti QSPI Registers

Offset Identifier Name		
0x00	CR	—
0x08	DCR	—
0x10	SR	—
0x18	FCR	—
0x20	DLR	—
0x28	CCR	—
0x30	AR	—
0x38	ABR	—
0x40	DR	—
0x48	PSMKR	—
0x50	PSMAR	—
0x58	PIR	—
0x60	LPTR	—

**CR register**

- Absolute Address: 0x2003000
- Base Offset: 0x0
- Size: 0x4

Bits	Identifier	Access	Reset	Name
0	qspi_enable	rw	0x0	—
1	abort	rw	0x0	—
2	dmaen	rw	0x0	—
3	tcen	rw	0x0	—
4	sshift	rw	0x0	—
6	dfm	rw	0x0	—
7	fsel	rw	0x0	—
12:8	fthres	rw	0x0	—
16	teie	rw	0x0	—
17	tcie	rw	0x0	—
18	ftie	rw	0x0	—
19	smie	rw	0x0	—
20	toie	rw	0x0	—
22	apms	rw	0x0	—
23	pmm	rw	0x0	—
31:24	prescaler	rw	0x0	—

qspi_enable field QSPI enable

abort field Abort request

dmaen field DMA enable

tcen field Timeout counter enable

sshift field Sample shift

dfm field Dual flash mode enable

fsel field Flash select



ftbres field FIFO threshold level

teie field Transfer error interrupt enable

tcie field Transfer complete interrupt enable

ftie field FIFO threshold interrupt enable

smie field Status match interrupt enable

toie field Timeout interrupt enable

apms field Automatic polling mode stop

pmm field Polling match mode (0=AND,1=OR)

prescaler field Clock prescaler (PRESCALER[7:0])

DCR register

- Absolute Address: 0x2003008
- Base Offset: 0x8
- Size: 0x4

Bits	Identifier	Access	Reset	Name
0	ckmode	rw	0x0	—
10:8	csht	rw	0x0	—
20:16	fsize	rw	0x0	—

ckmode field Clock mode (0=Mode0,1=Mode3)

csht field Chip select high time

fsize field Flash memory size

**SR register**

- Absolute Address: 0x2003010
- Base Offset: 0x10
- Size: 0x4

Bits Identifier Access Reset Name				
0	tef	r	—	—
1	tcf	r	—	—
2	ftf	r	—	—
3	smf	r	—	—
4	tof	r	—	—
5	busy	r	—	—
13:8	flevel	r	—	—

tef field Transfer error flag

tcf field Transfer complete flag

ftf field FIFO threshold flag

smf field Status match flag

tof field Timeout flag

busy field Busy flag

flevel field FIFO level

FCR register

- Absolute Address: 0x2003018
- Base Offset: 0x18
- Size: 0x4



Bits Identifier Access Reset Name				
0	ctef	rw	0x0	—
1	ctcf	rw	0x0	—
3	csmf	rw	0x0	—
4	ctof	rw	0x0	—

ctef field Clear transfer error flag

ctcf field Clear transfer complete flag

csmf field Clear status match flag

ctof field Clear timeout flag

DLR register

- Absolute Address: 0x2003020
- Base Offset: 0x20
- Size: 0x4

Bits Identifier Access Reset Name				
31:0	dl	rw	0x0	—

dl field Number of bytes to transfer (value+1)

CCR register

- Absolute Address: 0x2003028
- Base Offset: 0x28
- Size: 0x4

Bits Identifier Access Reset Name				
7:0	instr	rw	0x0	—
9:8	imode	rw	0x0	—
11:10	admode	rw	0x0	—
13:12	adsize	rw	0x0	—



CONTENTS

Bits	Identifier	Access	Reset	Name
15:14	abmode	rw	0x0	—
17:16	absize	rw	0x0	—
22:18	dcyc	rw	0x0	—
23	d_conf	rw	0x0	—
25:24	dmode	rw	0x0	—
27:26	fmode	rw	0x0	—
28	sioo	rw	0x0	—
30	dhhc	rw	0x0	—
31	ddrm	rw	0x0	—

instr field Instruction opcode

imode field Instruction mode

admode field Address mode

adsize field Address size

abmode field Alternate byte mode

absize field Alternate byte size

dcyc field Dummy cycles

d_conf field Dummy confirmation

dmode field Data mode

fmode field Functional mode

sioo field Send instruction only once

dhhc field DDR hold quarter cycle

ddrm field DDR mode enable

**AR register**

- Absolute Address: 0x2003030
- Base Offset: 0x30
- Size: 0x4

Bits Identifier Access Reset Name			
31:0 address	rw	0x0	—

address field Flash address

ABR register

- Absolute Address: 0x2003038
- Base Offset: 0x38
- Size: 0x4

Bits Identifier Access Reset Name			
31:0 alternate_bytes	rw	0x0	—

alternate_bytes field Alternate bytes

DR register

- Absolute Address: 0x2003040
- Base Offset: 0x40
- Size: 0x4

Bits Identifier Access Reset Name			
31:0 data	rw	0x0	—

data field Data register (FIFO access)

**PSMKR register**

- Absolute Address: 0x2003048
- Base Offset: 0x48
- Size: 0x4

Bits Identifier Access Reset Name			
31:0 mask	rw	0x0	—

mask field Polling status mask

PSMAR register

- Absolute Address: 0x2003050
- Base Offset: 0x50
- Size: 0x4

Bits Identifier Access Reset Name			
31:0 match	rw	0x0	—

match field Polling status match value

PIR register

- Absolute Address: 0x2003058
- Base Offset: 0x58
- Size: 0x4

Bits Identifier Access Reset Name			
31:0 interval	rw	0x0	—

interval field Polling interval in clock cycles



LPTR register

- Absolute Address: 0x2003060
- Base Offset: 0x60
- Size: 0x4

Bits Identifier Access Reset Name			
31:0	timeout	rw	0x0 —

timeout field Low power timeout value

uart_registers address map

- Absolute Address: 0x2004000
- Base Offset: 0x2004000
- Size: 0x5C

UART Registers Refer shakti GC doc for details

Offset	Identifier	Name
0x00	BaudReg	—
0x08	TxReg	—
0x10	RxReg	—
0x18	StatusReg	—
0x20	DelayReg	—
0x28	ControlReg	—
0x30	IQC	—
0x38	Rx_Threshold	—
0x40	NotUsed	—
0x48	InterruptRaw	—
0x50	InterruptMask	—
0x58	InterruptStatus	—

BaudReg register

- Absolute Address: 0x2004000
- Base Offset: 0x0
- Size: 0x4



CONTENTS

Bits Identifier	Access	Reset	Name
15:0 baud_value	rw	0x5	—

baud_value field $\text{Baud} = \text{Clk_Freq} / (16 * \text{baud_value})$

TxReg register

- Absolute Address: 0x2004008
- Base Offset: 0x8
- Size: 0x4

Bits Identifier	Access	Reset	Name
31:0 data	w	0x0	—

data field

RxReg register

- Absolute Address: 0x2004010
- Base Offset: 0x10
- Size: 0x4

Bits Identifier	Access	Reset	Name
31:0 data	r	0x0	—

data field

StatusReg register

- Absolute Address: 0x2004018
- Base Offset: 0x18
- Size: 0x4



Bits	Identifier	Access	Reset	Name
0	tx_empty	rw, woclr 0x0	—	
1	tx_full	rw, woclr 0x0	—	
2	rx_notEmpty	rw, woclr 0x0	—	
3	rx_full	rw, woclr 0x0	—	
4	parity_error	rw, woclr 0x0	—	
5	overrun_error	rw, woclr 0x0	—	
6	frame_error	rw, woclr 0x0	—	
7	break_error	rw, woclr 0x0	—	
8	rx_fifo_threshold	rw, woclr 0x0	—	

tx_empty field Tx FIFO is Empty

tx_full field Tx FIFO is Full

rx_notEmpty field Rx FIFO is Not Empty

rx_full field Rx FIFO is Full

parity_error field Rx Parity error

overrun_error field Rx FIFO overflow

frame_error field Rx stop bit is zero

break_error field Rx Data and stop bits are zero

rx_fifo_threshold field Threshold level crossed on Rx.

DelayReg register

- Absolute Address: 0x2004020
- Base Offset: 0x20
- Size: 0x4



CONTENTS

Bits Identifier	Access	Reset	Name
15:0	delay_control	rw	0x0 —

delay_control field Transmit delay count

ControlReg register

- Absolute Address: 0x2004028
- Base Offset: 0x28
- Size: 0x4

Bits Identifier	Access	Reset	Name
2:1	stopbits	rw	0x0 —
4:3	parity	rw	0x0 —
10:5	charsize	rw	0x8 —

stopbits field 00 -> 1 stop bit; 01 -> 1.5 stop bits ; 10 -> 2 stop bits

parity field 0-> no parity, 1 -> odd parity; 2 -> even parity

charsize field char size of data (max 32b)

IQC register

- Absolute Address: 0x2004030
- Base Offset: 0x30
- Size: 0x4

Bits Identifier	Access	Reset	Name
7:0	qual_cycles	rw	0x0 —

qual_cycles field

**Rx_Threshold register**

- Absolute Address: 0x2004038
- Base Offset: 0x38
- Size: 0x4

Bits Identifier Access Reset Name				
4:0	rx_level	rw	0x5	—

rx_level field**NotUsed register**

- Absolute Address: 0x2004040
- Base Offset: 0x40
- Size: 0x4

Bits Identifier Access Reset Name				
7:0	notused	r	0x0	—

notused field notused**InterruptRaw register**

- Absolute Address: 0x2004048
- Base Offset: 0x48
- Size: 0x4

RAW interrupt register, the bit is sticky and set when the condition for interrupt occurs and remains set until a value of 1 is written to it.

Bits Identifier		Access	Reset Name
0	tx_done	rw, woclr	0x0 —
1	tx_not_full	rw, woclr	0x0 —
2	rx_not_empty	rw, woclr	0x0 —
3	rx_full	rw, woclr	0x0 —
4	parity_error	rw, woclr	0x0 —



Bits	Identifier	Access	Reset	Name
5	overrun_error	rw, woclr	0x0	—
6	frame_error	rw, woclr	0x0	—
7	break_error	rw, woclr	0x0	—
8	rx_fifo_threshold	rw, woclr	0x0	—

tx_done field Transmission Done Interrupt

tx_not_full field Transmitter Not Full Interrupt

rx_not_empty field Receiver Not Empty Interrupt

rx_full field Receiver Full Interrupt

parity_error field Parity Error Interrupt

overrun_error field Overrun Interrupt

frame_error field Frame Error Interrupt

break_error field Break Error Interrupt

rx_fifo_threshold field RX FIFO ~80% Full Interrupt

InterruptMask register

- Absolute Address: 0x2004050
- Base Offset: 0x50
- Size: 0x4

interrupt Mask Register, this is anded with the raw interrupt bit. if the result is 1 an interrupt is generated.

Bits	Identifier	Access	Reset	Name
0	tx_done_en	rw	0x0	—
1	tx_not_full_en	rw	0x0	—
2	rx_not_empty_en	rw	0x0	—



Bits Identifier		Access Reset Name		
3	rx_full_en	rw	0x0	—
4	parity_error_en	rw	0x0	—
5	overrun_error_en	rw	0x0	—
6	frame_error_en	rw	0x0	—
7	break_error_en	rw	0x0	—
8	rx_fifo_threshold_en	rw	0x0	—

tx_done_en field Transmission Done Interrupt Mask

tx_not_full_en field Transmitter Not Full Interrupt Mask

rx_not_empty_en field Receiver Not Empty Interrupt Mask

rx_full_en field Receiver Full Interrupt Mask

parity_error_en field Parity Error Interrupt Mask

overrun_error_en field Overrun Interrupt Mask

frame_error_en field Frame Error Interrupt Mask

break_error_en field Break Error Interrupt Mask

rx_fifo_threshold_en field RX FIFO ~80% Full Interrupt Mask

InterruptStatus register

- Absolute Address: 0x2004058
- Base Offset: 0x58
- Size: 0x4

Bits Identifier		Access Reset Name		
0	tx_done	r	0x0	—
1	tx_not_full	r	0x0	—
2	rx_not_empty	r	0x0	—



CONTENTS

Bits Identifier		Access Reset Name		
3	rx_full	r	0x0	—
4	parity_error	r	0x0	—
5	overrun_error	r	0x0	—
6	frame_error	r	0x0	—
7	break_error	r	0x0	—
8	rx_fifo_threshold	r	0x0	—

tx_done field Transmission Done Interrupt

tx_not_full field Transmitter Not Full Interrupt

rx_not_empty field Receiver Not Empty Interrupt

rx_full field Receiver Full Interrupt

parity_error field Parity Error Interrupt

overrun_error field Overrun Interrupt

frame_error field Frame Error Interrupt

break_error field Break Error Interrupt

rx_fifo_threshold field RX FIFO ~80% Full Interrupt

ROMRAM address map

- Absolute Address: 0x2008000
- Base Offset: 0x2008000
- Size: 0x5000

Offset	Identifier	Name
0x0000	ROM	BootROM
0x4000	SRAM	SRAM



ROM memory

- Absolute Address: 0x2008000
- Base Offset: 0x0
- Size: 0x2000

No supported members.

SRAM memory

- Absolute Address: 0x200C000
- Base Offset: 0x4000
- Size: 0x1000

No supported members.

xspi_registers address map

- Absolute Address: 0x200F000
- Base Offset: 0x200F000
- Size: 0x38

Status and control Registers

Offset Identifier		Name
0x00	ID0	id0
0x08	ID1	ID1
0x10	CFG	—
0x18	xspi_status	—
0x20	xspi_control	—
0x28	xspi_rates	—
0x30	interrupt_status	—

ID0 register

- Absolute Address: 0x200F000
- Base Offset: 0x0
- Size: 0x8



CONTENTS

Bits	Identifier	Access	Reset	Name
3:0	mgf_id	rw	0x2	—
15:4	devid0	rw	0x0	—

mgf_id field Manufacturer: TODO check how to obtain this number

devid0 field DEVID0 TODO Use as suitable

ID1 register

- Absolute Address: 0x200F008
- Base Offset: 0x8
- Size: 0x8

Bits	Identifier	Access	Reset	Name
3:0	dev_type	rw	0x0	—
15:4	devid1	rw	0x0	—

dev_type field Device Type set to hyperram

devid1 field TODO Use as appropriate

CFG register

- Absolute Address: 0x200F010
- Base Offset: 0x10
- Size: 0x8

Bits	Identifier	Access	Reset	Name
1:0	BurstLength	rw	0x2	—
2	HybridBurstEnable	r	0x0	—
3	FixedLatency	r	0x1	—
7:4	InitialLatency	rw	0x8	—
11:8	Reserved	r	0x1	—
14:12	DriveStrength	r	0x3	—
15	DeepPowerDown	rw	0x0	—



Bits	Identifier	Access	Reset	Name
16	BurstEnable	rw	0x0	—
17	UltraDeepPowerDown	rw	0x0	—

BurstLength field Burst Length

HybridBurstEnable field Burst Enable

FixedLatency field Fixed Latency

InitialLatency field Initial Latency.

DriveStrength field Drive Strength.

DeepPowerDown field Deep Power down, Not too deep, not too shallow

BurstEnable field Enable bust access. Applicable only to memory access.
Register access are one at a time

UltraDeepPowerDown field Ultra Deep Power down.

xspi_status register

- Absolute Address: 0x200F018
- Base Offset: 0x18
- Size: 0x8

Bits	Identifier	Access	Reset	Name
0	wip	r	0x0	—

wip field write in progress; Valid only after a write mem transaction

**xspi_control register**

- Absolute Address: 0x200F020
- Base Offset: 0x20
- Size: 0x8

Bits Identifier		Access Reset Name		
0	use_xspi_clk	rw	0x0	—
1	interrupt_enable	rw	0x0	—

use_xspi_clk field Use xspi clock as system clock

interrupt_enable field Use xspi clock as system clock

xspi_rates register

- Absolute Address: 0x200F028
- Base Offset: 0x28
- Size: 0x8

Bits	Identifier	Access	Reset	Name
7:0	cmd_rate	rw	0x0	—
15:8	addr_rate	rw	0x0	—
23:16	data_rate	rw	0x0	—

cmd_rate field CMD Rate

addr_rate field CMD Rate

data_rate field CMD Rate

interrupt_status register

- Absolute Address: 0x200F030
- Base Offset: 0x30
- Size: 0x8



Bits	Identifier	Access	Reset	Name
1:0	axi_resp	r, rclr	0x0	—
2	read_underflow	r, rclr	0x0	—
3	write_overflow	r, rclr	0x0	—

axi_resp field AXI Resp Error

read_underflow field Read FIFO Underflow Error

write_overflow field Write FIFO Overflow Error

mram memory

- Absolute Address: 0x40000000
- Base Offset: 0x40000000
- Size: 0x1000000

No supported members.

cpu_registers address map

- Absolute Address: 0x80000000
- Base Offset: 0x80000000
- Size: 0xF40318

Offset	Identifier	Name
0x100000	User_neigh	U_neigh
0x340000	User_cpu	U_cpu
0x740000	Supervisor_cpu	S_cpu
0x800000	D_hart_esr	D_hart_esr
0x900000	D_neigh	D_neigh
0xB5F000	D_cpu	D_cpu
0xD00000	Machine_neigh	M_neigh
0xF40000	Machine_cpu	M_cpu



User__neigh address map

- Absolute Address: 0x80100000
- Base Offset: 0x100000
- Size: 0x48

Offset Identifier	Name
0x40 ipi_redirect_pc	ipi_redirect_pc

ipi_redirect_pc register

- Absolute Address: 0x80100040
- Base Offset: 0x40
- Size: 0x8

Bits Identifier	Access	Reset	Name
47:0 ipi_redirect_pc	rw	0x0	—

ipi_redirect_pc field In bits 47:0 of this register, user-level software writes the 48 least significant bits of a target virtual PC of a redirect IPI for all the harts in a given neighborhood. Also, bit 0 of the PC is always 0

User__cpu address map

- Absolute Address: 0x80340000
- Base Offset: 0x340000
- Size: 0x300

Offset Identifier	Name
0x080 ipi_redirect_trigger	ipi_redirect_trigger
0x0C0 CREDINC0	CREDINC0
0x0C8 CREDINC1	CREDINC1
0x0D0 CREDINC2	CREDINC2
0x0D8 CREDINC3	CREDINC3
0x100 fast_local_barrier0	fast_local_barrier0
0x108 fast_local_barrier1	fast_local_barrier1



Offset	Identifier	Name
0x110	fast_local_barrier2	fast_local_barrier2
0x118	fast_local_barrier3	fast_local_barrier3
0x120	fast_local_barrier4	fast_local_barrier4
0x128	fast_local_barrier5	fast_local_barrier5
0x130	fast_local_barrier6	fast_local_barrier6
0x138	fast_local_barrier7	fast_local_barrier7
0x140	fast_local_barrier8	fast_local_barrier8
0x148	fast_local_barrier9	fast_local_barrier9
0x150	fast_local_barrier10	fast_local_barrier10
0x158	fast_local_barrier11	fast_local_barrier11
0x160	fast_local_barrier12	fast_local_barrier12
0x168	fast_local_barrier13	fast_local_barrier13
0x170	fast_local_barrier14	fast_local_barrier14
0x178	fast_local_barrier15	fast_local_barrier15
0x180	fast_local_barrier16	fast_local_barrier16
0x188	fast_local_barrier17	fast_local_barrier17
0x190	fast_local_barrier18	fast_local_barrier18
0x198	fast_local_barrier19	fast_local_barrier19
0x1A0	fast_local_barrier20	fast_local_barrier20
0x1A8	fast_local_barrier21	fast_local_barrier21
0x1B0	fast_local_barrier22	fast_local_barrier22
0x1B8	fast_local_barrier23	fast_local_barrier23
0x1C0	fast_local_barrier24	fast_local_barrier24
0x1C8	fast_local_barrier25	fast_local_barrier25
0x1D0	fast_local_barrier26	fast_local_barrier26
0x1D8	fast_local_barrier27	fast_local_barrier27
0x1E0	fast_local_barrier28	fast_local_barrier28
0x1E8	fast_local_barrier29	fast_local_barrier29
0x1F0	fast_local_barrier30	fast_local_barrier30
0x1F8	fast_local_barrier31	fast_local_barrier31
0x2F8	icache_uprefetch	icache_uprefetch

ipi_redirect_trigger register

- Absolute Address: 0x80340080
- Base Offset: 0x80
- Size: 0x8



Bits Identifier	Access	Reset	Name
15:0	ipi_redirect_trigger	rw	— —

ipi_redirect_trigger field User-level software writes a bitmask to this register to request an IPI redirect to all harts indicated by the mask. Before delivery, the bitmask is unconditionally ANDed with the value in IPI_REDIRECT_FILTER. The register is encoded as follows: Bit 0: Minion 0, hart 0 Bit 1: Minion 0, hart 1 Bit 2: Minion 1, hart 0 Bit 3: Minion 1, hart 1 ... Bit 12: Minion 6, hart 0 Bit 13: Minion 6, hart 1 Bit 14: Minion 7, hart 0 Bit 15: Minion 7, hart 1 When a redirect IPI is delivered to a hart, that hart must satisfy two conditions, It must be executing in user mode. It must be capable of blocking on a stall CSR read instruction. If either condition is violated, the hart must take a trap into M-mode with, mcause set to Bad IPI redirect interrupt (5'h10), mtval set to the redirection PC. Reading this register returns a 0x0 value

CREDINC0 register

- Absolute Address: 0x803400C0
- Base Offset: 0xC0
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0	mask0	r	— —

mask0 field Writing this register increments the COUNTER0 credit counter for the first hart of all the cores selected in MASK0. Reading this register returns a value of 0.

CREDINC1 register

- Absolute Address: 0x803400C8
- Base Offset: 0xC8
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0	mask1	r	— —



mask1 field Writing this register increments the COUNTER1 credit counter for the first hart of all the cores selected in MASK1.

CREDINC2 register

- Absolute Address: 0x803400D0
- Base Offset: 0xD0
- Size: 0x8

Bits Identifier Access Reset Name				
7:0	mask2	r	—	—

mask2 field Writing this register increments the COUNTER2 credit counter for the first hart of all the cores selected in MASK2. Reading this register returns a value of 0.

CREDINC3 register

- Absolute Address: 0x803400D8
- Base Offset: 0xD8
- Size: 0x8

Bits Identifier Access Reset Name				
7:0	mask3	r	—	—

mask3 field Writing this register increments the COUNTER3 credit counter for the first hart of all the cores selected in MASK3. Reading this register returns a value of 0.

fast_local_barrier0 register

- Absolute Address: 0x80340100
- Base Offset: 0x100
- Size: 0x8



Bits Identifier	Access Reset Name
7:0 fast_local_barrier0 rw	0x0 —

fast_local_barrier0 field The CORE-ET Fast Local Barrier (FLB) extension registers provide fast barrier capabilities across the Minions within a Shire. Multiple barrier counters allow a subset of threads in a Shire to atomically modify a barrier counter and determine whether all participating threads have reached the barrier. The FLB extension provides 32 barrier counters, each 8 bits wide, supporting an all-thread (64 threads) barrier in the Shire. Barrier counter *i* should be initialized by software before starting the barrier operation using a regular RISC-V store instruction to address (address + *i* * 8). Software typically initializes barrier counters to zero, but any value from 0 to 255 is legal. For maximum performance, FLB counters are directly accessible by threads in the Shire using CSRRW instructions. Threads participating in a barrier use CSRRW to atomically increment the barrier counter. If a thread is the last to join the barrier, the CSRRW returns 0x1; otherwise, it returns 0x0.

fast_local_barrier1 register

- Absolute Address: 0x80340108
- Base Offset: 0x108
- Size: 0x8

Bits Identifier	Access Reset Name
7:0 fast_local_barrier1 rw	0x0 —

fast_local_barrier1 field see description for fast_local_barrier_0

fast_local_barrier2 register

- Absolute Address: 0x80340110
- Base Offset: 0x110
- Size: 0x8



Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier2	rw	0x0	—

fast_local_barrier2 field see description for fast_local_barrier_0

fast_local_barrier3 register

- Absolute Address: 0x80340118
- Base Offset: 0x118
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier3	rw	0x0	—

fast_local_barrier3 field see description for fast_local_barrier_0

fast_local_barrier4 register

- Absolute Address: 0x80340120
- Base Offset: 0x120
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier4	rw	0x0	—

fast_local_barrier4 field see description for fast_local_barrier_0

fast_local_barrier5 register

- Absolute Address: 0x80340128
- Base Offset: 0x128
- Size: 0x8



Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier5	rw	0x0	—

fast_local_barrier5 field see description for fast_local_barrier_0

fast_local_barrier6 register

- Absolute Address: 0x80340130
- Base Offset: 0x130
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier6	rw	0x0	—

fast_local_barrier6 field see description for fast_local_barrier_0

fast_local_barrier7 register

- Absolute Address: 0x80340138
- Base Offset: 0x138
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier7	rw	0x0	—

fast_local_barrier7 field see description for fast_local_barrier_0

fast_local_barrier8 register

- Absolute Address: 0x80340140
- Base Offset: 0x140
- Size: 0x8



Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier8	rw	0x0	—

fast_local_barrier8 field see description for fast_local_barrier_0

fast_local_barrier9 register

- Absolute Address: 0x80340148
- Base Offset: 0x148
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier9	rw	0x0	—

fast_local_barrier9 field see description for fast_local_barrier_0

fast_local_barrier10 register

- Absolute Address: 0x80340150
- Base Offset: 0x150
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier10	rw	0x0	—

fast_local_barrier10 field see description for fast_local_barrier_0

fast_local_barrier11 register

- Absolute Address: 0x80340158
- Base Offset: 0x158
- Size: 0x8



CONTENTS

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier11	rw	0x0	—

fast_local_barrier11 field see description for fast_local_barrier_0

fast_local_barrier12 register

- Absolute Address: 0x80340160
- Base Offset: 0x160
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier12	rw	0x0	—

fast_local_barrier12 field see description for fast_local_barrier_0

fast_local_barrier13 register

- Absolute Address: 0x80340168
- Base Offset: 0x168
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier13	rw	0x0	—

fast_local_barrier13 field see description for fast_local_barrier_0

fast_local_barrier14 register

- Absolute Address: 0x80340170
- Base Offset: 0x170
- Size: 0x8



Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier14	rw	0x0	—

fast_local_barrier14 field see description for fast_local_barrier_0

fast_local_barrier15 register

- Absolute Address: 0x80340178
- Base Offset: 0x178
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier15	rw	0x0	—

fast_local_barrier15 field see description for fast_local_barrier_0

fast_local_barrier16 register

- Absolute Address: 0x80340180
- Base Offset: 0x180
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier16	rw	0x0	—

fast_local_barrier16 field see description for fast_local_barrier_0

fast_local_barrier17 register

- Absolute Address: 0x80340188
- Base Offset: 0x188
- Size: 0x8



CONTENTS

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier17	rw	0x0	—

fast_local_barrier17 field see description for fast_local_barrier_0

fast_local_barrier18 register

- Absolute Address: 0x80340190
- Base Offset: 0x190
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier18	rw	0x0	—

fast_local_barrier18 field see description for fast_local_barrier_0

fast_local_barrier19 register

- Absolute Address: 0x80340198
- Base Offset: 0x198
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier19	rw	0x0	—

fast_local_barrier19 field see description for fast_local_barrier_0

fast_local_barrier20 register

- Absolute Address: 0x803401A0
- Base Offset: 0x1A0
- Size: 0x8



Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier20	rw	0x0	—

fast_local_barrier20 field see description for fast_local_barrier_0

fast_local_barrier21 register

- Absolute Address: 0x803401A8
- Base Offset: 0x1A8
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier21	rw	0x0	—

fast_local_barrier21 field see description for fast_local_barrier_0

fast_local_barrier22 register

- Absolute Address: 0x803401B0
- Base Offset: 0x1B0
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier22	rw	0x0	—

fast_local_barrier22 field see description for fast_local_barrier_0

fast_local_barrier23 register

- Absolute Address: 0x803401B8
- Base Offset: 0x1B8
- Size: 0x8



CONTENTS

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier23	rw	0x0	—

fast_local_barrier23 field see description for fast_local_barrier_0

fast_local_barrier24 register

- Absolute Address: 0x803401C0
- Base Offset: 0x1C0
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier24	rw	0x0	—

fast_local_barrier24 field see description for fast_local_barrier_0

fast_local_barrier25 register

- Absolute Address: 0x803401C8
- Base Offset: 0x1C8
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier25	rw	0x0	—

fast_local_barrier25 field see description for fast_local_barrier_0

fast_local_barrier26 register

- Absolute Address: 0x803401D0
- Base Offset: 0x1D0
- Size: 0x8



Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier26	rw	0x0	—

fast_local_barrier26 field see description for fast_local_barrier_0

fast_local_barrier27 register

- Absolute Address: 0x803401D8
- Base Offset: 0x1D8
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier27	rw	0x0	—

fast_local_barrier27 field see description for fast_local_barrier_0

fast_local_barrier28 register

- Absolute Address: 0x803401E0
- Base Offset: 0x1E0
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier28	rw	0x0	—

fast_local_barrier28 field see description for fast_local_barrier_0

fast_local_barrier29 register

- Absolute Address: 0x803401E8
- Base Offset: 0x1E8
- Size: 0x8



Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier29	rw	0x0	—

fast_local_barrier29 field see description for fast_local_barrier_0

fast_local_barrier30 register

- Absolute Address: 0x803401F0
- Base Offset: 0x1F0
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier30	rw	0x0	—

fast_local_barrier30 field see description for fast_local_barrier_0

fast_local_barrier31 register

- Absolute Address: 0x803401F8
- Base Offset: 0x1F8
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0 fast_local_barrier31	rw	0x0	—

fast_local_barrier31 field see description for fast_local_barrier_0

icache_uprefetch register

- Absolute Address: 0x803402F8
- Base Offset: 0x2F8
- Size: 0x8

If no prefetch with a higher privilege mode associated with it is in progress, a write in this ESR prefetches Count+1 consecutive lines starting at address Address to the instruction cache. The privilege mode associated with this



operation is User-mode, and all memory accesses are performed with User-mode privileges. If any of the memory requests generated by the prefetch engine generates an exception, the line is not prefetched and the operation continues with the next line. Reading this register returns the idle status of the prefetch engine.

Bits Identifier Access Reset Name				
5:0	count	rw	0x0	—
47:6	address	rw	—	—

count field Number of lines to be prefetched (minus 1) starting at the given Address.

address field Bits [47:6] of the address where prefetching should start from (aligned to a cacheline).

Supervisor__cpu address map

- Absolute Address: 0x80740000
- Base Offset: 0x740000
- Size: 0x308

Offset Identifier	Name
0x290 shire__coop__mode	shire__coop__mode
0x300 icache__sprefetch	icache__sprefetch

shire__coop__mode register

- Absolute Address: 0x80740290
- Base Offset: 0x290
- Size: 0x8

Bits Identifier	Access Reset Name
0 coop__mode	rw 0x0 —



coop_mode field Software can set this bit to enable cooperative prefetches, loads, and stores.

icache_sprefetch register

- Absolute Address: 0x80740300
- Base Offset: 0x300
- Size: 0x8

If no prefetch with a higher privilege mode associated with it is in progress, a write in this ESR prefetches Count+1 consecutive lines starting at address Address to the instruction caches. The privilege mode associated with this operation is Supervisor-mode, and all memory accesses are performed with Supervisor-mode privileges. If any of the memory requests generated by the prefetch engine generates an exception, the line is not prefetched and the operation continues with the next line. Reading this register returns the idle status of the prefetch engine.

Bits Identifier Access Reset Name				
5:0	count	rw	0x0	—
47:6	address	rw	—	—

count field Number of lines to be prefetched (minus 1) starting at the given Address.

address field Bits [47:6] of the address where prefetching should start from (aligned to a cacheline).

D_hart__esr address map

- Absolute Address: 0x80800000
- Base Offset: 0x800000
- Size: 0x10000

These registers are per hart. ([15:12] bits) in the address indicates hart_id. bits[15:13] minion id and bit [12] is thread id.



Offset	Identifier	Name
0x0000	dbg_hart[0]	—
0x1000	dbg_hart[1]	—
0x2000	dbg_hart[2]	—
0x3000	dbg_hart[3]	—
0x4000	dbg_hart[4]	—
0x5000	dbg_hart[5]	—
0x6000	dbg_hart[6]	—
0x7000	dbg_hart[7]	—
0x8000	dbg_hart[8]	—
0x9000	dbg_hart[9]	—
0xA000	dbg_hart[10]	—
0xB000	dbg_hart[11]	—
0xC000	dbg_hart[12]	—
0xD000	dbg_hart[13]	—
0xE000	dbg_hart[14]	—
0xF000	dbg_hart[15]	—

dbg_hart register file

- Absolute Address: 0x80800000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000

Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

**NXDATA0 register**

- Absolute Address: 0x80800780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x80800788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0

NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x80800790
- Base Offset: 0x790
- Size: 0x8



Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x80800798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x808007A0
- Base Offset: 0x7A0
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXPROGBUFF0	rw	—	—



AXPROGBUFF0 field This register shadows the nxprogbuf0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x808007A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 AXPROGBUFF1	rw	—	—

AXPROGBUFF1 field This register shadows the nxprogbuf1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x808007B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF0	rw	—	—

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x808007B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x808007C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x80801000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x80801780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x80801788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x80801790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x80801798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x808017A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuff0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x808017A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuff1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x808017B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x808017B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x808017C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x80802000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



CONTENTS

Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x80802780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x80802788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x80802790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x80802798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x808027A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuf0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x808027A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuf1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x808027B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x808027B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x808027C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x80803000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



CONTENTS

Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x80803780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x80803788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x80803790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x80803798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x808037A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuf0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x808037A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuf1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x808037B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x808037B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x808037C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x80804000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x80804780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x80804788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x80804790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x80804798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x808047A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuf0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x808047A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuf1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x808047B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x808047B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x808047C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x80805000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x80805780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x80805788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x80805790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x80805798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x808057A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuf0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x808057A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuf1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x808057B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x808057B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x808057C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x80806000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x80806780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x80806788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x80806790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x80806798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x808067A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuf0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x808067A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuf1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x808067B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x808067B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x808067C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x80807000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x80807780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x80807788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x80807790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x80807798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x808077A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuf0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x808077A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuf1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x808077B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x808077B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x808077C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x80808000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x80808780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x80808788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x80808790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x80808798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x808087A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuf0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x808087A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuf1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x808087B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x808087B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x808087C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x80809000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x80809780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x80809788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x80809790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x80809798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x808097A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuf0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x808097A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuf1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x808097B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x808097B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x808097C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x8080A000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x8080A780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x8080A788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x8080A790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x8080A798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x8080A7A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuf0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x8080A7A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuf1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x8080A7B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x8080A7B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x8080A7C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x8080B000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x8080B780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x8080B788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x8080B790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x8080B798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x8080B7A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuf0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x8080B7A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuf1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x8080B7B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x8080B7B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x8080B7C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x8080C000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x8080C780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x8080C788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x8080C790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x8080C798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x8080C7A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuf0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x8080C7A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuf1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x8080C7B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x8080C7B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x8080C7C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x8080D000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



CONTENTS

Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x8080D780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x8080D788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x8080D790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x8080D798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x8080D7A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuf0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x8080D7A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuf1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x8080D7B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x8080D7B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x8080D7C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x8080E000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x8080E780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x8080E788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x8080E790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x8080E798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x8080E7A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuf0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x8080E7A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuf1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x8080E7B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x8080E7B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x8080E7C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

dbg_hart register file

- Absolute Address: 0x8080F000
- Base Offset: 0x0
- Size: 0x7C8
- Array Dimensions: [16]
- Array Stride: 0x1000
- Total Size: 0x10000



Offset	Identifier	Name
0x780	NXDATA0	NXDATA0
0x788	NXDATA1	NXDATA1
0x790	AXDATA0	AXDATA0
0x798	AXDATA1	AXDATA1
0x7A0	AXPROGBUFF0	AXPROGBUFF0
0x7A8	AXPROGBUFF1	AXPROGBUFF1
0x7B0	NXPROGBUFF0	NXPROGBUFF0
0x7B8	NXPROGBUFF1	NXPROGBUFF1
0x7C0	ABSCMD	ABSCMD

NXDATA0 register

- Absolute Address: 0x8080F780
- Base Offset: 0x780
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA0_f0	rw	—	—
63:32	NXDATA0_f1	rw	—	—

NXDATA0_f0 field The low (nxdata0) of ddata0

NXDATA0_f1 field WIRI

NXDATA1 register

- Absolute Address: 0x8080F788
- Base Offset: 0x788
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	NXDATA1_f0	rw	—	—
63:32	NXDATA1_f1	rw	—	—

NXDATA1_f0 field The high (nxdata0) of ddata0



NXDATA1_f1 field WIRI

AXDATA0 register

- Absolute Address: 0x8080F790
- Base Offset: 0x790
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA0_f0	rw	—	—
63:32	AXDATA0_f1	rw	—	—

AXDATA0_f0 field This register is shadow, and have the same layout as the nxdata0 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer

AXDATA1 register

- Absolute Address: 0x8080F798
- Base Offset: 0x798
- Size: 0x8

Bits	Identifier	Access	Reset	Name
31:0	AXDATA1_f0	rw	—	—
63:32	AXDATA1_f1	rw	—	—

AXDATA1_f0 field This register is shadow, and have the same layout as the nxdata1 register. A write to this register “wakes up” a halted hart, after ddata0 is written, and starts execution from the beginning of the instruction buffer.

AXPROGBUFF0 register

- Absolute Address: 0x8080F7A0
- Base Offset: 0x7A0
- Size: 0x8



Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF0	rw	— —

AXPROGBUFF0 field This register shadows the nxprogbuff0 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

AXPROGBUFF1 register

- Absolute Address: 0x8080F7A8
- Base Offset: 0x7A8
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 AXPROGBUFF1	rw	— —

AXPROGBUFF1 field This register shadows the nxprogbuff1 register. A write to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an ebreak instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

NXPROGBUFF0 register

- Absolute Address: 0x8080F7B0
- Base Offset: 0x7B0
- Size: 0x8

Bits Identifier	Access	Reset Name
31:0 NXPROGBUFF0	rw	— —

NXPROGBUFF0 field This register holds bits 64-95 of the instruction buffer from where instructions can be executed when the hart is halted (see also abscmd), encoded in the lower half of the register.

**NXPROGBUFF1 register**

- Absolute Address: 0x8080F7B8
- Base Offset: 0x7B8
- Size: 0x8

Bits Identifier	Access	Reset	Name
31:0 NXPROGBUFF1	rw	—	—

NXPROGBUFF1 field This register holds bits 96-127 of the instruction buffer from where instructions can be executed when the hart is halted (see also `abscmd`), encoded in the lower half of the register.

ABSCMD register

- Absolute Address: 0x8080F7C0
- Base Offset: 0x7C0
- Size: 0x8

This register holds bits 0-63 of the instruction buffer from where instructions can be executed when the hart is halted. The first instruction in the buffer is in the lower order 16 or 32 bits of the ABSCMD register. Writing to this register wakes up a halted hart, after the instruction buffer is written, and starts execution from the beginning of the instruction buffer, until either an `ebreak` instruction is executed or the end of the instruction buffer is reached, at which point the hart goes back to being halted.

Bits Identifier	Access	Reset	Name
63:0 ABSCMD	rw	—	—

D_neigh address map

- Absolute Address: 0x80900000
- Base Offset: 0x900000
- Size: 0xFFA0



Offset	Identifier	Name
0xFF80	hactrl	hactrl
0xFF88	hastatus0	hastatus0
0xFF90	hastatus1	hastatus1
0xFF98	and_or_tree_IO	and_or_tree_IO

hactrl register

- Absolute Address: 0x8090FF80
- Base Offset: 0xFF80
- Size: 0x8

Bits	Identifier	Access	Reset	Name
15:0	hawindow	rw	0x0	—
31:16	hart_mask	rw	0x0	—
47:32	resethalt	r	0x0	—

hawindow field The n-th bit selects the n-th hart in the neighborhood, if hasel is 1. Non-selected harts ignore the reset (except ndmreset), halt and resume requests from the DM.

hart_mask field The n-th bit selects the n-th hart in the neighborhood (independent of the value of hasel). Non-selected harts ignore the reset (except ndmreset), halt and resume requests from the DM.

resethalt field Each bit in this field corresponds to one hart in the neighborhood. While a given resethalt bit is asserted, the corresponding hart will halt upon the next deassertion of its reset. These bits are set for all selected harts when the setresethaltreq bit in the RISC-V Debug Module Control register (DMCTRL) is written. This field is cleared for all selected harts when the clrresethaltreq bit in the DMCTRL register is written.

hastatus0 register

- Absolute Address: 0x8090FF88
- Base Offset: 0xFF88
- Size: 0x8



Bits	Identifier	Access	Reset	Name
15:0	halted	r	0x0	—
31:16	running	r	0x0	—
47:32	resumeack	r	0x0	—
63:48	havereset	r	0x0	—

halted field Each bit in this field corresponds to one of the 16 harts in the neighborhood. The sticky halted bit for a hart is asserted when the hart is in Debug Mode, and hardware clears this bit when the hart receives a reset or resume request.

running field Each bit in this field corresponds to one of the 16 harts in the neighborhood. The sticky running bit for each hart is asserted when a hart finishes resuming execution, and hardware clears this bit when a hart receives a reset or halt request.

resumeack field Each bit in this field corresponds to one of the 16 harts in the neighborhood. The sticky resumeack bit for a hart is asserted when a hart finishes resuming execution, and hardware clears this bit whenever the resumereq bit in the RISC-V Debug Module Control register (DMCTRL) is deasserted.

havereset field Each bit in this field corresponds to one of the 16 harts in the neighborhood. The sticky havereset bit for a hart is asserted when a hart exits reset, and hardware clears this bit whenever the ackhavereset bit in the RISC-V Debug Module Control register (DMCTRL) is written. Each ET-Minion core contains two harts; however, it is not possible to reset an individual hart within an ET-Minion core. Therefore, the havereset field can be implemented as eight physical bits, with bits $2m$ and $2m+1$ corresponding to the m -th Minion hardwired to the same value. For example, bits 0 and 1 correspond to the two harts in Minion core 0, but only one of these bits is required to reset both threads. Similarly, bits 2 and 3 correspond to ET-Minion core 1, and so on

hastatus1 register

- Absolute Address: 0x8090FF90
- Base Offset: 0xFF90



- Size: 0x8

Bits	Identifier	Access	Reset	Name
15:0	busy	r	0x0	—
31:16	exception	rw	0x0	—
47:32	error	rw	0x0	—

busy field Each bit in this field corresponds to one of the 16 harts in the neighborhood, and a bit is set to 1 while the corresponding hart executes an Abstract Command or a Program Buffer program.

exception field Each bit in this field corresponds to one of the 16 harts in the neighborhood. A bit is set to 1 if the execution of an Abstract Command or a Program Buffer program by the corresponding hart generates an exception. These bits are sticky and must be cleared using an explicit write to the register.

error field Each bit in this field corresponds to one of the 16 harts in the neighborhood. A bit is set to 1 if the execution of an Abstract Command or a Program Buffer program by the corresponding hart cannot complete, but no exception is generated. Examples of this include writes to ET-Minion core registers such as `abscmd`, `axprogbuf0`, `axprogbuf1`, `nxprogbuf0`, `nxprogbuf1`, and so on. These bits are sticky and must be cleared using an explicit write to the register.

and_or_tree_IO register

- Absolute Address: 0x8090FF98
- Base Offset: 0xFF98
- Size: 0x8

Bits	Identifier	Access	Reset	Name
0	anyhalted	r	—	—
1	allhalted	r	—	—
2	anyrunning	r	—	—
3	allrunning	r	—	—
4	anyresumeack	r	—	—
5	allresumeack	r	—	—



Bits Identifier		Access Reset Name		
6	anyhavereset	r	—	—
7	allhavereset	r	—	—
8	anyunavailable	r	—	—
9	anyselected	r	—	—

anyhalted field Hardware sets this bit whenever any of the 16 bits of the halted field in the CORE-ET Neighborhood Hart Status 0 register for the selected harts is set, and the hasel bit in the RISC-V Debug Module Control register (DMCTRL) is 1.

allhalted field Hardware sets this bit when all 16 bits of the halted field in the CORE-ET Neighborhood Hart Status 0 register for the selected harts are set, and the hasel bit in the RISC-V Debug Module Control register (DMCTRL) is 1.

anyrunning field Hardware sets this bit whenever any of the 16 bits of the running field in the CORE-ET Neighborhood Hart Status 0 register for the selected harts is set, and the hasel bit in the RISC-V Debug Module Control register (DMCTRL) is 1.

allrunning field Hardware sets this bit whenever all 16 bits of the running field in the CORE-ET Neighborhood Hart Status 0 register for the selected harts are set, and the hasel bit in the RISC-V Debug Module Control register (DMCTRL) is 1.

anyresumeack field Hardware sets this bit whenever any of the 16 bits of the resumeack field in the CORE-ET Neighborhood Hart Status 0 register for the selected harts is set, and the hasel bit in the RISC-V Debug Module Control register (DMCTRL) is 1.

allresumeack field Hardware sets this bit whenever any of the 16 bits of the resumeack field in the CORE-ET Neighborhood Hart Status 0 register for the selected harts is set, and the hasel bit in the RISC-V Debug Module Control register (DMCTRL) is 1.



anyhavereset field Hardware sets this bit whenever any 16 bits of the havereset field in the CORE-ET Neighborhood Hart Status 0 register for the selected harts are set, and the hasel bit in the RISC-V Debug Module Control register (DMCTRL) is 1.

allhavereset field Hardware sets this bit whenever all of the 16 bits of the havereset field in the CORE-ET Neighborhood Hart Status 0 register for the selected harts is set, and the hasel bit in the RISC-V Debug Module Control register (DMCTRL) is 1.

anyunavailable field Hardware sets this bit when any of the selected harts in the neighborhood are unavailable, and the hasel bit in the RISC-V Debug Module Control register (DMCTRL) is 1. A selected hart is considered unavailable if both HASTATUS0.halted[n] and HASTATUS0.running[n] are 0.

anyselected field Hardware sets this bit when any of the harts in the neighborhood are selected, and the hasel bit in the RISC-V Debug Module Control register (DMCTRL) is 1.

D_cpu address map

- Absolute Address: 0x80B5F000
- Base Offset: 0xB5F000
- Size: 0xFD8

Offset	Identifier	Name
0xF88	dmctrl	dmctrl
0xF90	sm_config	sm_config
0xF98	sm_trigger	sm_trigger
0xFA8	sm_match	sm_match
0xFB0	sm_filter0	sm_filter0
0xFB8	sm_filter1	sm_filter1
0xFC0	sm_filter2	sm_filter2
0xFC8	sm_data0	sm_data0
0xFD0	sm_data1	sm_data1

**dmctrl register**

- Absolute Address: 0x80B5FF88
- Base Offset: 0xF88
- Size: 0x8

Bits	Identifier	Access	Reset	Name
0	dmactive	rw	0x0	—
1	ndmreset	rw	—	—
2	clrresethaltreq	rw	—	—
3	setresethaltreq	rw	—	—
25:4	reserved25_4	r	—	—
26	hasel	rw	—	—
27	reserved27	r	—	—
28	ackhavereset	rw	—	—
29	hartreset	rw	—	—
30	resumereq	rw	—	—
31	haltreq	rw	—	—

dmactive field This bit controls the dmactive signal from the DM; it is broadcast to all Minion shires. When the signal is low it should hold at reset the debug logic of the receiving harts, and of the DM. After writing 1, the debugger should poll DMCTRL until dmactive is high. Hardware may take an arbitrarily long time to initialize and will indicate completion by setting dmactive to 1

ndmreset field This bit controls the ndmreset signal from the DM; it is broadcast to all Minion shires. The signal should reset every part of the receiving shire, except for any logic required for debug.

clrresethaltreq field This bit controls the clrresethaltreq signal from the DM; it is broadcast to all Minion shires. Writing 1 to this bit clears the HACTRL.resethalt bits of all currently selected harts. This bit always reads as 0.

setresethaltreq field This bit controls the setresethaltreq signal from the DM; it is broadcast to all Minion shires. Writing 1 to this bit sets the HACTRL.resethalt bits of all currently selected harts, unless clrresethaltreq is simultaneously set to 1. This bit always reads as 0.



reserved25__4 field WARL(0)

hasel field This bit controls the hasel signal from the DM; it is broadcast to all Minion shires. If set to 0, then DM requests, such as halt and resume apply only to the harts that are selected by HACTRL.hartmask. If set to 1, then DM requests apply to all harts selected by the union of HACTRL.hartmask and HACTRL.hawindow.

reserved27 field WARL(0)

ackhavereset field This bit controls the ackhavereset signal from the DM; it is broadcast to all Minion shires. Writing 1 to this bit clears HASTATUS0.havereset for all currently selected harts. This bit always reads as 0.

hartreset field This bit controls the hartreset signal from the DM; it is broadcast to all Minion shires.

resumereq field This bit controls the resumereq signal from the DM; it is broadcast to all Minion shires. Writing 1 to this bit causes the currently selected harts to resume and clears the HASTATUS0.resumeack bit for those harts. This bit always reads as 0.

haltreq field Bits [31:31]: This bit controls the haltreq signal from the DM; it is broadcast to all Minion shires. Writing 1 to this bit sets the halt request bit for all currently selected harts. Writing 0 to this bit clears the halt request bit for all currently selected harts. This may cancel outstanding halt requests for those harts. This bit always reads as 0.

sm_config register

- Absolute Address: 0x80B5FF90
- Base Offset: 0xF90
- Size: 0x8

Bits Identifier		Access		Reset Name	
6:0	sm_data_sel	rw	0x0	—	—
10:7	sm_target_sel	rw	—	—	—
11	sm_enable	rw	—	—	—



sm_data_sel field Data selector within each target

sm_target_sel field Target selector:

sm_enable field Status Monitor logic enable.

sm_trigger register

- Absolute Address: 0x80B5FF98
- Base Offset: 0xF98
- Size: 0x8

Bits Identifier	Access	Reset	Name
0	sm_trigger	r	— —

sm_trigger field Writing this bit triggers a simultaneous snapshot captured in all the SM ESRs

sm_match register

- Absolute Address: 0x80B5FFA8
- Base Offset: 0xFA8
- Size: 0x8

Bits Identifier	Access	Reset	Name
63:0	sm_match	r	— —

sm_match field Match bus

sm_filter0 register

- Absolute Address: 0x80B5FFB0
- Base Offset: 0xFB0
- Size: 0x8



CONTENTS

Bits	Identifier	Access	Reset	Name
63:0	sm_filter0	r	—	—

sm_filter0 field Filter bus [63:0]

sm_filter1 register

- Absolute Address: 0x80B5FFB8
- Base Offset: 0xFB8
- Size: 0x8

Bits	Identifier	Access	Reset	Name
63:0	sm_filter1	r	—	—

sm_filter1 field Filter bus [127:64]

sm_filter2 register

- Absolute Address: 0x80B5FFC0
- Base Offset: 0xFC0
- Size: 0x8

Bits	Identifier	Access	Reset	Name
63:0	sm_filter2	r	—	—

sm_filter2 field Filter bus [191:128]

sm_data0 register

- Absolute Address: 0x80B5FFC8
- Base Offset: 0xFC8
- Size: 0x8



Bits	Identifier	Access	Reset	Name
63:0	sm_data0	r	—	—

sm_data0 field Data bus [63:0]

sm_data1 register

- Absolute Address: 0x80B5FFD0
- Base Offset: 0xFD0
- Size: 0x8

Bits	Identifier	Access	Reset	Name
63:0	sm_data1	r	—	—

sm_data1 field Data bus [127:64]

Machine_neigh address map

- Absolute Address: 0x80D00000
- Base Offset: 0xD00000
- Size: 0x98

Offset	Identifier	Name
0x18	minion_boot	minion_boot
0x38	mprot	mprot
0x68	pmu_ctrl	pmu_ctrl
0x70	neigh_chicken	neigh_chicken
0x78	icache_err_log_ctl	icache_err_log_ctl
0x80	icache_err_log_info	icache_err_log_info
0x88	icache_err_log_address	icache_err_log_address
0x90	icache_sbe_dbe_counts	icache_sbe_dbe_counts

minion_boot register

- Absolute Address: 0x80D00018
- Base Offset: 0x18



- Size: 0x8

Bits Identifier	Access	Reset	Name
47:0	minion_boot	rw	0x2008000 —

minion_boot field Each neighborhood contains a MINION_BOOT ESR that determines the initial PC of the Minions in the neighborhood after reset. This register holds the 48 least significant bits of the reset PC, bits 63:48 of the PC are ignored on writes. Software must make sure that fetching from the PC value written in this register will not generate any faults.

mprot register

- Absolute Address: 0x80D00038
- Base Offset: 0x38
- Size: 0x8

Bits Identifier	Access	Reset	Name
3:0	smode_size	rw	0x0 —
7:4	mmode_size	rw	0x0 —
8	mprot_en	rw	0x0 —

smode_size field Size of the MRAM region which can be only accessed by Smode. The size is $4KB(2^{SMODE_SIZE}) - 4KB(2^{MMODE_SIZE})$ starting at offset $4KB \cdot (2^{MMODE_SIZE})$ in the MRAM.

mmode_size field Size of the MRAM region which can be only accessed by Mmode. The size is $4KB \cdot (2^{MMODE_SIZE})$ starting at offset 0 in the MRAM.

mprot_en field Enable privilege based memory protection.

pmu_ctrl register

- Absolute Address: 0x80D00068
- Base Offset: 0x68
- Size: 0x8



Bits Identifier		Access		Reset	Name
0	disable_clock	rw	0x0	—	

disable_clock field Set this bit to disable the clock

neigh_chicken register

- Absolute Address: 0x80D00070
- Base Offset: 0x70
- Size: 0x8

Bits Identifier		Access		Reset	Name
0	bypass_icache	rw	0x0	—	
1	bypass_dcache	rw	0x0	—	
2	force_dest_fifo	rw	0x0	—	
4:3	dest_fifo	rw	0x0	—	
5	force_all_agents	rw	0x0	—	
6	agent_forced	rw	0x0	—	

bypass_icache field Setting this bit bypasses the instruction cache.

bypass_dcache field Setting this bit bypasses the data cache.

force_dest_fifo field If enabled, all traffic from agents specified by the force_all_agents and agent_forced fields in the neighborhood will be routed to the destination specified in the dest_fifo field.

dest_fifo field Destination to which requests are routed when the force_dest_fifo bit of this register is set:

force_all_agents field When the force_dest_fifo bit and this bit are set, requests from all agents in the neighborhood go to the FIFO selected by the dest_fifo field.



agent_forced field When the force_dest_fifo bit is set and the force_all_agents bit is clear, this field selects agents from which requests will be forced to go to the FIFO selected by the dest_fifo field. 0: Requests are routed to the corresponding bank FIFO, according to the destination address. 1: Requests are routed to the UC FIFO. 2-3: Reserved

icache_err_log_ctl register

- Absolute Address: 0x80D00078
- Base Offset: 0x78
- Size: 0x8

Bits Identifier		Access		Reset	Name
2:0	err_interrupt_enable	rw	0x6	—	

err_interrupt_enable field This 3-bit field is used to enable or prevent certain categories of logged errors from generating interrupts or sending error responses. In general, single-bit ECC errors should not generate an interrupt. Software may choose whether the ECC error counter saturation should generate an interrupt. Each bit has a specific meaning: Bit 0 enables single-bit ECC errors, Bit 1 enables double-bit ECC errors, and Bit 2 enables ECC counter saturation. Error codes that are not enabled are still logged but have lower priority than errors that generate interrupts. Masked interrupts do not prevent subsequent unmasked errors from being recorded. When an enabled interrupt error overwrites a masked error, the multiple error bit is not set, and multiple masked errors also do not cause the multiple error bit to be set. The multiple error bit is intended to indicate that a fatal error was missed.

icache_err_log_info register

- Absolute Address: 0x80D00080
- Base Offset: 0x80
- Size: 0x8

Bits Identifier		Access		Reset	Name
0	valid	rw	0x0	—	
1	multi	rw	0x0	—	



Bits	Identifier	Access	Reset	Name
2	enabled	rw	0x0	—
3	reserved	r	—	—
7:4	err_code	rw	0x0	—
63:8	info	rw	—	—

valid field When set, this bit indicates an error has occurred and that the register contents are valid. This can be cleared by writing a 1 to this field and writing the matching code to `err_code`.

multi field When set, this bit indicates that multiple errors have occurred. When an error is detected, the `esr_ichache_err_log_info` register records the details of the first error encountered. Any subsequent errors set this bit to indicate that another fatal error occurred, but their details are not recorded.

enabled field The error detected was enabled by `esr_ichache_err_log_ctl.err_interrupt_enable`. Error codes that are not enabled are still logged, but they have lower priority than errors that generate interrupts.

err_code field This 4-bit field indicates the type of error. This field is used to determine how to decode the info bits in positions 63:8. 0: ECC single-bit error 1: ECC double-bit error 2: ECC error counter saturation

info field Additional information about the error. The format of this field is dependent upon the type of error. SBE_DBE format: - Bits 14:8: set. This field indicates the way that contained the error - Bits 16:15: way. This field indicates the way that contained the error - Bits 47:40: error_bits. Indicates which doubleword contained the error. The error_bits field is multi-hot. ECC_Saturation format: - Bit 52: double. This bit indicates the type of error counter that saturated and is encoded as follows: 0: The single-bit error counter saturated. 1: The double-bit error counter saturated.

ichache_err_log_address register

- Absolute Address: 0x80D00088
- Base Offset: 0x88
- Size: 0x8



Bits Identifier	Access	Reset	Name
33:0 icache_err_log_address_f0 r	0x0	—	

icache_err_log_address_f0 field This register contains the physical address (PA) associated with the error if it is available.

icache_sbe_dbe_counts register

- Absolute Address: 0x80D00090
- Base Offset: 0x90
- Size: 0x8

Single-bit errors (SBEs) are corrected so normal processing can continue, and no error response is sent to the requesting core because the request was processed correctly. SBEs should be logged locally, and no action should be taken. Double-bit errors (DBEs) cannot be corrected and are considered fatal, requiring immediate reporting. SBE and DBE responses occur once per ECC protection, meaning one error bit per dword in the cache line (8 bits). The SBE counter is 8 bits and saturates at 255, while the DBE counter is 3 bits and saturates at 7. All fields of this register are read-only. Writing all ones to a field clears its respective counter. If a counter saturates, it can be error-logged as described in the icache_err_log_info register.

Bits Identifier	Access	Reset	Name
7:0 sbe_counts rw	0x0	—	
10:8 db_e_counts rw	—	—	

sbe_counts field This field stores the data RAM single-bit error count.

db_e_counts field This field stores the data RAM double-bit error count.

Machine_cpu address map

- Absolute Address: 0x80F40000
- Base Offset: 0xF40000
- Size: 0x318



Offset Identifier	Name
0x000 minion_feature	minion_feature
0x010 thread1_disable	thread1_disable
0x088 ipi_redirect_filter	ipi_redirect_filter
0x090 ipi_trigger	ipi_trigger
0x098 ipi_trigger_clear	ipi_trigger_clear
0x200 mtime	mtime
0x208 mtime_cmp	mtime_cmp
0x210 time_config	time_config
0x218 mtime_local_target	mtime_local_target
0x240 thread0_disable	thread0_disable
0x308 icache_mprefetch	icache_mprefetch
0x310 clk_gate_ctrl	clk_gate_ctrl

minion_feature register

- Absolute Address: 0x80F40000
- Base Offset: 0x0
- Size: 0x8

Bits Identifier	Access	Reset	Name
0 trap_on_gfx	rw	0x1	—
1 trap_on_ml	rw	0x0	—
2 trap_on_u_cacheops	rw	0x0	—
3 trap_on_u_scp	rw	0x0	—
4 disable_multithreading	rw	0x0	—
5 disable_lock_unlock	rw	0x0	—

trap_on_gfx field Set this bit to trap on GFX. When this bit is enabled, GFX instructions generate an illegal instruction exception.

trap_on_ml field Set this bit to trap on ML. When this bit is set, tensor pseudo-instructions generate an illegal instruction exception.

trap_on_u_cacheops field Setting this bit disables user cacheops. If set, user mode will not have access to the cache op pseudo-instructions.



trap_on_u_scp field Setting this bit disables the user scratchpad. When set, user mode cannot enable or disable the scratchpad.

disable_multithreading field Set this bit to disable multithreading, making only thread 0 available. The hart IDs for all thread 0s in a Shire must be contiguous.

disable_lock_unlock field Set this bit to disable lock/unlock. When set, user mode cannot use the LockVA and UnlockVA operations.

thread1_disable register

- Absolute Address: 0x80F40010
- Base Offset: 0x10
- Size: 0x8

Bits Identifier	Access	Reset	Name
7:0	thread1_disable_f0	rw	0xFF —

thread1_disable_f0 field Each bit of this register corresponds to thread 1 of each Minion core in the Neighborhood. When a bit is set, thread 1 of the corresponding Minion is disabled. The field is encoded as follows: Bit 0: Minion 0, thread 1 Bit 1: Minion 1, thread 1 Bit 2: Minion 2, thread 1 ... Bit 7: Minion 7, thread 1

ipi_redirect_filter register

- Absolute Address: 0x80F40088
- Base Offset: 0x88
- Size: 0x8

Bits Identifier	Access	Reset	Name
15:0	ipi_redirect_filter	rw	0xFFFF —



ipi_redirect_filter field ipi_redirect_filter Writing a 16-bit value to this register sets the hart ID filter mask for subsequent writes to the IPI_REDIRECT_TRIGGER register. This register is used by privileged software (M-Mode) to prevent accidental user-level redirections. Its value is ANDed with the IPI_REDIRECT_TRIGGER ESR before sending any user-level redirections; setting this register to 0x0 disables all user-level redirections. The encoding of this register is as follows: Bit 0: Minion 0, hart 0 Bit 1: Minion 0, hart 1 Bit 2: Minion 1, hart 0 Bit 3: Minion 1, hart 1 ... Bit 12: Minion 6, hart 0 Bit 13: Minion 6, hart 1 Bit 14: Minion 7, hart 0 Bit 15: Minion 7, hart 1

ipi_trigger register

- Absolute Address: 0x80F40090
- Base Offset: 0x90
- Size: 0x8

Bits Identifier	Access	Reset	Name
15:0 ipi_trigger	rw	0x0	—

ipi_trigger field Machine-level software writes a a bit-mask into this register to set the MSIP bit of one or more harts in the Neighborhood. The encoding of this register is as follows: Bit 0: Minion 0, hart 0 Bit 1: Minion 0, hart 1 Bit 2: Minion 1, hart 0 Bit 3: Minion 1, hart 1 ... Bit 12: Minion 6, hart 0 Bit 13: Minion 6, hart 1 Bit 14: Minion 7, hart 0 Bit 15: Minion 7, hart 1 Reading this register reads the value of the MSIP bits of the harts in the Neighborhood. Writing a value with the i-th bit set to 0, will have no effect on the value of the MSIP bit of the corresponding hart.

ipi_trigger_clear register

- Absolute Address: 0x80F40098
- Base Offset: 0x98
- Size: 0x8

Bits Identifier	Access	Reset	Name
15:0 ipi_trigger_clear	r	0x0	—



ipi_trigger_clear field Machine-level software writes a bit-mask into this register to clear the MSIP bit of one or more harts in the Neighborhood. The encoding of this register is as follows: Bit 0: Minion 0, hart 0 Bit 1: Minion 0, hart 1 Bit 2: Minion 1, hart 0 Bit 3: Minion 1, hart 1 ... Bit 12: Minion 6, hart 0 Bit 13: Minion 6, hart 1 Bit 14: Minion 7, hart 0 Bit 15: Minion 7, hart 1 Reading this register returns a 0x0 value.

mtime register

- Absolute Address: 0x80F40200
- Base Offset: 0x200
- Size: 0x8

Bits	Identifier	Access	Reset	Name
63:0	mtime	rw	0x0	—

mtime field This register matches the definition on the RISC-V Privilege v1.1 specification. 64-bit timer counter. This timer increments in 100ns intervals. The mtimecmp register is compared to the value in this register. When the two are equal, a timer interrupt is generated.

mtime_cmp register

- Absolute Address: 0x80F40208
- Base Offset: 0x208
- Size: 0x8

Bits	Identifier	Access	Reset	Name
63:0	mtime_cmp	rw	0x0	—

mtime_cmp field This register matches the definition on the RISC-V Privilege v1.1 specification. 64-bit timer counter compare. Software sets the desired timer interval using this register. The value is then compared to the value in the mtime register. When the two are equal, a timer interrupt is generated.

**time_config register**

- Absolute Address: 0x80F40210
- Base Offset: 0x210
- Size: 0x8

Bits Identifier		Access	Reset	Name
0:6	prescalar_threshold	rw	0x28	—
7	ref_clock_mux	rw	0x0	—

prescalar_threshold field The value in this field will be used as the timer prescalar threshold.

ref_clock_mux field When this field holds 0 the timer is driven by the external reference clock. When this field holds 1 the timer is driven by the same clock that drives the cpu subsystem (external clock or ring oscillator)

mtime_local_target register

- Absolute Address: 0x80F40218
- Base Offset: 0x218
- Size: 0x8

Bits Identifier		Access	Reset	Name
15:0	mtime_local_target	rw	0xFFFF	—

mtime_local_target field This register controls which harts receive a timer interrupt. If bit N in mtime_local_target register is 1, hart N would be able to observe a timer interrupt in mip CSR if the mtimecmp is greater than mtime. If bit N is 0, the interrupt will be masked and not observable on hart N mip CSR.

thread0_disable register

- Absolute Address: 0x80F40240
- Base Offset: 0x240
- Size: 0x8



Bits Identifier	Access	Reset	Name
7:0	thread0_disable	rw	0xFE —

thread0_disable field Each bit of this register corresponds to thread 0 of each Minion core in the Neighborhood. When a bit is set, thread 0 of the corresponding Minion is disabled. The field is encoded as follows: Bit 0: Minion 0, thread 0 Bit 1: Minion 1, thread 0 Bit 2: Minion 2, thread 0 Bit 7: Minion 7, thread 0

icache__mprefetch register

- Absolute Address: 0x80F40308
- Base Offset: 0x308
- Size: 0x8

A write in this ESR prefetches Count+1 consecutive lines starting at address Address to the instruction cache. The privilege mode associated with this operation is Machine-mode, and all memory accesses are performed with Machine-mode privileges. If any of the memory requests generated by the prefetch engine generates an exception, the line is not prefetched and the operation continues with the next line. Reading this register returns the idle status of the prefetch engine.

Bits Identifier	Access	Reset	Name
5:0	count	rw	0x0 —
47:6	address	rw	0x0 —

count field Number of lines to be prefetched (minus 1) starting at the given Address.

address field Bits [47:6] of the address where prefetching should start from (aligned to a cacheline).

clk_gate_ctrl register

- Absolute Address: 0x80F40310
- Base Offset: 0x310
- Size: 0x8



Bits Identifier		Access Reset Name		
0	min_intpipe_clock_gate_disable	rw	0x0	—
1	min_vpulane_clock_gate_disable	rw	0x0	—
2	min_vputima_clock_gate_disable	rw	0x0	—
3	min_vputrans_clock_gate_disable	rw	0x0	—
4	min_dcache_clock_gate_disable	rw	—	—
5	reserved0	r	—	—
6	min_frontend_clock_gate_disable	rw	0x0	—
10:7	reserved1	r	—	—

min_intpipe_clock_gate_disable field .Setting this bit disables integer pipeline clock gating

min_vpulane_clock_gate_disable field Setting this bit disables VPU lane clock gating.

min_vputima_clock_gate_disable field Setting this bit disables VPUtima clock gating

min_vputrans_clock_gate_disable field Setting this bit disables VPU-trans clock gating

min_dcache_clock_gate_disable field Setting this bit disables data cache clock gating

reserved0 field Write as zero. Returns 0 on read.

min_frontend_clock_gate_disable field Setting this bit disables front end clock gating.

reserved1 field Write as zero. Returns 0 on read.

plic address map

- Absolute Address: 0xA0000000
- Base Offset: 0xA0000000
- Size: 0x21F008



RISC-V Platform-Level Interrupt Controller register map.

Offset	Identifier	Name
0x000004	priority_src01	—
0x000008	priority_src02	—
0x00000C	priority_src03	—
0x000010	priority_src04	—
0x000014	priority_src05	—
0x000018	priority_src06	—
0x001000	pending	—
0x002000	enable_ctx00	Interrupt Enable – Hart 0, M-mode
0x002080	enable_ctx01	Interrupt Enable – Hart 0, S-mode
0x002100	enable_ctx02	Interrupt Enable – Hart 1, M-mode
0x002180	enable_ctx03	Interrupt Enable – Hart 1, S-mode
0x002200	enable_ctx04	Interrupt Enable – Hart 2, M-mode
0x002280	enable_ctx05	Interrupt Enable – Hart 2, S-mode
0x002300	enable_ctx06	Interrupt Enable – Hart 3, M-mode
0x002380	enable_ctx07	Interrupt Enable – Hart 3, S-mode
0x002400	enable_ctx08	Interrupt Enable – Hart 4, M-mode
0x002480	enable_ctx09	Interrupt Enable – Hart 4, S-mode
0x002500	enable_ctx10	Interrupt Enable – Hart 5, M-mode
0x002580	enable_ctx11	Interrupt Enable – Hart 5, S-mode
0x002600	enable_ctx12	Interrupt Enable – Hart 6, M-mode
0x002680	enable_ctx13	Interrupt Enable – Hart 6, S-mode
0x002700	enable_ctx14	Interrupt Enable – Hart 7, M-mode
0x002780	enable_ctx15	Interrupt Enable – Hart 7, S-mode
0x002800	enable_ctx16	Interrupt Enable – Hart 8, M-mode
0x002880	enable_ctx17	Interrupt Enable – Hart 8, S-mode
0x002900	enable_ctx18	Interrupt Enable – Hart 9, M-mode
0x002980	enable_ctx19	Interrupt Enable – Hart 9, S-mode
0x002A00	enable_ctx20	Interrupt Enable – Hart 10, M-mode
0x002A80	enable_ctx21	Interrupt Enable – Hart 10, S-mode
0x002B00	enable_ctx22	Interrupt Enable – Hart 11, M-mode
0x002B80	enable_ctx23	Interrupt Enable – Hart 11, S-mode
0x002C00	enable_ctx24	Interrupt Enable – Hart 12, M-mode
0x002C80	enable_ctx25	Interrupt Enable – Hart 12, S-mode
0x002D00	enable_ctx26	Interrupt Enable – Hart 13, M-mode
0x002D80	enable_ctx27	Interrupt Enable – Hart 13, S-mode
0x002E00	enable_ctx28	Interrupt Enable – Hart 14, M-mode



Offset	Identifier	Name
0x002E80	enable_ctx29	Interrupt Enable – Hart 14, S-mode
0x002F00	enable_ctx30	Interrupt Enable – Hart 15, M-mode
0x002F80	enable_ctx31	Interrupt Enable – Hart 15, S-mode
0x200000	threshold_ctx00	Priority Threshold – Hart 0, M-mode
0x200004	claim_complete_ctx00	Claim/Complete – Hart 0, M-mode
0x201000	threshold_ctx01	Priority Threshold – Hart 0, S-mode
0x201004	claim_complete_ctx01	Claim/Complete – Hart 0, S-mode
0x202000	threshold_ctx02	Priority Threshold – Hart 1, M-mode
0x202004	claim_complete_ctx02	Claim/Complete – Hart 1, M-mode
0x203000	threshold_ctx03	Priority Threshold – Hart 1, S-mode
0x203004	claim_complete_ctx03	Claim/Complete – Hart 1, S-mode
0x204000	threshold_ctx04	Priority Threshold – Hart 2, M-mode
0x204004	claim_complete_ctx04	Claim/Complete – Hart 2, M-mode
0x205000	threshold_ctx05	Priority Threshold – Hart 2, S-mode
0x205004	claim_complete_ctx05	Claim/Complete – Hart 2, S-mode
0x206000	threshold_ctx06	Priority Threshold – Hart 3, M-mode
0x206004	claim_complete_ctx06	Claim/Complete – Hart 3, M-mode
0x207000	threshold_ctx07	Priority Threshold – Hart 3, S-mode
0x207004	claim_complete_ctx07	Claim/Complete – Hart 3, S-mode
0x208000	threshold_ctx08	Priority Threshold – Hart 4, M-mode
0x208004	claim_complete_ctx08	Claim/Complete – Hart 4, M-mode
0x209000	threshold_ctx09	Priority Threshold – Hart 4, S-mode
0x209004	claim_complete_ctx09	Claim/Complete – Hart 4, S-mode
0x20A000	threshold_ctx10	Priority Threshold – Hart 5, M-mode
0x20A004	claim_complete_ctx10	Claim/Complete – Hart 5, M-mode
0x20B000	threshold_ctx11	Priority Threshold – Hart 5, S-mode
0x20B004	claim_complete_ctx11	Claim/Complete – Hart 5, S-mode
0x20C000	threshold_ctx12	Priority Threshold – Hart 6, M-mode
0x20C004	claim_complete_ctx12	Claim/Complete – Hart 6, M-mode
0x20D000	threshold_ctx13	Priority Threshold – Hart 6, S-mode
0x20D004	claim_complete_ctx13	Claim/Complete – Hart 6, S-mode
0x20E000	threshold_ctx14	Priority Threshold – Hart 7, M-mode
0x20E004	claim_complete_ctx14	Claim/Complete – Hart 7, M-mode
0x20F000	threshold_ctx15	Priority Threshold – Hart 7, S-mode
0x20F004	claim_complete_ctx15	Claim/Complete – Hart 7, S-mode
0x210000	threshold_ctx16	Priority Threshold – Hart 8, M-mode
0x210004	claim_complete_ctx16	Claim/Complete – Hart 8, M-mode
0x211000	threshold_ctx17	Priority Threshold – Hart 8, S-mode



Offset	Identifier	Name
0x211004	claim_complete_ctx17	Claim/Complete – Hart 8, S-mode
0x212000	threshold_ctx18	Priority Threshold – Hart 9, M-mode
0x212004	claim_complete_ctx18	Claim/Complete – Hart 9, M-mode
0x213000	threshold_ctx19	Priority Threshold – Hart 9, S-mode
0x213004	claim_complete_ctx19	Claim/Complete – Hart 9, S-mode
0x214000	threshold_ctx20	Priority Threshold – Hart 10, M-mode
0x214004	claim_complete_ctx20	Claim/Complete – Hart 10, M-mode
0x215000	threshold_ctx21	Priority Threshold – Hart 10, S-mode
0x215004	claim_complete_ctx21	Claim/Complete – Hart 10, S-mode
0x216000	threshold_ctx22	Priority Threshold – Hart 11, M-mode
0x216004	claim_complete_ctx22	Claim/Complete – Hart 11, M-mode
0x217000	threshold_ctx23	Priority Threshold – Hart 11, S-mode
0x217004	claim_complete_ctx23	Claim/Complete – Hart 11, S-mode
0x218000	threshold_ctx24	Priority Threshold – Hart 12, M-mode
0x218004	claim_complete_ctx24	Claim/Complete – Hart 12, M-mode
0x219000	threshold_ctx25	Priority Threshold – Hart 12, S-mode
0x219004	claim_complete_ctx25	Claim/Complete – Hart 12, S-mode
0x21A000	threshold_ctx26	Priority Threshold – Hart 13, M-mode
0x21A004	claim_complete_ctx26	Claim/Complete – Hart 13, M-mode
0x21B000	threshold_ctx27	Priority Threshold – Hart 13, S-mode
0x21B004	claim_complete_ctx27	Claim/Complete – Hart 13, S-mode
0x21C000	threshold_ctx28	Priority Threshold – Hart 14, M-mode
0x21C004	claim_complete_ctx28	Claim/Complete – Hart 14, M-mode
0x21D000	threshold_ctx29	Priority Threshold – Hart 14, S-mode
0x21D004	claim_complete_ctx29	Claim/Complete – Hart 14, S-mode
0x21E000	threshold_ctx30	Priority Threshold – Hart 15, M-mode
0x21E004	claim_complete_ctx30	Claim/Complete – Hart 15, M-mode
0x21F000	threshold_ctx31	Priority Threshold – Hart 15, S-mode
0x21F004	claim_complete_ctx31	Claim/Complete – Hart 15, S-mode

priority_src01 register

- Absolute Address: 0xA0000004
- Base Offset: 0x4
- Size: 0x4

Interrupt source priority. Hardwired to 1 — no priority levels are implemented.



Bits Identifier		Access Reset Name	
0	PRIORITY r	0x1	—

PRIORITY field Priority level (always 1)

priority_src02 register

- Absolute Address: 0xA0000008
- Base Offset: 0x8
- Size: 0x4

Interrupt source priority. Hardwired to 1 — no priority levels are implemented.

Bits Identifier		Access Reset Name	
0	PRIORITY r	0x1	—

PRIORITY field Priority level (always 1)

priority_src03 register

- Absolute Address: 0xA000000C
- Base Offset: 0xC
- Size: 0x4

Interrupt source priority. Hardwired to 1 — no priority levels are implemented.

Bits Identifier		Access Reset Name	
0	PRIORITY r	0x1	—

PRIORITY field Priority level (always 1)

priority_src04 register

- Absolute Address: 0xA0000010
- Base Offset: 0x10
- Size: 0x4



Interrupt source priority. Hardwired to 1 — no priority levels are implemented.

Bits Identifier		Access Reset Name	
0	PRIORITY r	0x1	—

PRIORITY field Priority level (always 1)

priority_src05 register

- Absolute Address: 0xA0000014
- Base Offset: 0x14
- Size: 0x4

Interrupt source priority. Hardwired to 1 — no priority levels are implemented.

Bits Identifier		Access Reset Name	
0	PRIORITY r	0x1	—

PRIORITY field Priority level (always 1)

priority_src06 register

- Absolute Address: 0xA0000018
- Base Offset: 0x18
- Size: 0x4

Interrupt source priority. Hardwired to 1 — no priority levels are implemented.

Bits Identifier		Access Reset Name	
0	PRIORITY r	0x1	—

PRIORITY field Priority level (always 1)

**pending register**

- Absolute Address: 0xA0001000
- Base Offset: 0x1000
- Size: 0x4

Interrupt pending bits. Hardware-managed; read-only for software.

Bits Identifier		Access		Reset	Name
0	RESERVED	r	0x0	—	
6:1	PENDING	r	0x0	—	

RESERVED field Reserved — source 0 does not exist

PENDING field Pending flags for interrupt sources 1..6

enable__ctx00 register

- Absolute Address: 0xA0002000
- Base Offset: 0x2000
- Size: 0x4

Interrupt enable bits for context 0 (hart 0, Machine mode). Bit N enables source N for this context.

Bits Identifier		Access		Reset	Name
0	RESERVED	r	0x0	—	
6:1	ENABLE	rw	0x0	—	

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable__ctx01 register

- Absolute Address: 0xA0002080
- Base Offset: 0x2080
- Size: 0x4



Interrupt enable bits for context 1 (hart 0, Supervisor mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx02 register

- Absolute Address: 0xA0002100
- Base Offset: 0x2100
- Size: 0x4

Interrupt enable bits for context 2 (hart 1, Machine mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx03 register

- Absolute Address: 0xA0002180
- Base Offset: 0x2180
- Size: 0x4

Interrupt enable bits for context 3 (hart 1, Supervisor mode). Bit N enables source N for this context.



Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable__ctx04 register

- Absolute Address: 0xA0002200
- Base Offset: 0x2200
- Size: 0x4

Interrupt enable bits for context 4 (hart 2, Machine mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable__ctx05 register

- Absolute Address: 0xA0002280
- Base Offset: 0x2280
- Size: 0x4

Interrupt enable bits for context 5 (hart 2, Supervisor mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—



RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx06 register

- Absolute Address: 0xA0002300
- Base Offset: 0x2300
- Size: 0x4

Interrupt enable bits for context 6 (hart 3, Machine mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx07 register

- Absolute Address: 0xA0002380
- Base Offset: 0x2380
- Size: 0x4

Interrupt enable bits for context 7 (hart 3, Supervisor mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

**enable_ctx08 register**

- Absolute Address: 0xA0002400
- Base Offset: 0x2400
- Size: 0x4

Interrupt enable bits for context 8 (hart 4, Machine mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx09 register

- Absolute Address: 0xA0002480
- Base Offset: 0x2480
- Size: 0x4

Interrupt enable bits for context 9 (hart 4, Supervisor mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx10 register

- Absolute Address: 0xA0002500
- Base Offset: 0x2500
- Size: 0x4



Interrupt enable bits for context 10 (hart 5, Machine mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx11 register

- Absolute Address: 0xA0002580
- Base Offset: 0x2580
- Size: 0x4

Interrupt enable bits for context 11 (hart 5, Supervisor mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx12 register

- Absolute Address: 0xA0002600
- Base Offset: 0x2600
- Size: 0x4

Interrupt enable bits for context 12 (hart 6, Machine mode). Bit N enables source N for this context.



Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable__ctx13 register

- Absolute Address: 0xA0002680
- Base Offset: 0x2680
- Size: 0x4

Interrupt enable bits for context 13 (hart 6, Supervisor mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable__ctx14 register

- Absolute Address: 0xA0002700
- Base Offset: 0x2700
- Size: 0x4

Interrupt enable bits for context 14 (hart 7, Machine mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—



RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx15 register

- Absolute Address: 0xA0002780
- Base Offset: 0x2780
- Size: 0x4

Interrupt enable bits for context 15 (hart 7, Supervisor mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx16 register

- Absolute Address: 0xA0002800
- Base Offset: 0x2800
- Size: 0x4

Interrupt enable bits for context 16 (hart 8, Machine mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

**enable_ctx17 register**

- Absolute Address: 0xA0002880
- Base Offset: 0x2880
- Size: 0x4

Interrupt enable bits for context 17 (hart 8, Supervisor mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx18 register

- Absolute Address: 0xA0002900
- Base Offset: 0x2900
- Size: 0x4

Interrupt enable bits for context 18 (hart 9, Machine mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx19 register

- Absolute Address: 0xA0002980
- Base Offset: 0x2980
- Size: 0x4



Interrupt enable bits for context 19 (hart 9, Supervisor mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx20 register

- Absolute Address: 0xA0002A00
- Base Offset: 0x2A00
- Size: 0x4

Interrupt enable bits for context 20 (hart 10, Machine mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx21 register

- Absolute Address: 0xA0002A80
- Base Offset: 0x2A80
- Size: 0x4

Interrupt enable bits for context 21 (hart 10, Supervisor mode). Bit N enables source N for this context.



Bits Identifier		Access Reset Name		
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable__ctx22 register

- Absolute Address: 0xA0002B00
- Base Offset: 0x2B00
- Size: 0x4

Interrupt enable bits for context 22 (hart 11, Machine mode). Bit N enables source N for this context.

Bits Identifier		Access Reset Name		
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable__ctx23 register

- Absolute Address: 0xA0002B80
- Base Offset: 0x2B80
- Size: 0x4

Interrupt enable bits for context 23 (hart 11, Supervisor mode). Bit N enables source N for this context.

Bits Identifier		Access Reset Name		
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—



RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx24 register

- Absolute Address: 0xA0002C00
- Base Offset: 0x2C00
- Size: 0x4

Interrupt enable bits for context 24 (hart 12, Machine mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx25 register

- Absolute Address: 0xA0002C80
- Base Offset: 0x2C80
- Size: 0x4

Interrupt enable bits for context 25 (hart 12, Supervisor mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

**enable_ctx26 register**

- Absolute Address: 0xA0002D00
- Base Offset: 0x2D00
- Size: 0x4

Interrupt enable bits for context 26 (hart 13, Machine mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx27 register

- Absolute Address: 0xA0002D80
- Base Offset: 0x2D80
- Size: 0x4

Interrupt enable bits for context 27 (hart 13, Supervisor mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx28 register

- Absolute Address: 0xA0002E00
- Base Offset: 0x2E00
- Size: 0x4



CONTENTS

Interrupt enable bits for context 28 (hart 14, Machine mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx29 register

- Absolute Address: 0xA0002E80
- Base Offset: 0x2E80
- Size: 0x4

Interrupt enable bits for context 29 (hart 14, Supervisor mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx30 register

- Absolute Address: 0xA0002F00
- Base Offset: 0x2F00
- Size: 0x4

Interrupt enable bits for context 30 (hart 15, Machine mode). Bit N enables source N for this context.



Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

enable_ctx31 register

- Absolute Address: 0xA0002F80
- Base Offset: 0x2F80
- Size: 0x4

Interrupt enable bits for context 31 (hart 15, Supervisor mode). Bit N enables source N for this context.

Bits Identifier		Access	Reset	Name
0	RESERVED	r	0x0	—
6:1	ENABLE	rw	0x0	—

RESERVED field Reserved — source 0 does not exist

ENABLE field Enable bits for interrupt sources 1..6

threshold_ctx00 register

- Absolute Address: 0xA0200000
- Base Offset: 0x200000
- Size: 0x4

Priority threshold for context 0 (hart 0, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access	Reset	Name
0	THRESHOLD	r	0x0	—

THRESHOLD field Threshold (always 0)

**claim_complete_ctx00 register**

- Absolute Address: 0xA0200004
- Base Offset: 0x200004
- Size: 0x4

Claim/Complete for context 0 (hart 0, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx01 register

- Absolute Address: 0xA0201000
- Base Offset: 0x201000
- Size: 0x4

Priority threshold for context 1 (hart 0, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD	r	0x0 —

THRESHOLD field Threshold (always 0)

claim_complete_ctx01 register

- Absolute Address: 0xA0201004
- Base Offset: 0x201004
- Size: 0x4

Claim/Complete for context 1 (hart 0, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.



Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx02 register

- Absolute Address: 0xA0202000
- Base Offset: 0x202000
- Size: 0x4

Priority threshold for context 2 (hart 1, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD	r	0x0 —

THRESHOLD field Threshold (always 0)

claim_complete_ctx02 register

- Absolute Address: 0xA0202004
- Base Offset: 0x202004
- Size: 0x4

Claim/Complete for context 2 (hart 1, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

**threshold_ctx03 register**

- Absolute Address: 0xA0203000
- Base Offset: 0x203000
- Size: 0x4

Priority threshold for context 3 (hart 1, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx03 register

- Absolute Address: 0xA0203004
- Base Offset: 0x203004
- Size: 0x4

Claim/Complete for context 3 (hart 1, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier		Access Reset Name	
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx04 register

- Absolute Address: 0xA0204000
- Base Offset: 0x204000
- Size: 0x4

Priority threshold for context 4 (hart 2, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.



Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx04 register

- Absolute Address: 0xA0204004
- Base Offset: 0x204004
- Size: 0x4

Claim/Complete for context 4 (hart 2, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier		Access Reset Name	
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx05 register

- Absolute Address: 0xA0205000
- Base Offset: 0x205000
- Size: 0x4

Priority threshold for context 5 (hart 2, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

**claim_complete_ctx05 register**

- Absolute Address: 0xA0205004
- Base Offset: 0x205004
- Size: 0x4

Claim/Complete for context 5 (hart 2, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx06 register

- Absolute Address: 0xA0206000
- Base Offset: 0x206000
- Size: 0x4

Priority threshold for context 6 (hart 3, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD	r	0x0 —

THRESHOLD field Threshold (always 0)

claim_complete_ctx06 register

- Absolute Address: 0xA0206004
- Base Offset: 0x206004
- Size: 0x4

Claim/Complete for context 6 (hart 3, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.



Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx07 register

- Absolute Address: 0xA0207000
- Base Offset: 0x207000
- Size: 0x4

Priority threshold for context 7 (hart 3, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx07 register

- Absolute Address: 0xA0207004
- Base Offset: 0x207004
- Size: 0x4

Claim/Complete for context 7 (hart 3, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

**threshold_ctx08 register**

- Absolute Address: 0xA0208000
- Base Offset: 0x208000
- Size: 0x4

Priority threshold for context 8 (hart 4, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx08 register

- Absolute Address: 0xA0208004
- Base Offset: 0x208004
- Size: 0x4

Claim/Complete for context 8 (hart 4, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier		Access Reset Name	
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx09 register

- Absolute Address: 0xA0209000
- Base Offset: 0x209000
- Size: 0x4

Priority threshold for context 9 (hart 4, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.



Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx09 register

- Absolute Address: 0xA0209004
- Base Offset: 0x209004
- Size: 0x4

Claim/Complete for context 9 (hart 4, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier		Access Reset Name	
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx10 register

- Absolute Address: 0xA020A000
- Base Offset: 0x20A000
- Size: 0x4

Priority threshold for context 10 (hart 5, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

**claim_complete_ctx10 register**

- Absolute Address: 0xA020A004
- Base Offset: 0x20A004
- Size: 0x4

Claim/Complete for context 10 (hart 5, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx11 register

- Absolute Address: 0xA020B000
- Base Offset: 0x20B000
- Size: 0x4

Priority threshold for context 11 (hart 5, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD	r	0x0 —

THRESHOLD field Threshold (always 0)

claim_complete_ctx11 register

- Absolute Address: 0xA020B004
- Base Offset: 0x20B004
- Size: 0x4

Claim/Complete for context 11 (hart 5, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.



Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx12 register

- Absolute Address: 0xA020C000
- Base Offset: 0x20C000
- Size: 0x4

Priority threshold for context 12 (hart 6, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD	r	0x0 —

THRESHOLD field Threshold (always 0)

claim_complete_ctx12 register

- Absolute Address: 0xA020C004
- Base Offset: 0x20C004
- Size: 0x4

Claim/Complete for context 12 (hart 6, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

**threshold_ctx13 register**

- Absolute Address: 0xA020D000
- Base Offset: 0x20D000
- Size: 0x4

Priority threshold for context 13 (hart 6, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx13 register

- Absolute Address: 0xA020D004
- Base Offset: 0x20D004
- Size: 0x4

Claim/Complete for context 13 (hart 6, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier		Access Reset Name	
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx14 register

- Absolute Address: 0xA020E000
- Base Offset: 0x20E000
- Size: 0x4

Priority threshold for context 14 (hart 7, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.



Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx14 register

- Absolute Address: 0xA020E004
- Base Offset: 0x20E004
- Size: 0x4

Claim/Complete for context 14 (hart 7, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier		Access Reset Name	
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx15 register

- Absolute Address: 0xA020F000
- Base Offset: 0x20F000
- Size: 0x4

Priority threshold for context 15 (hart 7, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

**claim_complete_ctx15 register**

- Absolute Address: 0xA020F004
- Base Offset: 0x20F004
- Size: 0x4

Claim/Complete for context 15 (hart 7, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx16 register

- Absolute Address: 0xA0210000
- Base Offset: 0x210000
- Size: 0x4

Priority threshold for context 16 (hart 8, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD	r	0x0 —

THRESHOLD field Threshold (always 0)

claim_complete_ctx16 register

- Absolute Address: 0xA0210004
- Base Offset: 0x210004
- Size: 0x4

Claim/Complete for context 16 (hart 8, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.



Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx17 register

- Absolute Address: 0xA0211000
- Base Offset: 0x211000
- Size: 0x4

Priority threshold for context 17 (hart 8, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx17 register

- Absolute Address: 0xA0211004
- Base Offset: 0x211004
- Size: 0x4

Claim/Complete for context 17 (hart 8, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

**threshold_ctx18 register**

- Absolute Address: 0xA0212000
- Base Offset: 0x212000
- Size: 0x4

Priority threshold for context 18 (hart 9, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx18 register

- Absolute Address: 0xA0212004
- Base Offset: 0x212004
- Size: 0x4

Claim/Complete for context 18 (hart 9, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier		Access Reset Name	
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx19 register

- Absolute Address: 0xA0213000
- Base Offset: 0x213000
- Size: 0x4

Priority threshold for context 19 (hart 9, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.



Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx19 register

- Absolute Address: 0xA0213004
- Base Offset: 0x213004
- Size: 0x4

Claim/Complete for context 19 (hart 9, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier		Access Reset Name	
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx20 register

- Absolute Address: 0xA0214000
- Base Offset: 0x214000
- Size: 0x4

Priority threshold for context 20 (hart 10, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

**claim_complete_ctx20 register**

- Absolute Address: 0xA0214004
- Base Offset: 0x214004
- Size: 0x4

Claim/Complete for context 20 (hart 10, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx21 register

- Absolute Address: 0xA0215000
- Base Offset: 0x215000
- Size: 0x4

Priority threshold for context 21 (hart 10, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD	r	0x0 —

THRESHOLD field Threshold (always 0)

claim_complete_ctx21 register

- Absolute Address: 0xA0215004
- Base Offset: 0x215004
- Size: 0x4

Claim/Complete for context 21 (hart 10, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.



Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx22 register

- Absolute Address: 0xA0216000
- Base Offset: 0x216000
- Size: 0x4

Priority threshold for context 22 (hart 11, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD	r	0x0 —

THRESHOLD field Threshold (always 0)

claim_complete_ctx22 register

- Absolute Address: 0xA0216004
- Base Offset: 0x216004
- Size: 0x4

Claim/Complete for context 22 (hart 11, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

**threshold_ctx23 register**

- Absolute Address: 0xA0217000
- Base Offset: 0x217000
- Size: 0x4

Priority threshold for context 23 (hart 11, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx23 register

- Absolute Address: 0xA0217004
- Base Offset: 0x217004
- Size: 0x4

Claim/Complete for context 23 (hart 11, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier		Access Reset Name	
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx24 register

- Absolute Address: 0xA0218000
- Base Offset: 0x218000
- Size: 0x4

Priority threshold for context 24 (hart 12, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.



Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx24 register

- Absolute Address: 0xA0218004
- Base Offset: 0x218004
- Size: 0x4

Claim/Complete for context 24 (hart 12, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier		Access Reset Name	
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx25 register

- Absolute Address: 0xA0219000
- Base Offset: 0x219000
- Size: 0x4

Priority threshold for context 25 (hart 12, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

**claim_complete_ctx25 register**

- Absolute Address: 0xA0219004
- Base Offset: 0x219004
- Size: 0x4

Claim/Complete for context 25 (hart 12, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx26 register

- Absolute Address: 0xA021A000
- Base Offset: 0x21A000
- Size: 0x4

Priority threshold for context 26 (hart 13, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD	r	0x0 —

THRESHOLD field Threshold (always 0)

claim_complete_ctx26 register

- Absolute Address: 0xA021A004
- Base Offset: 0x21A004
- Size: 0x4

Claim/Complete for context 26 (hart 13, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.



Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx27 register

- Absolute Address: 0xA021B000
- Base Offset: 0x21B000
- Size: 0x4

Priority threshold for context 27 (hart 13, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx27 register

- Absolute Address: 0xA021B004
- Base Offset: 0x21B004
- Size: 0x4

Claim/Complete for context 27 (hart 13, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

**threshold_ctx28 register**

- Absolute Address: 0xA021C000
- Base Offset: 0x21C000
- Size: 0x4

Priority threshold for context 28 (hart 14, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx28 register

- Absolute Address: 0xA021C004
- Base Offset: 0x21C004
- Size: 0x4

Claim/Complete for context 28 (hart 14, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier		Access Reset Name	
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx29 register

- Absolute Address: 0xA021D000
- Base Offset: 0x21D000
- Size: 0x4

Priority threshold for context 29 (hart 14, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.



Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

claim_complete_ctx29 register

- Absolute Address: 0xA021D004
- Base Offset: 0x21D004
- Size: 0x4

Claim/Complete for context 29 (hart 14, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier		Access Reset Name	
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold_ctx30 register

- Absolute Address: 0xA021E000
- Base Offset: 0x21E000
- Size: 0x4

Priority threshold for context 30 (hart 15, Machine mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD r	0x0	—

THRESHOLD field Threshold (always 0)

**claim__complete__ctx30 register**

- Absolute Address: 0xA021E004
- Base Offset: 0x21E004
- Size: 0x4

Claim/Complete for context 30 (hart 15, Machine mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.

Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

threshold__ctx31 register

- Absolute Address: 0xA021F000
- Base Offset: 0x21F000
- Size: 0x4

Priority threshold for context 31 (hart 15, Supervisor mode). Hardwired to 0 — all enabled pending interrupts are forwarded to this hart.

Bits Identifier		Access Reset Name	
0	THRESHOLD	r	0x0 —

THRESHOLD field Threshold (always 0)

claim__complete__ctx31 register

- Absolute Address: 0xA021F004
- Base Offset: 0x21F004
- Size: 0x4

Claim/Complete for context 31 (hart 15, Supervisor mode). Read to claim the highest-priority pending interrupt (returns 0 if none); write the claimed ID to signal completion.



Bits Identifier Access Reset Name			
2:0	ID	rw	0x0 —

ID field Interrupt ID: 0 = no interrupt pending, 1..6 = valid interrupt source

nic_config address map

- Absolute Address: 0xFE000000
- Base Offset: 0xFE000000
- Size: 0x16000

Offset Identifier		Name
0x0	ni700_registers	—

ni700_registers memory

- Absolute Address: 0xFE000000
- Base Offset: 0x0
- Size: 0x16000

Refer to arm_corelink_ni_700_network_on_chip_trm_101566_0201_09_en-3.pdf for details

No supported members.



Packaging

Erbium is packaged in a 72 pin QFN (QFN72-10MM-0.50MM)

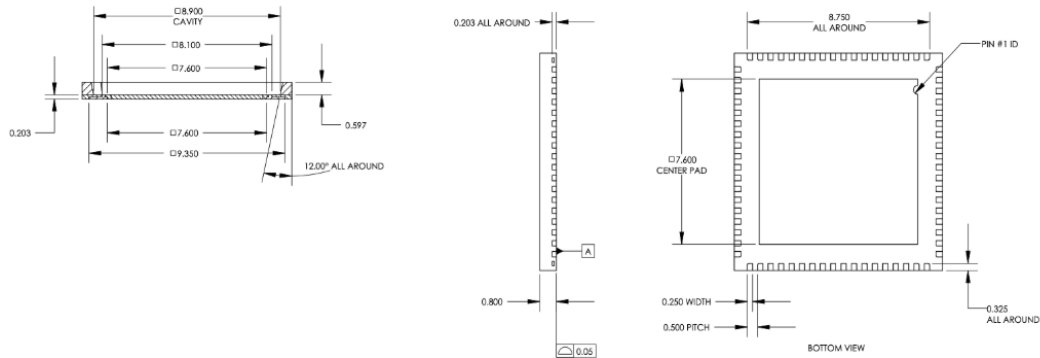


Figure 13: Packaging diagram

Pin Name and description

1. HB_DB[0] – xspi data bus 0
2. HB_DB[1] – xspi data bus 1
3. VSS
4. HB_DB[2] – xspi data bus 2
5. HB_DB[3] – xspi data bus 3
6. HB_DB[4] – xspi data bus 4
7. VDDO – Supply for output pads (Nominal 1.8V)
8. VSS
9. ANATEST0 – Analog test pin 0
10. ANATEST1 – Analog test pin 1



11. HB_DB[5] – xspi data bus 5
12. HB_DB[6] – xspi data bus 6
13. VDDIO – Core VDD for I/O pads and toplevel control circuitry (Nominal 0.8V)
14. HB_DB[7] – xspi data bus 7
15. HB_RWDS – xspi read write data select
16. VSS
17. HB_CS – xspi chip select
18. VDD_A – Core VDD supply for power domain A
19. XSPI_MODE[1] – xspi control mode pin 1
20. RESETN – Chip reset/b pin
21. TestMode – Chip test mode select pin
22. VSS
23. CONFIG[0] – Chip config select pin 0 for selecting active bank
24. CONFIG[1] – Chip config select pin 1 for selecting active bank
25. CONFIG[2] – Chip config select pin 2 for selecting active bank
26. JTAG_TMS - ??
27. VDDO – Supply for output pads (Nominal 1.8V)
28. VDD_A – Core VDD supply for power domain A (Nominal 0.8V)
29. VDD18_A – I/O supply for power domain A (Nominal 1.8V)
30. VDD_A – Core VDD supply for power domain A (Nominal 0.8V)



CONTENTS

- 31. VDD_C – Core VDD supply for power domain C (Nominal 0.8V)
- 32. VDD_B – Core VDD supply for power domain B (Nominal 0.8V)
- 33. VDD18_B – I/O supply for power domain B (Nominal 1.8V)
- 34. VSS
- 35. JTAG_TDO - Out
- 36. JTAG_TDI - In
- 37. GPIO[2] – ET16M 8 core general purpose input/output pin
- 38. GPIO[0] – ET16M 8 core general purpose input/output pin
- 39. VSS
- 40. GPIO[1] – ET16M 8 core general purpose input/output pin
- 41. VDDO – Supply for output pads (Nominal 1.8V)
- 42. VDDIO – Core VDD for I/O pads and toplevel control circuitry (Nominal 0.8V)
- 43. VDD18_B – I/O supply for power domain B (Nominal 1.8V)
- 44. VDD_B – Core VDD supply for power domain B (Nominal 0.8V)
- 45. VDD_D – Core VDD supply for power domain D (Nominal 0.8V)
- 46. VDD_C – Core VDD supply for power domain C (Nominal 0.8V)
- 47. VSS
- 48. VDDO – Supply for output pads (Nominal 1.8V)
- 49. GPIO[3] – ET16M 8 core general purpose input/output pin



- 50. CLK – System clock
- 51. VSS
- 52. VSS
- 53. GPIO[4] – ET16M 8 core general purpose input/output pin
- 54. GPIO[5] – ET16M 8 core general purpose input/output pin
- 55. GPIO[6] – ET16M 8 core general purpose input/output pin
- 56. GPIO[8] – ET16M 8 core general purpose input/output pin
- 57. GPIO[9] – ET16M 8 core general purpose input/output pin
- 58. GPIO[7] – ET16M 8 core general purpose input/output pin
- 59. VSS
- 60. VDD18_D – I/O supply for power domain D (Nominal 1.8V)
- 61. VDD_D – Core VDD supply for power domain D (Nominal 0.8V)
- 62. VDD_C – Core VDD supply for power domain C (Nominal 0.8V)
- 63. VDD_C – Core VDD supply for power domain C (Nominal 0.8V)
- 64. VDD_C – Core VDD supply for power domain C (Nominal 0.8V)
- 65. VDD18_D – I/O supply for power domain D (Nominal 1.8V)
- 66. VDD_D – Core VDD supply for power domain D (Nominal 0.8V)
- 67. VDD18_A – I/O supply for power domain A (Nominal 1.8V)
- 68. VDD_A – Core VDD supply for power domain A (Nominal 0.8V)
- 69. VSS



CONTENTS

70. GPIO[10] – ET16M 8 core general purpose input/output pin

71. XSPI_MODE[0] – xspi control mode pin 0

72. VSS

Power Domains

1. 5 – iRAM banks of 4MB of MRAM each
2. 1 -iRAM bank of 4MB
3. 8 core RISC-V domain
4. 8 core RISC-V memory domain including 16MB of MRAM



Changelog

Commit SHA	Author	Date	Message
246c4e0	Vijayvithal Jahagirdar	2026-06-22	Doc updates from oss.
a4250ac	Michael Jagers	2026-06-01	adding documentation for the MRAM.
fe106e5	Ying Hao Xu Lin	2026-04-09	Add more images to the CPU SS docs
4e7efb4	Ying Hao Xu Lin	2026-04-09	Update CPU SS diagram
02b17e5	Ying Hao Xu Lin	2026-04-09	Add CPU Subsystem high-level architecture and programming model
4043721	Kumaran S	2026-04-08	add gpio and i2c docs
ad7089f	Lavanya Jagadeeswaran	2026-04-08	added usage flow
0a94149	Lavanya Jagadeeswaran	2026-04-08	added qspi and uart docs from ip sheets
949411e	Vijayvithal	2026-04-07	Added cpu subsystem, updated docs.
61d496d	albert	2026-03-26	Update interrupts documentation
0475829	albert	2026-03-17	Update autogenerated documentation and registers DV files
ebd6202	Vijayvithal Jahagirdar	2026-03-11	Cleaned up r/w bits for system registers.



CONTENTS

Commit SHA	Author	Date	Message
f4c5154	Vijayvithal Jahagirdar	2026-03-10	Updated cpu subsystem to get the ral fix.
7a2d073	Vijayvithal Jahagirdar	2026-03-09	Updated address map for mram width and nic_config address
37b838b	Vijayvithal Jahagirdar	2026-03-04	(doc) Mram Registers, SRAM address is updated. regenerated pdf.
e0c4362	Vijayvithal Jahagirdar	2026-03-02	Updated sram base address.
fa117b3	Vijayvithal Jahagirdar	2026-02-28	updates xspi addresss spacing.
32c603a	Vijayvithal Jahagirdar	2026-02-27	Updated memory map.
cab7c6c	Vijayvithal Jahagirdar	2026-03-02	renamed ramrom as romram and ensured consistent name across files.
1cb8f89	Vijayvithal Jahagirdar	2026-03-02	Regenerated doc.
3683044	Vijayvithal Jahagirdar	2026-03-02	Updated Reference Manual with qspi and uart rdl
965f224	Vijayvithal Jahagirdar	2026-03-02	Updated QSPI.rdl
fbf7fac	Lavanya	2026-03-02	Update uart.rdl
0e7ca8b	Lavanya	2026-03-02	Update qspi.rdl
bec3cfb	Lavanya	2026-02-27	added qspi
32e874f	Kumaran	2026-02-27	update uart rdl
aecbb14	Vijayvithal	2026-02-21	Copied Registers from Shakti to uart and qspi rdl files.



Erbium Minion Isotope TRM

Commit SHA	Author	Date	Message
8d06f07	Michael Jaggers	2026-02-20	Updating documentation of the mram_pd and mram_dsleep_en.
50e9a2a	Vijayvithal	2026-02-20	Feature/integration
08bd83c	Vijayvithal	2026-02-13	Feature/integration
1523996	Michael Jaggers	2026-01-14	Updating top level Digital module.
8121462	Vijayvithal Jahagirdar	2026-01-28	added cpu and xspi memory maps, Added Yings DV Masterplan
8bf516e	Vijayvithal Jahagirdar	2026-01-27	Modified reset cause registers.
585290b	Vijayvithal Jahagirdar	2026-01-26	Initial Release of Erbium ET RTL.
dc37dbe	Vijayvithal Jahagirdar	2026-01-05	Added tentative schedule.
30ab7c2	Vijayvithal Jahagirdar	2025-12-16	Added Ying's ET Minion Overview
85d4400	Vijayvithal Jahagirdar	2025-11-26	Updated Interconnect chapter and System Registers
1898ae4	Vijayvithal Jahagirdar	2025-11-13	Document updated till bringup sequence.